Competitive Programmer's Handbook

竞赛程序员手册（简体中文版）

Antti Laaksonen 著
ChatGPT 中文翻译

原版：2018 年 7 月 3 日草稿
中文 AI 翻译稿：2026 年 10 月

目录

前言 **ix**

第一部分　基础技术 **1**

第一章　简介 **3**
1.1　编程语言 3
1.2　输入与输出 4
1.3　数值处理 6
1.4　缩短代码 8
1.5　数学基础 10
1.6　竞赛与资源 15

第二章　时间复杂度 **17**
2.1　计算规则 17
2.2　复杂度类别 20
2.3　估算效率 21
2.4　最大子数组和 22

第三章　排序 **25**
3.1　排序理论 25
3.2　C++ 中的排序 29
3.3　二分查找 31

第四章　数据结构 **35**
4.1　动态数组 35
4.2　集合结构 37
4.3　映射结构 38
4.4　迭代器与区间 39

4.5 其他结构 42
4.6 与排序的比较 46

第五章 完整搜索 **49**
5.1 生成子集 49
5.2 生成排列 51
5.3 回溯法 52
5.4 搜索剪枝 54
5.5 折半搜索 57

第六章 贪心算法 **59**
6.1 硬币问题 59
6.2 活动安排 60
6.3 任务与截止时间 62
6.4 最小化总和 63
6.5 数据压缩 64

第七章 动态规划 **67**
7.1 硬币问题 67
7.2 最长递增子序列 72
7.3 网格中的路径 73
7.4 背包问题 75
7.5 编辑距离 76
7.6 铺砖方案计数 78

第八章 摊还分析 **81**
8.1 双指针法 81
8.2 最近更小元素 83
8.3 滑动窗口最小值 84

第九章 区间查询 **87**
9.1 静态数组查询 88
9.2 树状数组 90
9.3 线段树 93
9.4 其他技术 97

第十章　位运算 **99**
10.1 二进制表示 99
10.2 位运算 100
10.3 集合的表示 102
10.4 位运算优化 104
10.5 动态规划 106

第二部分　图算法 **113**

第十一章　图论基础 **115**
11.1 图论术语 115
11.2 图的表示 115

第十二章　图的遍历 **123**
12.1 深度优先搜索 123
12.2 广度优先搜索 125
12.3 应用 127

第十三章　最短路 **131**
13.1 Bellman–Ford 算法 131
13.2 Dijkstra 算法 134
13.3 Floyd–Warshall 算法 137

第十四章　树算法 **141**
14.1 树的遍历 142
14.2 直径 143
14.3 所有最长路径 145
14.4 二叉树 147

第十五章　生成树 **149**
15.1 Kruskal 算法 150
15.2 并查集 153
15.3 Prim 算法 155

第十六章　有向图 **157**
16.1 拓扑排序 157
16.2 动态规划 159

16.3 后继路径 . . . 162
16.4 环检测 . . . 163

第十七章 强连通性 **165**
17.1 Kosaraju 算法 . . . 166
17.2 2SAT 问题 . . . 168

第十八章 树上查询 **171**
18.1 寻找祖先 . . . 171
18.2 子树与路径 . . . 172
18.3 最近公共祖先 . . . 175
18.4 离线算法 . . . 178

第十九章 路径与回路 **183**
19.1 欧拉路径 . . . 183
19.2 哈密顿路径 . . . 187
19.3 De Bruijn 序列 . . . 188
19.4 骑士巡游 . . . 189

第二十章 流与割 **191**
20.1 Ford–Fulkerson 算法 . . . 192
20.2 不相交路径 . . . 195
20.3 最大匹配 . . . 197
20.4 路径覆盖 . . . 200

第三部分 高级主题 203

第二十一章 数论 **205**
21.1 素数与因数 . . . 205
21.2 模运算 . . . 210
21.3 求解方程 . . . 212
21.4 其他结论 . . . 214

第二十二章 组合数学 **217**
22.1 二项式系数 . . . 218
22.2 Catalan 数 . . . 220
22.3 容斥原理 . . . 222

22.4 Burnside 引理 224
22.5 Cayley 公式 225

第二十三章 矩阵 **227**
23.1 运算 227
23.2 线性递推 230
23.3 图与矩阵 232

第二十四章 概率 **237**
24.1 概率计算 237
24.2 事件 238
24.3 随机变量 240
24.4 Markov 链 242
24.5 随机化算法 243

第二十五章 博弈论 **247**
25.1 博弈状态 247
25.2 Nim 博弈 249
25.3 Sprague–Grundy 定理 250

第二十六章 字符串算法 **255**
26.1 字符串术语 255
26.2 Trie 结构 256
26.3 字符串哈希 257
26.4 Z 算法 259

第二十七章 平方根算法 **263**
27.1 组合算法 264
27.2 整数拆分 266
27.3 莫队算法 267

第二十八章 线段树进阶 **269**
28.1 懒惰传播 270
28.2 动态线段树 273
28.3 节点中的数据结构 275
28.4 二维线段树 276

第二十九章　计算几何 **277**
29.1 复数 278
29.2 点与直线 280
29.3 多边形面积 283
29.4 距离函数 284

第三十章　扫描线算法 **287**
30.1 交点 288
30.2 最近点对问题 289
30.3 凸包问题 290

参考文献 **293**

前言

本书旨在为读者系统地介绍算法竞赛。我们假定读者已经掌握编程基础，但不要求具备任何算法竞赛经验。

本书尤其适合希望学习算法，并可能参加国际信息学奥林匹克竞赛（IOI）或国际大学生程序设计竞赛（ICPC）的学生。当然，任何对算法竞赛感兴趣的读者也都适合阅读本书。

成为一名优秀的竞赛程序员需要很长时间，但这个过程也提供了学习大量知识的机会。只要投入时间阅读本书、解决问题并参加比赛，你一定能够对算法建立良好而全面的理解。

本书仍在持续完善中。读者可随时将意见发送至 ahslaaks@cs.helsinki.fi。

赫尔辛基，2019 年 8 月

Antti Laaksonen

中文版说明

本简体中文版本由 ChatGPT 基于作者公开的官方 LaTeX 源码翻译并排版。中文稿力求忠实保留原书的结构、公式、代码、表格、插图与编号，但并非作者发布的官方译本。如中文表述与英文原文存在差异，应以英文原文为准。

原作署名为 Antti Laaksonen，采用知识共享署名-非商业性使用-相同方式共享 4.0 国际许可协议（CC BY-NC-SA 4.0）。本中文翻译稿依照同一许可协议发布，并保留原作者署名与许可声明。

第一部分

基础技术

第一章　简介

算法竞赛结合了两个主题：（1）算法设计；（2）算法实现。

算法设计包括问题求解与数学思维，需要分析问题并创造性地解决问题的能力。用于解决问题的算法必须既正确又高效，而题目的核心往往正是设计一种高效算法。

算法理论知识对竞赛程序员十分重要。通常，一道题的解法由知名技巧与新的洞察共同组成。算法竞赛中出现的技巧，也构成了算法科学研究的基础。

算法实现要求良好的编程能力。在算法竞赛中，评测系统会用一组测试数据检验所实现的算法。因此，仅有正确的算法思路还不够，实现也必须正确。

竞赛中的良好编码风格应当直接、简洁。由于时间有限，程序必须快速写出。与传统软件工程不同，竞赛程序很短（通常最多几百行代码），赛后也无须维护。

1.1　编程语言

目前，算法竞赛中最流行的编程语言是 C++、Python 和 Java。例如，在 2017 年 Google Code Jam 排名前 3000 的参赛者中，79% 使用 C++，16% 使用 Python，8% 使用 Java [29]；部分参赛者使用了多种语言。

许多人认为 C++ 是竞赛程序员的最佳选择，而且几乎所有竞赛系统都支持 C++。C++ 的优势在于执行效率很高，并且标准库包含大量数据结构与算法。

另一方面，掌握多种语言并了解各自优势也很有帮助。例如，若题目需要大整数运算，Python 可能是不错的选择，因为它内置了大整数运算。不过，大多数竞赛题都会避免让某一种特定编程语言获得不公平优势。

本书所有示例程序均使用 C++ 编写，并经常使用标准库中的数据结构和算法。程序遵循 C++11 标准，目前绝大多数竞赛均可使用该标准。如果你还不会 C++，现在正是开始学习的好时机。

C++ 代码模板

算法竞赛中典型的 C++ 代码模板如下：

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // solution comes here
}
```

代码开头的 #include 行是 g++ 编译器提供的功能，可一次包含整个标准库。因此，无须分别包含 iostream、vector、algorithm 等库，它们都会自动可用。

using 行声明可以在代码中直接使用标准库的类和函数。没有这一行时，例如必须写成 std::cout；有了它，只写 cout 即可。

可以使用以下命令编译代码：

```
g++ -std=c++11 -O2 -Wall test.cpp -o test
```

该命令由源代码 test.cpp 生成二进制文件 test。编译器遵循 C++11 标准（-std=c++11），优化代码（-O2），并显示潜在错误的警告（-Wall）。

1.2 输入与输出

大多数竞赛使用标准流读入和输出。在 C++ 中，标准输入流是 cin，标准输出流是 cout；也可以使用 C 语言函数 scanf 和 printf。

程序输入通常由以空格和换行符分隔的数字与字符串组成，可以按如下方式从 cin 读取：

```
int a, b;
string x;
cin >> a >> b >> x;
```

只要输入的各元素之间至少有一个空格或换行符，这种代码就能正常工作。例如，上述代码可以读取下面两种输入：

```
123 456 monkey
```

```
123 456
monkey
```

cout 流的输出方式如下：

```
int a = 123, b = 456;
string x = "monkey";
cout << a << " " << b << " " << x << "\n";
```

输入输出有时会成为程序的性能瓶颈。在代码开头加入以下两行可以提高输入输出效率：

```
ios::sync_with_stdio(0);
cin.tie(0);
```

注意，换行符 "\n" 比 endl 更快，因为 endl 总会触发刷新缓冲区操作。

C 语言函数 scanf 和 printf 是 C++ 标准流的替代方案。它们通常稍快，但也更难使用。下面的代码读取两个整数：

```
int a, b;
scanf("%d %d", &a, &b);
```

下面的代码输出两个整数：

```
int a = 123, b = 456;
printf("%d %d\n", a, b);
```

有时程序需要读取可能包含空格的整行输入，可以使用 getline 函数：

```
string s;
getline(cin, s);
```

若数据量未知，可以使用以下循环：

```
while (cin >> x) {
    // code
}
```

该循环逐个读取输入元素，直到没有更多数据。

有些竞赛系统使用文件输入输出。一种简单做法是照常使用标准流编写代

码，但在代码开头加入以下两行：

```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```

此后，程序从文件 “input.txt” 读取输入，并把输出写入文件 “output.txt”。

1.3 数值处理

整数

算法竞赛中最常用的整数类型是 32 位的 int，取值范围为 $-2^{31} \ldots 2^{31}-1$，约为 $-2 \cdot 10^9 \ldots 2 \cdot 10^9$。若 int 不够用，可以采用 64 位的 long long，其取值范围为 $-2^{63} \ldots 2^{63}-1$，约为 $-9 \cdot 10^{18} \ldots 9 \cdot 10^{18}$。

下面的代码定义了一个 long long 变量：

```
long long x = 123456789123456789LL;
```

后缀 LL 表示该数的类型为 long long。

使用 long long 时的常见错误是，代码中的某处仍使用了 int。例如，下面的代码包含一个不易察觉的错误：

```
int a = 123456789;
long long b = a*a;
cout << b << "\n"; // -1757895751
```

虽然变量 b 的类型是 long long，但表达式 a*a 中的两个数都是 int，结果也会按 int 计算。因此，变量 b 得到错误结果。可以把 a 的类型改为 long long，或把表达式改为 (long long)a*a。

竞赛题通常会保证 long long 足够使用。不过，g++ 编译器还提供 128 位类型 __int128_t，其取值范围为 $-2^{127} \ldots 2^{127}-1$，约为 $-10^{38} \ldots 10^{38}$。但并非所有竞赛系统都支持该类型。

模运算

用 $x \bmod m$ 表示 x 除以 m 的余数。例如，$17 \bmod 5 = 2$，因为 $17 = 3 \cdot 5 + 2$。

有时一道题的答案非常大，但只需输出 “模 m” 的结果，即答案除以 m 的余数（例如 “模 10^9+7”）。这样即使实际答案很大，也只需使用 int 和 long long。

余数的一个重要性质是，在加法、减法和乘法中，可以先取模再进行运算：

$$\begin{aligned}(a+b) \bmod m &= (a \bmod m + b \bmod m) \bmod m \\ (a-b) \bmod m &= (a \bmod m - b \bmod m) \bmod m \\ (a \cdot b) \bmod m &= (a \bmod m \cdot b \bmod m) \bmod m\end{aligned}$$

因此，可以在每次运算后取模，使数值始终不会变得过大。

例如，下面的代码计算 $n!$（n 的阶乘）模 m 的结果：

```
long long x = 1;
for (int i = 2; i <= n; i++) {
    x = (x*i)%m;
}
cout << x%m << "\n";
```

通常希望余数始终位于 $0 \ldots m-1$。然而，在 C++ 等语言中，负数取模的结果为零或负数。为确保余数非负，可以先照常取模，若结果为负再加上 m：

```
x = x%m;
if (x < 0) x += m;
```

不过，只有代码中存在减法、余数可能变为负数时才需要这样处理。

浮点数

算法竞赛中常用的浮点类型是 64 位 double，以及 g++ 扩展提供的 80 位 long double。大多数情况下 double 已经足够，但 long double 精度更高。

题目通常会说明答案所需精度。一种简单的输出方法是使用 printf，并在格式字符串中指定小数位数。例如，下面的代码以 9 位小数输出 x：

```
printf("%.9f\n", x);
```

使用浮点数的困难在于，有些数无法被精确表示，从而产生舍入误差。例如，下面代码的结果令人意外：

```
double x = 0.3*3+0.1;
printf("%.20f\n", x); // 0.99999999999999988898
```

由于舍入误差，x 的值略小于 1，而正确值应为 1。

使用 == 运算符比较浮点数存在风险，因为两个理论上相等的值可能因精度误差而不相等。更好的做法是：若两数之差小于一个很小的数 ε，就认为它

们相等。

实践中可按如下方式比较（$\varepsilon = 10^{-9}$）：

```
if (abs(a-b) < 1e-9) {
    // a and b are equal
}
```

注意，尽管浮点数可能不精确，但一定范围内的整数仍可精确表示。例如，double 能精确表示绝对值不超过 2^{53} 的所有整数。

1.4 缩短代码

算法竞赛要求尽快写出程序，因此短小的代码最为理想。竞赛程序员常常为数据类型和其他代码元素定义更短的名称。

类型名称

使用 typedef 可以为数据类型指定更短的名称。例如，long long 较长，可以定义简称 ll：

```
typedef long long ll;
```

此后，代码

```
long long a = 123456789;
long long b = 987654321;
cout << a*b << "\n";
```

可以缩写为：

```
ll a = 123456789;
ll b = 987654321;
cout << a*b << "\n";
```

typedef 也可用于更复杂的类型。例如，下面的代码用 vi 表示整数向量，用 pi 表示包含两个整数的二元组。

```
typedef vector<int> vi;
typedef pair<int,int> pi;
```

宏

另一种缩短代码的方法是定义**宏**。宏会使代码中的特定字符串在编译前被替换。C++ 使用关键字 #define 定义宏。

例如，可以定义以下宏：

```
#define F first
#define S second
#define PB push_back
#define MP make_pair
```

此后，代码

```
v.push_back(make_pair(y1,x1));
v.push_back(make_pair(y2,x2));
int d = v[i].first+v[i].second;
```

可以缩写为：

```
v.PB(MP(y1,x1));
v.PB(MP(y2,x2));
int d = v[i].F+v[i].S;
```

宏也可以带参数，从而缩短循环等结构。例如，可以定义如下宏：

```
#define REP(i,a,b) for (int i = a; i <= b; i++)
```

此后，代码

```
for (int i = 1; i <= n; i++) {
    search(i);
}
```

可以缩写为：

```
REP(i,1,n) {
    search(i);
}
```

宏有时会引入难以发现的错误。例如，考虑下面这个计算平方的宏：

```
#define SQ(a) a*a
```

这个宏并非总能按预期工作。例如，代码

```
cout << SQ(3+3) << "\n";
```

等价于

```
cout << 3+3*3+3 << "\n"; // 15
```

更好的宏定义如下：

```
#define SQ(a) (a)*(a)
```

此时代码

```
cout << SQ(3+3) << "\n";
```

等价于

```
cout << (3+3)*(3+3) << "\n"; // 36
```

1.5 数学基础

数学在算法竞赛中扮演重要角色；没有良好的数学能力，就很难成为成功的竞赛程序员。本节介绍本书后续内容所需的一些重要数学概念与公式。

求和公式

形如

$$\sum_{x=1}^{n} x^k = 1^k + 2^k + 3^k + \ldots + n^k,$$

的和，其中 k 为正整数，都有一个 $k+1$ 次多项式形式的闭式公式。例如[1]，

$$\sum_{x=1}^{n} x = 1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$$

以及

$$\sum_{x=1}^{n} x^2 = 1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

[1]这类求和甚至存在通用公式，称为 **Faulhaber 公式**，但它过于复杂，此处不作介绍。

等差数列是相邻两项之差恒定的数列。例如，

$$3, 7, 11, 15$$

是公差为 4 的等差数列。等差数列之和可用下式计算：

$$\underbrace{a + \cdots + b}_{n \text{ 项}} = \frac{n(a+b)}{2}$$

其中 a 是首项，b 是末项，n 是项数。例如，

$$3 + 7 + 11 + 15 = \frac{4 \cdot (3+15)}{2} = 36.$$

该公式基于如下事实：总和由 n 个数构成，每个数的平均值为 $(a+b)/2$。

等比数列是相邻两项之比恒定的数列。例如，

$$3, 6, 12, 24$$

是公比为 2 的等比数列。等比数列之和可用下式计算：

$$a + ak + ak^2 + \cdots + b = \frac{bk - a}{k - 1}$$

其中 a 是首项，b 是末项，相邻两项之比为 k。例如，

$$3 + 6 + 12 + 24 = \frac{24 \cdot 2 - 3}{2 - 1} = 45.$$

该公式可按如下方式推导。令

$$S = a + ak + ak^2 + \cdots + b.$$

两边同时乘以 k，得到

$$kS = ak + ak^2 + ak^3 + \cdots + bk,$$

解方程

$$kS - S = bk - a$$

即可得到上述公式。

等比数列求和的一个特殊情况是

$$1 + 2 + 4 + 8 + \ldots + 2^{n-1} = 2^n - 1.$$

调和和形如

$$\sum_{x=1}^{n} \frac{1}{x} = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n}.$$

调和和的一个上界是 $\log_2(n) + 1$。具体而言，可以把每一项 $1/k$ 的分母 k 改为不超过它的最大 2 的幂。例如，当 $n = 6$ 时，可作如下估计：

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} \leq 1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}.$$

该上界由 $\log_2(n) + 1$ 个部分组成（1、$2 \cdot 1/2$、$4 \cdot 1/4$ 等），每部分的值至多为 1。

集合论

集合是若干元素的汇集。例如，集合

$$X = \{2, 4, 7\}$$

包含元素 2、4、7。符号 $\emptyset$ 表示空集，$|S|$ 表示集合 S 的大小，即集合中的元素个数。例如，上述集合满足 $|X| = 3$。

若集合 S 包含元素 x，记为 $x \in S$；否则记为 $x \notin S$。例如，上述集合满足

$$4 \in X \quad \text{且} \quad 5 \notin X.$$

可以通过集合运算构造新集合：

- **交集** $A \cap B$ 由同时属于 A 和 B 的元素组成。例如，若 $A = \{1, 2, 5\}$、$B = \{2, 4\}$，则 $A \cap B = \{2\}$。
- **并集** $A \cup B$ 由属于 A 或 B（或同时属于二者）的元素组成。例如，若 $A = \{3, 7\}$、$B = \{2, 3, 8\}$，则 $A \cup B = \{2, 3, 7, 8\}$。
- **补集** $\bar{A}$ 由不属于 A 的元素组成。补集的含义取决于包含所有可能元素的**全集**。例如，若 $A = \{1, 2, 5, 7\}$，全集为 $\{1, 2, \ldots, 10\}$，则 $\bar{A} = \{3, 4, 6, 8, 9, 10\}$。
- **差集** $A \setminus B = A \cap \bar{B}$ 由属于 A 但不属于 B 的元素组成。注意，B 可以包含不属于 A 的元素。例如，若 $A = \{2, 3, 7, 8\}$、$B = \{3, 5, 8\}$，则 $A \setminus B = \{2, 7\}$。

若 A 的每个元素都属于 S，则称 A 是 S 的**子集**，记作 $A \subset S$。集合 S 总共有 $2^{|S|}$ 个子集，其中包括空集。例如，集合 $\{2, 4, 7\}$ 的子集为

$$\emptyset,\ \{2\},\ \{4\},\ \{7\},\ \{2, 4\},\ \{2, 7\},\ \{4, 7\} \text{ and } \{2, 4, 7\}.$$

常用集合包括 $\mathbb{N}$（自然数）、$\mathbb{Z}$（整数）、$\mathbb{Q}$（有理数）和 $\mathbb{R}$（实数）。根据具体语境，$\mathbb{N}$ 有两种定义：$\mathbb{N} = \{0, 1, 2, \ldots\}$ 或 $\mathbb{N} = \{1, 2, 3, ...\}$。

还可以使用如下规则构造集合：

$$\{f(n) : n \in S\},$$

其中 $f(n)$ 是某个函数。该集合包含所有形如 $f(n)$ 的元素，其中 n 属于 S。例如，集合

$$X = \{2n : n \in \mathbb{Z}\}$$

包含所有偶数。

逻辑

逻辑表达式的值为**真**（1）或**假**（0）。最重要的逻辑运算符是 ¬（**否定**）、∧（**合取**）、∨（**析取**）、⇒（**蕴含**）和 ⇔（**等价**）。下表给出这些运算符的含义：

A	B	$\neg A$	$\neg B$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
0	0	1	1	0	0	1	1
0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0
1	1	0	0	1	1	1	1

表达式 $\neg A$ 的值与 A 相反。$A \wedge B$ 在 A、B 均为真时为真；$A \vee B$ 在 A、B 至少一个为真时为真。若 A 为真时 B 也总为真，则 $A \Rightarrow B$ 为真。若 A、B 同真或同假，则 $A \Leftrightarrow B$ 为真。

谓词是其真假取决于参数的表达式，通常用大写字母表示。例如，可定义谓词 $P(x)$：当且仅当 x 为素数时为真。按此定义，$P(7)$ 为真，而 $P(8)$ 为假。

量词把逻辑表达式与集合中的元素联系起来。最重要的量词是 ∀（**任意**）和 ∃（**存在**）。例如，

$$\forall x(\exists y(y < x))$$

表示对集合中的每个元素 x，都存在集合中的元素 y 满足 $y < x$。该命题在整数集中为真，在自然数集中为假。

利用上述记号可以表达多种逻辑命题。例如，

$$\forall x((x > 1 \wedge \neg P(x)) \Rightarrow (\exists a(\exists b(a > 1 \wedge b > 1 \wedge x = ab))))$$

表示若整数 $x > 1$ 且不是素数，则存在大于 1 的整数 a、b，使得 $x = ab$。该命题在整数集中为真。

函数

函数 $\lfloor x \rfloor$ 把 x 向下取整，函数 $\lceil x \rceil$ 把 x 向上取整。例如，

$$\lfloor 3/2 \rfloor = 1 \quad \text{且} \quad \lceil 3/2 \rceil = 2.$$

函数 $\min(x_1, x_2, \ldots, x_n)$ 与 $\max(x_1, x_2, \ldots, x_n)$ 分别返回 $x_1, x_2, \ldots, x_n$ 中的最小值和最大值。例如，

$$\min(1, 2, 3) = 1 \quad \text{且} \quad \max(1, 2, 3) = 3.$$

阶乘 $n!$ 可定义为

$$\prod_{x=1}^{n} x = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n$$

或递归定义为

$$\begin{aligned} 0! &= 1 \\ n! &= n \cdot (n-1)! \end{aligned}$$

Fibonacci 数会出现在许多问题中，可递归定义如下：

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

最初若干个 Fibonacci 数为

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

计算 Fibonacci 数还有一个闭式公式，有时称为**Binet 公式**：

$$f(n) = \frac{(1+\sqrt{5})^n - (1-\sqrt{5})^n}{2^n\sqrt{5}}.$$

对数

数 x 的**对数**记作 $\log_k(x)$，其中 k 是底数。根据定义，$\log_k(x) = a$ 当且仅当 $k^a = x$。

对数的一项实用性质是：$\log_k(x)$ 等于把 x 连续除以 k 直至得到 1 所需的次数。例如，$\log_2(32) = 5$，因为需要除以 2 五次：

$$32 \to 16 \to 8 \to 4 \to 2 \to 1$$

对数常用于算法分析，因为许多高效算法每一步都会把某个量减半。因此，可以用对数估计算法效率。

乘积的对数为

$$\log_k(ab) = \log_k(a) + \log_k(b),$$

因此，

$$\log_k(x^n) = n \cdot \log_k(x).$$

商的对数为

$$\log_k\left(\frac{a}{b}\right) = \log_k(a) - \log_k(b).$$

另一个实用公式是

$$\log_u(x) = \frac{\log_k(x)}{\log_k(u)},$$

借助该公式，只要能计算某个固定底数的对数，就能计算任意底数的对数。

数 x 的**自然对数** $\ln(x)$ 以 $e \approx 2.71828$ 为底。对数的另一项性质是：整数 x 的 b 进制表示位数为 $\lfloor \log_b(x) + 1 \rfloor$。例如，123 的二进制表示为 1111011，并且 $\lfloor \log_2(123) + 1 \rfloor = 7$。

1.6 竞赛与资源

IOI

国际信息学奥林匹克竞赛（IOI）是面向中学生的年度编程竞赛。每个国家可派出由四名学生组成的代表队，通常约有来自 80 个国家的 300 名选手参赛。

IOI 包含两场各五小时的比赛，每场要求选手解决三道难度不一的算法题。题目划分为若干子任务，每个子任务有相应分值。尽管选手以代表队形式参赛，比赛仍按个人进行。

IOI 教学大纲 [41] 规定了题目可以涉及的主题。本书覆盖了大纲中的几乎所有主题。

各国通过国内竞赛选拔 IOI 参赛者。IOI 之前还会举办许多区域竞赛，例如波罗的海信息学奥林匹克竞赛（BOI）、中欧信息学奥林匹克竞赛（CEOI）和亚太信息学奥林匹克竞赛（APIO）。

一些国家会为未来的 IOI 选手举办在线训练赛，例如克罗地亚信息学公开赛 [11] 和美国计算机奥林匹克竞赛 [68]。此外，网上还提供大量波兰竞赛题目 [60]。

ICPC

国际大学生程序设计竞赛（ICPC）是面向大学生的年度编程竞赛。每支队伍由三名学生组成；与 IOI 不同，队员共同协作，每队只有一台计算机。

ICPC 分为多个阶段，最终最优秀的队伍受邀参加全球总决赛。虽然参赛者多达数万人，总决赛名额却很少[2]，因此在一些赛区，仅仅晋级总决赛就已是重大成就。

每场 ICPC 比赛持续五小时，队伍需要解决约十道算法题。只有高效通过全部测试数据的解答才会被接受。比赛期间，选手可以查看其他队伍的结果；但最后一小时封榜，无法看到最新提交的评测结果。

[2]总决赛名额每年不同；2017 年共有 133 个名额。

ICPC 可能涉及的主题不像 IOI 那样有明确规定。可以确定的是，ICPC 要求更广泛的知识，尤其需要更强的数学能力。

在线竞赛

还有许多面向所有人的在线竞赛。目前最活跃的竞赛网站是 Codeforces，基本每周都会举办比赛。Codeforces 将参赛者分为两个组别：初学者参加 Div2，经验更丰富的程序员参加 Div1。其他竞赛网站包括 AtCoder、CS Academy、HackerRank 和 Topcoder。

一些公司会举办设有线下决赛的在线竞赛，例如 Facebook Hacker Cup、Google Code Jam 和 Yandex.Algorithm。当然，公司也会借助这些竞赛招聘人才；在比赛中取得好成绩是证明能力的好方法。

书籍

除本书之外，已有一些专注于算法竞赛和算法问题求解的书籍：

- S. S. Skiena、M. A. Revilla：《编程挑战：程序设计竞赛训练手册》[59]
- S. Halim、F. Halim：《算法竞赛 3：程序设计竞赛的新下界》[33]
- K. Diks 等：《寻找挑战？华沙大学程序设计竞赛终极题集》[15]

前两本书面向初学者，最后一本包含高级内容。

当然，通用算法书籍也适合竞赛程序员。较为流行的书籍包括：

- T. H. Cormen、C. E. Leiserson、R. L. Rivest、C. Stein：《算法导论》[13]
- J. Kleinberg、É. Tardos：《算法设计》[45]
- S. S. Skiena：《算法设计手册》[58]

第二章　时间复杂度

算法的效率在算法竞赛中十分重要。通常，设计一个能缓慢解决问题的算法并不困难，真正的挑战在于构造快速算法。如果算法太慢，就只能得到部分分数，甚至完全无法得分。

算法的**时间复杂度**估计该算法处理某个输入时所需的时间。其思想是用一个以输入规模为参数的函数来表示效率。通过计算时间复杂度，我们无需实现算法就能判断它是否足够快。

2.1　计算规则

算法的时间复杂度记作 $O(\cdots)$，其中省略号表示某个函数。变量 n 通常表示输入规模。例如，若输入是一个数值数组，则 n 表示数组大小；若输入是一个字符串，则 n 表示字符串长度。

循环

算法运行缓慢的常见原因之一，是其中包含许多遍历输入的循环。嵌套循环越多，算法就越慢。如果有 k 层嵌套循环，时间复杂度就是 $O(n^k)$。

例如，以下代码的时间复杂度为 $O(n)$：

```
for (int i = 1; i <= n; i++) {
    // code
}
```

而以下代码的时间复杂度为 $O(n^2)$：

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // code
    }
}
```

数量级

时间复杂度并不表示循环内代码执行的精确次数，只表示其数量级。在以下示例中，循环内代码分别执行 $3n$、$n+5$ 和 $\lceil n/2 \rceil$ 次，但每段代码的时间复杂度都是 $O(n)$。

```
for (int i = 1; i <= 3*n; i++) {
    // code
}
```

```
for (int i = 1; i <= n+5; i++) {
    // code
}
```

```
for (int i = 1; i <= n; i += 2) {
    // code
}
```

再看一个例子，以下代码的时间复杂度为 $O(n^2)$：

```
for (int i = 1; i <= n; i++) {
    for (int j = i+1; j <= n; j++) {
        // code
    }
}
```

多个阶段

如果算法由若干依次执行的阶段组成，总时间复杂度等于单个阶段中最大的时间复杂度。这是因为最慢的阶段通常就是代码的瓶颈。

例如，以下代码由时间复杂度分别为 $O(n)$、$O(n^2)$ 和 $O(n)$ 的三个阶段组成，因此总时间复杂度为 $O(n^2)$。

```
for (int i = 1; i <= n; i++) {
    // code
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
```

```
        // code
    }
}
for (int i = 1; i <= n; i++) {
    // code
}
```

多个变量

有时，时间复杂度取决于多个因素。此时时间复杂度公式中会包含多个变量。

例如，以下代码的时间复杂度为 $O(nm)$：

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        // code
    }
}
```

递归

递归函数的时间复杂度取决于函数的调用次数和单次调用的时间复杂度，总时间复杂度是这两个量的乘积。

例如，考虑以下函数：

```
void f(int n) {
    if (n == 1) return;
    f(n-1);
}
```

调用 f(n) 会产生 n 次函数调用，每次调用的时间复杂度为 $O(1)$，因此总时间复杂度为 $O(n)$。

再考虑以下函数：

```
void g(int n) {
    if (n == 1) return;
    g(n-1);
    g(n-1);
```

```
}
```

除 $n = 1$ 外，每次函数调用都会再产生两次调用。下面考察以参数 n 调用 g 时的情况。下表列出了这一次调用所产生的全部函数调用：

函数调用	调用次数
$g(n)$	1
$g(n-1)$	2
$g(n-2)$	4
$\cdots$	$\cdots$
$g(1)$	2^{n-1}

由此可知，时间复杂度为

$$1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1 = O(2^n).$$

2.2 复杂度类别

下面列出算法中常见的时间复杂度：

$O(1)$ **常数时间**算法的运行时间与输入规模无关。典型的常数时间算法是直接用公式计算答案。

$O(\log n)$ **对数**算法通常在每一步把输入规模减半。此类算法的运行时间是对数级的，因为 $\log_2 n$ 等于把 n 不断除以 2 直至得到 1 所需的次数。

$O(\sqrt{n})$ **平方根算法**比 $O(\log n)$ 慢，但比 $O(n)$ 快。平方根具有一个特殊性质：$\sqrt{n} = n/\sqrt{n}$，因此在某种意义上，$\sqrt{n}$ 位于输入规模的中间位置。

$O(n)$ **线性**算法以常数次遍历整个输入。这通常是可能达到的最佳时间复杂度，因为报告答案之前一般至少要访问每个输入元素一次。

$O(n \log n)$ 这种时间复杂度往往意味着算法对输入进行了排序，因为高效排序算法的时间复杂度是 $O(n \log n)$。另一种可能是，算法使用了每次操作耗时 $O(\log n)$ 的数据结构。

$O(n^2)$ **平方级**算法通常包含两层嵌套循环。可以在 $O(n^2)$ 时间内遍历输入元素的所有二元组。

$O(n^3)$ **立方级**算法通常包含三层嵌套循环。可以在 $O(n^3)$ 时间内遍历输入元素的所有三元组。

$O(2^n)$ 这种时间复杂度往往意味着算法遍历输入元素的所有子集。例如，$\{1,2,3\}$ 的子集为 $\emptyset$、$\{1\}$、$\{2\}$、$\{3\}$、$\{1,2\}$、$\{1,3\}$、$\{2,3\}$ 和 $\{1,2,3\}$。

$O(n!)$ 这种时间复杂度往往意味着算法遍历输入元素的所有排列。例如，$\{1,2,3\}$ 的排列为 $(1,2,3)$、$(1,3,2)$、$(2,1,3)$、$(2,3,1)$、$(3,1,2)$ 和 $(3,2,1)$。

若算法的时间复杂度至多为 $O(n^k)$，其中 k 是常数，则称该算法是**多项式时间**算法。上面除 $O(2^n)$ 和 $O(n!)$ 以外的时间复杂度都是多项式时间。实际中常数 k 通常很小，因此，多项式时间复杂度大体意味着算法是高效的。

本书中的大多数算法都是多项式时间算法。然而，许多重要问题目前还没有已知的多项式时间算法，也就是说，人们尚不知道如何高效地解决它们。**NP困难**问题就是一类重要的、尚无已知多项式时间算法的问题[1]。

2.3 估算效率

通过计算算法的时间复杂度，可以在实现之前判断它对当前问题而言是否足够高效。估算的出发点是：现代计算机每秒可以执行数亿次操作。

例如，假设某问题的时间限制为一秒，输入规模为 $n = 10^5$。若时间复杂度为 $O(n^2)$，算法将执行大约 $(10^5)^2 = 10^{10}$ 次操作。这至少需要几十秒，因此该算法似乎太慢，无法解决这个问题。

另一方面，我们也可以根据输入规模推测解决该问题所需的时间复杂度。假设时间限制为一秒，下表给出了一些实用估计。

输入规模	所需时间复杂度
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$ 或 $O(n)$
n 很大	$O(1)$ 或 $O(\log n)$

例如，当输入规模为 $n = 10^5$ 时，题目很可能要求算法的时间复杂度为 $O(n)$ 或 $O(n \log n)$。这一信息排除了会产生更差时间复杂度的方案，使算法设计更容易。

[1]这一主题的经典著作是 M. R. Garey 和 D. S. Johnson 的 *Computers and Intractability: A Guide to the Theory of NP-Completeness* [28]。

不过，必须记住时间复杂度只是效率的估计，因为它隐藏了常数因子。例如，某个 $O(n)$ 算法可能执行 $n/2$ 次操作，也可能执行 $5n$ 次操作。这会显著影响算法的实际运行时间。

2.4 最大子数组和

解决同一个问题通常有多种算法，它们的时间复杂度可能不同。本节讨论一个经典问题，它有一个直观的 $O(n^3)$ 解法；通过改进算法，又可以把时间降到 $O(n^2)$，甚至 $O(n)$。

给定一个包含 n 个数的数组，我们的任务是计算**最大子数组和**，即数组中一段连续元素序列可能取得的最大和[2]。当数组中可能含有负数时，这个问题才有意义。例如，在数组

-1	2	4	-3	5	2	-5	2

中，下面的子数组产生最大和 10：

-1	2	4	-3	5	2	-5	2

我们允许空子数组，因此最大子数组和始终至少为 0。

算法 1

一种直接的解法是遍历所有可能的子数组，计算每个子数组的元素和，并维护最大值。以下代码实现了该算法：

```
int best = 0;
for (int a = 0; a < n; a++) {
    for (int b = a; b < n; b++) {
        int sum = 0;
        for (int k = a; k <= b; k++) {
            sum += array[k];
        }
        best = max(best,sum);
    }
}
cout << best << "\n";
```

[2]J. Bentley 的著作 *Programming Pearls* [8] 推广了这个问题。

变量 a 和 b 确定子数组的首、尾下标，元素和存入变量 sum。变量 best 保存搜索过程中找到的最大和。

算法包含三层遍历输入的嵌套循环，因此时间复杂度为 $O(n^3)$。

算法 2

去掉一层循环就能轻易提高算法 1 的效率。我们可以在子数组右端点移动时同步计算其元素和，得到以下代码：

```
int best = 0;
for (int a = 0; a < n; a++) {
    int sum = 0;
    for (int b = a; b < n; b++) {
        sum += array[b];
        best = max(best,sum);
    }
}
cout << best << "\n";
```

修改之后，时间复杂度为 $O(n^2)$。

算法 3

令人惊讶的是，这个问题可以在 $O(n)$ 时间内解决[3]，也就是说，只需一层循环。其思想是对每个数组位置，计算以该位置结尾的子数组所能取得的最大和；问题的答案就是这些和中的最大值。

考虑寻找以位置 k 结尾的最大和子数组这一子问题，有两种可能：

1. 子数组只包含位置 k 上的元素。
2. 子数组由一个以位置 $k-1$ 结尾的子数组，再接上位置 k 的元素构成。

在后一种情况下，为了找到和最大的子数组，以位置 $k-1$ 结尾的部分也应当具有最大和。因此，只要从左到右依次计算每个结束位置对应的最大子数组和，就能高效解决问题。

以下代码实现了该算法：

```
int best = 0, sum = 0;
for (int k = 0; k < n; k++) {
```

[3]在 [8] 中，这个线性时间算法归功于 J. B. Kadane，因此它有时称为**Kadane 算法**。

```
    sum = max(array[k],sum+array[k]);
    best = max(best,sum);
}
cout << best << "\n";
```

该算法只有一层遍历输入的循环，所以时间复杂度为 $O(n)$。这也是可能达到的最佳时间复杂度，因为任何算法都必须至少检查每个数组元素一次。

效率比较

考察算法在实际中的效率很有意义。下表列出了在现代计算机上，以上算法面对不同 n 值时的运行时间。

每次测试的输入均随机生成，读取输入所需的时间未计入测量。

数组大小 n	算法 1	算法 2	算法 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	$>$ 10.0 s	0.1 s	0.0 s
10^5	$>$ 10.0 s	5.3 s	0.0 s
10^6	$>$ 10.0 s	$>$ 10.0 s	0.0 s
10^7	$>$ 10.0 s	$>$ 10.0 s	0.0 s

比较结果表明，当输入规模较小时，所有算法都很高效；但输入增大后，各算法的运行时间就会出现显著差异。算法 1 在 $n = 10^4$ 时开始变慢，算法 2 在 $n = 10^5$ 时开始变慢。只有算法 3 即使面对最大的输入，也能立即处理完毕。

第三章　排序

排序是一个基本的算法设计问题。许多高效算法把排序作为子程序，因为元素有序后，数据往往更容易处理。

例如，“数组中是否包含两个相等元素?”可以借助排序轻松解决。若存在两个相等元素，排序后它们会彼此相邻，因而很容易找到。“数组中出现次数最多的元素是什么?”也可用类似方法解决。

排序算法种类繁多，也是展示不同算法设计技术的良好范例。高效的通用排序算法时间复杂度为 $O(n \log n)$；许多以排序为子程序的算法也具有这一复杂度。

3.1　排序理论

排序的基本问题如下：

给定一个包含 n 个元素的数组，将其中元素按递增顺序排列。

例如，数组

1	3	8	2	9	2	5	6

排序后变为：

1	2	2	3	5	6	8	9

$O(n^2)$ 算法

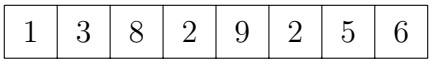
简单排序算法通常需要 $O(n^2)$ 时间。这类算法代码很短，一般由两个嵌套循环组成。著名的 $O(n^2)$ 排序算法是**冒泡排序**，其中元素会按数值大小在数组中逐渐“冒泡”。

冒泡排序进行 n 轮。每一轮遍历数组元素，只要发现相邻两元素顺序不正确，就交换它们。实现如下：

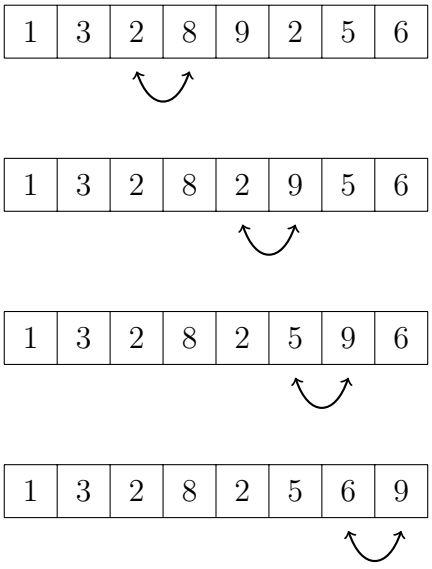
```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n-1; j++) {
        if (array[j] > array[j+1]) {
            swap(array[j], array[j+1]);
        }
    }
}
```

第一轮结束后，最大元素会到达正确位置；一般而言，经过 k 轮后，最大的 k 个元素都会位于正确位置。因此，经过 n 轮后整个数组即已排序。

例如，对数组



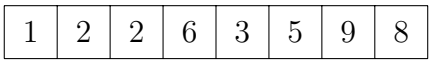
冒泡排序第一轮按如下过程交换元素：



逆序对

冒泡排序是一种始终交换数组中相邻元素的排序算法。这类算法的时间复杂度必然至少为 $O(n^2)$，因为最坏情况下需要 $O(n^2)$ 次交换才能把数组排好序。

分析排序算法时，一个实用概念是**逆序对**：一对数组元素 $(\text{array}[a], \text{array}[b])$，满足 $a < b$ 且 $\text{array}[a] > \text{array}[b]$，即两元素顺序错误。例如，数组



有三个逆序对：$(6,3)$、$(6,5)$ 和 $(9,8)$。逆序对数量反映了数组排序所需的工作量。数组完全有序时没有逆序对；若元素完全逆序，逆序对数量达到最大：

$$1+2+\cdots+(n-1)=\frac{n(n-1)}{2}=O(n^2)$$

交换一对顺序错误的相邻元素，恰好消除一个逆序对。因此，若排序算法只能交换相邻元素，每次交换至多消除一个逆序对，其时间复杂度至少为 $O(n^2)$。

$O(n\log n)$ 算法

不局限于交换相邻元素的算法，可以在 $O(n\log n)$ 时间内高效排序数组。其中一种是基于递归的**归并排序**[1]。

归并排序按如下步骤排序子数组 $\text{array}[a\ldots b]$：

1. 若 $a=b$，子数组已经有序，无须操作。
2. 计算中间位置：$k=\lfloor(a+b)/2\rfloor$。
3. 递归排序子数组 $\text{array}[a\ldots k]$。
4. 递归排序子数组 $\text{array}[k+1\ldots b]$。
5. 将两个有序子数组 $\text{array}[a\ldots k]$ 与 $\text{array}[k+1\ldots b]$ 归并成有序子数组 $\text{array}[a\ldots b]$。

归并排序十分高效，因为每一步都会把子数组大小减半。递归共有 $O(\log n)$ 层，每层处理需要 $O(n)$ 时间。由于两个子数组已经有序，可以在线性时间内完成归并。

例如，考虑排序以下数组：

1	3	6	2	8	2	5	9

数组先被分成两个子数组：

1	3	6	2

8	2	5	9

随后递归排序两个子数组：

1	2	3	6

2	5	8	9

最后，算法归并两个有序子数组，得到最终的有序数组：

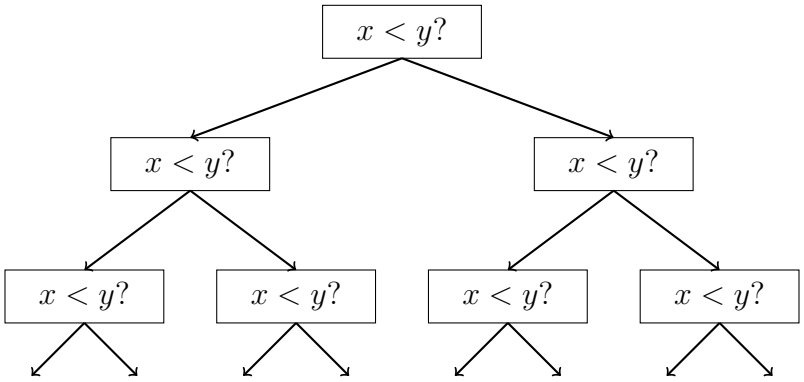
1	2	2	3	5	6	8	9

[1]据文献 [47]，归并排序由 J. von Neumann 于 1945 年发明。

排序下界

能否比 $O(n \log n)$ 更快地排序数组？若只考虑基于元素比较的排序算法，答案是不能。

可以把排序看作一个过程：每比较两个元素一次，就获得更多关于数组内容的信息。该过程形成如下决策树，由此可以证明时间复杂度下界：



其中 "$x < y?$" 表示比较元素 x 与 y。若 $x < y$，过程向左继续，否则向右继续。过程的结果对应数组所有可能的排列，共有 $n!$ 种。因此，树高至少为

$$\log_2(n!) = \log_2(1) + \log_2(2) + \cdots + \log_2(n).$$

取和式最后 $n/2$ 项，并把每项改为 $\log_2(n/2)$，可得到下界估计

$$\log_2(n!) \geq (n/2) \cdot \log_2(n/2),$$

因此，决策树高度以及排序算法最坏情况下所需的最少步骤数，至少为 $n \log n$ 量级。

计数排序

对于不比较数组元素、而利用其他信息的算法，$n \log n$ 下界并不成立。**计数排序**就是一例：若数组每个元素都是 $0 \ldots c$ 范围内的整数，且 $c = O(n)$，它可以在 $O(n)$ 时间内完成排序。

算法创建一个*计数*数组，其下标对应原数组中的元素。遍历原数组，统计每个元素的出现次数。

例如，数组

1	3	6	9	9	3	5	9

对应如下计数数组：

1	2	3	4	5	6	7	8	9
1	0	2	0	1	1	0	0	3

例如，计数数组位置 3 的值为 2，因为元素 3 在原数组中出现了两次。

构造计数数组需要 $O(n)$ 时间。随后，可根据每个元素的出现次数，在 $O(n)$ 时间内生成有序数组。因此，计数排序的总时间复杂度为 $O(n)$。

计数排序非常高效，但只有当常数 c 足够小、数组元素能够作为计数数组下标时才能使用。

3.2 C++ 中的排序

竞赛中几乎不应自行实现排序算法，因为编程语言已经提供了优秀实现。例如，C++ 标准库中的 sort 函数可轻松用于排序数组及其他数据结构。

使用库函数有很多好处。首先，无须自行实现，能够节省时间；其次，库实现可以确信是正确且高效的，自写排序函数通常不会更好。

本节介绍 C++ 的 sort 函数。下面的代码将向量按递增顺序排序：

```
vector<int> v = {4,2,5,3,5,8,3};
sort(v.begin(),v.end());
```

排序后，向量内容为 $[2,3,3,4,5,5,8]$。默认排序顺序为递增，也可按如下方式改为递减：

```
sort(v.rbegin(),v.rend());
```

普通数组可按如下方式排序：

```
int n = 7; // array size
int a[] = {4,2,5,3,5,8,3};
sort(a,a+n);
```

下面的代码对字符串 s 排序：

```
string s = "monkey";
sort(s.begin(), s.end());
```

对字符串排序是指排列其中的字符。例如，字符串 “monkey” 会变为 “ekmnoy”。

比较运算符

sort 要求待排序元素的数据类型定义了**比较运算符**。排序过程中，每当需要确定两个元素的先后顺序时，就会使用该运算符。

大多数 C++ 数据类型都内置了比较运算符，可以自动排序。例如，数字按数值排序，字符串按字典序排序。

二元组（pair）首先按第一元素（first）排序；若第一元素相等，再按第二元素（second）排序：

```
vector<pair<int,int>> v;
v.push_back({1,5});
v.push_back({2,3});
v.push_back({1,2});
sort(v.begin(), v.end());
```

此后二元组顺序为 $(1,2)$、$(1,5)$、$(2,3)$。

类似地，元组（tuple）依次按第一元素、第二元素等排序[2]：

```
vector<tuple<int,int,int>> v;
v.push_back({2,1,4});
v.push_back({1,5,3});
v.push_back({2,1,3});
sort(v.begin(), v.end());
```

此后元组顺序为 $(1,5,3)$、$(2,1,3)$、$(2,1,4)$。

自定义结构体

自定义结构体不会自动拥有比较运算符。应在结构体内部定义函数 operator<，其参数为同类型的另一个元素。若当前元素小于参数，运算符应返回 true，

[2]注意，在某些旧编译器中，必须用函数 make_tuple 而非花括号创建元组（例如用 make_tuple(2,1,4) 代替 {2,1,4}）。

否则返回 false。

例如，下面的结构体 P 包含一个点的 x、y 坐标。比较运算符使各点首先按 x 坐标排序，其次按 y 坐标排序。

```
struct P {
    int x, y;
    bool operator<(const P &p) {
        if (x != p.x) return x < p.x;
        else return y < p.y;
    }
};
```

比较函数

也可以把外部**比较函数**作为回调函数传给 sort。例如，下面的比较函数 comp 首先按长度、其次按字典序排列字符串：

```
bool comp(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
```

此时可按如下方式排序字符串向量：

```
sort(v.begin(), v.end(), comp);
```

3.3 二分查找

在数组中查找元素的一般方法是用 for 循环遍历数组。例如，下面的代码查找元素 x：

```
for (int i = 0; i < n; i++) {
    if (array[i] == x) {
        // x found at index i
    }
}
```

该方法时间复杂度为 $O(n)$，因为最坏情况下必须检查所有元素。若元素顺序任意，这也是最优方法，因为没有额外信息能指示应在数组何处寻找 x。

若数组有序，情况就不同了。元素顺序可以指导搜索，使查找大幅加速。下面的二**分查找**能在 $O(\log n)$ 时间内高效查找有序数组中的元素。

方法一

二分查找的常见实现类似于在词典中查找单词。查找过程维护数组中的一个活动区间，初始包含全部元素；随后执行若干步，每一步都把区间大小减半。

每一步检查活动区间的中间元素。若它就是目标元素，搜索结束；否则根据中间元素的值，继续搜索左半或右半区间。

上述思想可实现如下：

```
int a = 0, b = n-1;
while (a <= b) {
    int k = (a+b)/2;
    if (array[k] == x) {
        // x found at index k
    }
    if (array[k] > x) b = k-1;
    else a = k+1;
}
```

该实现中的活动区间为 $a \ldots b$，初始为 $0 \ldots n-1$。算法每一步都把区间大小减半，所以时间复杂度为 $O(\log n)$。

方法二

另一种二分查找实现基于高效遍历数组元素。其思想是进行跳跃，并在接近目标元素时逐渐缩短步长。

搜索从左向右进行，初始跳跃长度为 $n/2$。每一步把跳跃长度减半，依次变为 $n/4$、$n/8$、$n/16$ 等，直至 1。跳跃结束后，要么已经找到目标元素，要么确定它不在数组中。

实现如下：

```
int k = 0;
for (int b = n/2; b >= 1; b /= 2) {
    while (k+b < n && array[k+b] <= x) k += b;
}
```

```
if (array[k] == x) {
    // x found at index k
}
```

搜索过程中，变量 b 存储当前跳跃长度。算法时间复杂度为 $O(\log n)$，因为对每种跳跃长度，while 循环中的代码至多执行两次。

C++ 函数

C++ 标准库提供以下基于二分查找、在对数时间内运行的函数：

- lower_bound 返回指向第一个值不小于 x 的数组元素的指针。
- upper_bound 返回指向第一个值大于 x 的数组元素的指针。
- equal_range 同时返回上述两个指针。

这些函数假设数组已经有序。若不存在满足条件的元素，指针会指向数组末尾元素之后的位置。例如，下面的代码判断数组是否包含值为 x 的元素：

```
auto k = lower_bound(array,array+n,x)-array;
if (k < n && array[k] == x) {
    // x found at index k
}
```

下面的代码统计值为 x 的元素数量：

```
auto a = lower_bound(array, array+n, x);
auto b = upper_bound(array, array+n, x);
cout << b-a << "\n";
```

使用 equal_range 可以缩短代码：

```
auto r = equal_range(array, array+n, x);
cout << r.second-r.first << "\n";
```

寻找最小解

二分查找的一项重要用途是找到某个函数值发生变化的位置。假设要找出问题的最小可行解 k。给定函数 ok(x)：若 x 是可行解则返回 true，否则返回 false。还已知当 $x < k$ 时函数为 false，当 $x \geq k$ 时为 true。情况如下：

x	0	1	$\cdots$	$k-1$	k	$k+1$	$\cdots$
ok(x)	false	false	$\cdots$	false	true	true	$\cdots$

可以用二分查找求出 k：

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (!ok(x+b)) x += b;
}
int k = x+1;
```

搜索会找到满足 ok(x) 为 false 的最大 x。因此，下一个值 $k = x + 1$ 就是使 ok(k) 为 true 的最小值。初始跳跃长度 z 必须足够大，例如预先知道 ok(z) 为 true 的某个值。

算法调用函数 ok 共 $O(\log z)$ 次，因此总时间复杂度取决于 ok。例如，若该函数运行时间为 $O(n)$，则总复杂度为 $O(n \log z)$。

寻找最大值

二分查找还可用于求一个先增后减函数的最大值。目标是找出位置 k，使得

- $f(x) < f(x+1)$ when $x < k$, and
- $f(x) > f(x+1)$ when $x \geq k$.

方法是用二分查找找到满足 $f(x) < f(x+1)$ 的最大 x。由于此时 $f(x+1) > f(x+2)$，所以 $k = x + 1$。实现如下：

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (f(x+b) < f(x+b+1)) x += b;
}
int k = x+1;
```

注意，与普通二分查找不同，这里不允许函数的相邻值相等，否则无法判断搜索应向哪一侧继续。

第四章　数据结构

数据结构是在计算机内存中存储数据的方式。为问题选择合适的数据结构很重要，因为每种数据结构都有各自的优缺点。关键问题是：所选数据结构能够高效支持哪些操作?

本章介绍 C++ 标准库中最重要的数据结构。应尽可能使用标准库，这能节省大量时间。本书后面还会介绍标准库中没有的更复杂数据结构。

4.1　动态数组

动态数组是在程序运行期间可以改变大小的数组。C++ 中最常用的动态数组是 vector，其用法与普通数组几乎相同。

下面的代码创建一个空向量，并加入三个元素：

```
vector<int> v;
v.push_back(3); // [3]
v.push_back(2); // [3,2]
v.push_back(5); // [3,2,5]
```

随后可以像普通数组一样访问元素：

```
cout << v[0] << "\n"; // 3
cout << v[1] << "\n"; // 2
cout << v[2] << "\n"; // 5
```

函数 size 返回向量中的元素数量。下面的代码遍历并输出全部元素：

```
for (int i = 0; i < v.size(); i++) {
    cout << v[i] << "\n";
}
```

更简短的遍历方式如下：

```
for (auto x : v) {
    cout << x << "\n";
}
```

函数 back 返回向量的最后一个元素，函数 pop_back 删除最后一个元素：

```
vector<int> v;
v.push_back(5);
v.push_back(2);
cout << v.back() << "\n"; // 2
v.pop_back();
cout << v.back() << "\n"; // 5
```

下面的代码创建含五个元素的向量：

```
vector<int> v = {2,4,2,5,1};
```

另一种创建方式是指定元素数量及每个元素的初始值：

```
// size 10, initial value 0
vector<int> v(10);
```

```
// size 10, initial value 5
vector<int> v(10, 5);
```

向量内部使用普通数组实现。当向量扩大而原数组容量不足时，会分配新数组并把全部元素移入其中。不过这种情况并不频繁，因此 push_back 的平均时间复杂度为 $O(1)$。

string 也是动态数组，用法与向量相近；此外，它还提供其他数据结构没有的字符串专用语法。字符串可以用 + 拼接。函数 substr(k, x) 返回从位置 k 开始、长度为 x 的子串；函数 find(t) 查找子串 t 首次出现的位置。

下面展示若干字符串操作：

```
string a = "hatti";
string b = a+a;
cout << b << "\n"; // hattihatti
b[5] = 'v';
cout << b << "\n"; // hattivatti
string c = b.substr(3,4);
```

```
cout << c << "\n"; // tiva
```

4.2 集合结构

集合是维护一组元素的数据结构，基本操作包括插入、查找和删除。

C++ 标准库提供两种集合实现。set 基于平衡二叉树，操作时间为 $O(\log n)$；unordered_set 使用哈希，操作平均为 $O(1)$。

选择哪一种实现往往取决于偏好。set 能维护元素顺序，并提供 unordered_set 没有的函数；另一方面，unordered_set 可能更高效。

下面的代码创建整数集合并演示若干操作。函数 insert 插入元素，count 返回元素的出现次数，erase 删除元素。

```
set<int> s;
s.insert(3);
s.insert(2);
s.insert(5);
cout << s.count(3) << "\n"; // 1
cout << s.count(4) << "\n"; // 0
s.erase(3);
s.insert(4);
cout << s.count(3) << "\n"; // 0
cout << s.count(4) << "\n"; // 1
```

集合的多数用法类似向量，但不能用 [] 访问元素。下面的代码创建集合，输出元素数量，再遍历所有元素：

```
set<int> s = {2,5,6,8};
cout << s.size() << "\n"; // 4
for (auto x : s) {
    cout << x << "\n";
}
```

集合的重要性质是所有元素都互不相同。因此，count 总是返回 0（元素不在集合中）或 1（元素在集合中）；若元素已存在，insert 不会重复插入。示例如下：

```
set<int> s;
s.insert(5);
```

```
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 1
```

C++ 还提供 multiset 和 unordered_multiset，用法分别类似 set 和 unordered_set，但允许元素重复。例如，下面的代码向多重集合加入三个 5：

```
multiset<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 3
```

函数 erase 会删除多重集合中该元素的所有实例：

```
s.erase(5);
cout << s.count(5) << "\n"; // 0
```

若只想删除一个实例，可以写成：

```
s.erase(s.find(5));
cout << s.count(5) << "\n"; // 2
```

4.3 映射结构

映射是由键值对组成的广义数组。普通数组的键总是连续整数 $0, 1, \ldots, n-1$，而映射的键可以是任意数据类型，也不必连续。

C++ 标准库提供与集合对应的两种映射实现：map 基于平衡二叉树，访问元素需要 $O(\log n)$ 时间；unordered_map 使用哈希，访问元素平均需要 $O(1)$ 时间。

下面的代码创建一个以字符串为键、整数为值的映射：

```
map<string,int> m;
m["monkey"] = 4;
m["banana"] = 3;
m["harpsichord"] = 9;
cout << m["banana"] << "\n"; // 3
```

若请求一个不存在的键所对应的值，该键会自动以默认值加入映射。例如，下面会加入值为 0 的键 “aybabtu”。

```
map<string,int> m;
cout << m["aybabtu"] << "\n"; // 0
```

函数 count 检查映射中是否存在某个键：

```
if (m.count("aybabtu")) {
    // key exists
}
```

下面的代码输出映射中的所有键和值：

```
for (auto x : m) {
    cout << x.first << " " << x.second << "\n";
}
```

4.4 迭代器与区间

许多 C++ 标准库函数使用迭代器操作。**迭代器**是指向数据结构中某个元素的变量。

常用迭代器 begin 和 end 定义包含数据结构全部元素的区间。begin 指向第一个元素，end 指向最后一个元素之后的位置。如下所示：

```
{  3,  4,  6,  8,  12,  13,  14,  17  }
   ↑                                   ↑
   s.begin()                           s.end()
```

注意二者的不对称性：s.begin() 指向数据结构中的元素，而 s.end() 指向结构之外。因此，迭代器定义的是*左闭右开*区间。

使用区间

迭代器用于向 C++ 标准库函数提供数据结构中的元素区间。通常需要处理全部元素，因此把 begin 和 end 传给函数。

例如，下面的代码先用 sort 排序向量，再用 reverse 反转元素顺序，最后用 random_shuffle 随机打乱元素：

```
sort(v.begin(), v.end());
reverse(v.begin(), v.end());
random_shuffle(v.begin(), v.end());
```

这些函数也可用于普通数组，此时传入数组指针而非迭代器：

```
sort(a, a+n);
reverse(a, a+n);
random_shuffle(a, a+n);
```

集合迭代器

迭代器常用于访问集合元素。下面创建指向集合最小元素的迭代器 it：

```
set<int>::iterator it = s.begin();
```

更简短的写法是：

```
auto it = s.begin();
```

使用 * 可访问迭代器所指元素。例如，下面输出集合第一个元素：

```
auto it = s.begin();
cout << *it << "\n";
```

可以用运算符 ++（向前）和 --（向后）移动迭代器，即移至集合中的下一个或上一个元素。

下面按递增顺序输出所有元素：

```
for (auto it = s.begin(); it != s.end(); it++) {
    cout << *it << "\n";
}
```

下面输出集合中的最大元素：

```
auto it = s.end(); it --;
cout << *it << "\n";
```

函数 find(x) 返回指向值为 x 的元素的迭代器。若集合不包含 x，则返回 end。

```
auto it = s.find(x);
if (it == s.end()) {
    // x is not found
}
```

函数 lower_bound(x) 返回指向集合中最小的、值不小于 x 的元素的迭代器；upper_bound(x) 返回指向最小的、值大于 x 的元素的迭代器。若不存在这样的元素，两者都返回 end。不维护元素顺序的 unordered_set 不支持这些函数。

例如，下面的代码寻找最接近 x 的元素：

```
auto it = s.lower_bound(x);
if (it == s.begin()) {
    cout << *it << "\n";
} else if (it == s.end()) {
    it --;
    cout << *it << "\n";
} else {
    int a = *it; it --;
    int b = *it;
    if (x-b < a-x) cout << b << "\n";
    else cout << a << "\n";
}
```

代码假设集合非空，并用迭代器 it 处理所有情况。首先令迭代器指向最小的不小于 x 的元素。若 it 等于 begin，对应元素最接近 x；若等于 end，则集合最大元素最接近 x；否则，答案是 it 所指元素或其前一个元素。

4.5 其他结构

位集

位集是每个元素只能为 0 或 1 的数组。例如，下面创建一个含 10 个元素的位集：

```
bitset<10> s;
s[1] = 1;
s[3] = 1;
s[4] = 1;
s[7] = 1;
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

位集比普通数组更节省内存，因为每个元素只占一位。例如，用 int 数组存储 n 个二进制值需要 $32n$ 位，而位集只需 n 位。此外，位运算可以高效地批量操作位集元素，从而优化算法。

下面展示创建上述位集的另一种方式：

```
bitset<10> s(string("0010011010")); // from right to left
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

函数 count 返回位集中 1 的数量：

```
bitset<10> s(string("0010011010"));
cout << s.count() << "\n"; // 4
```

下面是位运算示例：

```
bitset<10> a(string("0010110110"));
bitset<10> b(string("1011011000"));
cout << (a&b) << "\n"; // 0010010000
cout << (a|b) << "\n"; // 1011111110
cout << (a^b) << "\n"; // 1001101110
```

双端队列

双端队列是一种可在两端高效改变大小的动态数组。与向量一样，它提供 push_back 和 pop_back，同时还提供向量没有的 push_front 和 pop_front。

双端队列用法如下：

```
deque<int> d;
d.push_back(5); // [5]
d.push_back(2); // [5,2]
d.push_front(3); // [3,5,2]
d.pop_back(); // [3,5]
d.pop_front(); // [5]
```

双端队列的内部实现比向量复杂，因此速度较慢。不过，在两端添加和删除元素的平均时间仍为 $O(1)$。

栈

栈支持两种 $O(1)$ 操作：向栈顶添加元素，以及从栈顶删除元素。只能访问栈顶元素。

栈的用法如下：

```
stack<int> s;
s.push(3);
s.push(2);
s.push(5);
cout << s.top(); // 5
s.pop();
cout << s.top(); // 2
```

队列

队列也支持两种 $O(1)$ 操作：向队尾添加元素，以及删除队首元素。只能访问队首和队尾元素。

队列的用法如下：

```
queue<int> q;
q.push(3);
q.push(2);
q.push(5);
cout << q.front(); // 3
q.pop();
cout << q.front(); // 2
```

优先队列

优先队列维护一组元素。它支持插入，并根据队列类型查询和删除最小或最大元素。插入和删除需要 $O(\log n)$ 时间，查询需要 $O(1)$ 时间。

有序集合也能高效支持优先队列的全部操作，但优先队列的常数因子更小。优先队列通常用堆实现，其结构比有序集合所用的平衡二叉树简单得多。

C++ 优先队列默认按递减顺序排列，可以查询并删除最大元素。示例如下：

```
priority_queue<int> q;
```

```
q.push(3);
q.push(5);
q.push(7);
q.push(2);
cout << q.top() << "\n"; // 7
q.pop();
cout << q.top() << "\n"; // 5
q.pop();
q.push(6);
cout << q.top() << "\n"; // 6
q.pop();
```

若要创建支持查询和删除最小元素的优先队列，可以写成：

```
priority_queue<int,vector<int>,greater<int>> q;
```

策略型数据结构

g++ 编译器还支持一些不属于 C++ 标准库的策略型数据结构。使用它们时，需在代码中加入：

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

随后可以定义 indexed_set：它类似 set，但能像数组一样按秩访问。整数版本定义如下：

```
typedef tree<int,null_type,less<int>,rb_tree_tag,
             tree_order_statistics_node_update> indexed_set;
```

现在可以按如下方式创建集合：

```
indexed_set s;
s.insert(2);
s.insert(3);
s.insert(7);
s.insert(9);
```

该集合的特点是可以访问元素在有序数组中对应的下标。函数 find_by_order

返回指向指定位置元素的迭代器：

```
auto x = s.find_by_order(2);
cout << *x << "\n"; // 7
```

函数 order_of_key 返回指定元素的位置：

```
cout << s.order_of_key(7) << "\n"; // 2
```

若元素不在集合中，则返回它若被插入后应处的位置：

```
cout << s.order_of_key(6) << "\n"; // 2
cout << s.order_of_key(8) << "\n"; // 3
```

两个函数都在对数时间内运行。

4.6 与排序的比较

一个问题往往既可用数据结构解决，也可用排序解决。两种方法的渐进复杂度可能掩盖实际效率上的显著差异。

考虑如下问题：给定两个均含 n 个元素的列表 A、B，计算同时属于两者的元素数量。例如，

$$A = [5, 2, 8, 9] \quad \text{且} \quad B = [3, 2, 9, 5],$$

答案为 3，因为 2、5、9 同时属于两个列表。

直接做法是以 $O(n^2)$ 时间遍历所有元素对，下面讨论更高效的算法。

算法一

把 A 中元素存入集合，再遍历 B，逐个检查元素是否也属于 A。由于 A 的元素保存在集合中，检查很高效。使用 set 时，算法复杂度为 $O(n \log n)$。

算法二

这里无须维护有序集合，因此可用 unordered_set 代替 set。只需更换底层数据结构，就能轻松提高效率，新算法的时间复杂度为 $O(n)$。

算法三

也可以不用数据结构而采用排序。先分别排序 A、B，再同时遍历两个列表找出公共元素。排序需要 $O(n \log n)$ 时间，其余部分为 $O(n)$，总复杂度为 $O(n \log n)$。

效率比较

下表展示当 n 变化、列表元素为 $1 \ldots 10^9$ 之间的随机整数时，上述算法的效率：

n	算法一	算法二	算法三
10^6	1.5 s	0.3 s	0.2 s
$2 \cdot 10^6$	3.7 s	0.8 s	0.3 s
$3 \cdot 10^6$	5.7 s	1.3 s	0.5 s
$4 \cdot 10^6$	7.7 s	1.7 s	0.7 s
$5 \cdot 10^6$	10.0 s	2.3 s	0.9 s

算法一和算法二除使用不同集合结构外完全相同。在本问题中，这一选择对运行时间影响很大：算法二比算法一快 4–5 倍。

不过，效率最高的是使用排序的算法三，其耗时仅为算法二的一半。有趣的是，算法一与算法三的复杂度同为 $O(n \log n)$，但算法三却快十倍。这是因为排序过程简单，且只在算法三开始时执行一次，其余部分在线性时间内运行；算法一则在整个执行过程中都要维护复杂的平衡二叉树。

第五章　完整搜索

完整搜索是一种几乎可用于解决任何算法问题的通用方法。其思想是用暴力方法生成所有可能解，再根据题意选出最优解或统计解的数量。

若有足够时间遍历全部解，完整搜索是很好的技术，因为它通常易于实现且一定能得到正确答案。若完整搜索太慢，则需要贪心、动态规划等其他技术。

5.1　生成子集

首先考虑生成一个 n 元集合的全部子集。例如，$\{0,1,2\}$ 的子集为 $\emptyset$, $\{0\}$, $\{1\}$, $\{2\}$, $\{0,1\}$, $\{0,2\}$, $\{1,2\}$ and $\{0,1,2\}$. 生成子集有两种常见方法：递归搜索，或利用整数的二进制表示。

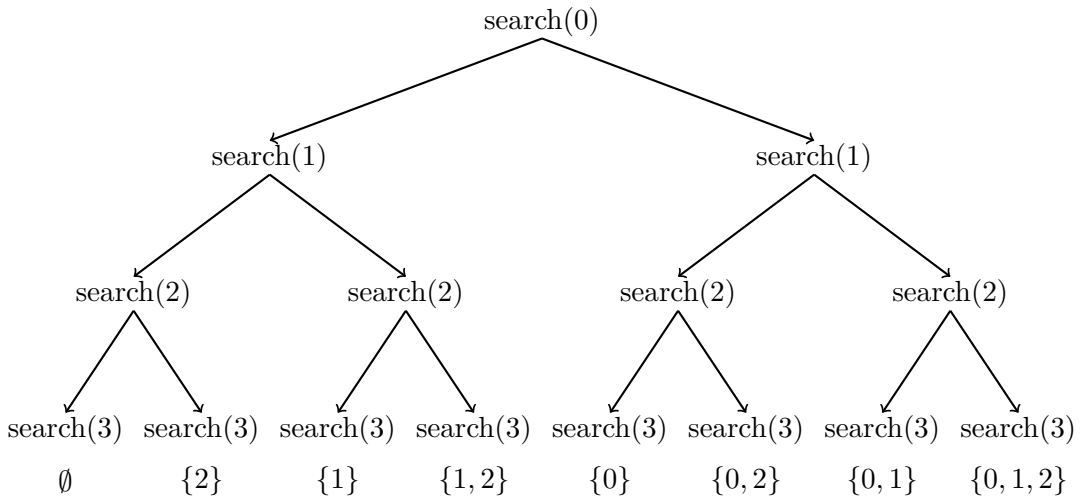
方法一

一种优雅的子集遍历方式是递归。下面的函数 search 生成集合 $\{0,1,\ldots,n-1\}$ 的全部子集。函数维护向量 subset 存放当前子集元素，从参数 0 开始调用。

```
void search(int k) {
    if (k == n) {
        // process subset
    } else {
        search(k+1);
        subset.push_back(k);
        search(k+1);
        subset.pop_back();
    }
}
```

调用 search(k) 时，函数决定是否把元素 k 加入子集，两种情况下都再以 $k+1$ 递归调用自身。若 $k=n$，说明所有元素均已处理，一个子集生成完毕。

下图展示 $n = 3$ 时的函数调用。每次可选择左分支（不加入 k）或右分支（加入 k）。



方法二

另一种方法基于整数的二进制表示。n 元集合的每个子集都可表示为长度为 n 的二进制序列，对应 $0 \ldots 2^n - 1$ 之间的整数；序列中的 1 表示哪些元素属于子集。

通常规定最低位对应元素 0，次低位对应元素 1，依此类推。例如，25 的二进制表示为 11001，对应子集 $\{0, 3, 4\}$。

下面的代码遍历 n 元集合的所有子集：

```
for (int b = 0; b < (1<<n); b++) {
    // process subset
}
```

下面展示如何找出一个二进制序列对应子集中的元素。处理每个子集时，代码构造一个向量保存其中元素。

```
for (int b = 0; b < (1<<n); b++) {
    vector<int> subset;
    for (int i = 0; i < n; i++) {
        if (b&(1<<i)) subset.push_back(i);
    }
}
```

5.2 生成排列

下面考虑生成 n 元集合的全部排列。例如，$\{0,1,2\}$ 的排列为 $(0,1,2)$, $(0,2,1)$, $(1,0,2)$, $(1,2,0)$, $(2,0,1)$ and $(2,1,0)$. 同样有两种方法：递归生成，或迭代遍历排列。

方法一

与子集一样，排列也可递归生成。下面的 search 遍历集合 $\{0,1,\ldots,n-1\}$ 的排列，并在向量 permutation 中构造当前排列；搜索从无参数调用开始。

```
void search() {
    if (permutation.size() == n) {
        // process permutation
    } else {
        for (int i = 0; i < n; i++) {
            if (chosen[i]) continue;
            chosen[i] = true;
            permutation.push_back(i);
            search();
            chosen[i] = false;
            permutation.pop_back();
        }
    }
}
```

每次函数调用向 permutation 加入一个新元素。数组 chosen 记录哪些元素已在排列中。当 permutation 大小等于集合大小时，一个排列生成完毕。

方法二

另一种方法从排列 $\{0,1,\ldots,n-1\}$ 开始，反复调用函数构造字典序中的下一个排列。C++ 标准库提供 next_permutation：

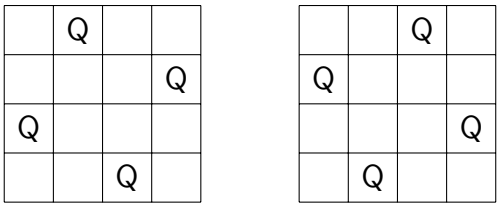
```
vector<int> permutation;
for (int i = 0; i < n; i++) {
    permutation.push_back(i);
}
do {
```

```
    // process permutation
} while (next_permutation(permutation.begin(),permutation.end()));
```

5.3 回溯法

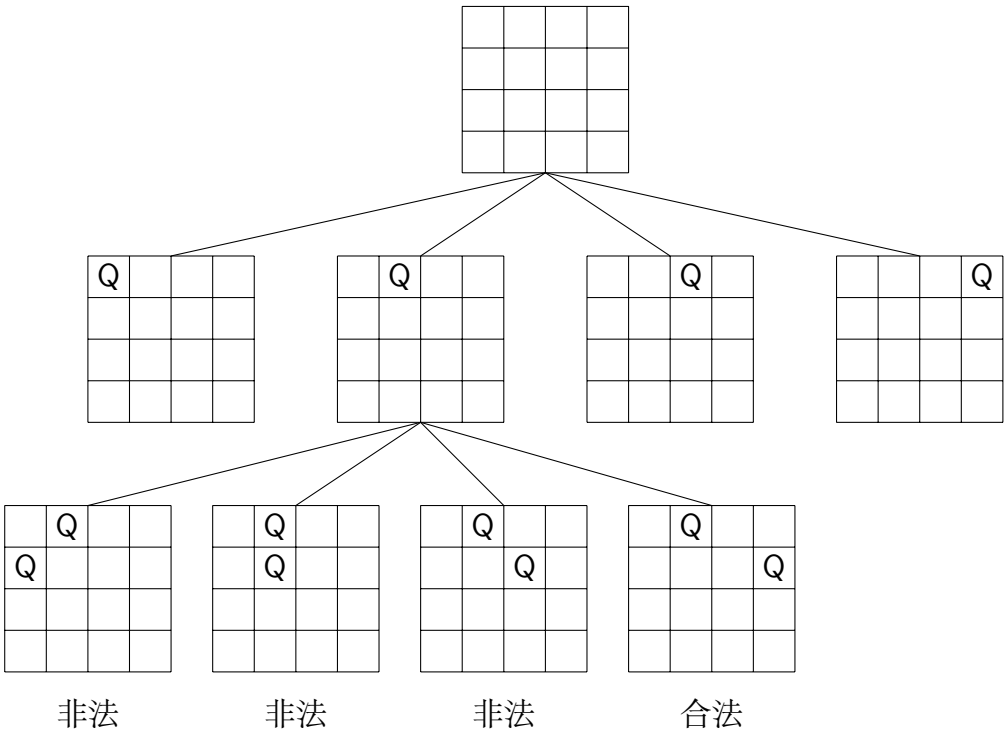
回溯法从一个空解开始，逐步扩展这个解。搜索过程递归地遍历构造解的所有不同方式。

例如，考虑计算在 $n \times n$ 的棋盘上放置 n 个皇后且任意两个皇后互不攻击的方案数。当 $n = 4$ 时，有两种可能的解：



可以使用回溯法逐行在棋盘上放置皇后来解决这个问题。更准确地说，每一行恰好放置一个皇后，并保证它不会攻击此前放置的任何皇后。当 n 个皇后都已放到棋盘上时，就找到了一个解。

例如，当 $n = 4$ 时，回溯算法生成的一些部分解如下：



在最下层，前三种布局都是非法的，因为其中的皇后会互相攻击。第四种布局则是合法的；再向棋盘放置两个皇后即可将其扩展为完整解，而这两个剩余皇后的放置方式只有一种。

该算法可以实现如下：

```
void search(int y) {
    if (y == n) {
        count++;
        return;
    }
    for (int x = 0; x < n; x++) {
        if (column[x] || diag1[x+y] || diag2[x-y+n-1]) continue;
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 1;
        search(y+1);
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 0;
    }
}
```

搜索从调用 search(0) 开始。棋盘大小为 $n \times n$，代码将解的数量计算到 count 中。

代码假设棋盘的行和列均从 0 编号到 $n-1$。以参数 y 调用函数 search 时，它会在第 y 行放置一个皇后，然后以参数 $y+1$ 递归调用自身。当 $y = n$ 时，说明已经找到一个解，于是变量 count 加一。

数组 column 记录哪些列中已有皇后，数组 diag1 和 diag2 则记录两组对角线。不能在已经包含皇后的列或对角线上再放置皇后。例如，4×4 棋盘的列和对角线编号如下：

0	1	2	3
0	1	2	3
0	1	2	3
0	1	2	3

column

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

diag1

3	4	5	6
2	3	4	5
1	2	3	4
0	1	2	3

diag2

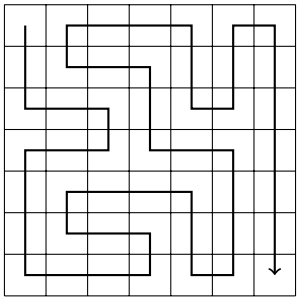
令 $q(n)$ 表示在 $n \times n$ 棋盘上放置 n 个皇后的方案数。上述回溯算法告诉我们，例如 $q(8) = 92$。随着 n 增大，搜索会迅速变慢，因为解的数量呈指数增长。例如，在现代计算机上使用上述算法计算 $q(16) = 14772512$ 已经需要约一分钟[1]。

[1]目前还没有已知方法能够高效计算更大的 $q(n)$。当前纪录是 2016 年计算出的 $q(27) =$

5.4 搜索剪枝

通常可以通过对搜索树进行**剪枝**来优化回溯算法。其思想是让算法具备一定的“判断能力”，尽早发现某个部分解无法扩展为完整解。这类优化可能极大提高搜索效率。

考虑这样一个问题：计算 $n \times n$ 网格中从左上角到右下角，并且恰好访问每个方格一次的路径数。例如，在 7×7 网格中共有 111712 条这样的路径，其中一条如下：



我们重点研究 7×7 的情形，因为它的难度恰好适合这里的需要。先从一个直接的回溯算法开始，再根据对剪枝方式的观察逐步优化。每次优化后，我们都会测量算法运行时间与递归调用次数，从而清楚看到各项优化对搜索效率的影响。

基本算法

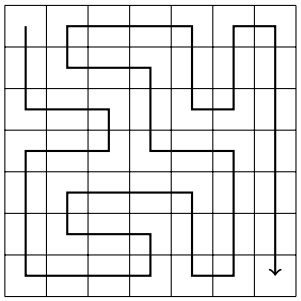
算法的第一个版本不包含任何优化。我们只是使用回溯法生成从左上角到右下角的所有可能路径，并统计其数量。

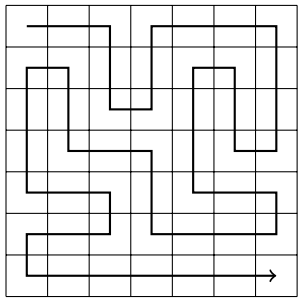
- 运行时间：483 秒
- 递归调用次数：760 亿

优化一

在任意解中，第一步都只能向下或向右走。第一步之后，路径总能按网格的对角线找到一条对称路径。例如，下列两条路径互为对称：

2349079671541225 28 [55]。



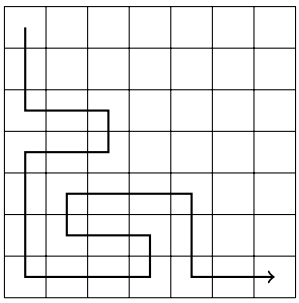


因此，可以规定第一步总是向下（或向右），最后再将解的数量乘以二。

- 运行时间：244 秒
- 递归调用次数：380 亿

优化二

如果路径在访问网格的所有其他方格之前就到达右下角，那么显然不可能再完成这个解。下面的路径就是一例：

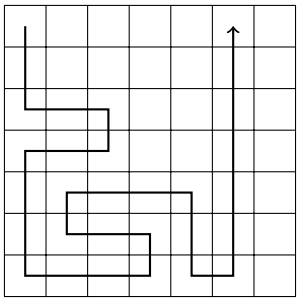


利用这一观察，如果过早到达右下角，就可以立即终止搜索。

- 运行时间：119 秒
- 递归调用次数：200 亿

优化三

如果路径抵达边界，并且既可以向左转也可以向右转，那么网格会被分成两个都含有未访问方格的部分。例如，在下列情形中，路径可以向左转或向右转：

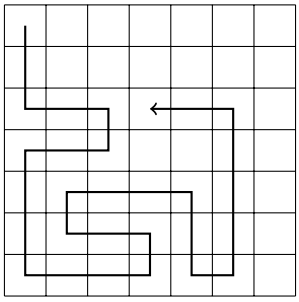


在这种情况下，已经无法访问所有方格，因此可以终止搜索。这项优化非常有效：

- 运行时间：1.8 秒
- 递归调用次数：2.21 亿

优化四

优化三的思想还可以推广：如果路径无法继续向前，却既可以向左转也可以向右转，那么网格同样会被分成两个都含有未访问方格的部分。例如，考虑下面的路径：



显然，此时已经无法访问所有方格，因此可以终止搜索。经过这项优化后，搜索已经非常高效：

- 运行时间：0.6 秒
- 递归调用次数：6900 万

现在正适合暂停优化，看看已经取得的成果。原始算法的运行时间是 483 秒，而经过这些优化后，运行时间仅为 0.6 秒。也就是说，优化后的算法速度提升了将近 1000 倍。

这在回溯算法中很常见：搜索树通常很大，即使简单的观察也能有效剪枝。尤其有用的是那些能在算法最初几步、也就是搜索树顶部生效的优化。

5.5 折半搜索

折半搜索是一种将搜索空间划分为两个大小大致相等的部分，分别搜索两部分，最后合并搜索结果的技术。

如果能够高效合并两次搜索的结果，就可以使用这种技术。在这种情况下，两次较小搜索所需的时间可能少于一次大搜索。通常，折半搜索可以把 2^n 这一因子降为 $2^{n/2}$。

例如，给定一个包含 n 个数的列表以及一个数 x，我们希望判断能否从列表中选出若干数，使它们的和为 x。对于列表 $[2,4,5,9]$ 和 $x=15$，可以选择 $[2,4,9]$，得到 $2+4+9=15$。但对于同一个列表，如果 $x=10$，就无法凑出这个和。

解决该问题的一种简单算法是遍历所有元素子集，检查是否有某个子集的元素和等于 x。由于共有 2^n 个子集，该算法的运行时间为 $O(2^n)$。使用折半搜索则可以得到更高效的 $O(2^{n/2})$ 时间算法[2]。注意，$O(2^n)$ 与 $O(2^{n/2})$ 是不同的复杂度，因为 $2^{n/2}=\sqrt{2^n}$。

其思路是把原列表分为 A 和 B 两个列表，使二者各包含约一半的数。第一次搜索生成 A 的所有子集，并将各子集的元素和存入列表 S_A；相应地，第二次搜索由 B 构造列表 S_B。之后，只需检查能否分别从 S_A 和 S_B 中各选一个元素，使二者之和为 x。这成立当且仅当能用原列表中的数凑出和 x。

例如，设列表为 $[2,4,5,9]$，且 $x=15$。首先将其分为 $A=[2,4]$ 和 $B=[5,9]$，再构造 $S_A=[0,2,4,6]$ 与 $S_B=[0,5,9,14]$。这里可以凑出 $x=15$，因为 S_A 中有和 6，S_B 中有和 9，且 $6+9=15$。这对应于解 $[2,4,9]$。

可以将该算法实现为 $O(2^{n/2})$ 时间复杂度。首先生成有序的列表 S_A 和 S_B，利用类似归并的技术可以在 $O(2^{n/2})$ 时间内完成。此后，由于两个列表已有序，可以在 $O(2^{n/2})$ 时间内检查能否从 S_A 和 S_B 中选出两个数凑成 x。

[2]这一思想由 E. Horowitz 和 S. Sahni 于 1974 年提出 [39]。

第六章　贪心算法

贪心算法总是作出当前看来最优的选择，以此构造问题的解。贪心算法从不撤销已经作出的选择，而是直接构造最终解。因此，贪心算法通常非常高效。

设计贪心算法的难点在于找到一种总能产生最优解的贪心策略。贪心算法中的局部最优选择还必须导向全局最优解。证明一个贪心算法正确往往并不容易。

6.1　硬币问题

作为第一个例子，考虑这样一个问题：给定一组硬币，需要用它们凑出金额 n。硬币面值为 $\text{coins} = \{c_1, c_2, \ldots, c_k\}$，每种硬币都可以使用任意多次。最少需要多少枚硬币?

例如，如果硬币是欧元硬币（单位为欧分）

$$\{1, 2, 5, 10, 20, 50, 100, 200\}$$

且 $n = 520$，那么至少需要四枚硬币。最优解是选择 $200 + 200 + 100 + 20$，其总和为 520。

贪心算法

解决该问题的一种简单贪心算法是始终选择不超过剩余金额的最大面值硬币，直到凑出所需金额。在上述例子中，这个算法会先选择两枚 200 欧分硬币，再选择一枚 100 欧分和一枚 20 欧分硬币，因而是正确的。但它是否总是正确?

事实证明，对于欧元硬币，这个贪心算法总是正确，也就是说，它总能得到硬币数最少的解。可以如下证明算法的正确性：

首先，在最优解中，面值 1、5、10、50 和 100 的硬币各自至多出现一次，因为若其中某种出现两次，就可以用一枚硬币替换它们，得到更好的解。例如，若解中含有 $5 + 5$，可以将其替换为一枚面值 10 的硬币。

同理，在最优解中，面值 2 和 20 的硬币各自至多出现两次，因为可以分别用 $5+1$ 替换 $2+2+2$，用 $50+10$ 替换 $20+20+20$。此外，最优解不可能含有 $2+2+1$ 或 $20+20+10$，因为可分别将它们替换为面值 5 和 50 的硬币。

利用这些观察，可以证明：对每个面值 x，仅使用小于 x 的硬币，不可能最优地凑出 x 或更大的金额。例如，当 $x=100$ 时，使用较小硬币能够组成的最大最优金额是 $50+20+20+5+2+2=99$。因此，始终选择最大面值硬币的贪心算法会产生最优解。

这个例子表明，即使贪心算法本身很简单，要证明它正确也可能很困难。

一般情形

在一般情形下，硬币集合可以包含任意面值，而贪心算法未必产生最优解。

只需给出一个算法答案错误的反例，就能证明某个贪心算法不正确。这里很容易找到反例：如果硬币面值为 $\{1,3,4\}$，目标金额为 6，贪心算法给出的解是 $4+1+1$，而最优解是 $3+3$。

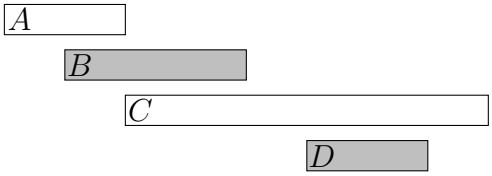
目前尚不清楚一般硬币问题能否用某种贪心算法解决[1]。不过，第 7 章将会看到，在某些情况下，可以使用总能给出正确答案的动态规划算法高效解决一般问题。

6.2 活动安排

许多调度问题可以用贪心算法解决。一个经典问题如下：给定 n 个活动及其开始和结束时间，找出包含尽可能多活动的安排。不能只选择活动的一部分。例如，考虑下列活动：

活动	开始时间	结束时间
A	1	3
B	2	5
C	3	9
D	6	8

此时最多可以选择两个活动。例如，可以如下选择活动 B 和 D：

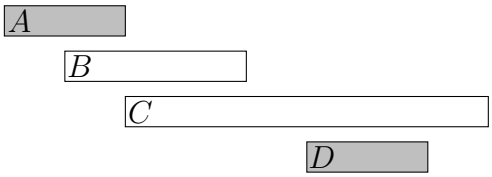


[1]不过，可以在多项式时间内检验本章介绍的贪心算法对给定硬币集合是否正确 [53]。

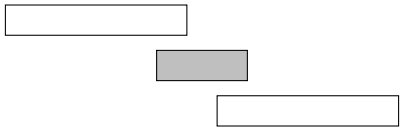
可以为这个问题设计多种贪心算法，但哪一种在所有情况下都正确?

算法一

第一种想法是选择尽可能短的活动。在示例中，该算法选择如下活动:



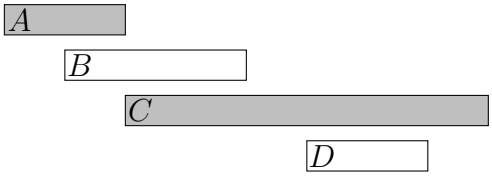
然而，选择短活动并不总是正确的策略。例如，该算法在下列情形中会失败:



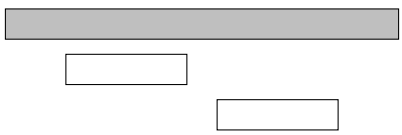
如果选择短活动，就只能选择一个活动；但实际上两个长活动可以同时选取。

算法二

另一种想法是始终选择可以选取且开始得尽可能早的下一个活动。该算法选择如下活动:



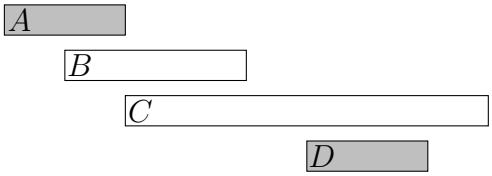
然而，这个算法同样存在反例。例如，在下列情形中，算法只会选择一个活动:



如果选择第一个活动，就无法再选择任何其他活动；但实际上可以选择另外两个活动。

算法三

第三种想法是始终选择可以选取且结束得尽可能早的下一个活动。该算法选择如下活动：



事实证明，该算法总是产生最优解。这是因为，首先选择结束时间最早的活动总是最优选择。之后继续用同样的策略选择下一个活动仍是最优的，如此反复，直到不能再选择活动。

证明该算法的一种方式，是考虑首先选择一个比最早结束活动结束得更晚的活动会怎样。此后可供选择的下一个活动数量至多与选择最早结束活动时相同。因此，选择结束更晚的活动绝不可能得到更好的解，贪心算法是正确的。

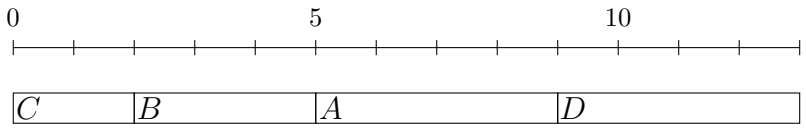
6.3 任务与截止时间

现在考虑一个问题：给定 n 个任务及其持续时间和截止时间，需要选择执行任务的顺序。对每个任务，可获得 $d-x$ 分，其中 d 是任务的截止时间，x 是完成该任务的时刻。能够得到的最大总分是多少?

例如，设任务如下：

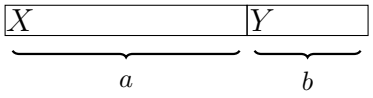
任务	持续时间	截止时间
A	4	2
B	3	5
C	2	7
D	4	5

此时，一种最优任务安排如下：

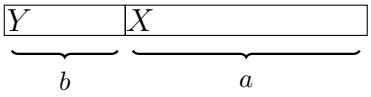


在这个解中，C 得 5 分，B 得 0 分，A 得 -7 分，D 得 -8 分，因此总分为 -10。

令人意外的是，该问题的最优解完全不依赖截止时间；正确的贪心策略只是按持续时间递增的顺序执行任务。原因在于，若连续执行的两个任务中前一个耗时更长，交换二者就能得到更好的解。例如，考虑如下安排：



这里 $a > b$，因此应交换两个任务：



现在，X 少得 b 分，Y 多得 a 分，所以总分增加 $a - b > 0$。在最优解中，任意两个相邻任务都必须是较短任务排在较长任务之前。因此，任务必须按持续时间排序后执行。

6.4 最小化总和

接下来考虑一个问题：给定 n 个数 $a_1, a_2, \ldots, a_n$，需要找到一个值 x，使下式最小：

$$|a_1 - x|^c + |a_2 - x|^c + \cdots + |a_n - x|^c.$$

我们重点讨论 $c = 1$ 和 $c = 2$ 两种情形。

$c = 1$ 的情形

此时需要最小化

$$|a_1 - x| + |a_2 - x| + \cdots + |a_n - x|.$$

例如，如果这些数为 $[1, 2, 9, 2, 6]$，最优选择是 $x = 2$，得到

$$|1 - 2| + |2 - 2| + |9 - 2| + |2 - 2| + |6 - 2| = 12.$$

一般而言，x 的最优选择是这些数的**中位数**，即排序后位于中间的数。例如，列表 $[1, 2, 9, 2, 6]$ 排序后为 $[1, 2, 2, 6, 9]$，所以中位数是 2。

中位数是最优选择，因为当 x 小于中位数时，增大 x 会使总和减小；当 x 大于中位数时，减小 x 会使总和减小。因此，最优解是令 x 等于中位数。若 n 为偶数而存在两个中位数，则这两个中位数及其之间的所有值都是最优选择。

$c = 2$ 的情形

此时需要最小化

$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2.$$

例如，如果这些数为 $[1,2,9,2,6]$，最优选择是 $x=4$，得到

$$(1-4)^2+(2-4)^2+(9-4)^2+(2-4)^2+(6-4)^2=46.$$

一般而言，x 的最优选择是这些数的**平均数**。示例中的平均数为 $(1+2+9+2+6)/5=4$。将总和写成下式即可推导出这一结论：

$$nx^2-2x(a_1+a_2+\cdots+a_n)+(a_1^2+a_2^2+\cdots+a_n^2)$$

最后一部分与 x 无关，因此可以忽略。其余部分构成函数 nx^2-2xs，其中 $s=a_1+a_2+\cdots+a_n$。这是一个开口向上的抛物线，两个零点为 $x=0$ 和 $x=2s/n$，最小值在两零点的平均位置 $x=s/n$ 处，也就是 $a_1,a_2,\ldots,a_n$ 的平均数。

6.5 数据压缩

二进制编码为字符串中的每个字符分配一个由比特组成的**码字**。将每个字符替换为对应码字，就可以用二进制编码压缩字符串。例如，下列二进制编码为字符 A–D 分配码字：

字符	码字
A	00
B	01
C	10
D	11

这是一种**定长编码**，即每个码字的长度都相同。例如，可以如下压缩字符串 AABACDACA：

00 00 01 00 10 11 00 10 00

使用这种编码，压缩后的字符串长 18 比特。不过，若使用码字长度可以不同的**变长编码**，还能进一步压缩：为频繁出现的字符分配短码字，为很少出现的字符分配长码字。事实证明，上述字符串的一种**最优**编码如下：

字符	码字
A	0
B	110
C	10
D	111

最优编码产生的压缩字符串尽可能短。此时使用最优编码得到的压缩字符串为

$$0\,0\,1\,1\,0\,0\,1\,0\,1\,1\,1\,0\,1\,0\,0,$$

因此只需 15 比特而非 18 比特。借助更好的编码，压缩字符串节省了 3 比特。

要求任意码字都不能是另一码字的前缀。例如，编码中不能同时包含码字 10 和 1011。这是因为我们希望能够从压缩字符串还原原字符串；如果一个码字可以是另一个码字的前缀，就不一定能够做到这一点。例如，下列编码是*无效*的：

字符	码字
A	10
B	11
C	1011
D	111

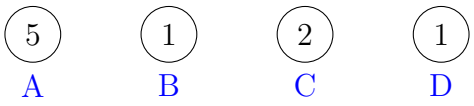
使用这种编码，无法判断压缩字符串 1011 对应字符串 AB 还是字符串 C。

Huffman 编码

Huffman 编码[2]是一种为给定字符串构造最优压缩编码的贪心算法。算法根据字符串中各字符的频率建立一棵二叉树；沿根结点到相应结点的路径即可读出每个字符的码字。向左移动对应比特 0，向右移动对应比特 1。

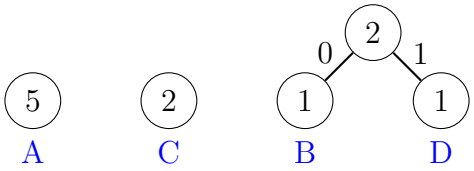
初始时，字符串中的每个字符由一个结点表示，结点权值是该字符在字符串中的出现次数。此后每一步合并权值最小的两个结点，创建一个新结点，其权值为原来两个结点的权值之和。不断重复，直到所有结点都被合并。

下面来看 Huffman 编码如何为字符串 AABACDACA 构造最优编码。初始时有四个结点，分别对应字符串中的四种字符：



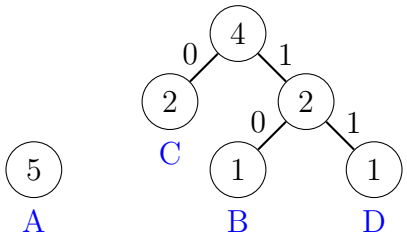
表示字符 A 的结点权值为 5，因为 A 在字符串中出现了 5 次。其他权值也以同样方式计算。

第一步合并分别对应字符 B 和 D 的结点，二者权值均为 1。结果如下：

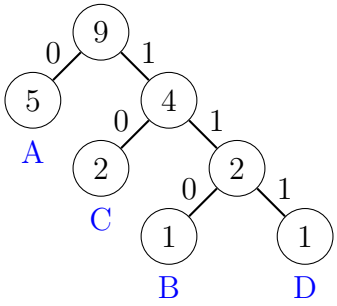


[2]D. A. Huffman 在完成大学课程作业时发现了这种方法，并于 1952 年发表该算法 [40]。

之后，合并两个权值为 2 的结点：



最后，合并剩余的两个结点：



现在所有结点都已进入树中，编码构造完毕。可以从树中读出下列码字：

字符	码字
A	0
B	110
C	10
D	111

第七章　动态规划

动态规划是一种兼具完整搜索的正确性与贪心算法效率的技术。当一个问题可以划分为相互重叠且能够独立求解的子问题时，就可以应用动态规划。

动态规划有两类用途：

- **寻找最优解**：希望找到尽可能大或尽可能小的解。
- **统计解的数量**：希望计算所有可能解的总数。

我们先了解如何用动态规划寻找最优解，再用相同思想统计解的数量。

理解动态规划是每位竞赛程序员成长过程中的里程碑。它的基本思想虽然简单，难点却在于如何将其应用到不同问题。本章介绍一组经典问题，作为学习动态规划的良好起点。

7.1　硬币问题

首先讨论第 6 章已经见过的问题：给定硬币面值集合 $\text{coins} = \{c_1, c_2, \ldots, c_k\}$ 和目标金额 n，需要用尽可能少的硬币凑出 n。

第 6 章使用始终选择最大可用面值的贪心算法解决了该问题。对于欧元硬币等情形，这个算法是正确的；但在一般情形下，贪心算法不一定产生最优解。

现在用动态规划高效解决这个问题，使算法适用于任意硬币集合。动态规划算法以一个递归函数为基础，像暴力算法一样遍历凑出金额的所有可能方式。不过，动态规划使用记忆化，每个子问题的答案只计算一次，因此十分高效。

递归形式

动态规划的思想是用递归形式描述问题，使原问题的解可以由更小子问题的解计算出来。对于硬币问题，一个自然的递归问题是：凑出金额 x 最少需要多少枚硬币？

令 $\text{solve}(x)$ 表示凑出金额 x 所需的最少硬币数。函数值取决于硬币的面值。例如，若 $\text{coins} = \{1, 3, 4\}$，函数开头的一些值如下：

$$
\begin{aligned}
\text{solve}(0) &= 0 \\
\text{solve}(1) &= 1 \\
\text{solve}(2) &= 2 \\
\text{solve}(3) &= 1 \\
\text{solve}(4) &= 1 \\
\text{solve}(5) &= 2 \\
\text{solve}(6) &= 2 \\
\text{solve}(7) &= 2 \\
\text{solve}(8) &= 2 \\
\text{solve}(9) &= 3 \\
\text{solve}(10) &= 3
\end{aligned}
$$

例如, $\text{solve}(10) = 3$, 因为凑出 10 至少需要 3 枚硬币, 最优解为 $3+3+4 = 10$。

solve 的关键性质是, 其值可以由较小的函数值递归计算出来。方法是关注为总金额选择的第一枚硬币。在上述情形中, 第一枚硬币可以是 1、3 或 4。如果首先选择面值 1, 剩余任务就是用最少硬币凑出 9, 这是原问题的子问题; 面值 3 和 4 也同理。因此, 可以用下列递推式计算最少硬币数:

$$
\begin{aligned}
\text{solve}(x) = \min(&\text{solve}(x-1)+1, \\
&\text{solve}(x-3)+1, \\
&\text{solve}(x-4)+1).
\end{aligned}
$$

递归的基本情形为 $\text{solve}(0) = 0$, 因为凑出空金额不需要硬币。例如,

$$
\text{solve}(10) = \text{solve}(7)+1 = \text{solve}(4)+2 = \text{solve}(0)+3 = 3.
$$

现在可以给出计算凑出金额 x 所需最少硬币数的一般递归函数:

$$
\text{solve}(x) = \begin{cases}
\infty & x < 0 \\
0 & x = 0 \\
\min_{c \in \text{coins}} \text{solve}(x-c)+1 & x > 0
\end{cases}
$$

首先, 若 $x < 0$, 函数值为 ∞, 因为不可能凑出负金额。若 $x = 0$, 函数值为 0, 因为凑出空金额不需要硬币。最后, 若 $x > 0$, 变量 c 遍历第一枚硬币的所有选择。

找到解决问题的递归函数后, 就可以直接用 C++ 实现 (常量 INF 表示无穷大):

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    return best;
}
```

不过，这个函数仍然不够高效，因为凑出金额的方式可能有指数级多种。下面将使用一种称为记忆化的技术使函数高效运行。

使用记忆化

动态规划使用**记忆化**高效计算递归函数的值，也就是把计算出的函数值存入数组。对每个参数只递归计算一次函数值，此后可直接从数组中取出。

在这个问题中，使用数组

```
bool ready[N];
int value[N];
```

其中 ready[x] 表示 solve(x) 是否已计算；若已计算，value[x] 保存其值。常量 N 的取值应确保所有需要的值都能放入数组。

现在可以高效地实现该函数：

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    if (ready[x]) return value[x];
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    value[x] = best;
    ready[x] = true;
    return best;
}
```

函数仍像之前一样处理 $x < 0$ 和 $x = 0$ 两种基本情形。然后通过 ready[x] 检查 solve(x) 是否已存入 value[x]；若已存入便直接返回，否则递归计算 solve(x)，并将结果存入 value[x]。

这个函数效率很高，因为每个参数 x 的答案只递归计算一次。solve(x) 的值存入 value[x] 后，再以参数 x 调用函数时即可高效取出。算法的时间复杂度为 $O(nk)$，其中 n 是目标金额，k 是硬币种数。

也可以用循环迭代构造数组 value，依次计算参数 $0 \ldots n$ 对应的所有 solve 值：

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0) {
            value[x] = min(value[x], value[x-c]+1);
        }
    }
}
```

实际上，大多数竞赛程序员更喜欢这种实现，因为它更短且常数因子更小。此后的示例也采用迭代实现。不过，以递归函数的形式思考动态规划解法通常更容易。

构造一个解

有时不仅要求求出最优解的值，还要给出如何构造这种解的示例。在硬币问题中，可以另设一个数组，对每个金额记录其某个最优解中的第一枚硬币：

```
int first[N];
```

随后将算法修改如下：

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0 && value[x-c]+1 < value[x]) {
            value[x] = value[x-c]+1;
            first[x] = c;
```

```
        }
    }
}
```

之后，可以用下列代码输出金额 n 的某个最优解中出现的硬币：

```
while (n > 0) {
    cout << first[n] << "\n";
    n -= first[n];
}
```

统计解的数量

现在考虑硬币问题的另一版本：计算使用这些硬币凑出金额 x 的方案总数。例如，若 $\text{coins} = \{1, 3, 4\}$ 且 $x = 5$，共有 6 种方案：

- $1+1+1+1+1$
- $3+1+1$
- $1+1+3$
- $1+4$
- $1+3+1$
- $4+1$

同样可以递归解决该问题。令 $\text{solve}(x)$ 表示凑出金额 x 的方案数。例如，若 $\text{coins} = \{1, 3, 4\}$，则 $\text{solve}(5) = 6$，递推式为

$$\begin{aligned}\text{solve}(x) =& \text{solve}(x-1)+ \\ & \text{solve}(x-3)+ \\ & \text{solve}(x-4).\end{aligned}$$

一般递归函数如下：

$$\text{solve}(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ \sum_{c \in \text{coins}} \text{solve}(x-c) & x > 0 \end{cases}$$

若 $x < 0$，函数值为 0，因为没有解。若 $x = 0$，函数值为 1，因为凑出空金额只有一种方式。否则，计算所有 $\text{solve}(x-c)$ 的总和，其中 c 遍历 coins。

下列代码构造数组 count，使得对 $0 \le x \le n$，$\text{count}[x]$ 等于 $\text{solve}(x)$：

```
count[0] = 1;
```

```
for (int x = 1; x <= n; x++) {
    for (auto c : coins) {
        if (x-c >= 0) {
            count[x] += count[x-c];
        }
    }
}
```

解的数量往往非常大，因此无需求出精确值，只需给出对 m 取模的结果，例如 $m = 10^9 + 7$。将所有计算改为模 m 意义下的运算即可。在上述代码中，只需加入一行

```
        count[x] %= m;
```

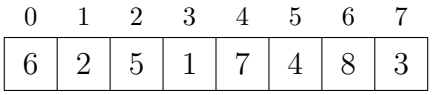
放在下面这行之后：

```
        count[x] += count[x-c];
```

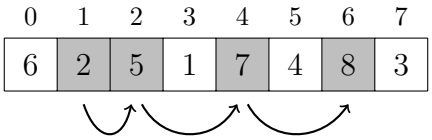
至此，我们已经讨论了动态规划的所有基本思想。动态规划可以用于许多不同情形，下面通过一组问题进一步展示它的应用方式。

7.2 最长递增子序列

第一个问题是在含有 n 个元素的数组中寻找**最长递增子序列**。它是一个按从左到右顺序选取的最长元素序列，序列中每个元素都大于前一个元素。例如，在数组



中，最长递增子序列包含 4 个元素：



令 length(k) 表示以位置 k 结尾的最长递增子序列长度。因此，计算出所有 $0 \le k \le n-1$ 的 length(k)，就能得到最长递增子序列的长度。对于上述数

组，函数值如下：

$$\begin{aligned}
\text{length}(0) &= 1 \\
\text{length}(1) &= 1 \\
\text{length}(2) &= 2 \\
\text{length}(3) &= 1 \\
\text{length}(4) &= 3 \\
\text{length}(5) &= 2 \\
\text{length}(6) &= 4 \\
\text{length}(7) &= 2
\end{aligned}$$

例如，length(6) = 4，因为以位置 6 结尾的最长递增子序列含有 4 个元素。

为计算 length(k)，应找到位置 $i < k$，使 array[i] < array[k] 且 length(i) 尽可能大。于是 length(k) = length(i) + 1，因为这是把 array[k] 加入子序列的最优方式。若不存在这样的位置 i，则 length(k) = 1，即子序列只含 array[k]。

由于所有函数值都能由较小的函数值计算，可以使用动态规划。下列代码将函数值存入数组 length。

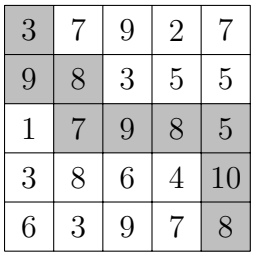
```
for (int k = 0; k < n; k++) {
    length[k] = 1;
    for (int i = 0; i < k; i++) {
        if (array[i] < array[k]) {
            length[k] = max(length[k],length[i]+1);
        }
    }
}
```

该代码包含两层嵌套循环，时间复杂度为 $O(n^2)$。不过，也可以在 $O(n \log n)$ 时间内更高效地完成这一动态规划计算。你能找到方法吗？

7.3 网格中的路径

下一个问题是在 $n \times n$ 网格中寻找一条从左上角到右下角、只能向下或向右移动的路径。每个方格包含一个正整数，需要让路径上数值之和尽可能大。

下图展示了网格中的一条最优路径：



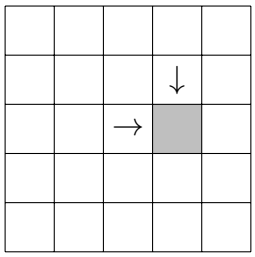
路径上的数值之和为 67，这是从左上角到右下角的路径所能取得的最大和。

假设网格行列均从 1 编号到 n，value$[y][x]$ 等于方格 (y, x) 中的值。令 $\text{sum}(y, x)$ 表示从左上角到方格 (y, x) 的路径最大和，则 $\text{sum}(n, n)$ 就是从左上角到右下角的最大和。例如，在上述网格中，$\text{sum}(5, 5) = 67$。

可以按如下递推式计算这些和：

$$\text{sum}(y, x) = \max(\text{sum}(y, x-1), \text{sum}(y-1, x)) + \text{value}[y][x]$$

该递推式基于如下观察：以方格 (y, x) 结尾的路径只能来自方格 $(y, x-1)$ 或 $(y-1, x)$：



因此，应选择使总和最大的方向。规定当 $y = 0$ 或 $x = 0$ 时 $\text{sum}(y, x) = 0$（因为不存在这样的路径），这样递推式在 $y = 1$ 或 $x = 1$ 时也适用。

由于函数 sum 有两个参数，动态规划数组也有两个维度。例如，可以使用数组

```
int sum[N][N];
```

并按如下方式计算：

```
for (int y = 1; y <= n; y++) {
    for (int x = 1; x <= n; x++) {
        sum[y][x] = max(sum[y][x-1],sum[y-1][x])+value[y][x];
    }
}
```

算法的时间复杂度为 $O(n^2)$。

7.4 背包问题

背包问题是指给定一组物品，要求找出具有某些性质的子集的一类问题。背包问题通常可以用动态规划解决。

本节重点讨论如下问题：给定重量列表 $[w_1, w_2, \ldots, w_n]$，确定用这些重量能够组成的所有总和。例如，当重量为 $[1, 3, 3, 5]$ 时，可以组成下列总和：

0	1	2	3	4	5	6	7	8	9	10	11	12
X	X		X	X	X	X	X	X	X		X	X

此时，$0 \ldots 12$ 之间除 2 和 10 外的所有总和都可以组成。例如，选择重量 $[1, 3, 3]$ 即可组成总和 7。

为解决该问题，考虑只使用前 k 个重量来组成总和的子问题。如果能用前 k 个重量组成总和 x，令 $\text{possible}(x, k) = \text{true}$，否则令其为 false。函数值可按如下方式递推：

$$\text{possible}(x, k) = \text{possible}(x - w_k, k - 1) \vee \text{possible}(x, k - 1)$$

这个公式基于这样一个事实：组成总和时可以选择使用或不使用重量 w_k。若使用 w_k，剩余任务是用前 $k-1$ 个重量组成 $x - w_k$；若不使用 w_k，则要用前 $k-1$ 个重量组成 x。基本情形为

$$\text{possible}(x, 0) = \begin{cases} \text{true} & x = 0 \\ \text{false} & x \neq 0 \end{cases}$$

因为不使用任何重量时只能组成总和 0。

下表给出了重量 $[1, 3, 3, 5]$ 对应的所有函数值（符号 “X” 表示值为真）：

$k \backslash x$	0	1	2	3	4	5	6	7	8	9	10	11	12
0	X												
1	X	X											
2	X	X		X	X								
3	X	X		X	X		X	X					
4	X	X		X	X	X	X	X	X	X		X	X

计算这些值后，$\text{possible}(x, n)$ 表示能否使用全部重量中的若干个组成总和 x。

令 W 表示所有重量之和。下列时间复杂度为 $O(nW)$ 的动态规划解法与递归函数相对应：

```
possible [0][0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = 0; x <= W; x++) {
        if (x-w[k] >= 0) possible[x][k] |= possible[x-w[k]][k-1];
        possible[x][k] |= possible[x][k-1];
    }
}
```

不过，还有一种更好的实现，只使用一维数组 possible[x] 表示能否构造出总和为 x 的子集。技巧是对每个新重量从右向左更新数组：

```
possible[0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = W; x >= 0; x--) {
        if (possible[x]) possible[x+w[k]] = true;
    }
}
```

这里的一般思想可用于许多背包问题。例如，若给定带有重量和价值的物品，可以对每个重量总和求出子集的最大价值总和。

7.5 编辑距离

编辑距离，也称 **Levenshtein 距离**[1]，是将一个字符串变为另一个字符串所需的最少编辑操作数。允许的编辑操作如下：

- 插入一个字符（例如 ABC → ABCA）
- 删除一个字符（例如 ABC → AC）
- 修改一个字符（例如 ABC → ADC）

例如，LOVE 与 MOVIE 之间的编辑距离为 2，因为可以先执行 LOVE → MOVE（修改），再执行 MOVE → MOVIE（插入）。这是最少操作数，因为显然一次操作不足以完成转换。

设给定长度为 n 的字符串 x 和长度为 m 的字符串 y，要计算二者的编辑距离。定义函数 distance(a, b) 表示前缀 x$[0 \ldots a]$ 与 y$[0 \ldots b]$ 之间的编辑距离，则 x 和 y 的编辑距离为 distance$(n-1, m-1)$。

[1]该距离以 V. I. Levenshtein 命名，他在研究二进制编码时研究了这一距离 [49]。

可以按如下方式计算 distance：

$$\begin{aligned}\text{distance}(a,b) = \min(&\text{distance}(a,b-1)+1,\\ &\text{distance}(a-1,b)+1,\\ &\text{distance}(a-1,b-1)+\text{cost}(a,b)).\end{aligned}$$

当 $\text{x}[a]=\text{y}[b]$ 时，$\text{cost}(a,b)=0$，否则 $\text{cost}(a,b)=1$。该公式考虑了如下编辑字符串 x 的方式：

- $\text{distance}(a,b-1)$：在 x 末尾插入一个字符
- $\text{distance}(a-1,b)$：删除 x 的最后一个字符
- $\text{distance}(a-1,b-1)$：匹配或修改 x 的最后一个字符

前两种情形需要一次编辑操作（插入或删除）。在最后一种情形中，若 $\text{x}[a]=\text{y}[b]$，无需编辑即可匹配最后一个字符；否则需要一次修改操作。

下表给出了示例中的 distance 值：

		M	O	V	I	E
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

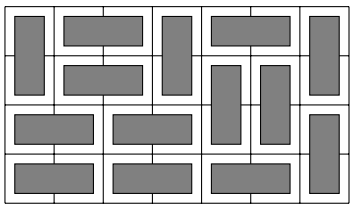
表格右下角表明 LOVE 与 MOVIE 的编辑距离为 2。该表还展示了如何构造最短编辑操作序列。此时路径如下：

		M	O	V	I	E
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

LOVE 与 MOVIE 的最后一个字符相同，所以二者的编辑距离等于 LOV 与 MOVI 的编辑距离。可以用一次编辑操作从 MOVI 中删除字符 I，因此该距离比 LOV 与 MOV 的编辑距离大一，依此类推。

7.6 铺砖方案计数

有时，动态规划状态会比固定的数字组合更加复杂。例如，考虑用 1×2 和 2×1 的砖铺满 $n \times m$ 网格的不同方案数。下面是 4×7 网格的一种合法方案：



而方案总数为 781。

可以逐行遍历网格，用动态规划解决该问题。解中的每一行都可以表示为一个长度为 m 的字符串，字符取自集合 $\{\sqcap, \sqcup, \sqsubset, \sqsupset\}$。例如，上述解由四行组成，对应下列字符串：

- ⊓ ⊏⊐ ⊓ ⊏⊐ ⊓
- ⊔ ⊏⊐ ⊔ ⊓ ⊓⊔
- ⊏⊐⊏⊐ ⊔ ⊔ ⊓
- ⊏⊐⊏⊐⊏⊐ ⊔

令 $\text{count}(k, x)$ 表示构造网格第 $1 \ldots k$ 行，且第 k 行对应字符串 x 的方案数。这里可以使用动态规划，因为一行的状态只受前一行状态的约束。

一个解合法，当且仅当第 1 行不含字符 ⊔，第 n 行不含字符 ⊓，并且所有相邻行都兼容。例如，行 ⊔ ⊏⊐ ⊔ ⊓ ⊓⊔ and ⊏⊐⊏⊐ ⊔ ⊔ ⊓ 彼此兼容，而行 ⊓ ⊏⊐ ⊓ ⊏⊐ ⊓ and ⊏⊐⊏⊐⊏⊐ ⊔ 彼此不兼容。

由于每行含有 m 个字符，每个字符有四种选择，不同行的数量至多为 4^m。因此，解法的时间复杂度为 $O(n4^{2m})$：每一行有 $O(4^m)$ 种可能状态，而对每种状态，前一行也有 $O(4^m)$ 种可能状态。实践中最好旋转网格，使较短边的长度为 m，因为因子 4^{2m} 主导时间复杂度。

使用更紧凑的行表示可以提高效率。事实上，只需知道前一行的哪些列包含竖砖的上半部分。因此，可以只用字符 ⊓ 和 □ 表示一行，其中 □ 综合表示字符 ⊔、⊏ 和 ⊐。采用这种表示后，只有 2^m 种不同的行，时间复杂度为 $O(n2^{2m})$。

最后，还有一个令人惊讶的直接公式可以计算铺砖方案数[2]：

$$\prod_{a=1}^{\lceil n/2 \rceil} \prod_{b=1}^{\lceil m/2 \rceil} 4 \cdot (\cos^2 \frac{\pi a}{n+1} + \cos^2 \frac{\pi b}{m+1})$$

[2]令人惊讶的是，该公式由两个彼此独立工作的研究团队于 1961 年发现 [43, 67]。

该公式非常高效，可以在 $O(nm)$ 时间内计算铺砖方案数。不过，由于答案是实数的乘积，使用该公式时的一个难题是如何精确保存中间结果。

第八章　摊还分析

通常只需考察算法的结构，例如包含哪些循环以及循环执行多少次，就很容易分析算法的时间复杂度。然而，有时直接分析并不能真实反映算法的效率。

摊还分析可用于分析包含耗时不一的操作的算法。其思想是不关注单次操作，而是估计算法执行期间所有这类操作耗费的总时间。

8.1　双指针法

双指针法使用两个指针遍历数组元素。两个指针都只能沿一个方向移动，从而保证算法高效运行。下面讨论两个可用双指针法解决的问题。

子数组和

第一个例子是：给定含有 n 个正整数的数组以及目标和 x，找出一个元素和为 x 的子数组，若不存在则报告无解。

例如，数组

1	3	2	5	1	1	2	3

包含一个元素和为 8 的子数组：

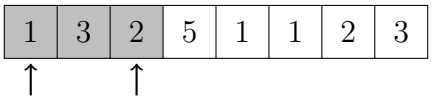
1	3	2	5	1	1	2	3

使用双指针法可以在 $O(n)$ 时间内解决该问题。维护两个分别指向子数组首尾元素的指针。每轮将左指针向右移动一步，同时只要所得子数组的和不超过 x，就继续向右移动右指针。若总和恰好为 x，就找到了一个解。

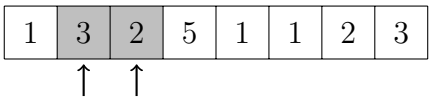
例如，考虑下列数组与目标和 $x = 8$：

1	3	2	5	1	1	2	3

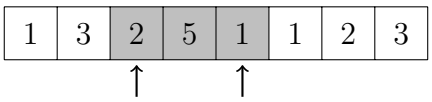
初始子数组包含 1、3 和 2，总和为 6：



然后左指针向右移动一步。右指针不动，因为否则子数组和会超过 x。



左指针再次向右移动一步，这次右指针向右移动三步。子数组和为 $2+5+1=8$，因此找到了元素和为 x 的子数组。



算法的运行时间取决于右指针移动的步数。虽然右指针在单轮中可能移动很多步，没有有用的上界，但它始终只向右移动，因此整个算法中总共只移动 $O(n)$ 步。

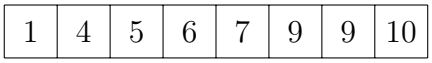
由于左右指针在算法中都移动 $O(n)$ 步，算法的时间复杂度为 $O(n)$。

两数之和问题

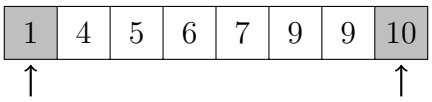
另一个可以用双指针法解决的问题称为**两数之和问题**（2SUM）：给定含有 n 个数的数组与目标和 x，找出两个和为 x 的数组元素，若不存在则报告无解。

先将数组元素按递增顺序排序，再用两个指针遍历数组。左指针从第一个元素开始，每轮向右移动一步。右指针从最后一个元素开始，不断向左移动，直到左右指针所指元素之和不超过 x。若总和恰好为 x，就找到了一个解。

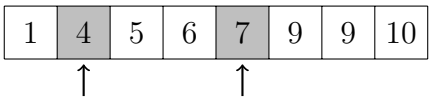
例如，考虑下列数组与目标和 $x=12$：



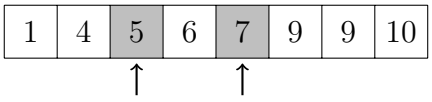
指针的初始位置如下。两元素之和为 $1+10=11$，小于 x。



随后左指针向右移动一步，右指针向左移动三步，总和变为 $4+7=11$。



之后左指针再次向右移动一步，右指针不动，于是找到解 $5+7=12$。



算法的时间复杂度为 $O(n\log n)$，因为先在 $O(n\log n)$ 时间内排序数组，随后两个指针各移动 $O(n)$ 步。

还可以使用二分查找，以另一种方式在 $O(n\log n)$ 时间内解决问题：遍历数组，对每个元素尝试找到另一个能使总和为 x 的元素。共执行 n 次二分查找，每次耗时 $O(\log n)$。

更困难的问题是**三数之和问题**（3SUM），要求找出三个和为 x 的数组元素。借鉴上述算法的思想，可以在 $O(n^2)$ 时间内解决该问题[1]。你能看出该怎么做吗?

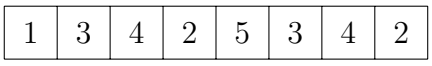
8.2 最近更小元素

摊还分析常用于估算数据结构上的操作次数。这些操作的分布可能不均匀，大多数操作集中发生在算法的某个阶段，但操作总数仍然受限。

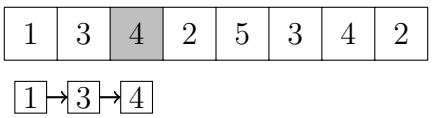
例如，考虑为每个数组元素寻找**最近更小元素**的问题，即在该元素之前出现、离它最近且比它小的元素。这样的元素可能不存在，此时算法应予以报告。下面介绍如何使用栈高效解决该问题。

从左到右遍历数组，并维护一个存放数组元素的栈。在每个位置，不断弹出栈中元素，直到栈顶元素小于当前元素或栈为空。此时若栈非空，栈顶就是当前元素的最近更小元素；若栈为空，则不存在这样的元素。最后将当前元素压入栈中。

例如，考虑下列数组：

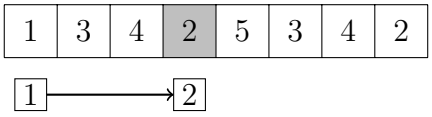


首先将元素 1、3 和 4 压入栈中，因为每个元素都大于前一个元素。因此，4 的最近更小元素是 3，3 的最近更小元素是 1。

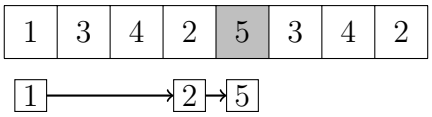


[1]很长一段时间里，人们认为不可能以优于 $O(n^2)$ 的时间复杂度解决 3SUM 问题。然而，2014 年的研究表明事实并非如此 [30]。

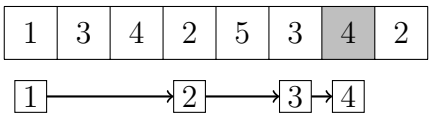
下一个元素 2 小于栈顶的两个元素。因此，3 和 4 被弹出栈，再将 2 压入栈中。它的最近更小元素是 1：



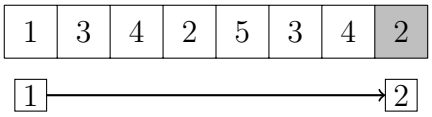
随后，元素 5 大于元素 2，因此将其压入栈中，它的最近更小元素是 2：



之后将元素 5 弹出栈，再将元素 3 和 4 压入栈：



最后，除 1 外的所有元素都被弹出栈，并将末尾元素 2 压入栈中：



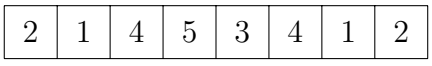
算法效率取决于栈操作总数。若当前元素大于栈顶元素，可直接压栈，十分高效。有时栈中会有多个较大元素，弹出它们需要一定时间。但每个元素都恰好入栈一次，并且至多出栈一次。因此，每个元素引起 $O(1)$ 次栈操作，算法的时间复杂度为 $O(n)$。

8.3 滑动窗口最小值

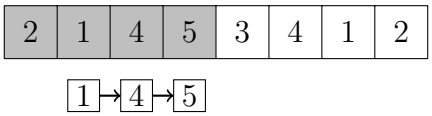
滑动窗口是一个大小固定、在数组中从左向右移动的子数组。在窗口的每个位置，都要计算窗口内元素的某些信息。本节重点讨论维护**滑动窗口最小值**，即报告每个窗口中的最小值。

计算滑动窗口最小值的思想与计算最近更小元素类似。维护一个元素严格递增的队列，队首始终对应窗口内的最小元素。每次窗口移动后，从队尾不断删除元素，直到队尾元素小于新进入窗口的元素或队列为空。如果队首元素已经不在窗口内，也将其删除。最后把新元素加入队尾。

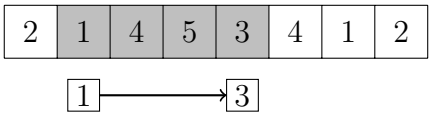
例如，考虑下列数组：



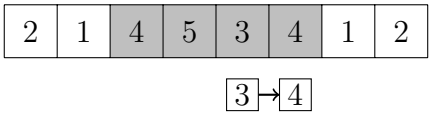
设滑动窗口大小为 4。在窗口的第一个位置，最小值为 1：



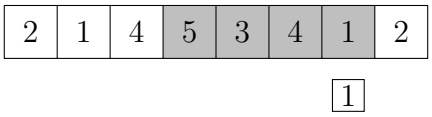
随后窗口向右移动一步。新元素 3 小于队列中的 4 和 5，因此将 4 和 5 从队列中删除，再将 3 加入队列。最小值仍为 1。



接着窗口再次移动，最小元素 1 已不在窗口中，因此将其从队列删除，此时最小值为 3；同时把新元素 4 加入队列。



下一个新元素 1 小于队列中的所有元素。因此删除队列中的全部元素，队列中只留下元素 1：



最后窗口到达末尾位置。将元素 2 加入队列，但窗口内的最小值仍为 1。



由于每个数组元素恰好入队一次、至多出队一次，算法的时间复杂度为 $O(n)$。

第九章　区间查询

本章讨论能够高效处理区间查询的数据结构。**区间查询**要求根据数组的某个子数组计算一个值。典型的区间查询包括：

- $\text{sum}_q(a,b)$：计算区间 $[a,b]$ 内元素之和
- $\min_q(a,b)$：求区间 $[a,b]$ 内的最小值
- $\max_q(a,b)$：求区间 $[a,b]$ 内的最大值

例如，考虑下列数组中的区间 $[3,6]$：

0	1	2	3	4	5	6	7
1	3	8	4	6	1	3	4

此时 $\text{sum}_q(3,6)=14$，$\min_q(3,6)=1$，$\max_q(3,6)=6$。

处理区间查询的一种简单方法是用循环遍历区间内的所有数组元素。例如，下列函数可用于处理数组上的区间和查询：

```
int sum(int a, int b) {
    int s = 0;
    for (int i = a; i <= b; i++) {
        s += array[i];
    }
    return s;
}
```

该函数的时间复杂度为 $O(n)$，其中 n 是数组大小。因此，使用该函数处理 q 次查询需要 $O(nq)$ 时间。若 n 和 q 都很大，这种方法会很慢。好在还有效率高得多的区间查询处理方法。

9.1 静态数组查询

首先考虑数组是静态的情形，即查询之间从不更新数组元素。此时，只需构造一个能够回答任意查询的静态数据结构。

区间和查询

构造**前缀和数组**即可轻松处理静态数组上的区间和查询。前缀和数组中每个元素等于原数组从开头到该位置的元素和，即位置 k 的值为 $\mathrm{sum}_q(0,k)$。前缀和数组可在 $O(n)$ 时间内构造。

例如，考虑下列数组：

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

相应的前缀和数组如下：

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

由于前缀和数组包含所有 $\mathrm{sum}_q(0,k)$ 的值，可以按下式在 $O(1)$ 时间内计算任意 $\mathrm{sum}_q(a,b)$：

$$\mathrm{sum}_q(a,b) = \mathrm{sum}_q(0,b) - \mathrm{sum}_q(0,a-1)$$

规定 $\mathrm{sum}_q(0,-1) = 0$ 后，上式在 $a = 0$ 时也成立。

例如，考虑区间 $[3,6]$：

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

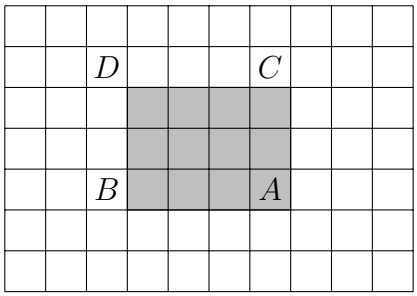
此时 $\mathrm{sum}_q(3,6) = 8+6+1+4 = 19$。可以用前缀和数组中的两个值计算该和：

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

因此，$\mathrm{sum}_q(3,6) = \mathrm{sum}_q(0,6) - \mathrm{sum}_q(0,2) = 27 - 8 = 19$。

这一思想还可推广到更高维。例如，可以构造二维前缀和数组，在 $O(1)$ 时间内计算任意矩形子数组的元素和。该数组中的每个和都对应一个从数组左上角开始的矩形子数组。

下图说明了这一思想：



灰色子数组的元素和可用下式计算：

$$S(A) - S(B) - S(C) + S(D),$$

其中 $S(X)$ 表示从左上角到 X 所在位置的矩形子数组元素和。

区间最小值查询

区间最小值查询比区间和查询更难处理。不过，有一种相当简单的方法，经过 $O(n \log n)$ 时间预处理后，即可在 $O(1)$ 时间内回答任意区间最小值查询[1]。区间最小值和最大值查询的处理方式相似，因此这里只讨论最小值查询。

其思想是预先计算所有区间长度 $b-a+1$ 为 2 的幂的 $\min_q(a,b)$。例如，对于数组

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

需要计算下列值：

a	b	$\min_q(a,b)$
0	0	1
1	1	3
2	2	4
3	3	8
4	4	6
5	5	1
6	6	4
7	7	2

a	b	$\min_q(a,b)$
0	1	1
1	2	3
2	3	4
3	4	6
4	5	1
5	6	1
6	7	2

a	b	$\min_q(a,b)$
0	3	1
1	4	3
2	5	1
3	6	1
4	7	1
0	7	1

[1]该技术见 [7]，有时称为**稀疏表**方法。还有更复杂的技术 [22] 能将预处理时间降到 $O(n)$，但算法竞赛中并不需要这些算法。

预计算值的数量为 $O(n \log n)$，因为作为 2 的幂的区间长度有 $O(\log n)$ 种。可以用下列递推式高效计算这些值：

$$\min_q(a, b) = \min(\min_q(a, a + w - 1), \min_q(a + w, b)),$$

其中 $b - a + 1$ 是 2 的幂，且 $w = (b - a + 1)/2$。计算所有这些值需要 $O(n \log n)$ 时间。

此后，任意 $\min_q(a, b)$ 都可取两个预计算值的最小值，在 $O(1)$ 时间内求出。令 k 为不超过 $b - a + 1$ 的最大 2 的幂，则可用下式计算 $\min_q(a, b)$：

$$\min_q(a, b) = \min(\min_q(a, a + k - 1), \min_q(b - k + 1, b)).$$

上式将区间 $[a, b]$ 表示为两个长度均为 k 的区间 $[a, a + k - 1]$ 与 $[b - k + 1, b]$ 的并集。

例如，考虑区间 $[1, 6]$：

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

该区间长度为 6，不超过 6 的最大 2 的幂为 4。因此，区间 $[1, 6]$ 是区间 $[1, 4]$ 与 $[3, 6]$ 的并集：

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

由于 $\min_q(1, 4) = 3$ 且 $\min_q(3, 6) = 1$，可知 $\min_q(1, 6) = 1$。

9.2 树状数组

树状数组，也称 **Fenwick 树**[2]，可以看作前缀和数组的动态版本。它支持两种 $O(\log n)$ 操作：区间和查询与单点更新。

树状数组的优势在于，可以在区间和查询之间高效更新数组元素。前缀和数组无法做到这一点，因为每次更新后都必须花费 $O(n)$ 时间重新构造整个前缀和数组。

[2]树状数组由 P. M. Fenwick 于 1994 年提出 [21]。

结构

尽管名称中含有“树”，树状数组通常仍用数组表示。本节假设所有数组从 1 开始编号，因为这样实现更简单。

令 $p(k)$ 表示能够整除 k 的最大 2 的幂。将树状数组存入数组 tree，使得

$$\text{tree}[k] = \text{sum}_q(k - p(k) + 1, k),$$

即每个位置 k 保存原数组中一个长度为 $p(k)$、以位置 k 结尾的区间元素和。例如，因为 $p(6) = 2$，tree[6] 保存 $\text{sum}_q(5, 6)$。

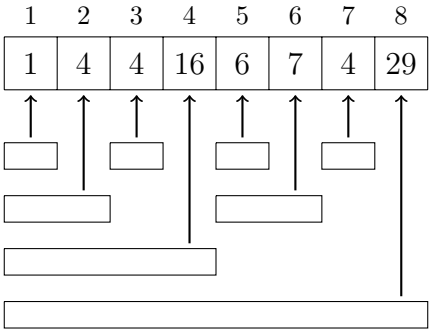
例如，考虑下列数组：

1	2	3	4	5	6	7	8
1	3	4	8	6	1	4	2

对应的树状数组如下：

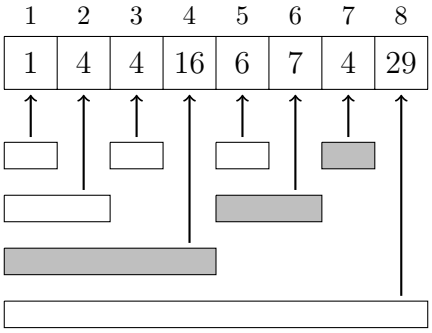
1	2	3	4	5	6	7	8
1	4	4	16	6	7	4	29

下图更清楚地展示了树状数组中的每个值如何对应原数组中的一个区间：



使用树状数组，可以在 $O(\log n)$ 时间内计算任意 $\text{sum}_q(1, k)$，因为区间 $[1, k]$ 总能划分成 $O(\log n)$ 个元素和已存入树中的区间。

例如，区间 $[1, 7]$ 由下列区间组成：



因此，可以按如下方式计算对应的和：

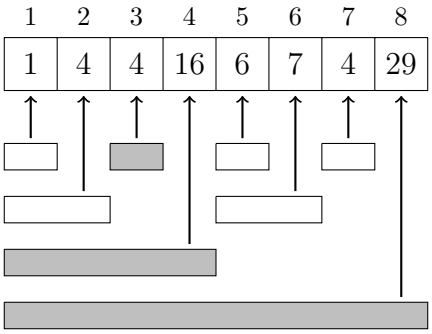
$$\text{sum}_q(1,7) = \text{sum}_q(1,4) + \text{sum}_q(5,6) + \text{sum}_q(7,7) = 16 + 7 + 4 = 27$$

要计算 $a > 1$ 时的 $\text{sum}_q(a,b)$，可以使用与前缀和数组相同的技巧：

$$\text{sum}_q(a,b) = \text{sum}_q(1,b) - \text{sum}_q(1,a-1).$$

由于 $\text{sum}_q(1,b)$ 和 $\text{sum}_q(1,a-1)$ 都可在 $O(\log n)$ 时间内计算，总时间复杂度为 $O(\log n)$。

更新原数组中的一个元素后，还需要更新树状数组中的若干值。例如，若位置 3 的元素改变，下列区间的和也会改变：



由于每个数组元素属于树状数组中的 $O(\log n)$ 个区间，只需更新树中的 $O(\log n)$ 个值。

实现

树状数组的操作可以用位运算高效实现。关键在于可用下式计算任意 $p(k)$：

$$p(k) = k\&-k.$$

下列函数计算 $\text{sum}_q(1,k)$：

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

下列函数将数组位置 k 的元素增加 x（x 可以为正或负）：

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

两个函数的时间复杂度均为 $O(\log n)$，因为它们访问树状数组中的 $O(\log n)$ 个值，而每次移动到下一位置只需 $O(1)$ 时间。

9.3 线段树

线段树[3]是一种支持区间查询与数组元素更新的数据结构。线段树可以支持区间和、区间最小值、区间最大值等许多查询，并使查询和更新都在 $O(\log n)$ 时间内完成。

与树状数组相比，线段树的优点是更加通用。树状数组主要支持区间和查询[4]，而线段树还支持其他查询。另一方面，线段树需要更多内存，实现也略为困难。

结构

线段树是一棵二叉树，最底层结点对应数组元素，其他结点则保存处理区间查询所需的信息。

本节假设数组大小为 2 的幂，且使用从 0 开始的编号，因为这样构造线段树很方便。若数组大小不是 2 的幂，可以在末尾补充额外元素。

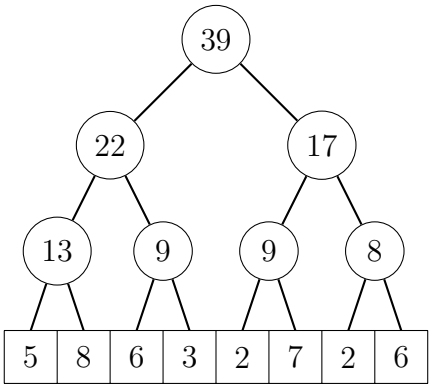
首先讨论支持区间和查询的线段树。例如，考虑下列数组：

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

对应的线段树如下：

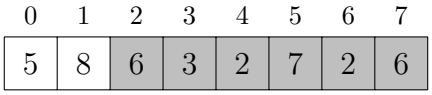
[3]本章自底向上的实现与 [62] 相对应。20 世纪 70 年代末已有类似结构用于解决几何问题 [9]。

[4]事实上，使用两个树状数组也可以支持最小值查询 [16]，但这比使用线段树复杂。

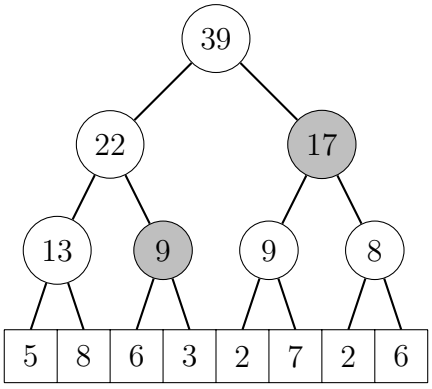


每个内部结点对应一个长度为 2 的幂的数组区间。在上图中，每个内部结点的值是对应区间的元素和，可以由左右子结点的值相加得到。

任意区间 $[a, b]$ 都可以划分成 $O(\log n)$ 个值存储在线段树结点中的区间。例如，考虑区间 $[2, 7]$：



这里 $\text{sum}_q(2, 7) = 6 + 3 + 2 + 7 + 2 + 6 = 26$。此时，下列两个树结点对应这个区间：

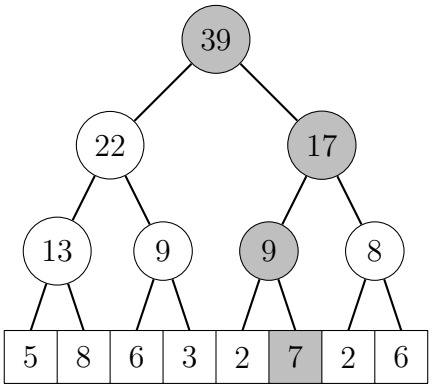


因此，也可以用 $9 + 17 = 26$ 计算该和。

若尽量使用树中层次较高的结点计算区间和，则每层至多需要两个结点，因此结点总数为 $O(\log n)$。

更新数组元素后，应更新所有依赖该元素的结点。沿被更新元素到根结点的路径向上遍历，并更新路径上的结点即可。

下图展示数组元素 7 改变时哪些树结点需要更新：

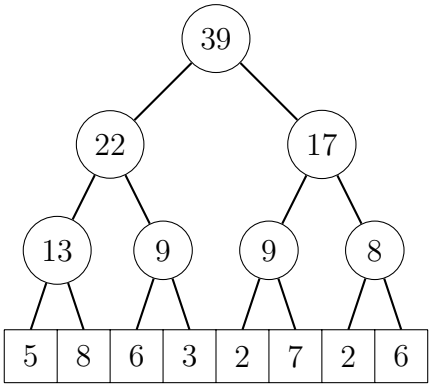


从底层到根的路径总是包含 $O(\log n)$ 个结点，因此每次更新会改变树中的 $O(\log n)$ 个结点。

实现

用含 $2n$ 个元素的数组存储线段树，其中 n 是原数组大小且为 2 的幂。树结点按从上到下的顺序存储：tree[1] 是根结点，tree[2] 和 tree[3] 是它的子结点，依此类推。最后，tree[n] 到 tree[$2n-1$] 对应树最底层的原数组元素。

例如，线段树



按如下方式存储：

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

在这种表示中，tree[k] 的父结点是 tree[$\lfloor k/2 \rfloor$]，子结点是 tree[$2k$] 和 tree[$2k+1$]。因此，左子结点的位置为偶数，右子结点的位置为奇数。

下列函数计算 $\text{sum}_q(a,b)$：

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
```

```
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

函数维护一个初始为 $[a+n, b+n]$ 的区间。每一步将区间上移一层，在此之前把不属于上一层区间的结点值加入总和。

下列函数将数组位置 k 的元素增加 x：

```
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}
```

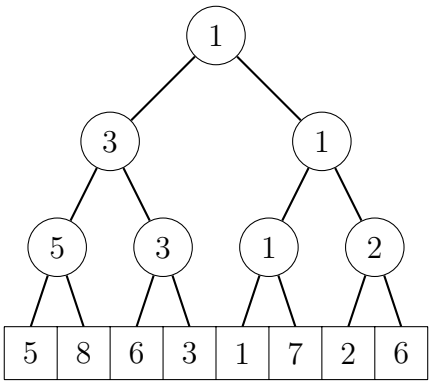
函数先更新树最底层的值，再更新所有相关内部结点，直到到达根结点。

上述两个函数的时间复杂度均为 $O(\log n)$，因为含 n 个元素的线段树有 $O(\log n)$ 层，而函数每步向上移动一层。

其他查询

线段树可以支持所有满足如下性质的区间查询：能把区间分为两部分，分别计算两部分的答案，再高效合并。此类查询包括最小值、最大值、最大公约数，以及按位与、或、异或。

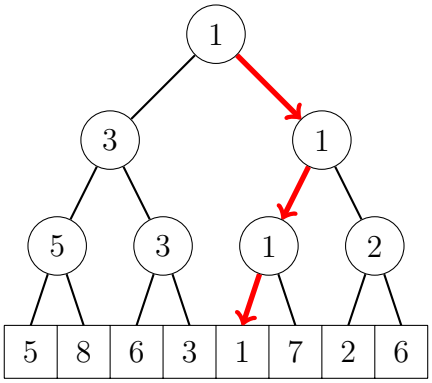
例如，下列线段树支持区间最小值查询：



此时，每个树结点保存对应数组区间内的最小值，根结点保存整个数组的最小值。操作实现与之前相同，只是把求和改为求最小值。

线段树结构还允许用二分思想定位数组元素。例如，若线段树支持最小值查询，就可以在 $O(\log n)$ 时间内找到某个最小元素的位置。

例如，在上面的树中，可以从根结点向下遍历一条路径，找到值为 1 的最小元素：



9.4 其他技术

坐标压缩

基于数组的数据结构有一个限制：元素必须用连续整数作为下标。当需要很大的下标时就会遇到困难。例如，若要使用下标 10^9，数组就需要包含 10^9 个元素，内存开销过大。

不过，如果预先知道算法运行中需要的所有下标，通常可以用**坐标压缩**绕过这一限制：把原下标替换为 $1, 2, 3, \ldots$。

其思想是用 $c(x)$ 替换每个原下标 x，其中 c 是压缩下标的函数。要求下标的相对次序不变，即若 $a < b$，则 $c(a) < c(b)$。这样即使下标经过压缩，也能方便地执行查询。

例如，若原下标为 555、10^9 和 8，则新下标为：

$$\begin{aligned} c(8) &= 1 \\ c(555) &= 2 \\ c(10^9) &= 3 \end{aligned}$$

区间更新

目前实现的数据结构支持区间查询与单点更新。现在考虑相反的情形：进行区间更新与单点查询。重点讨论将区间 $[a, b]$ 内所有元素增加 x 的操作。

令人意外的是，本章介绍的数据结构也能用于这种情形。构造一个**差分数组**，其中各元素表示原数组中相邻元素的差。这样，原数组就是差分数组的前缀和数组。例如，考虑下列数组：

0	1	2	3	4	5	6	7
3	3	1	1	1	5	2	2

上述数组的差分数组如下：

0	1	2	3	4	5	6	7
3	0	−2	0	0	4	−3	0

例如，原数组位置 6 的值 2 对应差分数组中的和 $3-2+4-3=2$。

差分数组的优势在于，只需修改其中两个元素，就能更新原数组的一个区间。例如，若要把原数组位置 1 到 4 的元素均增加 5，只需把差分数组位置 1 的值增加 5，并把位置 5 的值减少 5。结果如下：

0	1	2	3	4	5	6	7
3	5	−2	0	0	−1	−3	0

更一般地，要把区间 $[a,b]$ 内的元素增加 x，只需把位置 a 的差分值增加 x，并把位置 $b+1$ 的差分值减少 x。这样只需进行单点更新与前缀和查询，因此可以使用树状数组或线段树。

更困难的问题是同时支持区间查询与区间更新。第 28 章将会看到，这同样可以做到。

第十章　位运算

计算机程序中的所有数据在内部都以比特，也就是数字 0 和 1 的形式存储。本章讨论整数的二进制表示，并通过示例说明如何使用位运算。事实证明，算法编程中有许多位运算的应用。

10.1　二进制表示

在程序中，一个 n 位整数在内部存储为由 n 个比特组成的二进制数。例如，C++ 的 int 是 32 位类型，即每个 int 数由 32 个比特组成。

下面是 int 数 43 的二进制表示：

$$00000000000000000000000000101011$$

其中各位从右向左编号。要把二进制表示 $b_k \cdots b_2 b_1 b_0$ 转换为数值，可以使用公式

$$b_k 2^k + \ldots + b_2 2^2 + b_1 2^1 + b_0 2^0.$$

例如，

$$1 \cdot 2^5 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 43.$$

数的二进制表示分为**有符号**和**无符号**两种。通常使用有符号表示，因此正数和负数都能表示。一个 n 位有符号变量可保存 -2^{n-1} 到 $2^{n-1}-1$ 之间的任意整数。例如，C++ 的 int 是有符号类型，可保存 -2^{31} 到 $2^{31}-1$ 之间的任意整数。

有符号表示中的最高位是符号位（非负数为 0，负数为 1），其余 $n-1$ 位保存数值。实际使用**补码**表示，即先将一个数的所有位取反，再加一，就得到其相反数。

例如，int 数 -43 的二进制表示为

$$11111111111111111111111111010101.$$

无符号表示只能用于非负数，但数值上界更大。一个 n 位无符号变量可保存 0 到 2^n-1 之间的任意整数。例如，C++ 的 unsigned int 变量可保存 0 到 $2^{32}-1$ 之间的任意整数。

两种表示之间存在联系：有符号数 $-x$ 等同于无符号数 2^n-x。例如，下列代码表明有符号数 $x=-43$ 等同于无符号数 $y=2^{32}-43$：

```
int x = -43;
unsigned int y = x;
cout << x << "\n"; // -43
cout << y << "\n"; // 4294967253
```

若一个数超过二进制表示的上界，就会发生溢出。在有符号表示中，$2^{n-1}-1$ 之后的数是 -2^{n-1}；在无符号表示中，2^n-1 之后的数是 0。例如，考虑下列代码：

```
int x = 2147483647
cout << x << "\n"; // 2147483647
x++;
cout << x << "\n"; // -2147483648
```

初始时 $x=2^{31}-1$，这是 int 变量能保存的最大值，因此 $2^{31}-1$ 之后的数是 -2^{31}。

10.2 位运算

按位与

按位与运算 $x \,\&\, y$ 产生一个数，只有 x 和 y 对应位都为 1 时，结果的该位才为 1。例如，$22 \,\&\, 26 = 18$，因为

$$\begin{array}{rll} & 10110 & (22) \\ \& & 11010 & (26) \\ \hline = & 10010 & (18) \end{array}$$

利用按位与可以判断 x 的奇偶性：若 x 为偶数，则 $x \,\&\, 1 = 0$；若 x 为奇数，则 $x \,\&\, 1 = 1$。更一般地，x 能被 2^k 整除，当且仅当 $x \,\&\, (2^k-1) = 0$。

按位或

按位或运算 $x \mid y$ 产生一个数，只要 x 和 y 对应位中至少有一个为 1，结果的该位就为 1。例如，$22 \mid 26 = 30$，因为

$$\begin{array}{rll} & 10110 & (22) \\ \mid & 11010 & (26) \\ \hline = & 11110 & (30) \end{array}$$

按位异或

按位异或运算 $x \hat{} y$ 产生一个数，只有 x 和 y 对应位中恰好一个为 1 时，结果的该位才为 1。例如，$22 \hat{} 26 = 12$，因为

$$\begin{array}{rll} & 10110 & (22) \\ \hat{} & 11010 & (26) \\ \hline = & 01100 & (12) \end{array}$$

按位取反

按位取反运算 $\sim x$ 将 x 的所有位翻转。恒有 $\sim x = -x - 1$，例如 $\sim 29 = -30$。

取反运算在位层面的结果取决于二进制表示的长度，因为它会翻转所有位。例如，若使用 32 位 int，结果如下：

$$\begin{array}{rcrl} x & = & 29 & 00000000000000000000000000011101 \\ \sim x & = & -30 & 11111111111111111111111111100010 \end{array}$$

位移

左移 $x << k$ 在数的末尾补上 k 个零，右移 $x >> k$ 删除末尾 k 位。例如，$14 << 2 = 56$，因为 14 和 56 分别对应 1110 和 111000。同理，$49 >> 3 = 6$，因为 49 和 6 分别对应 110001 和 110。

注意，$x << k$ 等价于将 x 乘以 2^k，$x >> k$ 等价于将 x 除以 2^k 后向下取整。

应用

形如 $1 << k$ 的数只有第 k 位为 1，其余位均为 0，因此可用它访问单个位。具体而言，一个数的第 k 位为 1，当且仅当 $x \,\&\, (1 << k)$ 非零。下列代码输出 int 数 x 的二进制表示：

```
for (int i = 31; i >= 0; i--) {
    if (x&(1<<i)) cout << "1";
    else cout << "0";
}
```

利用类似思想也可以修改单个位。例如，$x \mid (1 << k)$ 将 x 的第 k 位置为 1；x & ~$(1 << k)$ 将第 k 位置为 0；$x \hat{} (1 << k)$ 翻转第 k 位。

表达式 x & $(x-1)$ 将 x 最低的 1 位清零；x & $-x$ 将除最低 1 位外的所有 1 位清零；$x \mid (x-1)$ 翻转最低 1 位之后的所有位。还要注意，正数 x 是 2 的幂，当且仅当 x & $(x-1) = 0$。

其他函数

g++ 编译器提供下列位计数函数：

- ___builtin_clz(x): 数的开头连续零的数量
- ___builtin_ctz(x): 数的末尾连续零的数量
- ___builtin_popcount(x): 数中 1 的数量
- ___builtin_parity(x): 数中 1 的数量的奇偶性

这些函数可按如下方式使用：

```
int x = 5328; // 00000000000000000001010011010000
cout << ___builtin_clz(x) << "\n"; // 19
cout << ___builtin_ctz(x) << "\n"; // 4
cout << ___builtin_popcount(x) << "\n"; // 5
cout << ___builtin_parity(x) << "\n"; // 1
```

上述函数只支持 int，但也有以 ll 为后缀、支持 long long 的版本。

10.3 集合的表示

集合 $\{0, 1, 2, \ldots, n-1\}$ 的每个子集都可表示为一个 n 位整数，其中为 1 的位指出哪些元素属于该子集。这是一种高效的集合表示方式，因为每个元素只占一个比特，并且集合运算可以实现为位运算。

例如，由于 int 是 32 位类型，一个 int 数可以表示集合 $\{0,1,2,\ldots,31\}$ 的任意子集。集合 $\{1,3,4,8\}$ 的二进制表示为

$$00000000000000000000000100011010,$$

对应数值 $2^8+2^4+2^3+2^1=282$。

集合实现

下列代码声明一个 int 变量 x，用于保存 $\{0,1,2,\ldots,31\}$ 的子集。随后，代码把元素 1、3、4 和 8 加入集合，并输出集合大小。

```
int x = 0;
x |= (1<<1);
x |= (1<<3);
x |= (1<<4);
x |= (1<<8);
cout << __builtin_popcount(x) << "\n"; // 4
```

下列代码输出属于该集合的所有元素：

```
for (int i = 0; i < 32; i++) {
    if (x&(1<<i)) cout << i << " ";
}
// output: 1 3 4 8
```

集合运算

集合运算可以实现为如下位运算：

	集合写法	位运算写法
交集	$a \cap b$	$a \,\&\, b$
并集	$a \cup b$	$a \mid b$
补集	$\bar{a}$	$\sim a$
差集	$a \setminus b$	$a \,\&\, (\sim b)$

例如，下列代码先构造集合 $x=\{1,3,4,8\}$ 与 $y=\{3,6,8,9\}$，再构造集合 $z=x\cup y=\{1,3,4,6,8,9\}$：

```
int x = (1<<1)|(1<<3)|(1<<4)|(1<<8);
```

```
int y = (1<<3)|(1<<6)|(1<<8)|(1<<9);
int z = x|y;
cout << __builtin_popcount(z) << "\n"; // 6
```

遍历子集

下列代码遍历 $\{0, 1, \ldots, n-1\}$ 的所有子集：

```
for (int b = 0; b < (1<<n); b++) {
    // process subset b
}
```

下列代码遍历恰含 k 个元素的子集：

```
for (int b = 0; b < (1<<n); b++) {
    if (__builtin_popcount(b) == k) {
        // process subset b
    }
}
```

下列代码遍历集合 x 的所有子集：

```
int b = 0;
do {
    // process subset b
} while (b=(b-x)&x);
```

10.4 位运算优化

许多算法都可以用位运算优化。这类优化不会改变算法的时间复杂度，却可能显著影响代码的实际运行时间。本节讨论几个这样的例子。

Hamming 距离

两个等长字符串 a 和 b 之间的 **Hamming 距离** $\text{hamming}(a, b)$ 是二者字符不同的位置数。例如，

$$\text{hamming}(01101, 11001) = 2.$$

考虑如下问题：给定 n 个长度均为 k 的比特串，计算列表中两个字符串之间的最小 Hamming 距离。例如，$[00111, 01101, 11110]$ 的答案为 2，因为

- $\text{hamming}(00111, 01101) = 2$,
- $\text{hamming}(00111, 11110) = 3$, and
- $\text{hamming}(01101, 11110) = 3$.

直接解法是遍历所有字符串对并计算 Hamming 距离，得到 $O(n^2k)$ 时间算法。下列函数可用于计算距离：

```
int hamming(string a, string b) {
    int d = 0;
    for (int i = 0; i < k; i++) {
        if (a[i] != b[i]) d++;
    }
    return d;
}
```

若 k 较小，可以把比特串存为整数，并用位运算计算 Hamming 距离来优化代码。特别地，若 $k \leq 32$，可以直接把字符串存为 int，并用下列函数计算距离：

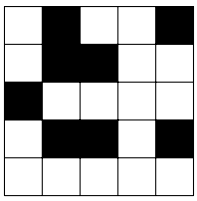
```
int hamming(int a, int b) {
    return __builtin_popcount(a^b);
}
```

上述函数中，异或运算构造出一个在 a 与 b 不同位置为 1 的比特串，再用 __builtin_popcount 统计 1 的数量。

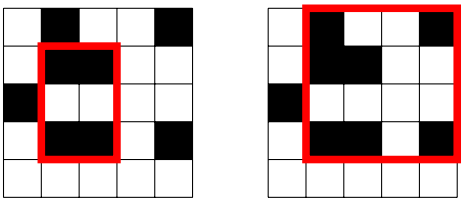
为比较两种实现，我们生成了 10000 个长度为 30 的随机比特串。第一种方法耗时 13.5 秒，位运算优化后只需 0.5 秒。因此，优化后的代码几乎比原代码快 30 倍。

子网格计数

再考虑如下问题：给定一个 $n \times n$ 网格，每个方格为黑色 (1) 或白色 (0)，计算四个角均为黑色的子网格数量。例如，网格



包含两个这样的子网格：



有一种 $O(n^3)$ 时间算法：遍历 $O(n^2)$ 对行，对每一对 (a, b)，在 $O(n)$ 时间内计算有多少列在这两行中都含黑色方格。下列代码假设 color$[y][x]$ 表示第 y 行第 x 列的颜色：

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (color[a][i] == 1 && color[b][i] == 1) count++;
}
```

这些列会形成 count(count $-$ 1)/2 个四角为黑色的子网格，因为可任选其中两列组成一个子网格。

为优化该算法，把网格按列分块，每块由连续的 N 列组成。随后将每一行存为一列 N 位整数，用以描述方格颜色。这样便可用位运算同时处理 N 列。下列代码中，color$[y][k]$ 以比特形式表示一块 N 个颜色：

```
int count = 0;
for (int i = 0; i <= n/N; i++) {
    count += __builtin_popcount(color[a][i]&color[b][i]);
}
```

所得算法的时间复杂度为 $O(n^3/N)$。

我们生成了一个 2500×2500 的随机网格，并比较原始实现与位运算优化实现。原代码耗时 29.6 秒，而优化版本在 $N = 32$ (int) 时只需 3.1 秒，在 $N = 64$ (long long) 时只需 1.7 秒。

10.5 动态规划

对于状态中包含元素子集的动态规划算法，位运算提供了一种高效方便的实现方式，因为这类状态可以存为整数。下面讨论结合位运算与动态规划的例子。

最优选择

第一个例子如下：已知 k 种商品在 n 天中的价格，希望每种商品恰好购买一次，但每天至多购买一种商品。最小总价是多少? 例如，考虑下列情形 ($k = 3$, $n = 8$)：

	0	1	2	3	4	5	6	7
商品 0	6	9	5	2	8	9	1	6
商品 1	8	2	6	2	7	5	7	2
商品 2	5	3	9	7	3	5	1	4

此时最小总价为 5：

	0	1	2	3	4	5	6	7
商品 0	6	9	5	**2**	8	9	1	6
商品 1	8	**2**	6	2	7	5	7	2
商品 2	5	3	9	7	3	5	**1**	4

令 $\text{price}[x][d]$ 表示商品 x 在第 d 天的价格。例如，上述情形中 $\text{price}[2][3] = 7$。再令 $\text{total}(S, d)$ 表示截至第 d 天购买商品子集 S 的最小总价，则问题答案为 $\text{total}(\{0 \ldots k-1\}, n-1)$。

首先，$\text{total}(\emptyset, d) = 0$，因为购买空集无需花费；且 $\text{total}(\{x\}, 0) = \text{price}[x][0]$，因为第一天购买一种商品只有一种方式。随后使用递推式：

$$\text{total}(S, d) = \min(\text{total}(S, d-1), \min_{x \in S}(\text{total}(S \setminus x, d-1) + \text{price}[x][d]))$$

这表示第 d 天要么不买任何商品，要么购买属于 S 的商品 x。后一种情形中，从 S 删除 x，并把 x 的价格加入总价。

下一步用动态规划计算函数值。声明数组保存函数值：

```
int total[1<<K][N];
```

其中 K 和 N 是足够大的常量。数组第一维对应子集的二进制表示。

首先，按如下方式处理 $d = 0$ 的情形：

```
for (int x = 0; x < k; x++) {
    total[1<<x][0] = price[x][0];
}
```

随后，递推式转化为下列代码：

```
for (int d = 1; d < n; d++) {
    for (int s = 0; s < (1<<k); s++) {
        total[s][d] = total[s][d-1];
        for (int x = 0; x < k; x++) {
            if (s&(1<<x)) {
                total[s][d] = min(total[s][d],
                                  total[s^(1<<x)][d-1]+price[x][d]);
            }
        }
    }
}
```

算法的时间复杂度为 $O(n2^k k)$。

从排列到子集

利用动态规划，常可把对排列的遍历转化为对子集的遍历[1]。这样做的好处是，排列数 $n!$ 远大于子集数 2^n。例如，当 $n = 20$ 时，$n! \approx 2.4 \cdot 10^{18}$，而 $2^n \approx 10^6$。因此，对某些 n，可以高效遍历子集，却无法遍历排列。

例如，考虑如下问题：一部电梯的最大载重为 x，有 n 个体重已知的人想从底层到顶层。若按最优顺序进入电梯，最少需要运行多少趟?

例如，设 $x = 10$、$n = 5$，各人体重如下：

人员	体重
0	2
1	3
2	3
3	5
4	6

此时最少需要 2 趟。一种最优顺序为 $\{0, 2, 3, 1, 4\}$，将人员分为两趟：先运送 $\{0, 2, 3\}$（总重 10），再运送 $\{1, 4\}$（总重 9）。

测试 n 个人的所有排列，可以轻松在 $O(n!n)$ 时间内解决问题。不过，使用动态规划可以得到更高效的 $O(2^n n)$ 时间算法。其思想是为每个人员子集计算两个值：所需的最少趟数，以及最后一趟的最小总重。

[1]该技术由 M. Held 和 R. M. Karp 于 1962 年提出 [34]。

令 weight[p] 表示人员 p 的体重。定义两个函数：rides(S) 是子集 S 所需的最少趟数，last(S) 是最后一趟的最小总重。例如，在上述情形中

$$\text{rides}(\{1,3,4\}) = 2 \quad \text{且} \quad \text{last}(\{1,3,4\}) = 5,$$

因为最优的两趟为 $\{1,4\}$ 和 $\{3\}$,第二趟总重为 5。最终目标当然是计算 rides($\{0 \ldots n-1\}$)。

可以递归计算函数值，再应用动态规划。遍历属于 S 的所有人员，最优地选择最后进入电梯的人 p。每种选择都会产生一个更小人员子集的子问题。若 last($S \setminus p$) + weight[p] $\le x$，可以把 p 加入最后一趟；否则必须另开一趟，起初只包含 p。

为实现动态规划，声明数组

```
pair<int,int> best[1<<N];
```

为每个子集 S 保存二元组 (rides(S), last(S))。空集的值如下：

```
best[0] = {1,0};
```

随后按如下方式填充数组：

```
for (int s = 1; s < (1<<n); s++) {
    // initial value: n+1 rides are needed
    best[s] = {n+1,0};
    for (int p = 0; p < n; p++) {
        if (s&(1<<p)) {
            auto option = best[s^(1<<p)];
            if (option.second+weight[p] <= x) {
                // add p to an existing ride
                option.second += weight[p];
            } else {
                // reserve a new ride for p
                option.first ++;
                option.second = weight[p];
            }
            best[s] = min(best[s], option);
        }
    }
}
```

注意，上述循环保证对任意满足 $S_1 \subset S_2$ 的两个子集 S_1 和 S_2，总是先处理 S_1。因此，动态规划值按正确顺序计算。

子集计数

本章最后一个问题如下：令 $X = \{0 \ldots n-1\}$，并为每个子集 $S \subset X$ 赋予整数 value$[S]$。需要对每个 S 计算

$$\text{sum}(S) = \sum_{A \subset S} \text{value}[A],$$

即 S 的所有子集的值之和。

例如，设 $n = 3$，各值如下：

- value$[\emptyset] = 3$
- value$[\{0\}] = 1$
- value$[\{1\}] = 4$
- value$[\{0,1\}] = 5$
- value$[\{2\}] = 5$
- value$[\{0,2\}] = 1$
- value$[\{1,2\}] = 3$
- value$[\{0,1,2\}] = 3$

此时，例如，

$$\begin{aligned}\text{sum}(\{0,2\}) &= \text{value}[\emptyset] + \text{value}[\{0\}] + \text{value}[\{2\}] + \text{value}[\{0,2\}] \\ &= 3 + 1 + 5 + 1 = 10.\end{aligned}$$

由于共有 2^n 个子集，一种解法是在 $O(2^{2n})$ 时间内遍历所有子集对。不过，使用动态规划可以在 $O(2^n n)$ 时间内解决问题。其思想是限制可以从 S 中删除的元素，并研究相应的和。

令 partial(S, k) 表示 S 的所有满足“只能删除元素 $0 \ldots k$”这一限制的子集的值之和。例如，

$$\text{partial}(\{0,2\}, 1) = \text{value}[\{2\}] + \text{value}[\{0,2\}],$$

因为只能删除元素 $0 \ldots 1$。可以用 partial 计算 sum，因为

$$\text{sum}(S) = \text{partial}(S, n-1).$$

函数的基本情形为

$$\text{partial}(S, -1) = \text{value}[S],$$

因为此时不能从 S 中删除任何元素。一般情形使用下列递推式：

$$\text{partial}(S,k) = \begin{cases} \text{partial}(S,k-1) & k \notin S \\ \text{partial}(S,k-1) + \text{partial}(S \setminus \{k\}, k-1) & k \in S \end{cases}$$

这里关注元素 k。若 $k \in S$，有两种选择：在 S 中保留 k，或从 S 删除 k。

有一种特别巧妙的方式实现这些和的计算。声明数组

```
int sum[1<<N];
```

保存每个子集的和。数组初始化如下：

```
for (int s = 0; s < (1<<n); s++) {
    sum[s] = value[s];
}
```

随后按如下方式填充数组：

```
for (int k = 0; k < n; k++) {
    for (int s = 0; s < (1<<n); s++) {
        if (s&(1<<k)) sum[s] += sum[s^(1<<k)];
    }
}
```

该代码把 $k = 0 \ldots n-1$ 时的 $\text{partial}(S,k)$ 依次计算到数组 sum 中。由于 $\text{partial}(S,k)$ 总是以 $\text{partial}(S,k-1)$ 为基础，可以复用数组 sum，从而得到非常高效的实现。

第二部分

图算法

第十一章　图论基础

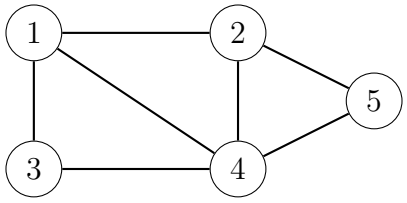
许多编程问题都可以通过把问题建模成图论问题，再使用合适的图算法来解决。图的一个典型例子是由一个国家的城市和道路组成的网络。不过，有时图隐藏在问题之中，并不容易识别。

本书这一部分讨论图算法，尤其关注算法竞赛中的重要主题。本章将介绍与图有关的概念，并研究在算法中表示图的不同方法。

11.1　图论术语

一个**图**由**节点**和**边**组成。本书用变量 n 表示图的节点数，用变量 m 表示边数。节点用整数 $1, 2, \ldots, n$ 编号。

例如，下图包含 5 个节点和 7 条边：



11.2　图的表示

算法中有多种表示图的方法。数据结构的选择取决于图的规模以及算法处理图的方式。下面介绍三种常见表示。

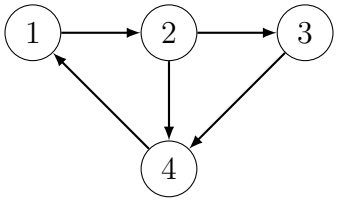
邻接表表示

在邻接表表示中，图中每个节点 x 都对应一个**邻接表**，其中包含从 x 出发有边相连的节点。邻接表是最常用的图表示方法，大多数算法都可以基于它高效实现。

一种方便的存储方式是声明一个向量数组：

```
vector<int> adj[N];
```

常量 N 应足够大，以容纳所有邻接表。例如，图



可以存储为：

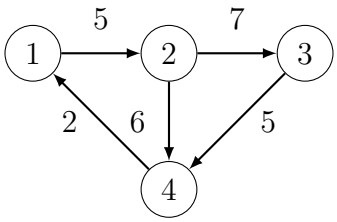
```
adj [1]. push_back(2);
adj [2]. push_back(3);
adj [2]. push_back(4);
adj [3]. push_back(4);
adj [4]. push_back(1);
```

如果图是无向图，也可用类似方法存储，但每条边要按两个方向各加入一次。

对于带权图，可将结构扩展为：

```
vector<pair<int,int>> adj[N];
```

此时，每当存在一条从节点 a 到节点 b、权值为 w 的边时，节点 a 的邻接表中就包含数对 (b, w)。例如，图



可以存储为：

```
adj [1]. push_back({2,5});
adj [2]. push_back({3,7});
adj [2]. push_back({4,6});
adj [3]. push_back({4,5});
adj [4]. push_back({1,2});
```

邻接表的优点是可以高效找到经一条边能从给定节点到达的所有节点。例如，下面的循环遍历从节点 s 出发能够直接到达的所有节点：

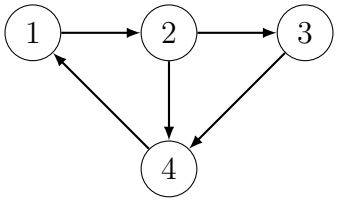
```
for (auto u : adj[s]) {
    // process node u
}
```

邻接矩阵表示

邻接矩阵是一个二维数组，用来表示图中包含哪些边。利用邻接矩阵，可以高效检查两个节点之间是否存在边。矩阵可存储为数组

```
int adj[N][N];
```

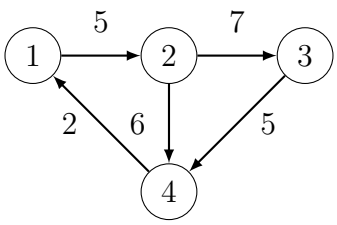
其中每个值 adj$[a][b]$ 表示图中是否含有从节点 a 到节点 b 的边。若该边存在，则 adj$[a][b] = 1$；否则 adj$[a][b] = 0$。例如，图



可表示为：

	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	1	0	0	0

如果图带权，则可扩展邻接矩阵表示，使存在边时矩阵中存放该边的权值。采用这种表示，图



对应下列矩阵：

	1	2	3	4
1	0	5	0	0
2	0	0	7	6
3	0	0	0	5
4	2	0	0	0

邻接矩阵表示的缺点是矩阵包含 n^2 个元素，而其中大多数通常为零。因此，当图很大时不能采用这种表示。

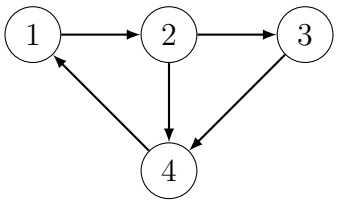
边列表表示

边列表按某种顺序包含图的所有边。当算法需要处理图中所有边，而不需要查找从某个给定节点出发的边时，这是一种方便的图表示方法。

边列表可以存放在向量

```
vector<pair<int,int>> edges;
```

中，其中每个数对 (a, b) 表示存在一条从节点 a 到节点 b 的边。因此，图



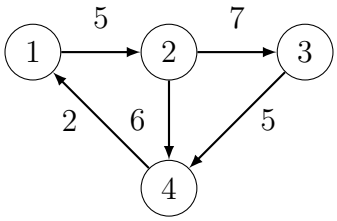
可以表示为：

```
edges.push_back({1,2});
edges.push_back({2,3});
edges.push_back({2,4});
edges.push_back({3,4});
edges.push_back({4,1});
```

若图带权，可将结构扩展为：

```
vector<tuple<int,int,int>> edges;
```

列表中的每个元素形如 (a, b, w)，表示存在一条从节点 a 到节点 b、权值为 w 的边。例如，图

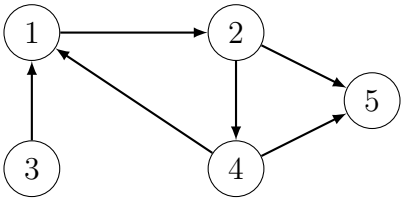


可以表示为[1]：

```
edges.push_back({1,2,5});
edges.push_back({2,3,7});
edges.push_back({2,4,6});
edges.push_back({3,4,5});
edges.push_back({4,1,2});
```

边的方向

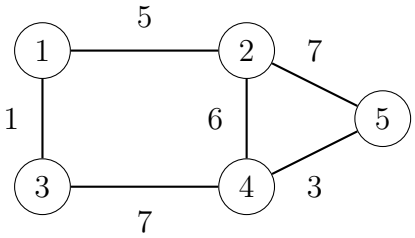
如果每条边只能沿一个方向通过，则称图为**有向图**。例如，下图是有向图：



上图包含从节点 3 到节点 5 的路径 $3 \to 1 \to 2 \to 5$，但不存在从节点 5 到节点 3 的路径。

边权

在**带权图**中，每条边都被赋予一个**权值**。权值通常解释为边的长度。例如，下图是带权图：

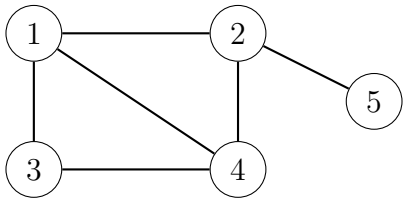


[1]在某些较旧的编译器中，必须用函数 make_tuple 代替花括号（例如用 make_tuple(1,2,5) 代替 {1,2,5}）。

带权图中一条路径的长度是路径上各边权值之和。例如，在上图中，路径 $1 \to 2 \to 5$ 的长度为 12，路径 $1 \to 3 \to 4 \to 5$ 的长度为 11。后者是从节点 1 到节点 5 的**最短**路径。

邻居与度数

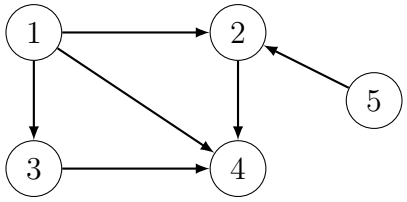
若两个节点之间存在一条边，则称它们互为**邻居**或彼此**相邻**。节点的**度数**是其邻居的数量。例如，在下图中，节点 2 的邻居是 1、4 和 5，所以它的度数为 3。



图中所有节点的度数之和总是 $2m$，其中 m 是边数，因为每条边都会使恰好两个节点的度数各增加一。因此，度数之和必为偶数。

如果每个节点的度数都等于同一个常数 d，则称图为**正则图**。如果每个节点的度数都是 $n-1$，即图中包含节点之间所有可能的边，则称图为**完全图**。

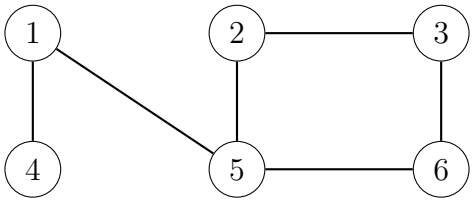
在有向图中，以某节点为终点的边数称为该节点的**入度**，以该节点为起点的边数称为它的**出度**。例如，在下图中，节点 2 的入度为 2，出度为 1。



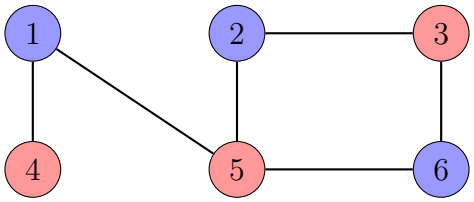
着色

图的**着色**是给每个节点分配一种颜色，使任意两个相邻节点的颜色都不同。

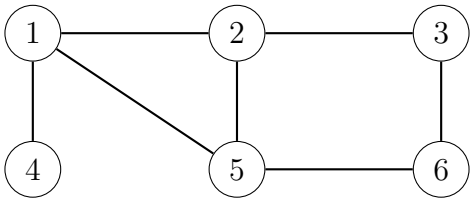
如果一个图能够用两种颜色完成着色，则称它为**二分图**。事实上，一个图是二分图，当且仅当它不含边数为奇数的环。例如，图



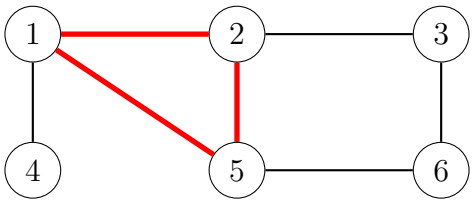
是二分图，因为可以按如下方式着色：



然而，图

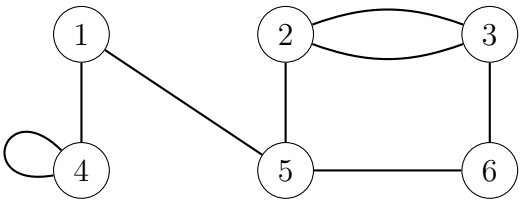


不是二分图，因为下面这个由三个节点组成的环无法用两种颜色着色：

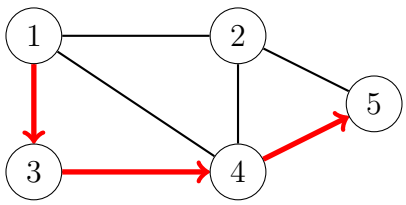


简单图

如果没有边的起点与终点是同一节点，并且任意两个节点之间都不存在重边，则称图为**简单图**。我们经常假定所讨论的图是简单图。例如，下图就不是简单图：



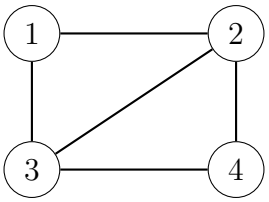
一条**路径**沿图中的边从节点 a 通向节点 b。路径的**长度**是其中所含的边数。例如，上图包含一条从节点 1 到节点 5、长度为 3 的路径 $1 \to 3 \to 4 \to 5$：



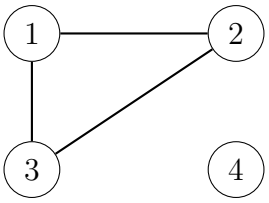
如果路径的第一个节点和最后一个节点相同，则该路径是一个**环**。例如，上图包含环 $1 \to 3 \to 4 \to 1$。如果每个节点在一条路径中至多出现一次，则称该路径为**简单路径**。

连通性

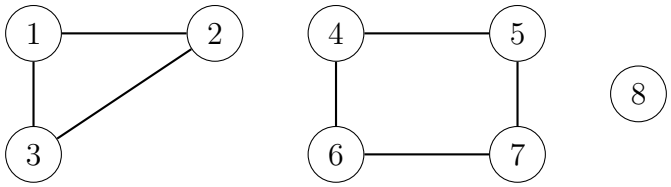
如果任意两个节点之间都存在路径，则称图是**连通的**。例如，下图是连通图：



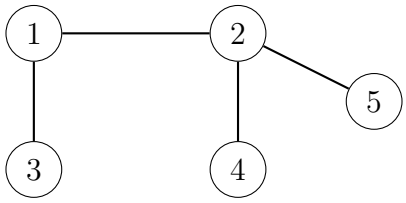
下图不是连通图，因为无法从节点 4 到达任何其他节点：



图中的各个连通部分称为它的**连通分量**。例如，下图包含三个连通分量：$\{1, 2, 3\}$、$\{4, 5, 6, 7\}$ 和 $\{8\}$。



树是由 n 个节点和 $n-1$ 条边组成的连通图。树中任意两个节点之间都有且仅有一条路径。例如，下图是一棵树：



第十二章　图的遍历

本章讨论两种基本的图算法：深度优先搜索和广度优先搜索。两种算法都从图中的一个起始节点出发，访问从该节点能够到达的所有节点。它们的区别在于访问节点的顺序。

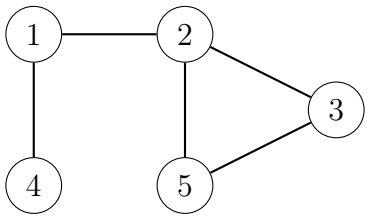
12.1　深度优先搜索

深度优先搜索（DFS）是一种直接的图遍历方法。算法从起始节点开始，沿图中的边前往从该节点能够到达的所有其他节点。

只要还能发现新节点，深度优先搜索就会沿图中的一条路径不断前进。随后，它返回先前的节点并开始探索图的其他部分。算法会记录已经访问过的节点，从而保证每个节点只处理一次。

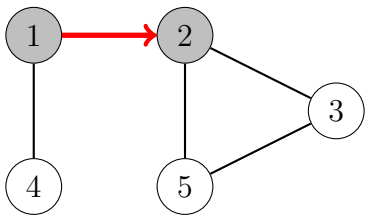
示例

考察深度优先搜索如何处理下图：

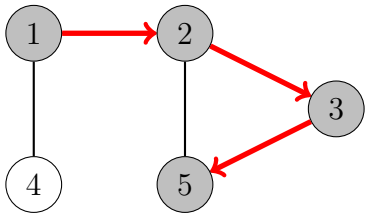


搜索可以从图中任意节点开始；这里从节点 1 开始。

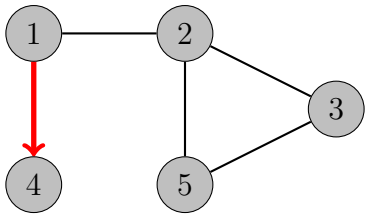
搜索首先前往节点 2：



随后访问节点 3 和 5：



节点 5 的邻居是 2 和 3，但搜索已经访问过二者，因此现在要返回先前的节点。节点 3 和 2 的邻居也都已访问，所以下一步从节点 1 前往节点 4：



至此所有节点都已访问，搜索结束。

深度优先搜索的时间复杂度为 $O(n+m)$，其中 n 为节点数，m 为边数，因为算法会把每个节点和每条边处理一次。

实现

使用递归可以很方便地实现深度优先搜索。下面的函数 dfs 从给定节点开始深度优先搜索。函数假定图以邻接表形式存储在数组

```
vector<int> adj[N];
```

中，并维护数组

```
bool visited[N];
```

来记录已经访问的节点。初始时数组中每个值都是 false；搜索到达节点 s 时，visited[s] 变为 true。函数可实现如下：

```
void dfs(int s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
    for (auto u: adj[s]) {
        dfs(u);
    }
}
```

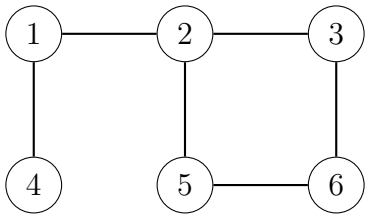
12.2 广度优先搜索

广度优先搜索（BFS）按照与起始节点的距离从小到大访问节点。因此，我们可以用广度优先搜索计算起始节点到所有其他节点的距离。不过，广度优先搜索的实现比深度优先搜索更复杂。

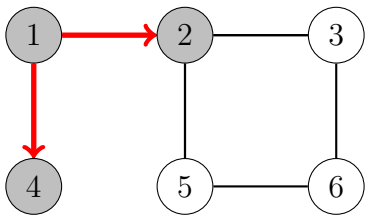
广度优先搜索逐层遍历节点。搜索首先探索与起始节点距离为 1 的节点，然后探索距离为 2 的节点，依此类推，直到所有节点均已访问。

示例

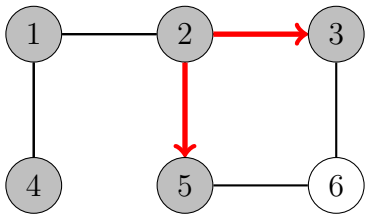
考察广度优先搜索如何处理下图：



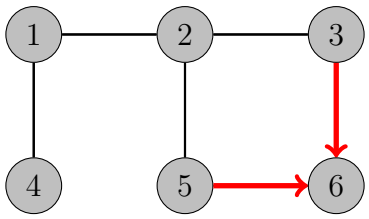
假设搜索从节点 1 开始。首先处理从节点 1 经过一条边即可到达的所有节点：



随后前往节点 3 和 5：



最后访问节点 6：



现在已经算出起始节点到图中所有节点的距离：

节点	距离
1	0
2	1
3	2
4	1
5	2
6	3

与深度优先搜索一样，广度优先搜索的时间复杂度也是 $O(n+m)$，其中 n 为节点数，m 为边数。

实现

广度优先搜索的实现比深度优先搜索更复杂，因为算法会访问图中不同区域的节点。典型实现基于一个存放节点的队列；每一步处理队首的下一个节点。

以下代码假定图以邻接表存储，并维护这些数据结构：

```
queue<int> q;
bool visited[N];
int distance[N];
```

队列 q 按距离递增的顺序保存待处理节点。新节点总是加入队尾，而队首节点是下一个待处理节点。数组 visited 表示搜索已经访问了哪些节点，数组 distance 则保存起始节点到图中所有节点的距离。

从节点 x 开始的搜索可实现如下：

```
visited[x] = true;
distance[x] = 0;
q.push(x);
while (!q.empty()) {
    int s = q.front(); q.pop();
    // process node s
    for (auto u : adj[s]) {
        if (visited[u]) continue;
        visited[u] = true;
        distance[u] = distance[s]+1;
```

```
            q.push(u);
        }
    }
```

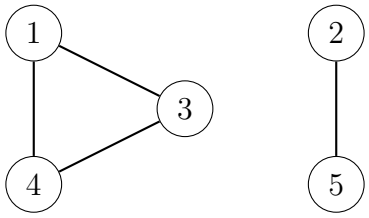
12.3 应用

利用图遍历算法，我们可以检查图的许多性质。通常深度优先搜索和广度优先搜索都能使用；实际中深度优先搜索往往更合适，因为它更容易实现。在以下应用中，我们假定图是无向图。

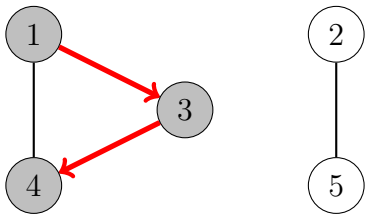
连通性检查

若图中任意两个节点之间都存在路径，则称图是连通的。因此，从任意节点出发，检查是否能到达所有其他节点，就可以判断图是否连通。

例如，在图



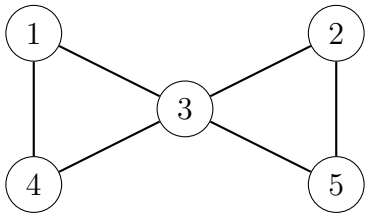
中，从节点 1 开始的深度优先搜索会访问以下节点：



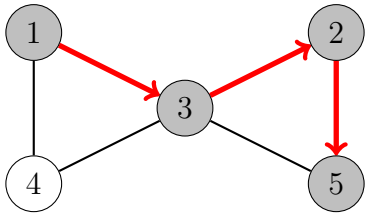
由于搜索没有访问所有节点，可以断定该图不连通。类似地，遍历各个节点，每当当前节点尚不属于任何分量时就启动一次新的深度优先搜索，还可以找出图的所有连通分量。

寻找环

在图遍历过程中，如果发现某个节点的一个邻居已经访问过，并且该邻居不是当前路径上的前一个节点，那么图中就存在环。例如，图



包含两个环。可以按如下方式找到其中一个：



从节点 2 移动到节点 5 后，我们发现节点 5 的邻居 3 已经访问过。因此，图中存在一个经过节点 3 的环，例如 $3 \to 2 \to 5 \to 3$。

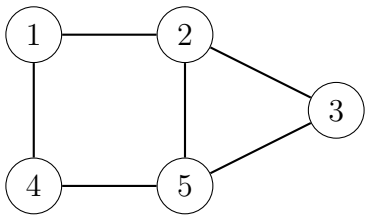
判断图中是否含环的另一种方法，是直接计算每个分量的节点数与边数。若一个分量含有 c 个节点且没有环，它必须恰好含有 $c-1$ 条边（因而是一棵树）。如果边数不少于 c，该分量一定含环。

二分图检查

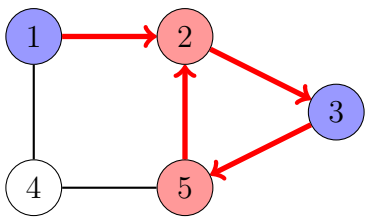
如果可以用两种颜色给图中的节点着色，使任意相邻节点颜色都不同，则该图是二分图。使用图遍历算法判断一个图是否为二分图出乎意料地简单。

基本思想是把起始节点染成蓝色，把它的所有邻居染成红色，再把这些邻居的邻居染成蓝色，依此类推。如果搜索过程中发现两个相邻节点颜色相同，图就不是二分图；否则图是二分图，并且我们已经得到一种合法着色。

例如，图



不是二分图，因为从节点 1 开始的搜索过程如下：



可以看到，节点 2 和 5 都被染成红色，但它们在图中相邻，因此该图不是二分图。

该算法总是正确的，因为只有两种颜色可用时，一个分量中起始节点的颜色就决定了该分量内所有其他节点的颜色。起始节点染成红色还是蓝色并无影响。

需要注意，一般情况下，要判断图中节点能否使用 k 种颜色着色、同时保证相邻节点颜色不同是很困难的。即使 $k = 3$，也没有已知的高效算法，因为这个问题是 NP 困难的。

第十三章　最短路

寻找图中两个节点之间的最短路径是一个重要问题，有许多实际应用。例如，在道路网络中，给定各条道路的长度，一个自然的问题就是计算两个城市之间可能的最短路线长度。

在无权图中，路径长度等于其边数，可以直接用广度优先搜索寻找最短路。本章则重点讨论带权图；在这种图中，需要更复杂的算法来寻找最短路。

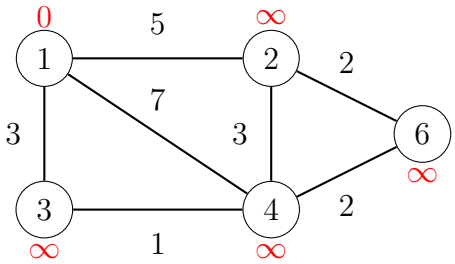
13.1　Bellman–Ford 算法

Bellman–Ford 算法[1]计算从一个起始节点到图中所有节点的最短路。只要图中不存在长度为负的环，算法可以处理各种图；如果图中存在负环，算法也能检测出来。

算法记录起始节点到图中所有节点的距离。初始时，到起始节点的距离为 0，到其他节点的距离为无穷大。算法不断寻找能够缩短路径的边来减小距离，直到任何距离都无法继续减小。

示例

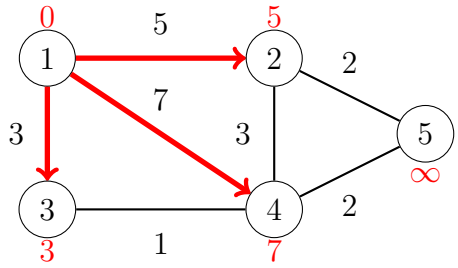
考察 Bellman–Ford 算法如何处理下图：



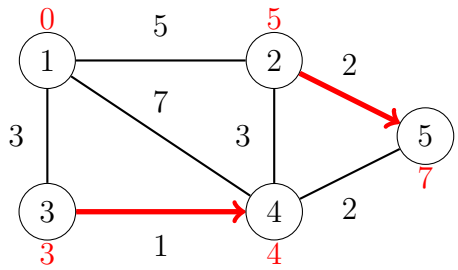
图中每个节点都被赋予一个距离。初始时，到起始节点的距离为 0，到其他节点的距离为无穷大。

[1]该算法以 R. E. Bellman 和 L. R. Ford 命名，两人分别于 1958 年和 1956 年独立发表了这一算法 [5, 24]。

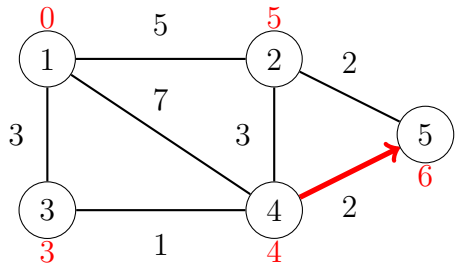
算法寻找能减小距离的边。首先，从节点 1 出发的所有边都会减小距离：



随后，边 $2 \to 5$ 和 $3 \to 4$ 会减小距离：

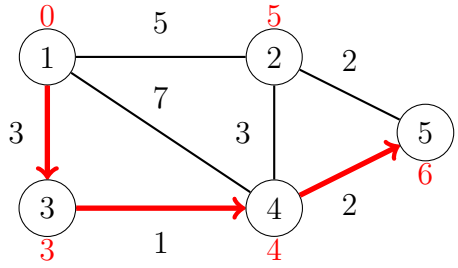


最后还会发生一次变化：



此后没有任何边能够减小距离，说明所有距离已确定，我们成功算出了起始节点到图中所有节点的最短距离。

例如，从节点 1 到节点 5 的最短距离 6 对应下列路径：



实现

下面的 Bellman–Ford 实现计算节点 x 到图中所有节点的最短距离。代码假设图用边列表 edges 存储，其中的每个三元组 (a, b, w) 表示一条从 a 到 b、权值为 w 的边。

算法共执行 $n-1$ 轮，每轮遍历图中所有边，并尝试减小距离。算法构造数组 distance，保存 x 到所有节点的距离；常量 INF 表示无穷距离。

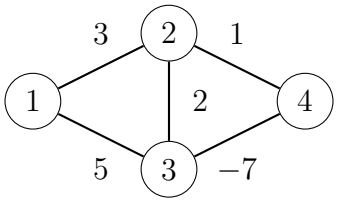
```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
for (int i = 1; i <= n-1; i++) {
    for (auto e : edges) {
        int a, b, w;
        tie(a, b, w) = e;
        distance[b] = min(distance[b], distance[a]+w);
    }
}
```

算法执行 $n-1$ 轮，每轮遍历全部 m 条边，因此时间复杂度为 $O(nm)$。若图中没有负环，$n-1$ 轮后所有距离必已确定，因为任意最短路径至多包含 $n-1$ 条边。

实际中，通常不需要 $n-1$ 轮就能得到最终距离。因此，可以在某一轮没有任何距离减小时提前停止，以提高效率。

负环

Bellman–Ford 算法还可以检查图中是否存在长度为负的环。例如，图



包含长度为 -4 的负环 $2 \to 3 \to 4 \to 2$。

如果图中含负环，任何包含该环的路径都能通过反复绕环而无限缩短，因此此时“最短路径”没有意义。

执行 n 轮 Bellman–Ford 即可检测负环。如果最后一轮仍有距离减小，图中就存在负环。注意，无论起始节点如何，这种算法都可以搜索整个图中的负环。

SPFA 算法

SPFA 算法（“Shortest Path Faster Algorithm”）[20] 是 Bellman–Ford 的一种变体，通常比原算法更高效。SPFA 不在每一轮遍历所有边，而是更智能

地选择要检查的边。

算法维护一个可能用于减小距离的节点队列。首先把起始节点 x 加入队列。随后，每次处理队首节点；当边 $a \to b$ 使某个距离减小时，把节点 b 加入队列。

SPFA 的效率取决于图的结构：它通常很高效，但最坏时间复杂度仍为 $O(nm)$，也可以构造使其和原始 Bellman–Ford 一样慢的输入。

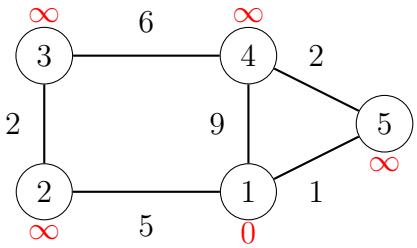
13.2 Dijkstra 算法

Dijkstra 算法[2]与 Bellman–Ford 一样，计算从起始节点到图中所有节点的最短路。Dijkstra 的优点是效率更高，能够处理大型图；但它要求图中不存在负权边。

与 Bellman–Ford 相同，Dijkstra 也维护到各节点的距离，并在搜索过程中减小这些距离。由于图中没有负权边，算法只需处理每条边一次，因此很高效。

示例

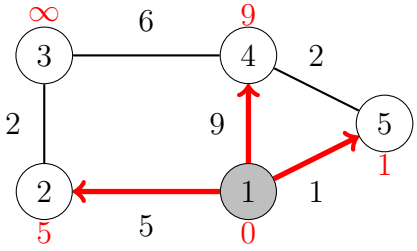
以节点 1 为起点，考察 Dijkstra 算法如何处理下图：



与 Bellman–Ford 一样，初始时到起始节点的距离为 0，到其他节点的距离为无穷大。

每一步，Dijkstra 选择一个尚未处理、且当前距离最小的节点。第一个这样的节点是距离为 0 的节点 1。

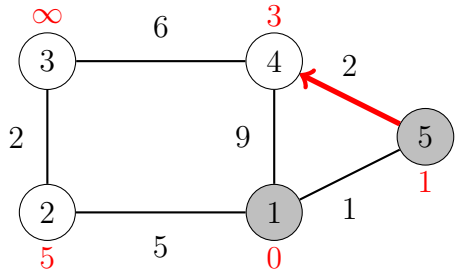
选中某节点后，算法遍历从它出发的所有边，并利用这些边减小距离：



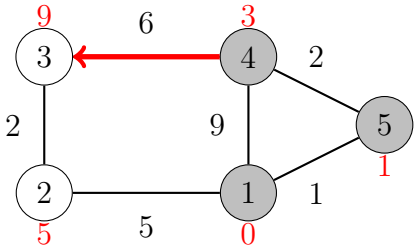
[2]E. W. Dijkstra 于 1959 年发表了该算法 [14]，但原始论文没有说明如何高效实现它。

本例中，从节点 1 出发的边使节点 2、4、5 的距离分别变为 5、9、1。

下一个处理的是距离为 1 的节点 5，它把节点 4 的距离从 9 降为 3：

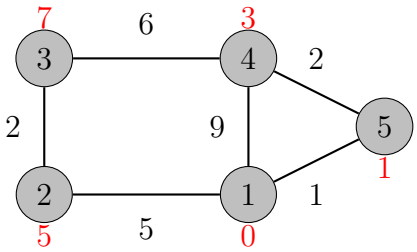


随后处理节点 4，把节点 3 的距离降为 9：



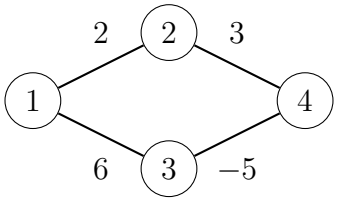
Dijkstra 算法的一个重要性质是：每当一个节点被选中时，它的距离就已是最终值。例如，此时到节点 1、5、4 的距离 0、1、3 都已确定。

随后处理余下两个节点，最终距离如下：



负权边

Dijkstra 的效率建立在图中不存在负权边这一事实之上。如果存在负权边，算法可能给出错误结果。例如，考虑下图：



从节点 1 到节点 4 的最短路是 $1 \to 3 \to 4$，长度为 1。然而，Dijkstra 会沿当前最小权边找到路径 $1 \to 2 \to 4$，没有考虑另一条路径上的权值 -5 会抵消之前较大的权值 6。

实现

下面的 Dijkstra 实现计算节点 x 到图中其他节点的最短距离。图用邻接表存储：每当存在从 a 到 b、权值为 w 的边时，adj[a] 中就包含数对 (b, w)。

高效实现要求能够迅速找出尚未处理且距离最小的节点。合适的数据结构是按距离排列节点的优先队列，使用它可以在对数时间内取出下一个待处理节点。

以下代码中，优先队列 q 存放形如 $(-d, x)$ 的数对，表示节点 x 的当前距离为 d。数组 distance 保存各节点距离，数组 processed 表示节点是否已处理。初始时，到 x 的距离为 0，到其他节点的距离为 ∞。

```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
q.push({0,x});
while (!q.empty()) {
    int a = q.top().second; q.pop();
    if (processed[a]) continue;
    processed[a] = true;
    for (auto u : adj[a]) {
        int b = u.first, w = u.second;
        if (distance[a]+w < distance[b]) {
            distance[b] = distance[a]+w;
            q.push({-distance[b],b});
        }
    }
}
```

注意，优先队列中保存的是距离的相反数。这是因为 C++ 默认优先队列取最大元素，而我们需要最小元素；使用负距离就能直接使用默认优先队列[3]。同一节点可能在队列中出现多次，但只会处理距离最小的那次。

上述实现的时间复杂度为 $O(n+m\log m)$，因为算法遍历图中所有节点，并且每条边至多向优先队列加入一个距离。

[3]当然，也可以像第 4.5 节那样声明优先队列并使用正距离，但实现会稍长。

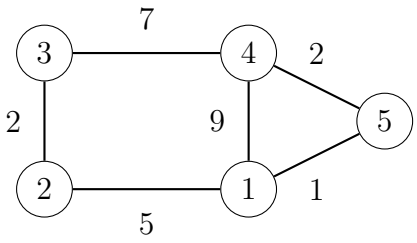
13.3 Floyd–Warshall 算法

Floyd–Warshall 算法[4]提供了另一种求最短路的方法。与本章其他算法不同，它一次运行就能求出所有节点对之间的最短路。

算法维护一个包含节点间距离的二维数组。首先只使用节点之间的直接边计算距离，随后允许路径经过中间节点，并据此不断减小距离。

示例

考察 Floyd–Warshall 如何处理下图：



初始时，每个节点到自身的距离为 0；若节点 a 与 b 之间有权值为 x 的边，则它们之间的距离为 x；其他距离均为无穷大。

该图的初始数组如下：

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

算法分若干轮执行。每一轮选择一个新的节点，使它从此可以作为路径的中间节点，并借助该节点减小距离。

第一轮把节点 1 作为新的中间节点。节点 1 连接了节点 2 和 4，于是出现长度为 14 的新路径；节点 2 和 5 之间也出现长度为 6 的新路径。

[4]该算法以 R. W. Floyd 和 S. Warshall 命名，两人于 1962 年独立发表了它 [23, 70]。

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	**14**	**6**
3	∞	2	0	7	∞
4	9	**14**	7	0	2
5	1	**6**	∞	2	0

第二轮把节点 2 作为新的中间节点，产生节点 1 与 3、节点 3 与 5 之间的新路径：

	1	2	3	4	5
1	0	5	**7**	9	1
2	5	0	2	14	6
3	**7**	2	0	7	**8**
4	9	14	7	0	2
5	1	6	**8**	2	0

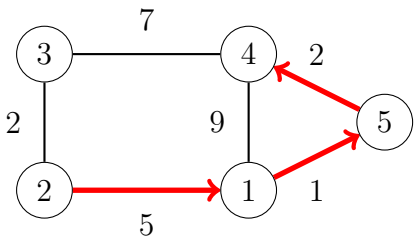
第三轮把节点 3 作为新的中间节点，节点 2 和 4 之间出现一条新路径：

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	**9**	6
3	7	2	0	7	8
4	9	**9**	7	0	2
5	1	6	8	2	0

算法以同样方式继续，直到所有节点都曾作为中间节点。结束后，数组包含任意两个节点之间的最短距离：

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

例如，数组表明节点 2 和 4 之间的最短距离为 8，对应下列路径：



实现

Floyd–Warshall 的优点是容易实现。以下代码构造距离矩阵，其中 distance[a][b] 是节点 a 与 b 之间的最短距离。首先使用图的邻接矩阵 adj 初始化 distance：

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == j) distance[i][j] = 0;
        else if (adj[i][j]) distance[i][j] = adj[i][j];
        else distance[i][j] = INF;
    }
}
```

随后按如下方式求出最短距离：

```
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            distance[i][j] = min(distance[i][j],
                                 distance[i][k]+distance[k][j]);
        }
    }
}
```

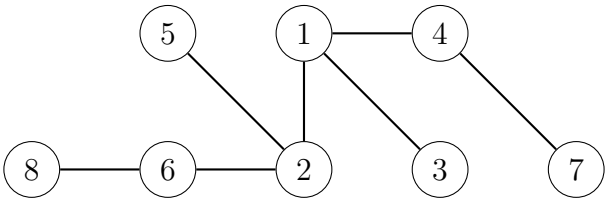
算法包含三层遍历图中节点的嵌套循环，因此时间复杂度为 $O(n^3)$。

由于 Floyd–Warshall 实现简单，即使只需寻找图中的一条最短路，它也可能是不错的选择。不过，只有当图足够小、立方时间复杂度可以接受时才能使用该算法。

第十四章　树算法

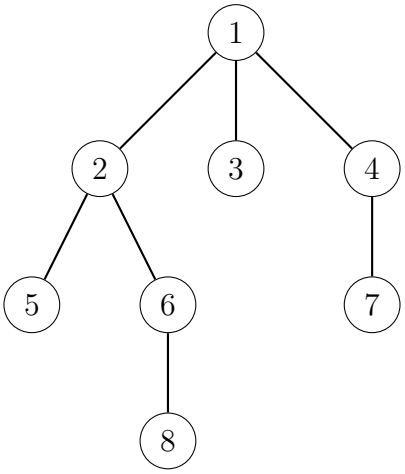
树是由 n 个节点和 $n-1$ 条边组成的连通无环图。从树中删除任意一条边都会把它分成两个连通分量，而向树中添加任意一条边都会形成环。此外，树中任意两个节点之间始终存在唯一一条路径。

例如，下图中的树包含 8 个节点和 7 条边：



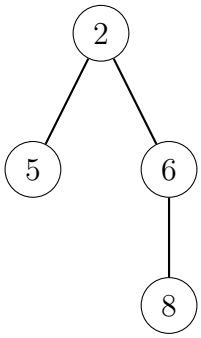
树的**叶节点**是度数为 1 的节点，也就是只有一个邻居的节点。例如，上述树的叶节点为 3、5、7 和 8。

在**有根树**中，选定一个节点作为树的**根**，其他节点都位于根的下方。例如，在下图中，节点 1 是根节点。



在有根树中，一个节点下方的邻居称为它的**子节点**，上方的邻居称为它的**父节点**。除没有父节点的根以外，每个节点恰有一个父节点。例如，在上图中，节点 2 的子节点是节点 5 和 6，它的父节点是节点 1。

有根树的结构是*递归的*：树中每个节点都可以作为一棵**子树**的根，这棵子树包含该节点本身，以及其所有子节点的子树中的节点。例如，在上图中，节点 2 的子树由节点 2、5、6 和 8 构成：



14.1 树的遍历

一般的图遍历算法也能用于遍历树的节点。不过，树的遍历比一般图的遍历更容易实现，因为树中没有环，也不可能从多个方向到达同一个节点。

遍历树的典型方法是从任意节点开始深度优先搜索。可以使用以下递归函数：

```
void dfs(int s, int e) {
    // process node s
    for (auto u : adj[s]) {
        if (u != e) dfs(u, s);
    }
}
```

函数接收两个参数：当前节点 s 和前一个节点 e。参数 e 用来保证搜索只移动到尚未访问的节点。

下面的函数调用从节点 x 开始搜索：

```
dfs(x, 0);
```

第一次调用时 $e = 0$，因为没有前一个节点，可以沿树中的任意方向前进。

动态规划

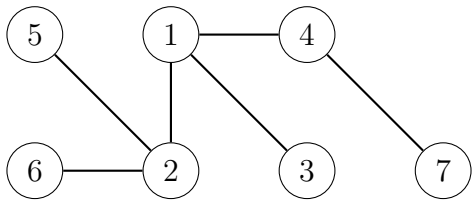
可以在树的遍历过程中使用动态规划计算某些信息。例如，利用动态规划，可以在 $O(n)$ 时间内为有根树中的每个节点计算其子树的节点数，或者从该节点到叶节点的最长路径长度。

例如，为每个节点 s 计算值 count[s]，即其子树中的节点数。子树包含节点本身以及其各个子节点的子树中的所有节点，因此可以用以下代码递归计算节点数：

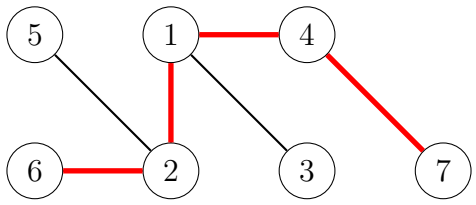
```
void dfs(int s, int e) {
    count[s] = 1;
    for (auto u : adj[s]) {
        if (u == e) continue;
        dfs(u, s);
        count[s] += count[u];
    }
}
```

14.2 直径

树的**直径**是任意两个节点之间路径长度的最大值。例如，考虑下图：



这棵树的直径为 4，对应下列路径：



最长路径可能不止一条。在上面的路径中，可以把节点 6 换成节点 5，得到另一条长度为 4 的路径。

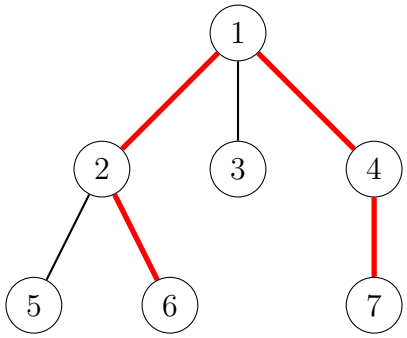
下面介绍两种在 $O(n)$ 时间内计算树直径的算法。第一种基于动态规划，第二种使用两次深度优先搜索。

算法 1

处理许多树上问题的一般方法，是先任意选根，再尝试分别解决每棵子树上的问题。第一种直径算法正是基于这一思想。

一个重要观察是，有根树中的每条路径都有一个最高点，即路径上位置最高的节点。因此，可以为每个节点计算以它为最高点的最长路径。这样的路径中必有一条对应树的直径。

例如，在下图中，节点 1 是直径所对应路径的最高点：



对每个节点 x 计算两个值：

- toLeaf(x)：从 x 到任意叶节点的路径的最大长度；
- maxLength(x)：以 x 为最高点的路径的最大长度。

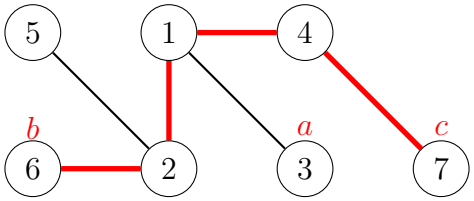
例如，在上图中，toLeaf(1) = 2，因为存在路径 $1 \to 2 \to 6$；maxLength(1) = 4，因为存在路径 $6 \to 2 \to 1 \to 4 \to 7$。在这里，maxLength(1) 就等于直径。

利用动态规划，可以在 $O(n)$ 时间内计算所有节点的上述值。计算 toLeaf(x) 时，遍历 x 的子节点，选择 toLeaf(c) 最大的子节点 c，并在该值上加一。计算 maxLength(x) 时，选择两个不同的子节点 a 和 b，使 toLeaf(a) + toLeaf(b) 最大，再给这个和加二。

算法 2

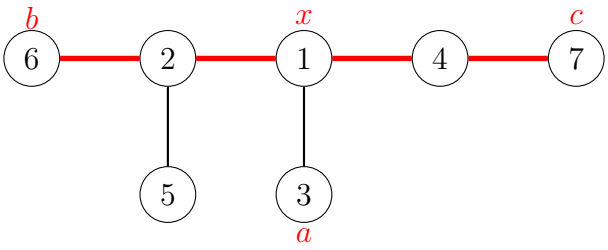
另一种计算树直径的高效方法基于两次深度优先搜索。首先在树中任取节点 a，找到离 a 最远的节点 b；然后找到离 b 最远的节点 c。树的直径就是 b 与 c 之间的距离。

在下图中，a、b 和 c 可以取为：



这个方法很巧妙，但它为什么正确？

把树换一种画法会更容易理解：让直径对应的路径水平放置，其他所有节点都悬挂在这条路径上：

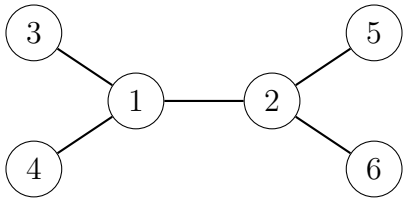


节点 x 表示从节点 a 出发的路径与直径路径汇合的位置。离 a 最远的节点要么是 b，要么是 c，要么是某个与节点 x 的距离至少同样远的其他节点。因此，这个节点总能作为某条直径路径的一个端点。

14.3 所有最长路径

下一个问题是：对树中的每个节点，计算从该节点出发的路径的最大长度。它可以看作树直径问题的推广，因为这些长度中的最大值就是树的直径。这个问题同样可以在 $O(n)$ 时间内解决。

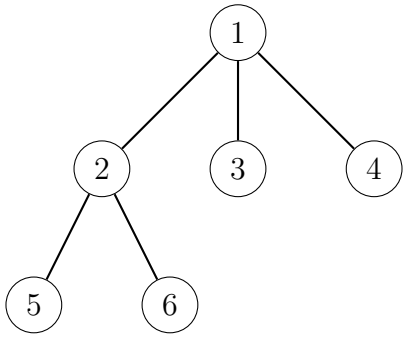
例如，考虑下图：



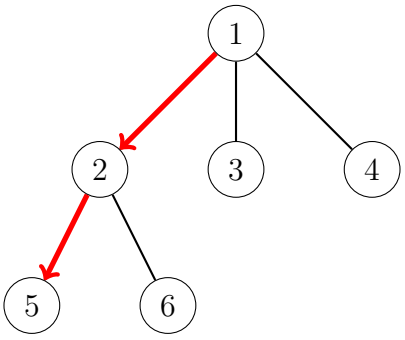
令 maxLength(x) 表示从节点 x 出发的路径的最大长度。例如，在上图中，maxLength(4) = 3，因为存在路径 $4 \to 1 \to 2 \to 6$。所有值如下表所示：

节点 x	1	2	3	4	5	6
maxLength(x)	2	2	3	3	3	3

同样，解决这个问题的良好起点是任意选定树根：

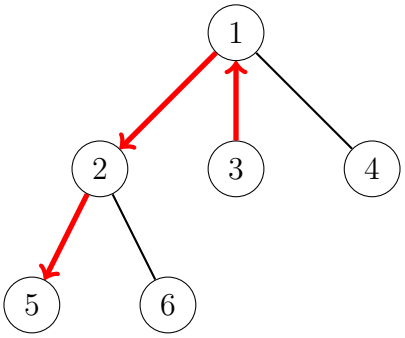


问题的第一部分是：对每个节点 x，计算经过 x 的某个子节点的路径的最大长度。例如，从节点 1 出发的最长路径经过其子节点 2：

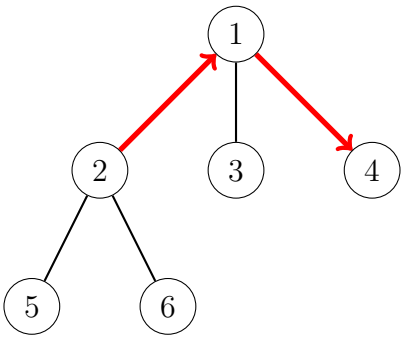


这一部分很容易在 $O(n)$ 时间内解决，因为可以像前面一样使用动态规划。

问题的第二部分是：对每个节点 x，计算经过其父节点 p 的路径的最大长度。例如，从节点 3 出发的最长路径经过其父节点 1：



乍看之下，似乎应选择从 p 出发的最长路径。然而，这种做法并不总是正确，因为从 p 出发的最长路径可能经过 x。下面就是这种情形：



不过，只要为每个节点 x 保存两个最大长度，仍可在 $O(n)$ 时间内解决第二部分：

- $\text{maxLength}_1(x)$：从 x 出发的路径的最大长度；
- $\text{maxLength}_2(x)$：从 x 出发、且方向不同于第一条路径的路径的最大长度。

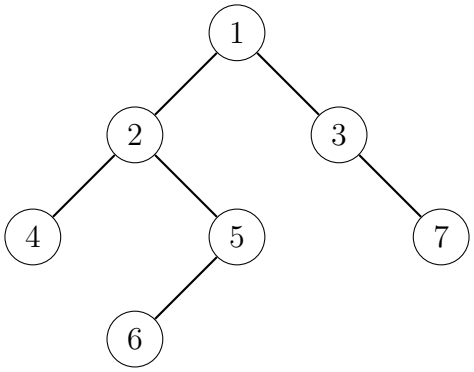
例如，在上图中，沿路径 $1 \to 2 \to 5$ 可得 $\text{maxLength}_1(1) = 2$，沿路径 $1 \to 3$ 可得 $\text{maxLength}_2(1) = 1$。

最后，如果 $\text{maxLength}_1(p)$ 对应的路径经过 x，则最大长度为 $\text{maxLength}_2(p)+1$；否则为 $\text{maxLength}_1(p) + 1$。

14.4 二叉树

二**叉树**是一种有根树，其中每个节点都有左子树和右子树。节点的某棵子树可以为空，因此二叉树中的每个节点有零个、一个或两个子节点。

例如，下图是一棵二叉树：



二叉树的节点有三种自然顺序，分别对应递归遍历树的不同方式：

- **前序遍历**：先处理根，再遍历左子树，最后遍历右子树；
- **中序遍历**：先遍历左子树，再处理根，最后遍历右子树；
- **后序遍历**：先遍历左子树，再遍历右子树，最后处理根。

对上图中的树，前序遍历为 $[1,2,4,5,6,3,7]$，中序遍历为 $[4,2,6,5,1,3,7]$，后序遍历为 $[4,6,5,2,7,3,1]$。

若知道一棵树的前序和中序遍历，就能重建其准确结构。例如，上图是前序遍历为 $[1,2,4,5,6,3,7]$、中序遍历为 $[4,2,6,5,1,3,7]$ 的唯一一棵树。类似地，后序与中序遍历也能确定树的结构。

然而，如果只知道前序和后序遍历，情况就不同了：可能有多棵树符合这两个顺序。例如，在下面两棵树中，

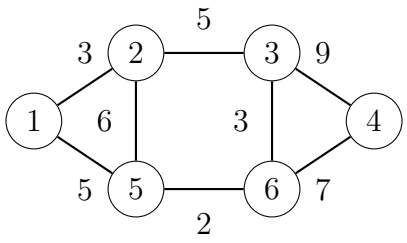


前序遍历都是 $[1,2]$，后序遍历都是 $[2,1]$，但树的结构不同。

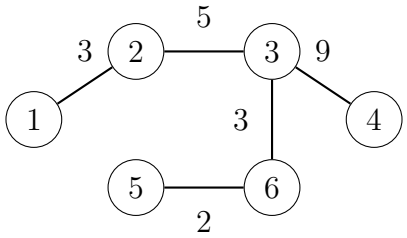
第十五章　生成树

图的一棵**生成树**包含图中的所有节点和部分边，并且任意两个节点之间都存在路径。和一般的树一样，生成树是连通且无环的。通常，一个图可以用多种方式构造生成树。

例如，考虑下图：

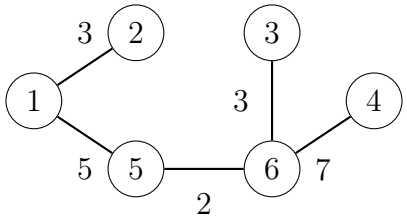


它的一棵生成树如下：

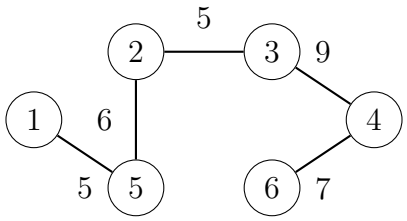


生成树的权值是其所有边权之和。例如，上述生成树的权值为 $3+5+9+3+2=22$。

最小生成树是权值尽可能小的生成树。示例图的最小生成树权值为 20，可以构造如下：



类似地，**最大生成树**是权值尽可能大的生成树。示例图的最大生成树权值为 32：



一个图可能有多棵最小或最大生成树，因此这些树不一定唯一。

事实证明，可以使用多种贪心方法构造最小和最大生成树。本章讨论两种按边权顺序处理图中各边的算法。我们重点寻找最小生成树；若按相反顺序处理边，同样的算法也能找到最大生成树。

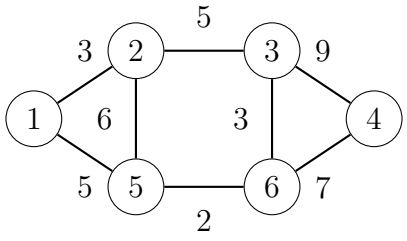
15.1 Kruskal 算法

Kruskal 算法[1]开始时的生成树只包含图的节点，不含任何边。随后，算法按边权顺序遍历所有边；如果加入一条边不会形成环，就把它加入树中。

算法维护树的各个连通分量。初始时，每个节点分别属于一个分量。每向树中加入一条边，就合并两个分量。最终所有节点都属于同一分量，也就找到了最小生成树。

示例

考察 Kruskal 算法如何处理下图：



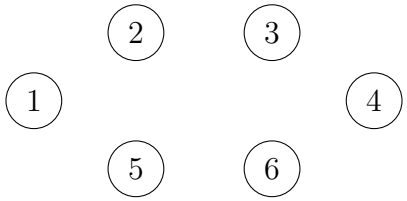
算法第一步按权值递增排序所有边，得到下表：

[1]J. B. Kruskal 于 1956 年发表了这一算法 [48]。

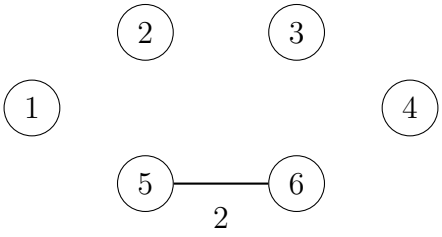
边	权值
5–6	2
1–2	3
3–6	3
1–5	5
2–3	5
2–5	6
4–6	7
3–4	9

随后，算法遍历该列表；若一条边连接两个不同分量，就将它加入树中。

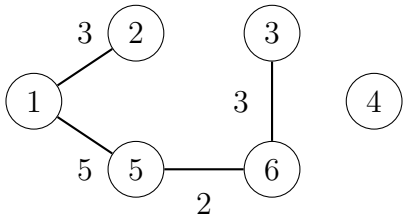
初始时，每个节点各自构成一个分量：



首先加入边 5–6，它合并分量 $\{5\}$ 和 $\{6\}$，形成分量 $\{5,6\}$：



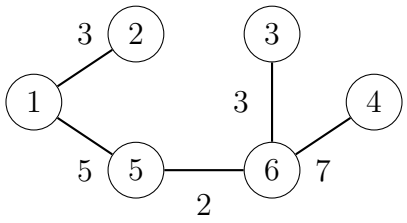
随后，以同样方式加入边 1–2、3–6 和 1–5：



此时大多数分量已经合并，树中只剩两个分量：$\{1,2,3,5,6\}$ 和 $\{4\}$。

列表中的下一条边是 2–3，但节点 2 和 3 已在同一分量中，因此不会加入树。出于同样原因，边 2–5 也不会加入。

最后，将边 4–6 加入树中：

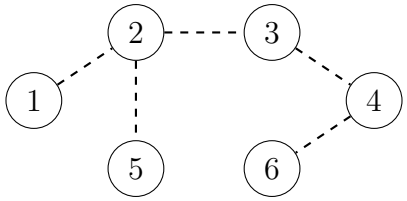


此后图已连通，任意两节点之间都有路径，算法不会再加入新边。所得图是一棵权值为 $2+3+3+5+7=20$ 的最小生成树。

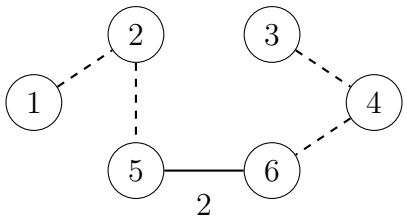
为什么正确？

Kruskal 算法为什么正确，是一个很好的问题。为什么这种贪心策略能够保证找到最小生成树?

考察图中权值最小的边没有包含在生成树中的情况。例如，假设前一个图的某棵生成树不包含权值最小的边 5–6。虽然不知道这棵生成树的确切结构，但它总要包含一些边。假设它如下所示：



然而，上面的树不可能是该图的最小生成树。原因是，可以从树中删除一条边，并用权值最小的边 5–6 替换它，从而得到一棵权值更小的生成树：



因此，为构造最小生成树，把权值最小的边加入树中总是最优的。使用类似论证，可以证明按权值顺序加入下一条边也是最优的，依此类推。于是 Kruskal 算法是正确的，并且总能生成一棵最小生成树。

实现

实现 Kruskal 算法时，使用图的边列表表示十分方便。算法第一阶段在 $O(m \log m)$ 时间内对列表中的边排序。随后，第二阶段按如下方式构造最小生成树：

```
for (...) {
  if (!same(a,b)) unite(a,b);
}
```

循环遍历边列表，每次处理连接两个节点 a、b 的一条边。需要两个函数：same 判断 a 与 b 是否位于同一分量，unite 合并分别包含 a 与 b 的两个分量。

问题在于如何高效实现 same 和 unite。一种方案是通过图遍历实现 same，检查是否能从节点 a 到达节点 b。但这样的函数时间复杂度为 $O(n+m)$；由于图中每条边都会调用 same，最终算法会很慢。

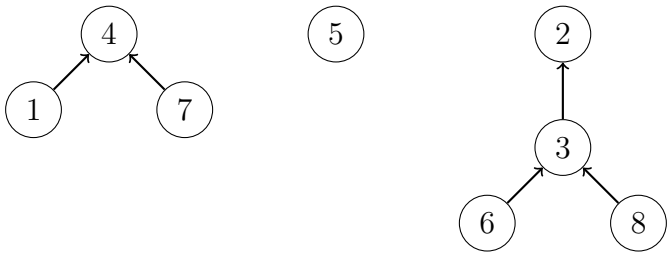
我们将使用并查集解决这一问题，它能在 $O(\log n)$ 时间内实现两个函数。因此，边列表排序之后，Kruskal 算法的时间复杂度为 $O(m \log n)$。

15.2 并查集

并查集维护一组集合。这些集合两两不交，因此没有元素同时属于多个集合。它支持两种 $O(\log n)$ 操作：unite 操作合并两个集合，find 操作寻找包含给定元素的集合代表元[2]。

结构

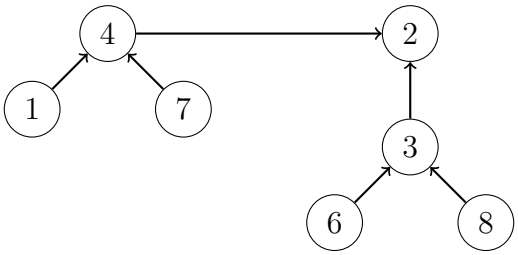
在并查集中，每个集合选取一个元素作为代表元，集合内任意其他元素都有一条链通向代表元。例如，假设集合为 $\{1,4,7\}$、$\{5\}$ 和 $\{2,3,6,8\}$：



[2]这里介绍的数据结构由 J. D. Hopcroft 和 J. D. Ullman 于 1971 年提出 [38]。1975 年，R. E. Tarjan 研究了一种更复杂的变体 [64]，如今许多算法教材都会介绍它。

这些集合的代表元分别是 4、5 和 2。沿着从某元素开始的链即可找到其代表元。例如，元素 6 的代表元是 2，因为沿链 $6 \to 3 \to 2$ 可到达它。两个元素属于同一集合，当且仅当它们的代表元相同。

把一个集合的代表元连接到另一个集合的代表元，就能合并两个集合。例如，集合 $\{1,4,7\}$ 和 $\{2,3,6,8\}$ 可以按如下方式合并：



所得集合包含元素 $\{1,2,3,4,6,7,8\}$。从此以后，元素 2 是整个集合的代表元，原代表元 4 指向元素 2。

并查集的效率取决于集合的合并方式。可以采用一个简单策略：始终把较小集合的代表元连接到较大集合的代表元（若大小相等，可以任意选择）。使用该策略，任意链的长度都是 $O(\log n)$，所以能够沿相应的链高效找到任意元素的代表元。

实现

并查集可以用数组实现。下面的实现中，数组 link 对每个元素保存链中的下一个元素；若该元素是代表元，则保存它自身。数组 size 对每个代表元保存相应集合的大小。

初始时，每个元素分别属于一个集合：

```
for (int i = 1; i <= n; i++) link[i] = i;
for (int i = 1; i <= n; i++) size[i] = 1;
```

函数 find 返回元素 x 的代表元。沿着从 x 开始的链即可找到代表元。

```
int find(int x) {
    while (x != link[x]) x = link[x];
    return x;
}
```

函数 same 检查元素 a 和 b 是否属于同一集合。使用 find 即可轻松完成：

```
bool same(int a, int b) {
    return find(a) == find(b);
```

```
}
```

函数 unite 合并包含元素 a 和 b 的集合（这两个元素必须属于不同集合）。函数先找到两个集合的代表元，再把较小集合连接到较大集合。

```
void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (size[a] < size[b]) swap(a,b);
    size[a] += size[b];
    link[b] = a;
}
```

假设每条链的长度都是 $O(\log n)$，函数 find 的时间复杂度就是 $O(\log n)$，函数 same 和 unite 也都在 $O(\log n)$ 时间内运行。unite 始终把较小集合连接到较大集合，从而保证每条链的长度为 $O(\log n)$。

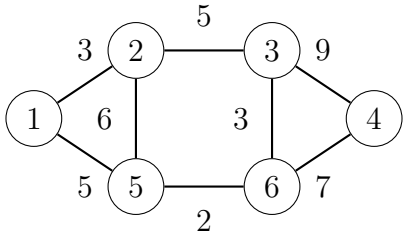
15.3 Prim 算法

Prim 算法[3]是寻找最小生成树的另一种方法。算法首先向树中加入任意节点。随后，每次选择一条权值最小、且能向树中加入一个新节点的边。最终所有节点都被加入树中，也就找到了一棵最小生成树。

Prim 算法与 Dijkstra 算法相似。区别是：Dijkstra 算法总是选择一条使节点到起点距离最小的边，而 Prim 算法只选择能向树中加入一个新节点的最小权边。

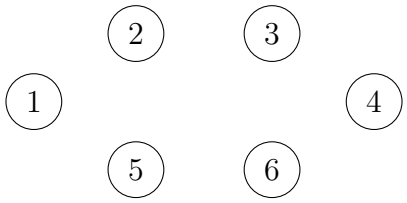
示例

考察 Prim 算法如何处理下图：

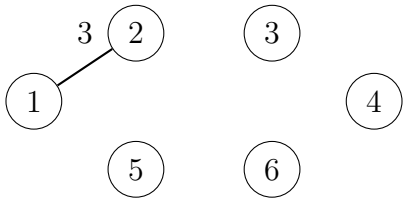


[3]该算法以 R. C. Prim 命名，他于 1957 年发表了这一算法 [54]。不过，V. Jarník 早在 1930 年就发现了同一算法。

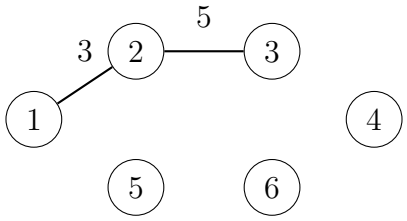
初始时，节点之间没有任何边：



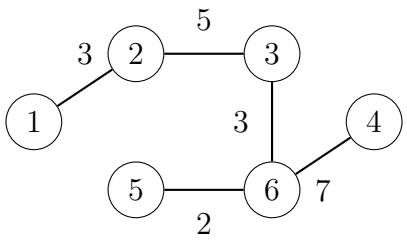
起始节点可以任意选择，这里选择节点 1。首先，加入由权值为 3 的边连接的节点 2：



此时有两条权值为 5 的边，因此可以把节点 3 或节点 5 加入树中。这里先加入节点 3：



不断重复这一过程，直到所有节点都加入树中：



实现

与 Dijkstra 算法一样，Prim 算法可以用优先队列高效实现。优先队列应包含所有能通过一条边连接到当前分量的节点，并按照相应边的权值递增排列。

Prim 算法的时间复杂度是 $O(n+m\log m)$，与 Dijkstra 算法相同。实际中，Prim 和 Kruskal 算法都很高效，选择哪一个主要取决于个人偏好。不过，大多数竞赛程序员使用 Kruskal 算法。

第十六章　有向图

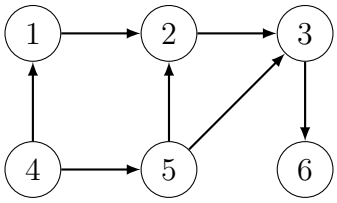
本章重点讨论两类有向图：

- **有向无环图**：图中没有环，因此不存在从任何节点回到自身的路径[1]。
- **后继图**：每个节点的出度都是 1，因此每个节点都有唯一的后继。

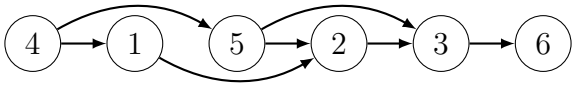
事实证明，在这两种情况下，都可以利用图的特殊性质设计高效算法。

16.1　拓扑排序

拓扑序是有向图节点的一种排列，满足：若存在从节点 a 到节点 b 的路径，则 a 在该排列中出现在 b 之前。例如，对于图



一种拓扑序是 $[4, 1, 5, 2, 3, 6]$：



有向无环图总是存在拓扑序。若图中含环，则无法构造拓扑序，因为环中的任何节点都不可能排在环内所有其他节点之前。利用深度优先搜索，既能检查有向图是否含环，也能在无环时构造拓扑序。

算法

算法依次考察图中所有节点；若当前节点尚未处理，就从它开始深度优先搜索。搜索过程中，节点有三种可能状态：

[1]有向无环图常简称为 DAG。

- 状态 0：节点尚未处理（白色）；
- 状态 1：节点正在处理（浅灰色）；
- 状态 2：节点已经处理（深灰色）。

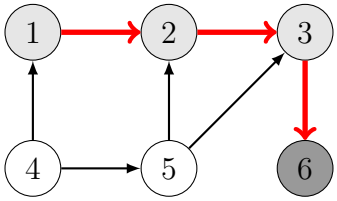
初始时，每个节点的状态都是 0。搜索第一次到达某节点时，其状态变为 1。该节点的所有后继都处理完毕后，其状态变为 2。

若图中含环，搜索迟早会到达一个状态为 1 的节点，从而发现这个环。此时无法构造拓扑序。

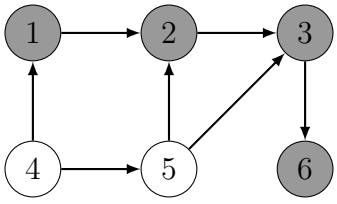
若图中不含环，可以在每个节点的状态变为 2 时把它加入列表。该列表的逆序就是一个拓扑序。

示例 1

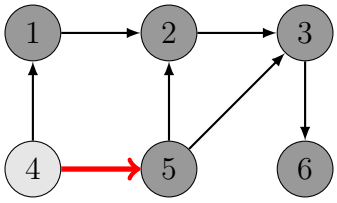
在示例图中，搜索首先从节点 1 前进到节点 6：



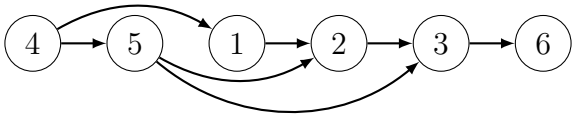
此时节点 6 已处理完毕，因此加入列表。随后，节点 3、2 和 1 也依次加入列表：



此时列表为 $[6, 3, 2, 1]$。下一次搜索从节点 4 开始：



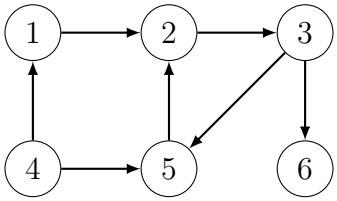
最终列表为 $[6, 3, 2, 1, 5, 4]$。所有节点均已处理，因此找到一个拓扑序，即该列表的逆序 $[4, 5, 1, 2, 3, 6]$：



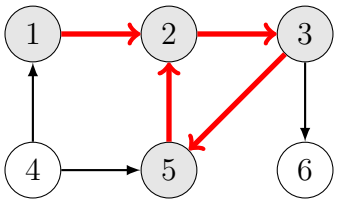
注意，拓扑序不一定唯一，一个图可能存在多个拓扑序。

示例 2

现在考察一个无法构造拓扑序的图，因为其中含环：



搜索过程如下：



搜索到达了状态为 1 的节点 2，说明图中含环。本例中的环为 $2 \to 3 \to 5 \to 2$。

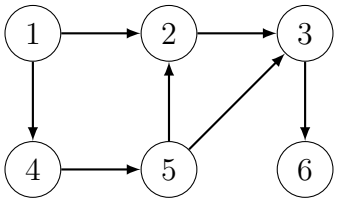
16.2 动态规划

如果一个有向图无环，就可以在图上应用动态规划。例如，对于从起始节点到终止节点的路径，可以高效解决以下问题：

- 有多少条不同路径？
- 最短或最长路径是什么？
- 一条路径中边数的最小值或最大值是多少？
- 哪些节点一定出现在所有路径中？

计算路径数量

例如，计算下图中从节点 1 到节点 6 的路径数量：



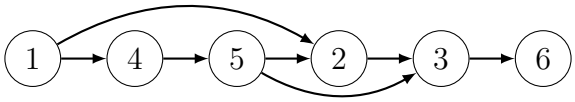
这样的路径共有三条：

- $1 \to 2 \to 3 \to 6$
- $1 \to 4 \to 5 \to 2 \to 3 \to 6$
- $1 \to 4 \to 5 \to 3 \to 6$

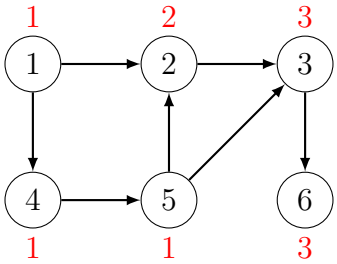
令 $\text{paths}(x)$ 表示从节点 1 到节点 x 的路径数。边界条件为 $\text{paths}(1) = 1$。其他 $\text{paths}(x)$ 值可由递推式

$$\text{paths}(x) = \text{paths}(a_1) + \text{paths}(a_2) + \cdots + \text{paths}(a_k)$$

计算，其中 $a_1, a_2, \ldots, a_k$ 是所有有边指向 x 的节点。由于图无环，可以按照拓扑序计算 $\text{paths}(x)$。上图的一种拓扑序如下：



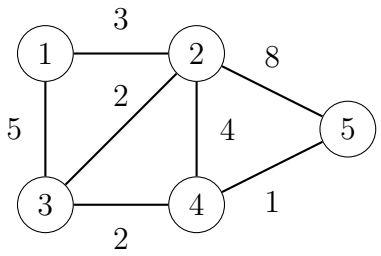
因此，各节点的路径数如下：



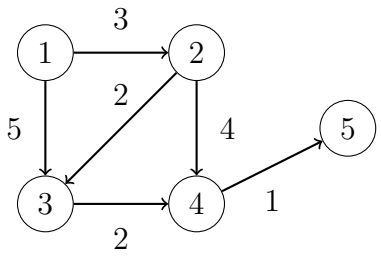
例如，计算 $\text{paths}(3)$ 时，可以使用公式 $\text{paths}(2) + \text{paths}(5)$，因为节点 2 和 5 都有边指向节点 3。由于 $\text{paths}(2) = 2$ 且 $\text{paths}(5) = 1$，可得 $\text{paths}(3) = 3$。

扩展 Dijkstra 算法

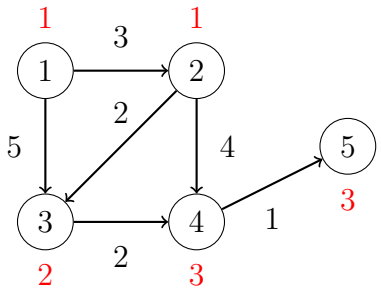
Dijkstra 算法的一个副产品是有向无环图：它对原图中每个节点表示从起始节点沿最短路径到达该节点的所有可能方式。可以在这个图上应用动态规划。例如，在图



中，从节点 1 出发的最短路径可能使用以下边：



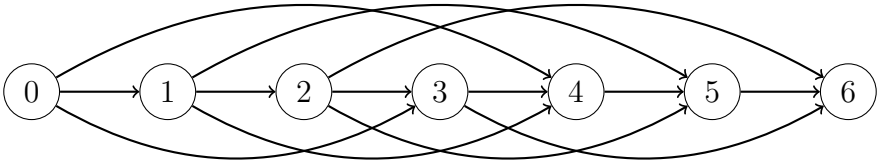
现在可以用动态规划计算从节点 1 到节点 5 的最短路径数量：



把问题表示成图

实际上，任何动态规划问题都能表示成有向无环图。在这种图中，每个节点对应一个动态规划状态，边表示状态之间的依赖关系。

例如，考虑使用硬币 $\{c_1, c_2, \ldots, c_k\}$ 凑出金额 n 的问题。可以构造一个图，其中每个节点对应一个金额，边表示可以如何选择硬币。对于硬币 $\{1, 3, 4\}$ 和 $n = 6$，图如下：



在这种表示中，从节点 0 到节点 n 的最短路径对应使用硬币数最少的解，而从节点 0 到节点 n 的路径总数就等于解的总数。

16.3 后继路径

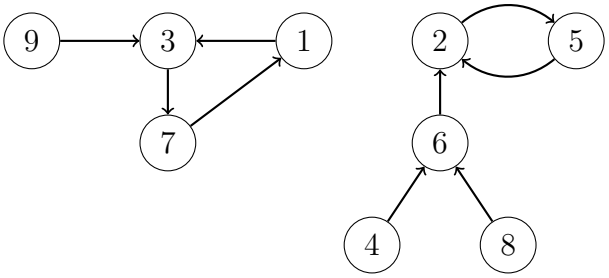
本章余下部分重点讨论**后继图**。在这种图中，每个节点的出度都是 1，即恰有一条边从每个节点出发。后继图由一个或多个分量构成，每个分量都包含一个环，以及若干通向该环的路径。

后继图有时也称为**函数图**，因为任何后继图都对应一个定义图中各边的函数。函数的参数是图中的节点，函数值是该节点的后继。

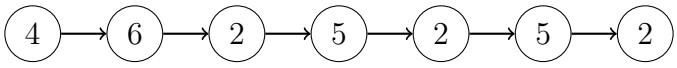
例如，函数

x	1	2	3	4	5	6	7	8	9
$\mathrm{succ}(x)$	3	5	7	6	2	2	1	6	3

定义了下图：



由于后继图中的每个节点都有唯一后继，还可以定义函数 $\mathrm{succ}(x,k)$，表示从节点 x 出发向前走 k 步所到达的节点。例如，在上图中 $\mathrm{succ}(4,6)=2$，因为从节点 4 走 6 步后到达节点 2：



直接计算 $\mathrm{succ}(x,k)$ 的方法是从 x 开始走 k 步，耗时 $O(k)$。不过，经过预处理后，任意 $\mathrm{succ}(x,k)$ 都能在 $O(\log k)$ 时间内求出。

基本思想是预先计算所有 $\mathrm{succ}(x,k)$，其中 k 是不超过 u 的 2 的幂，u 为可能行走的最大步数。利用以下递推式可以高效完成预处理：

$$\mathrm{succ}(x,k) = \begin{cases} \mathrm{succ}(x) & k = 1 \\ \mathrm{succ}(\mathrm{succ}(x,k/2),k/2) & k > 1 \end{cases}$$

每个节点要计算 $O(\log u)$ 个值，因此预处理耗时 $O(n \log u)$。在上图中，前几个值如下：

x	1	2	3	4	5	6	7	8	9
$\mathrm{succ}(x,1)$	3	5	7	6	2	2	1	6	3
$\mathrm{succ}(x,2)$	7	2	1	2	5	5	3	2	7
$\mathrm{succ}(x,4)$	3	2	7	2	5	5	1	2	3
$\mathrm{succ}(x,8)$	7	2	1	2	5	5	3	2	7
$\cdots$									

此后，把步数 k 表示成若干个 2 的幂之和，就能计算任意 $\mathrm{succ}(x,k)$。例如，要计算 $\mathrm{succ}(x,11)$，先写出 $11 = 8+2+1$，于是

$$\mathrm{succ}(x,11) = \mathrm{succ}(\mathrm{succ}(\mathrm{succ}(x,8),2),1).$$

在前述图中，

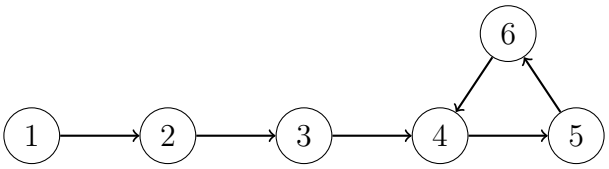
$$\mathrm{succ}(4,11) = \mathrm{succ}(\mathrm{succ}(\mathrm{succ}(4,8),2),1) = 5.$$

这种表示始终只有 $O(\log k)$ 个部分，因此计算 $\mathrm{succ}(x,k)$ 需要 $O(\log k)$ 时间。

16.4 环检测

考虑一个只包含一条最终进入环的路径的后继图。若从路径的起始节点开始行走，可以问：环中的第一个节点是什么？环包含多少个节点？

例如，在图



中，我们从节点 1 开始行走。环中的第一个节点是节点 4，环包含三个节点（4、5 和 6）。

一种简单的检测方法是沿图行走并记录所有访问过的节点。一旦某节点被第二次访问，就可以断定它是环中的第一个节点。该方法耗时 $O(n)$，并使用 $O(n)$ 内存。

不过，还有更好的环检测算法。它们的时间复杂度仍为 $O(n)$，但只使用 $O(1)$ 内存。当 n 很大时，这是重要改进。下面介绍具备这些性质的 Floyd 算法。

Floyd 算法

Floyd 算法[2]使用两个指针 a 和 b 在图中向前移动。两个指针都从图的起始节点 x 开始。每轮中，a 向前走一步，b 向前走两步，直到二者相遇：

```
a = succ(x);
b = succ(succ(x));
while (a != b) {
    a = succ(a);
    b = succ(succ(b));
}
```

此时，指针 a 走了 k 步，指针 b 走了 $2k$ 步，所以环长整除 k。把指针 a 移回节点 x，然后让两个指针每次各前进一步，直到再次相遇，即可找到环中的第一个节点：

```
a = x;
while (a != b) {
    a = succ(a);
    b = succ(b);
}
first = a;
```

随后可以如下计算环长：

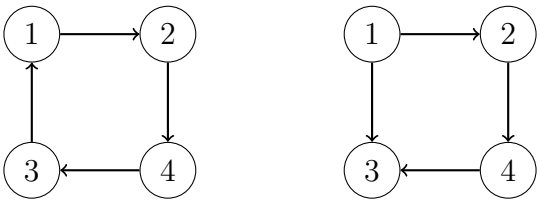
```
b = succ(a);
length = 1;
while (a != b) {
    b = succ(b);
    length++;
}
```

[2]该算法思想见于 [46]，并归功于 R. W. Floyd；但 Floyd 是否确实发现了这一算法并不确定。

第十七章　强连通性

在有向图中，每条边只能沿一个方向通过，因此，即使图是连通的，也不能保证从一个节点到另一个节点一定存在路径。为此，有必要定义一个比连通性要求更强的新概念。

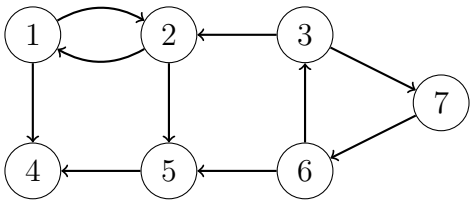
如果从图中任意节点出发都能到达所有其他节点，则称该图是**强连通的**。例如，下图左侧的图是强连通图，而右侧的图不是。



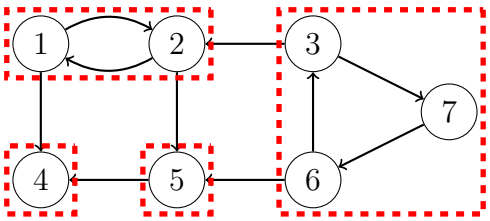
右图不是强连通图，因为例如不存在从节点 2 到节点 1 的路径。

图的**强连通分量**将图划分为尽可能大的强连通部分。所有强连通分量形成一个无环的**分量图**，它反映了原图的深层结构。

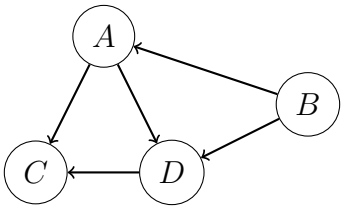
例如，对于图



其强连通分量如下：



对应的分量图如下：



各分量为 $A = \{1, 2\}$、$B = \{3, 6, 7\}$、$C = \{4\}$ 和 $D = \{5\}$。

分量图是无环有向图，因此比原图更容易处理。由于图中不含环，我们总能构造拓扑序，并使用第 16 章介绍的动态规划方法。

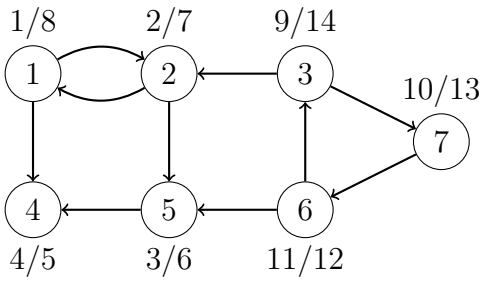
17.1 Kosaraju 算法

Kosaraju 算法[1]是一种寻找有向图强连通分量的高效方法。算法执行两次深度优先搜索：第一次搜索根据图的结构构造节点列表，第二次搜索形成强连通分量。

搜索 1

Kosaraju 算法的第一阶段按照深度优先搜索处理节点的顺序构造一个列表。算法依次考察所有节点，并从每个尚未处理的节点开始一次深度优先搜索。每个节点在处理完毕后加入列表。

在示例图中，节点的处理顺序如下：



记号 x/y 表示节点在时刻 x 开始处理，在时刻 y 处理完毕。因此，对应的列表如下：

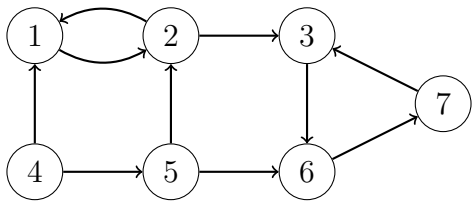
[1]据 [1] 记载，S. R. Kosaraju 于 1978 年发明了这一算法，但没有发表。1981 年，M. Sharir 再次发现并发表了同一算法 [57]。

节点	完成时刻
4	5
5	6
2	7
1	8
6	12
7	13
3	14

搜索 2

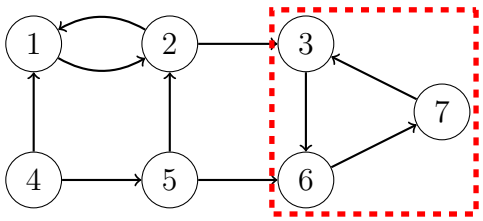
算法的第二阶段形成图的强连通分量。首先，算法把图中的每条边反向。这保证在第二次搜索中，我们找到的强连通分量不会混入额外节点。

所有边反向后，示例图如下：



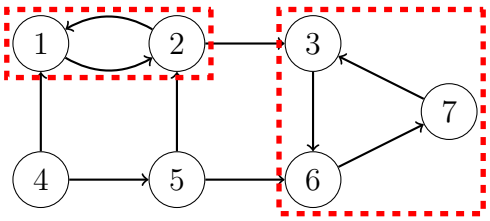
随后，算法按照第一次搜索所得节点列表的*逆序*处理节点。若某节点尚不属于任何分量，算法就创建一个新分量，并从该节点开始深度优先搜索，把搜索发现的所有新节点加入这个分量。

在示例图中，第一个分量从节点 3 开始：

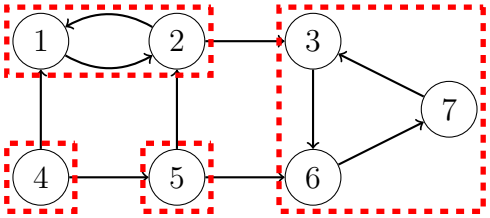


注意，由于所有边都已反向，这个分量不会“泄漏”到图的其他部分。

列表中的下两个节点是 7 和 6，但它们已经属于某个分量，因此下一个新分量从节点 1 开始：



最后，算法处理节点 5 和 4，建立余下的强连通分量：



算法执行两次深度优先搜索，因此时间复杂度为 $O(n+m)$。

17.2 2SAT 问题

强连通性还与 **2SAT 问题**[2]有关。在这个问题中，给定逻辑公式

$$(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \cdots \wedge (a_m \vee b_m),$$

其中每个 a_i 和 b_i 要么是逻辑变量 $(x_1, x_2, \ldots, x_n)$，要么是逻辑变量的否定 $(\neg x_1, \neg x_2, \ldots, \neg x_n)$。符号“$\wedge$”和“$\vee$”分别表示逻辑运算“与”和“或”。我们的任务是为每个变量赋值，使公式为真；或者说明无法做到。

例如，公式

$$L_1 = (x_2 \vee \neg x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_4)$$

在变量取值如下时为真：

$$\begin{cases} x_1 = \text{false} \\ x_2 = \text{false} \\ x_3 = \text{true} \\ x_4 = \text{true} \end{cases}$$

[2]这里介绍的算法最初见于 [4]。另有一种著名的线性时间算法 [19]，它基于回溯。

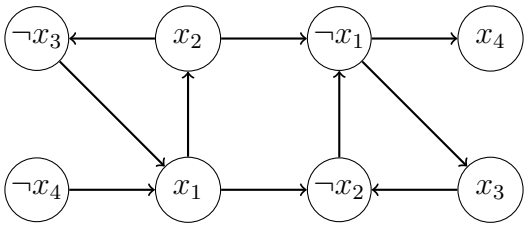
然而，无论如何赋值，公式

$$L_2 = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_1 \vee \neg x_3)$$

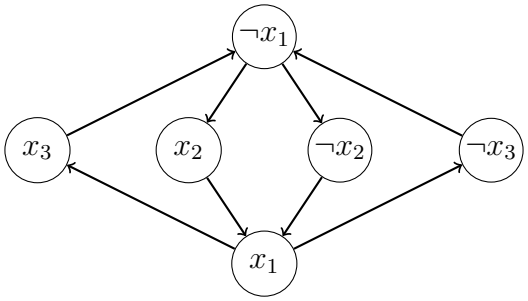
都为假。这是因为给 x_1 选择任何值都会产生矛盾。若 x_1 为假，则 x_2 和 $\neg x_2$ 必须同时为真，这是不可能的；若 x_1 为真，则 x_3 和 $\neg x_3$ 必须同时为真，同样不可能。

2SAT 问题可以表示为一个图，其中节点对应变量 x_i 及其否定 $\neg x_i$，边表示变量之间的蕴含关系。每个子句 $(a_i \vee b_i)$ 生成两条边：$\neg a_i \to b_i$ 和 $\neg b_i \to a_i$。这表示若 a_i 不成立，则 b_i 必须成立，反之亦然。

公式 L_1 对应的图为：



公式 L_2 对应的图为：

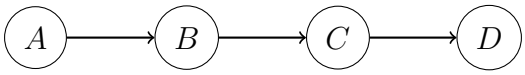


图的结构告诉我们是否能够为变量赋值，使公式为真。事实证明，当且仅当不存在某个 i，使节点 x_i 与 $\neg x_i$ 同属一个强连通分量时，才存在这样的赋值。如果存在这类节点，图中既有从 x_i 到 $\neg x_i$ 的路径，也有从 $\neg x_i$ 到 x_i 的路径，因此 x_i 和 $\neg x_i$ 都必须为真，这是不可能的。

公式 L_1 的图中不存在同属一个强连通分量的 x_i 和 $\neg x_i$，因此存在解。公式 L_2 的图中所有节点都属于同一个强连通分量，因此无解。

若解存在，可以按照逆拓扑序遍历分量图中的节点来确定变量值。每一步处理一个不存在指向未处理分量的边的分量。如果分量中的变量尚未赋值，就根据该分量内的文字确定它们的值；若已经赋值，则保持不变。重复此过程，直到每个变量都得到一个值。

公式 L_1 的分量图如下：



各分量为 $A = \{\neg x_4\}$、$B = \{x_1, x_2, \neg x_3\}$、$C = \{\neg x_1, \neg x_2, x_3\}$ 和 $D = \{x_4\}$。构造解时，先处理分量 D，令 x_4 为真；随后处理分量 C，令 x_1 和 x_2 为假，x_3 为真。所有变量都已赋值，因此余下的分量 A 和 B 不再改变变量值。

这种方法能够成立，是因为该图具有特殊结构：若存在从节点 x_i 到节点 x_j 的路径，以及从节点 x_j 到节点 $\neg x_j$ 的路径，那么节点 x_i 永远不会取真。原因是，此时也存在从节点 $\neg x_j$ 到节点 $\neg x_i$ 的路径，于是 x_i 和 x_j 都取假。

更困难的是 **3SAT 问题**，其中公式的每个部分都形如 $(a_i \vee b_i \vee c_i)$。该问题是 NP 困难的，目前没有已知的高效算法。

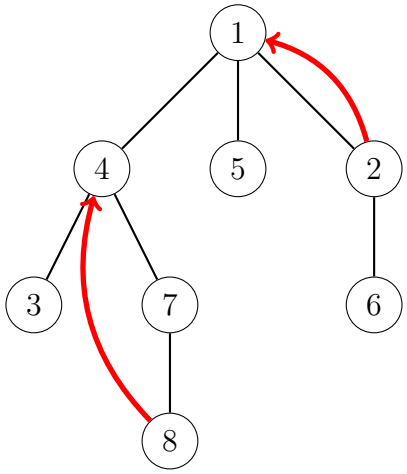
第十八章　树上查询

本章讨论处理有根树中子树与路径查询的技术。例如，这类查询包括：

- 一个结点的第 k 个祖先是谁？
- 一个结点的子树中所有结点的值之和是多少？
- 两个结点之间路径上所有结点的值之和是多少？
- 两个结点的最近公共祖先是谁？

18.1　寻找祖先

在有根树中，从结点 x 向上移动 k 层所到达的结点，称为 x 的第 k 个**祖先**。用 $\text{ancestor}(x,k)$ 表示结点 x 的第 k 个祖先（如果不存在这样的祖先，则值为 0）。例如，在下图中，$\text{ancestor}(2,1)=1$，且 $\text{ancestor}(8,2)=4$。



计算 $\text{ancestor}(x,k)$ 的一种简单方法是在树中连续向上移动 k 次。然而，这种方法的时间复杂度为 $O(k)$，可能很慢，因为一棵有 n 个结点的树可能包含一条由 n 个结点组成的链。

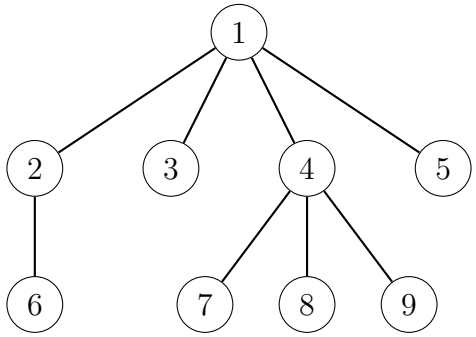
幸运的是，使用与第 16.3 节类似的技术，在预处理之后，可以在 $O(\log k)$ 时间内高效计算任意 ancestor(x,k)。其思想是预先计算所有满足 $k \le n$ 且 k 为 2 的幂的 ancestor(x,k)。例如，上图对应的值如下：

x	1	2	3	4	5	6	7	8
ancestor$(x,1)$	0	1	4	1	1	2	4	7
ancestor$(x,2)$	0	0	1	0	0	1	1	4
ancestor$(x,4)$	0	0	0	0	0	0	0	0
$\cdots$								

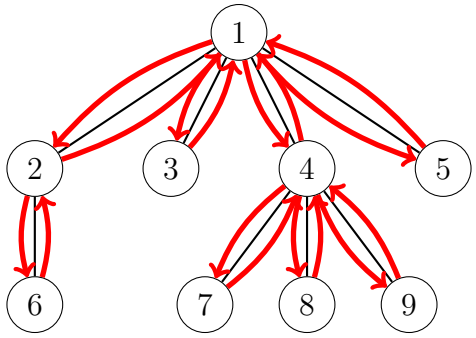
预处理需要 $O(n \log n)$ 时间，因为每个结点都要计算 $O(\log n)$ 个值。之后，把 k 表示成若干个 2 的幂之和，就能在 $O(\log k)$ 时间内计算任意 ancestor(x,k)。

18.2 子树与路径

树遍历数组按照从根结点开始进行深度优先搜索时的访问顺序，存放有根树中的各个结点。例如，在树



中，深度优先搜索按如下方式进行：



因此，对应的树遍历数组如下：

1	2	6	3	4	7	8	9	5

子树查询

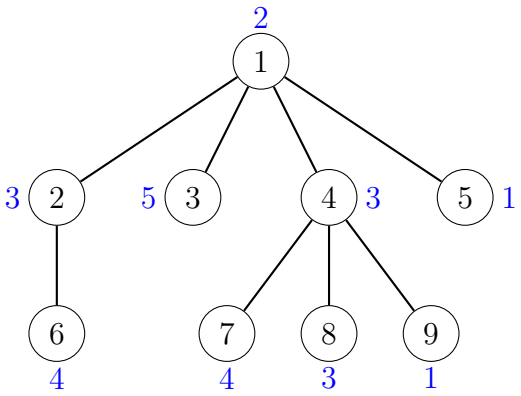
树中的每棵子树都对应树遍历数组中的一个子数组，并且该子数组的第一个元素就是子树的根结点。例如，下图标出的子数组包含结点 4 的子树中的所有结点：

1	2	6	3	4	7	8	9	5

利用这一性质，可以高效处理与子树有关的查询。例如，考虑这样一个问题：每个结点都有一个值，需要支持以下查询：

- 更新一个结点的值；
- 计算一个结点的子树中所有结点的值之和。

考虑下图，其中蓝色数字表示结点的值。例如，结点 4 的子树的值之和为 $3+4+3+1=11$。



其思想是构造一个树的遍历数组，为每个结点保存三个值：结点编号、子树大小和结点值。例如，上述树对应的数组如下：

node id	1	2	6	3	4	7	8	9	5
subtree size	9	2	1	1	4	1	1	1	1
node value	2	3	4	5	3	4	3	1	1

利用该数组，先求出子树大小，再取出对应结点的值，就能计算任意子树中所有结点的值之和。例如，可以按下图找到结点 4 的子树中各结点的值：

结点编号	1	2	6	3	4	7	8	9	5
子树大小	9	2	1	1	4	1	1	1	1
结点值	2	3	4	5	3	4	3	1	1

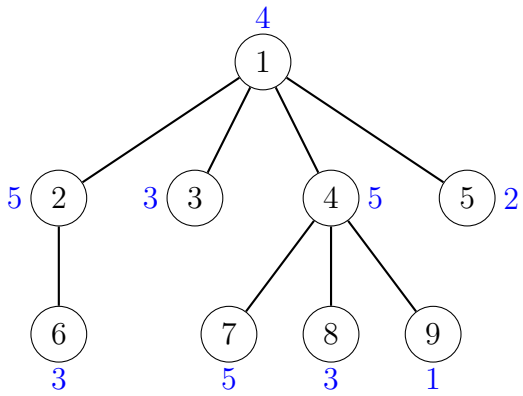
为了高效回答查询，只需把各结点的值存入树状数组或线段树。这样，更新一个值与计算区间和都能在 $O(\log n)$ 时间内完成。

路径查询

借助树遍历数组，还可以高效计算从根结点到树中任意结点的路径上的值之和。考虑需要支持以下查询的问题：

- 修改一个结点的值；
- 计算从根到某个结点的路径上的值之和。

例如，在下图中，从根结点到结点 7 的路径上的值之和为 $4+5+5=14$：



可以采用与前面类似的方法解决该问题，不过现在数组最后一行的每个值表示从根到对应结点的路径和。例如，上图对应的数组如下：

结点编号	1	2	6	3	4	7	8	9	5
子树大小	9	2	1	1	4	1	1	1	1
路径和	4	9	12	7	9	14	12	10	6

当一个结点的值增加 x 时，其子树中所有结点对应的路径和都会增加 x。例如，如果结点 4 的值增加 1，数组将变为：

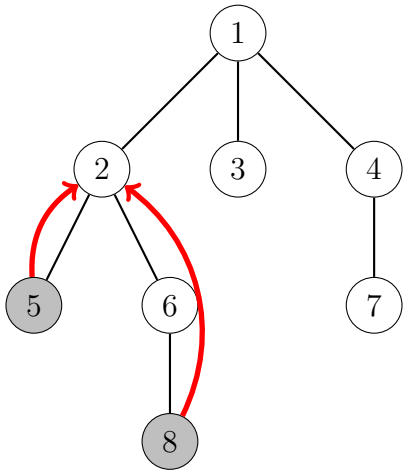
结点编号	1	2	6	3	4	7	8	9	5
子树大小	9	2	1	1	4	1	1	1	1
路径和	4	9	12	7	10	15	13	11	6

因此，为了支持这两种操作，需要能够把一个区间内的所有值同时增加，并查询某个单点的值。使用树状数组或线段树（参见第 9.4 节），可以在 $O(\log n)$ 时间内完成这些操作。

18.3 最近公共祖先

有根树中两个结点的**最近公共祖先**，是其子树同时包含这两个结点的最深结点。一个典型问题是高效处理求两个结点最近公共祖先的查询。

例如，在下图中，结点 5 和结点 8 的最近公共祖先是结点 2：



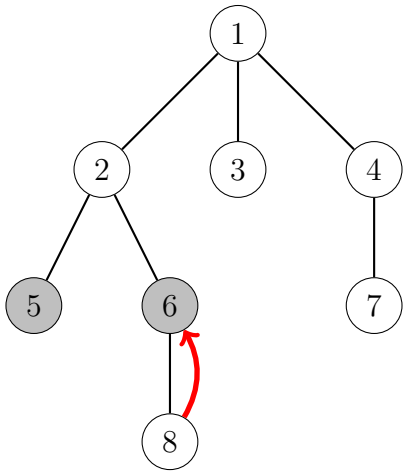
下面讨论两种高效求取两个结点最近公共祖先的技术。

方法一

一种解法利用了能够高效求出树中任意结点第 k 个祖先这一事实。借助这一操作，可以把寻找最近公共祖先分成两个阶段。

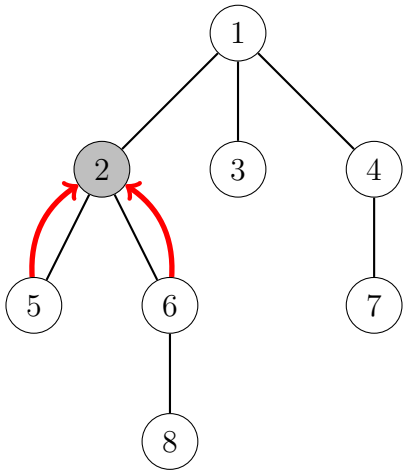
使用两个指针，初始时分别指向待求最近公共祖先的两个结点。首先把其中一个指针向上移动，使两个指针所指结点处于同一深度。

在上述示例中，把第二个指针向上移动一层，使其指向结点 6；此时它与结点 5 处于同一深度：



随后，求出需要把两个指针同时向上移动的最少步数，使它们指向同一结点。最终所指的结点就是最近公共祖先。

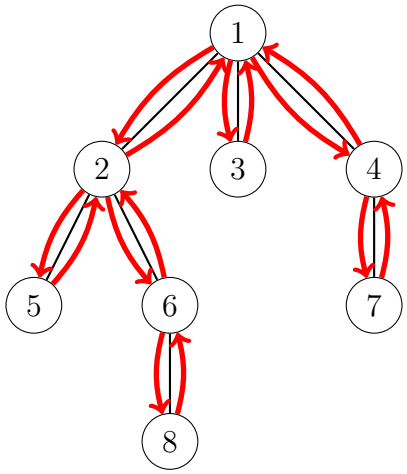
在该示例中，只需把两个指针同时向上移动一步到结点 2；结点 2 即为最近公共祖先：



利用预处理信息，算法的两个阶段都能在 $O(\log n)$ 时间内完成，因此可以在 $O(\log n)$ 时间内求出任意两个结点的最近公共祖先。

方法二

另一种解法基于树遍历数组[1]。其思想仍然是使用深度优先搜索遍历结点：



不过，这里使用的树遍历数组与前文不同：深度优先搜索每次经过一个结点时，都把它加入数组，而不只是第一次访问时加入。因此，一个有 k 个子结点的结点会在数组中出现 $k+1$ 次，数组中总共有 $2n-1$ 个结点。

数组为每个位置存储两个值：结点编号，以及该结点在树中的深度。上图对应的数组如下：

[1]这一最近公共祖先算法发表于 [7]。该技术有时称为**欧拉序技术** [66]。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
结点编号	1	2	5	2	6	8	6	2	1	3	1	4	7	4	1
深度	1	2	3	2	3	4	3	2	1	2	1	2	3	2	1

现在，只需在数组中结点 a 与结点 b 两次出现的位置之间，找出深度最小的结点，就能得到它们的最近公共祖先。例如，可以按下图求出结点 5 和结点 8 的最近公共祖先：

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
结点编号	1	2	5	2	6	8	6	2	1	3	1	4	7	4	1
深度	1	2	3	2	3	4	3	2	1	2	1	2	3	2	1
				↑											

结点 5 位于位置 2，结点 8 位于位置 5；在位置 $2 \ldots 5$ 之间，深度最小的是位置 3 上、深度为 2 的结点 2。因此，结点 5 和结点 8 的最近公共祖先是结点 2。

由此可见，求两个结点的最近公共祖先可以归约为区间最小值查询。由于数组是静态的，在进行 $O(n \log n)$ 的预处理后，每次查询可以在 $O(1)$ 时间内完成。

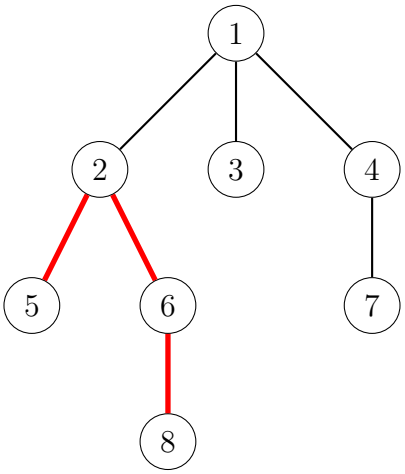
结点距离

结点 a 与结点 b 之间的距离，等于从 a 到 b 的路径长度。计算结点间距离的问题可以归约为寻找它们的最近公共祖先。

首先任选一个结点作为树根。随后，可以用以下公式计算结点 a 与结点 b 之间的距离：

$$\mathrm{depth}(a) + \mathrm{depth}(b) - 2 \cdot \mathrm{depth}(c),$$

其中 c 是 a 与 b 的最近公共祖先，$\mathrm{depth}(s)$ 表示结点 s 的深度。例如，考虑结点 5 与结点 8 之间的距离：



结点 5 和结点 8 的最近公共祖先是结点 2。各结点的深度为 $\text{depth}(5)=3$、$\text{depth}(8)=4$、$\text{depth}(2)=2$，因此结点 5 与结点 8 之间的距离为 $3+4-2\cdot 2=3$。

18.4 离线算法

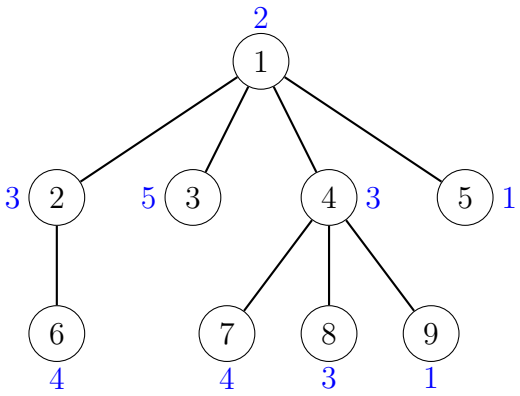
到目前为止，我们讨论的都是处理树上查询的*在线*算法。这类算法逐个处理查询，并且必须在接收下一个查询之前回答当前查询。

然而，许多问题并不要求在线处理。本节重点讨论*离线*算法：算法一次获得一组查询，并可按任意顺序回答。与在线算法相比，离线算法通常更容易设计。

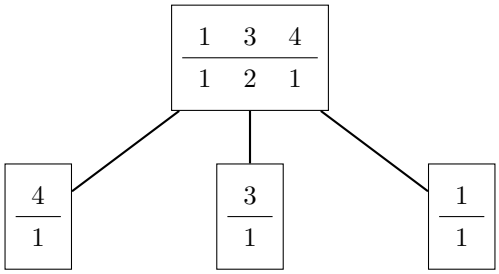
合并数据结构

构造离线算法的一种方法是对树进行深度优先遍历，并在各结点维护数据结构。对于每个结点 s，根据其子结点的数据结构建立 $\text{d}[s]$，随后使用这一数据结构处理所有与 s 有关的查询。

例如，考虑如下问题：给定一棵每个结点都有某个值的树，需要处理形如“计算结点 s 的子树中值为 x 的结点个数”的查询。在下图中，结点 4 的子树包含两个值为 3 的结点。



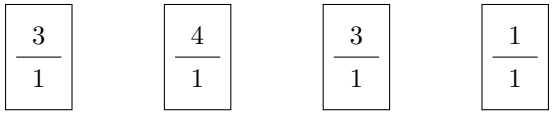
在这个问题中，可以使用映射（map）结构回答查询。例如，结点 4 及其各子结点对应的映射如下：



如果为每个结点建立这样的数据结构，就能轻松处理所有查询，因为在建立某结点的数据结构后，可以立即处理与该结点有关的全部查询。例如，上面结点 4 的映射表明，其子树中有两个值为 3 的结点。

不过，从头建立所有数据结构会过于缓慢。作为替代，对于每个结点 s，先建立只包含 s 自身值的初始数据结构 $\mathrm{d}[s]$。随后遍历 s 的所有子结点，把 $\mathrm{d}[s]$ 与每个子结点 u 的数据结构 $\mathrm{d}[u]$ 依次合并。

例如，在上图中，结点 4 的映射由以下映射合并而成：



其中第一个映射是结点 4 的初始数据结构，另外三个映射分别对应结点 7、8、9。

可以按如下方式在结点 s 处完成合并：遍历 s 的各个子结点，对每个子结点 u 合并 $\mathrm{d}[s]$ 与 $\mathrm{d}[u]$。总是把 $\mathrm{d}[u]$ 的内容复制到 $\mathrm{d}[s]$；但在此之前，如果 $\mathrm{d}[s]$ 小于 $\mathrm{d}[u]$，就先交换两者的内容。这样，在整次树遍历中，每个值只会被复制 $O(\log n)$ 次，从而保证算法高效。

要高效交换两个数据结构 a 与 b 的内容，只需使用以下代码：

```
swap(a,b);
```

当 a 和 b 是 C++ 标准库数据结构时，可以保证上述代码在常数时间内运行。

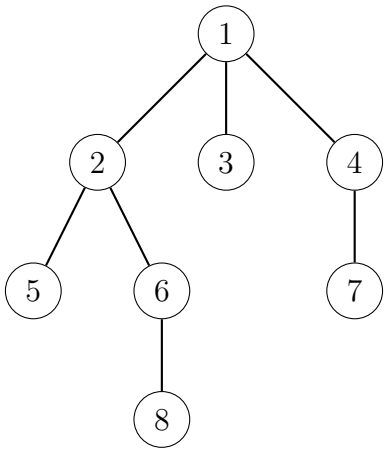
最近公共祖先

还有一种离线算法可用于处理一组最近公共祖先查询[2]。该算法基于并查集（参见第 15.2 节），其优点是比本章前面讨论的算法更容易实现。

算法的输入是一组结点对，任务是为每一对结点确定最近公共祖先。算法对树进行深度优先遍历，并维护若干互不相交的结点集合。初始时，每个结点各自属于一个集合；对于每个集合，还要保存其中在树上位置最高的结点。

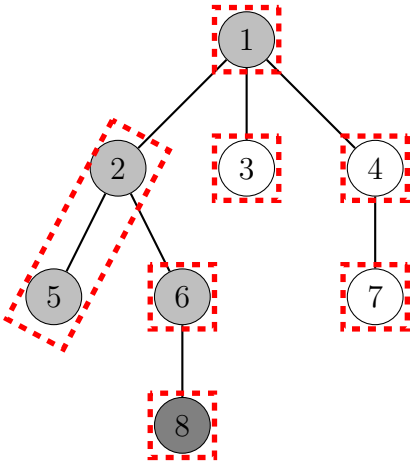
当算法访问结点 x 时，会遍历所有需要与 x 一起查询最近公共祖先的结点 y。如果 y 已经被访问，则 x 与 y 的最近公共祖先就是 y 所在集合中位置最高的结点。处理完结点 x 后，算法把 x 所在集合与其父结点所在集合合并。

例如，假设要在下图中求结点对 $(5, 8)$ 与 $(2, 7)$ 的最近公共祖先：

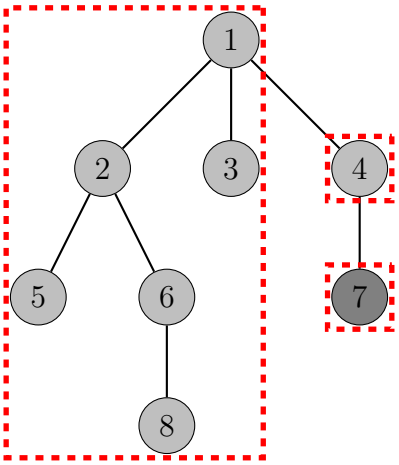


在下面的图中，灰色结点表示已访问结点，虚线框中的结点属于同一集合。当算法访问结点 8 时，会发现结点 5 已被访问，且其所在集合中位置最高的结点为 2。因此，结点 5 和结点 8 的最近公共祖先是 2：

[2]该算法由 R. E. Tarjan 于 1979 年发表 [65]。



之后，当算法访问结点 7 时，可以确定结点 2 与结点 7 的最近公共祖先是 1：



第十九章　路径与回路

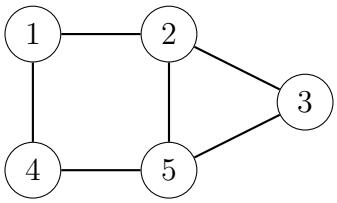
本章重点讨论图中的两类路径：

- **欧拉路径**是恰好经过每条边一次的路径。
- **哈密顿路径**是恰好访问每个结点一次的路径。

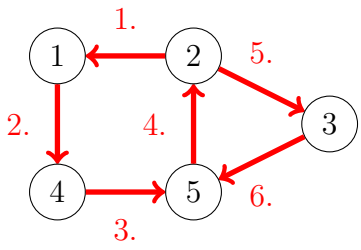
欧拉路径和哈密顿路径乍看之下十分相似，但与它们有关的计算问题却截然不同。判断一个图是否包含欧拉路径有一条简单的规则，而且如果欧拉路径存在，也有高效算法能够找到它。相反，判断哈密顿路径是否存在是一个 NP 困难问题，目前没有已知的高效算法。

19.1　欧拉路径

欧拉路径[1]是恰好经过图中每条边一次的路径。例如，图

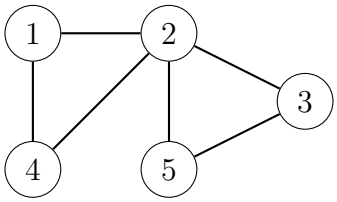


包含一条从结点 2 到结点 5 的欧拉路径：

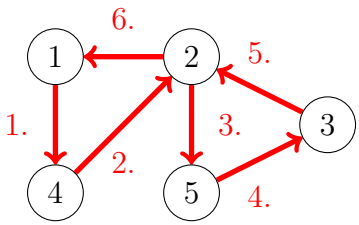


起点和终点为同一结点的欧拉路径称为**欧拉回路**。例如，图

[1]L. Euler 于 1736 年研究了这类路径，并解决了著名的柯尼斯堡七桥问题。这也标志着图论的诞生。



包含一条以结点 1 为起点和终点的欧拉回路：



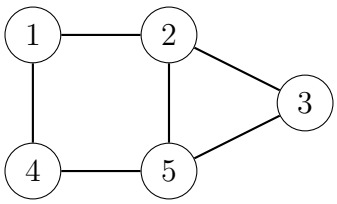
存在条件

欧拉路径和欧拉回路是否存在，取决于各结点的度。首先，无向图恰好在所有边都属于同一个连通分量，并且满足以下条件之一时存在欧拉路径：

- 每个结点的度均为偶数；*或*
- 恰有两个结点的度为奇数，其余结点的度均为偶数。

第一种情况下，每条欧拉路径也都是欧拉回路。第二种情况下，两个奇度结点分别是某条非欧拉回路的欧拉路径的起点和终点。

例如，在图



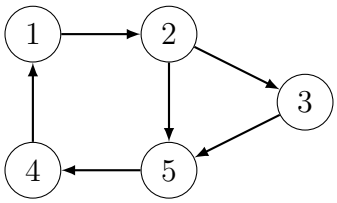
中，结点 1、3、4 的度为 2，结点 2、5 的度为 3。恰有两个结点的度为奇数，因此结点 2 与结点 5 之间存在欧拉路径，但该图不包含欧拉回路。

对于有向图，我们考察结点的入度与出度。有向图恰好在所有边都属于同一个连通分量，并且满足以下条件之一时包含欧拉路径：

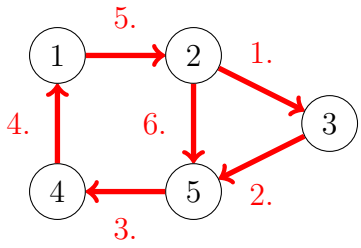
- 每个结点的入度都等于出度；*或*
- 有一个结点的入度比出度大 1，另一个结点的出度比入度大 1，其余结点的入度都等于出度。

第一种情况下，每条欧拉路径也都是欧拉回路；第二种情况下，图中存在一条欧拉路径，它从出度较大的结点出发，在入度较大的结点结束。

例如，在图



中，结点 1、3、4 的入度和出度均为 1，结点 2 的入度为 1、出度为 2，结点 5 的入度为 2、出度为 1。因此，该图包含一条从结点 2 到结点 5 的欧拉路径：



Hierholzer 算法

Hierholzer 算法[2]是一种构造欧拉回路的高效方法。算法分若干轮执行，每一轮都向回路中加入一些新边。当然，我们假设图中包含欧拉回路；否则 Hierholzer 算法不可能找到它。

首先，算法构造一个包含图中部分边（不一定是全部边）的回路。随后，算法逐步向该回路中插入子回路，以扩展原回路。这个过程持续到所有边都被加入回路为止。

扩展回路时，算法总是寻找一个属于当前回路、但仍有某条出边尚未包含在回路中的结点 x。然后从结点 x 出发，仅沿尚未加入回路的边构造一条新路径。这条路径迟早会返回结点 x，从而形成一个子回路。

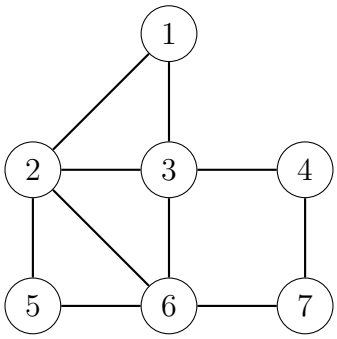
如果图中只有欧拉路径，我们仍可利用 Hierholzer 算法寻找它：先向图中加入一条额外的边，构造出欧拉回路后再删去这条边。例如，在无向图中，可在两个奇度结点之间加入这条额外边。

下面考察 Hierholzer 算法如何为一个无向图构造欧拉回路。

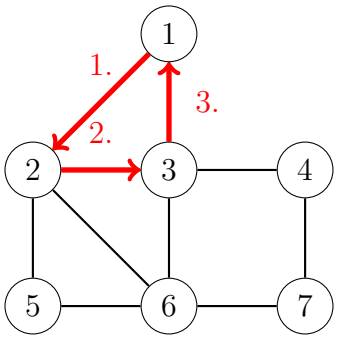
[2]该算法于 1873 年在 Hierholzer 去世后发表 [35]。

示例

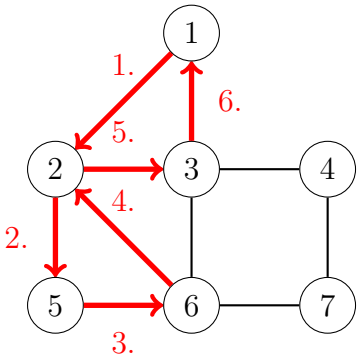
考虑下图：



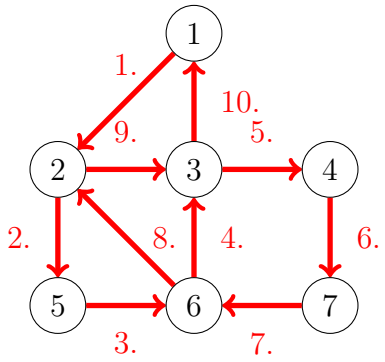
假设算法首先构造一条从结点 1 开始的回路。一种可能的回路是 $1 \to 2 \to 3 \to 1$：



随后，算法把子回路 $2 \to 5 \to 6 \to 2$ 加入当前回路：



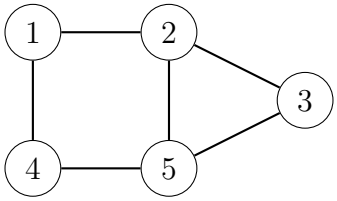
最后，算法把子回路 $6 \to 3 \to 4 \to 7 \to 6$ 加入当前回路：



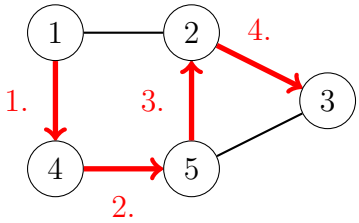
此时所有边都已包含在回路中，因此我们成功构造出了一条欧拉回路。

19.2 哈密顿路径

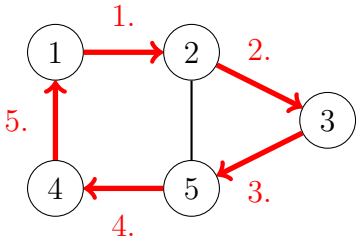
哈密顿路径是恰好访问图中每个结点一次的路径。例如，图



包含一条从结点 1 到结点 3 的哈密顿路径：



如果一条哈密顿路径的起点和终点是同一结点，就称它为**哈密顿回路**。上图还包含一条以结点 1 为起点和终点的哈密顿回路：



存在条件

目前没有已知的高效方法能够判断一个图是否包含哈密顿路径，而且该问题是 NP 困难的。不过在某些特殊情况下，我们可以确定图中一定存在哈密顿路径。

一个简单的观察是：如果图是完全图，即任意一对结点之间都有边，那么它一定包含哈密顿路径。此外，还有如下更强的结论：

- **Dirac 定理**：如果每个结点的度都至少为 $n/2$，则图中存在哈密顿路径。
- **Ore 定理**：如果每一对不相邻结点的度数之和都至少为 n，则图中存在哈密顿路径。

这些定理及其他相关结论有一个共同特征：当图中边的数量足够多时，它们便能保证哈密顿路径存在。这很自然，因为图中的边越多，构造哈密顿路径的可能方式也就越多。

构造

既然没有高效方法能够判断哈密顿路径是否存在，那么显然也不存在高效构造该路径的方法；否则，只要尝试构造路径，就能判断它是否存在。

搜索哈密顿路径的一种简单方法是使用回溯算法，枚举构造路径的所有可能方式。此类算法的时间复杂度至少为 $O(n!)$，因为 n 个结点共有 $n!$ 种不同的排列顺序。

一种更高效的解法基于动态规划（参见第 10.5 节）。其思想是计算函数 possible(S, x) 的值，其中 S 是结点集合的一个子集，x 是其中一个结点。该函数表示是否存在一条访问 S 中所有结点、并在结点 x 结束的哈密顿路径。这一解法可以在 $O(2^n n^2)$ 时间内实现。

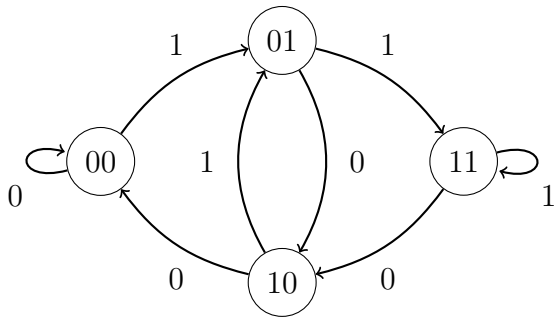
19.3 De Bruijn 序列

对于一个固定的、含 k 个字符的字母表，**De Bruijn 序列**是这样一个字符串：每个长度为 n 的字符串都恰好作为它的子串出现一次。这样的字符串长度为 $k^n + n - 1$。例如，当 $n = 3$、$k = 2$ 时，一个 De Bruijn 序列为

0001011100.

该字符串的子串包含三位二进制串的所有组合：000、001、010、011、100、101、110 和 111。

每个 De Bruijn 序列都对应某个图中的一条欧拉路径。构图方法是：每个结点表示一个长度为 $n-1$ 的字符串，每条边则向字符串添加一个字符。与上述情形对应的图如下：



该图中的一条欧拉路径对应一个包含所有长度为 n 的字符串的字符串。这个字符串由起始结点中的字符以及各条边上的字符组成。起始结点含有 $n-1$ 个字符，边上共有 k^n 个字符，因此字符串总长度为 k^n+n-1。

19.4 骑士巡游

骑士巡游是国际象棋中的骑士按照行棋规则，在 $n \times n$ 棋盘上移动并恰好访问每个方格一次的移动序列。如果骑士最终返回起始方格，就称为*闭巡游*；否则称为*开巡游*。

例如，下面是 5×5 棋盘上的一次开巡游：

1	4	11	16	25
12	17	2	5	10
3	20	7	24	15
18	13	22	9	6
21	8	19	14	23

骑士巡游对应于一个图中的哈密顿路径：图的结点表示棋盘方格，如果骑士能按照国际象棋规则在两个方格之间移动，就在对应的两个结点之间连边。

构造骑士巡游的一种自然方法是回溯。还可以使用*启发式方法*引导骑士移动，使其尽快找到完整巡游，从而提高搜索效率。

Warnsdorf 规则

Warnsdorf 规则是一种寻找骑士巡游的简单而有效的启发式方法[3]。利用该规则，即使在很大的棋盘上也能高效构造巡游。其思想是：骑士每次都移动到后续可选步数尽可能少的方格。

例如，在下图所示的局面中，骑士可以移动到五个方格（$a \ldots e$）：

1				a
		2		
b				e
	c		d	

在这种情况下，Warnsdorf 规则会把骑士移动到方格 a，因为作出该选择后，下一步只有一种可能。若选择其他方格，下一步都将有三种可能。

[3]这一启发式方法由 Warnsdorf 在 1823 年出版的著作 [69] 中提出。寻找骑士巡游也有多项式时间算法 [52]，但它们更为复杂。

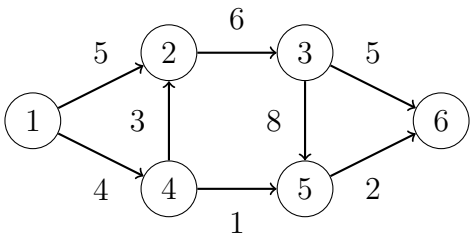
第二十章　流与割

本章重点讨论以下两个问题：

- **求最大流**：从一个结点向另一个结点最多能输送多少流量？
- **求最小割**：使图中两个结点分离的最小权值边集是什么？

这两个问题的输入都是一个带权有向图，其中包含两个特殊结点：源点没有入边，汇点没有出边。

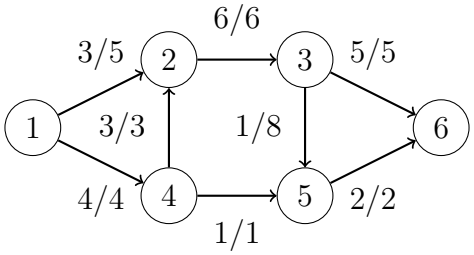
我们将以下图为例，其中结点 1 是源点，结点 6 是汇点：



最大流

在**最大流**问题中，任务是尽可能多地把流量从源点输送到汇点。每条边的权值表示容量，它限制了能够通过该边的流量。对于每个中间结点，流入量必须等于流出量。

例如，示例图中的最大流量为 7。下图展示了一种流量分配方式：

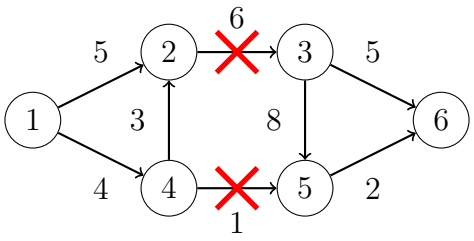


记号 v/k 表示一条容量为 k 的边上通过了 v 单位流量。该流的大小为 7，因为源点流出了 $3+4$ 单位流量，而汇点流入了 $5+2$ 单位流量。容易看出这已经是最大流，因为所有通向汇点的边的总容量就是 7。

最小割

在**最小割**问题中，需要从图中删除一组边，使得删除后不存在从源点到汇点的路径，并且被删除边的总权值最小。

示例图中的最小割大小为 7。只需删除边 $2 \to 3$ 和 $4 \to 5$：



删除这些边之后，就不存在从源点到汇点的路径。该割的大小为 7，因为两条被删除边的权值分别为 6 和 1。这个割是最小的，因为无法通过删除总权值小于 7 的边使源点与汇点分离。

在上述示例中，最大流与最小割的大小相同并非巧合。事实上，最大流与最小割的大小总是相等，因此二者是同一问题的两个侧面。

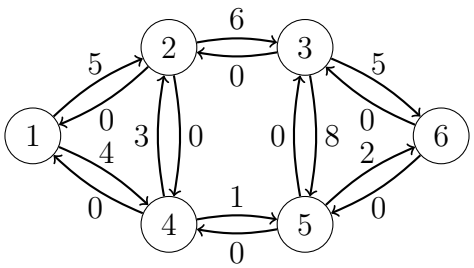
下面讨论 Ford–Fulkerson 算法，它可以求出图的最大流与最小割，也能帮助我们理解二者的大小为何相等。

20.1 Ford–Fulkerson 算法

Ford–Fulkerson 算法 [25] 用于求图中的最大流。算法从零流开始，每一步寻找一条从源点到汇点、能够增加流量的路径。当无法再增加流量时，就得到了最大流。

算法使用一种特殊的图表示：每条原始边都有一条方向相反的反向边。每条边的权值表示还可以沿该边增加多少流量。算法开始时，每条原始边的权值等于其容量，而每条反向边的权值为零。

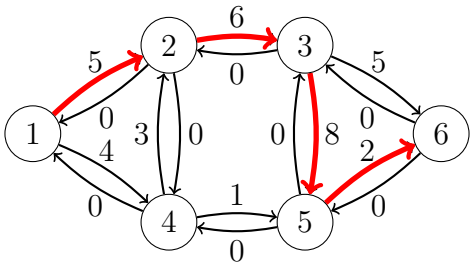
示例图的新表示如下：



算法描述

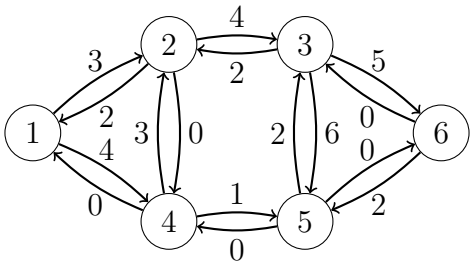
Ford–Fulkerson 算法分若干轮执行。每一轮中，算法寻找一条从源点到汇点、且路径上每条边的权值均为正的路径。如果存在多条可选路径，可以任选其一。

例如，假设选择下图所示的路径：



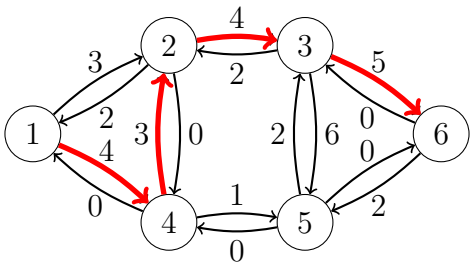
选定路径后，流量增加 x，其中 x 是路径上最小的边权。同时，路径上每条正向边的权值减小 x，相应反向边的权值增加 x。

上述路径上的边权依次为 5、6、8、2。最小边权为 2，因此流量增加 2，新图如下：



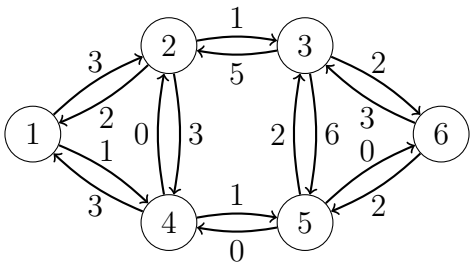
其含义是：增加流量会减少以后还能通过这些边的流量。另一方面，如果后来发现改用另一种流量分配更有利，也可以沿图中的反向边撤销已经发送的流量。

只要还存在一条由正权边组成的源点到汇点路径，算法就继续增加流量。在当前示例中，下一条路径可以是：

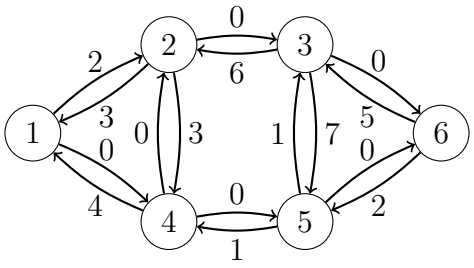


该路径上的最小边权为 3，因此它使流量增加 3；处理完这条路径后，总流量为 5。

新图如下：



还需再进行两轮才能达到最大流。例如，可以依次选择路径 $1 \to 2 \to 3 \to 6$ 和 $1 \to 4 \to 5 \to 3 \to 6$。两条路径都使流量增加 1，最终得到下图：



此时已经无法继续增加流量，因为不存在一条由正权边组成的源点到汇点路径。因此算法终止，最大流为 7。

寻找路径

Ford–Fulkerson 算法并未规定应如何选择增大流量的路径。无论如何选择，算法最终都会终止并正确求出最大流；不过，算法效率取决于路径的选择方式。

寻找路径的一种简单方法是深度优先搜索。它通常效果不错，但在最坏情况下，每条路径只能使流量增加 1，算法会很慢。可以使用以下技术避免这种情况：

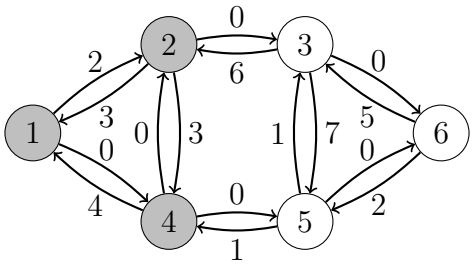
Edmonds–Karp 算法 [18] 每次都选择边数尽可能少的路径。为此，寻找路径时使用广度优先搜索而不是深度优先搜索。可以证明，这能保证流量快速增加，算法的时间复杂度为 $O(m^2n)$。

容量缩放算法 [2] 使用深度优先搜索寻找每条边权都至少达到某个阈值的路径。初始阈值取一个较大的数，例如图中所有边权之和；每当找不到路径时，就把阈值除以 2。算法的时间复杂度为 $O(m^2 \log c)$，其中 c 是初始阈值。

实践中，容量缩放算法更容易实现，因为可以使用深度优先搜索寻找路径。这两种算法对算法竞赛中的典型问题都足够高效。

最小割

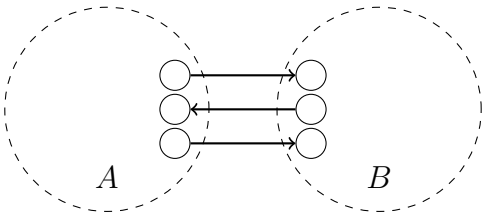
Ford–Fulkerson 算法求出最大流的同时，也确定了一个最小割。令 A 为从源点出发、沿正权边能够到达的结点集合。在示例图中，A 包含结点 1、2、4：



最小割由原图中满足以下条件的边构成：起点属于 A，终点不属于 A，并且该边的容量在最大流中已被完全用尽。在上图中，这些边是 $2 \to 3$ 和 $4 \to 5$，对应的最小割大小为 $6 + 1 = 7$。

为什么算法产生的流一定是最大流，而得到的割一定是最小割？原因在于，图中任意流的大小都不可能超过任意割的权值。因此，只要某个流与某个割大小相等，它们就分别是最大流和最小割。

考虑图的任意一个割：源点属于集合 A，汇点属于集合 B，两个集合之间存在一些边：



割的大小等于所有从 A 指向 B 的边的权值之和。它是图中流量的上界，因为流必须从 A 流向 B。因此，最大流的大小小于或等于图中任意割的大小。

另一方面，Ford–Fulkerson 算法产生的流，其大小与图中某个割的大小恰好相等。因此，该流必为最大流，该割也必为最小割。

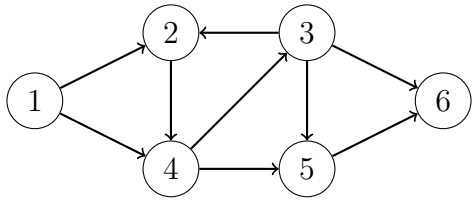
20.2 不相交路径

许多图论问题都可以归约为最大流问题。第一个例子如下：给定一个带有源点和汇点的有向图，任务是求从源点到汇点的不相交路径的最大条数。

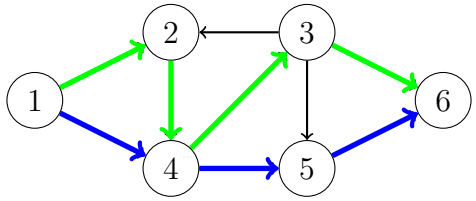
边不相交路径

首先考虑从源点到汇点的**边不相交路径**的最大条数。也就是说，需要构造一组路径，使每条边最多只出现在其中一条路径中。

例如，考虑下图：



该图中边不相交路径的最大条数为 2。可以选择路径 $1 \to 2 \to 4 \to 3 \to 6$ 和 $1 \to 4 \to 5 \to 6$，如下图所示：

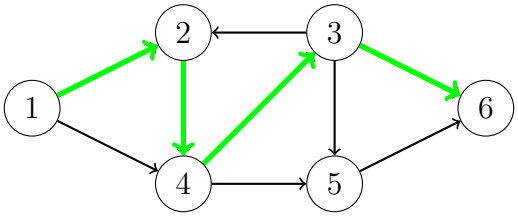


如果把每条边的容量都设为 1，那么边不相交路径的最大条数就等于图的最大流。构造出最大流后，可以从源点到汇点贪心地沿有流量的边提取出这些边不相交路径。

点不相交路径

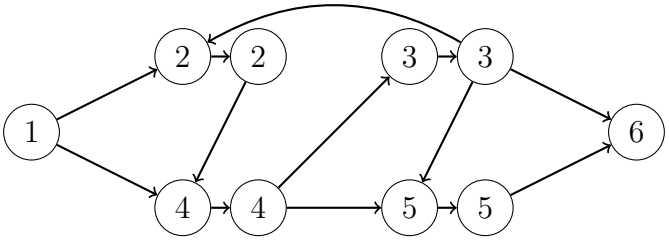
下面考虑另一个问题：求从源点到汇点的**点不相交路径**的最大条数。在这个问题中，除源点和汇点外，每个结点最多只能出现在一条路径中。点不相交路径的最大条数可能小于边不相交路径的最大条数。

例如，在前一幅图中，点不相交路径的最大条数为 1：

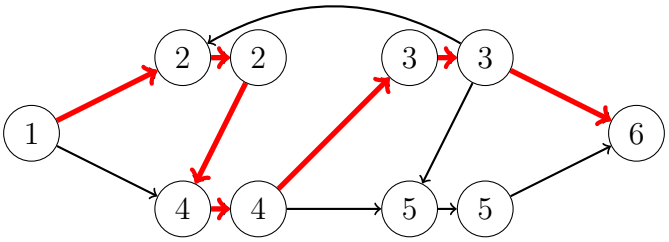


这个问题同样可以归约为最大流。由于每个结点最多只能出现在一条路径中，需要限制通过结点的流量。标准方法是把每个结点拆成两个结点：第一个结点连接原结点的所有入边，第二个结点连接原结点的所有出边，并在第一个结点到第二个结点之间添加一条容量为 1 的新边。

在本例中，图变为：



该图的最大流如下：



因此，从源点到汇点的点不相交路径最大条数为 1。

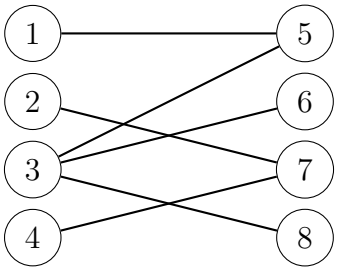
20.3 最大匹配

最大匹配问题要求在无向图中找出规模最大的结点对集合，使每对结点之间都有边相连，并且每个结点最多属于一个结点对。

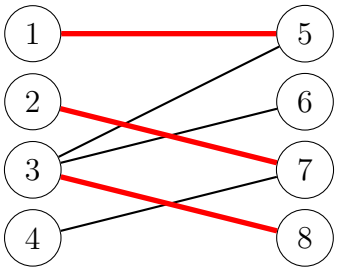
一般图的最大匹配存在多项式时间算法 [17]，但这些算法较为复杂，在算法竞赛中很少出现。对于二分图，最大匹配问题容易得多，因为它可以归约为最大流问题。

寻找最大匹配

二分图的结点总能划分为两组，使图中的所有边都连接左侧组与右侧组。例如，在下图中，两组分别为 $\{1,2,3,4\}$ 和 $\{5,6,7,8\}$。

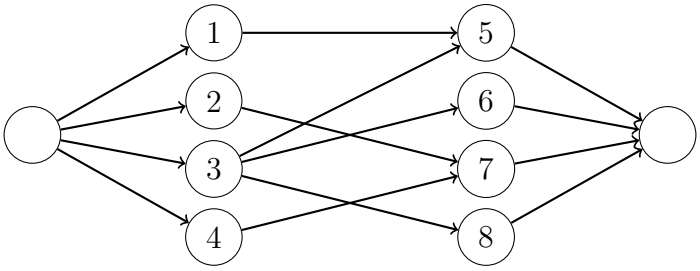


该图的最大匹配大小为 3：

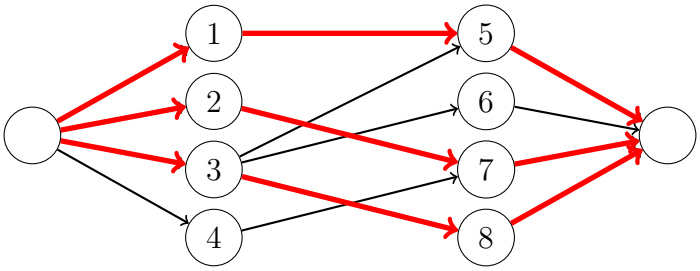


可以通过向图中加入源点和汇点两个新结点，把二分图最大匹配归约为最大流问题。再从源点向每个左侧结点连边，并从每个右侧结点向汇点连边。这样，新图的最大流大小就等于原图的最大匹配大小。

例如，上图的归约结果如下：



该图的最大流如下：

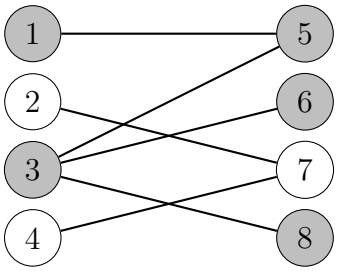


Hall 定理

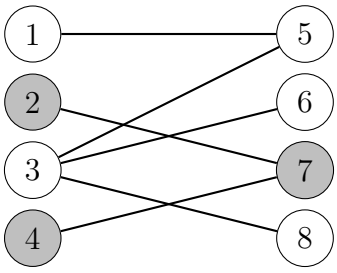
Hall 定理可用于判断二分图是否存在覆盖全部左侧结点或全部右侧结点的匹配。如果左右两侧结点数量相同，Hall 定理可以判断能否构造覆盖图中所有结点的**完美匹配**。

假设要寻找覆盖所有左侧结点的匹配。令 X 为任意左侧结点集合，$f(X)$ 为这些结点的邻居集合。根据 Hall 定理，覆盖所有左侧结点的匹配存在，当且仅当对每个 X 都有 $|X| \leq |f(X)|$。

在示例图中考察 Hall 定理。首先令 $X = \{1, 3\}$，则 $f(X) = \{5, 6, 8\}$：



因为 $|X| = 2$ 且 $|f(X)| = 3$，所以满足 Hall 定理的条件。接着令 $X = \{2, 4\}$，则 $f(X) = \{7\}$：



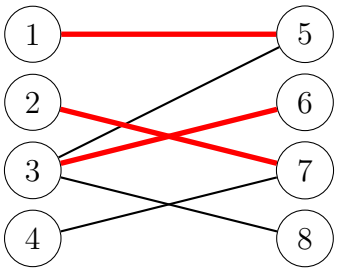
此时 $|X| = 2$ 而 $|f(X)| = 1$，不满足 Hall 定理的条件。这意味着该图不存在完美匹配。这个结果并不意外，因为已经知道该图的最大匹配大小是 3，而不是 4。

如果 Hall 定理的条件不成立，集合 X 也解释了为何无法构造这样的匹配。由于 X 中的结点数多于 $f(X)$，不可能为 X 中每个结点都找到不同的配对。例如，在上图中，结点 2 和结点 4 都只能与结点 7 配对，这是不可能的。

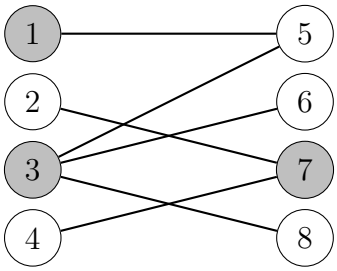
Kőnig 定理

图的**最小点覆盖**是规模最小的结点集合，使图中每条边至少有一个端点属于该集合。在一般图中，求最小点覆盖是 NP 困难问题。然而，对于二分图，**Kőnig 定理**指出最小点覆盖的大小总是等于最大匹配的大小。因此，可以利用最大流算法计算最小点覆盖的大小。

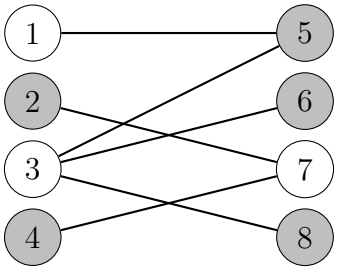
考虑下图，其最大匹配大小为 3：



由 Kőnig 定理可知，其最小点覆盖的大小也为 3。可以构造如下点覆盖：



不属于某个最小点覆盖的结点构成一个**最大独立集**。它是满足集合内任意两个结点之间都没有边的最大结点集合。一般图的最大独立集同样是 NP 困难问题，但在二分图中可以借助 Kőnig 定理高效求解。示例图中的最大独立集如下：

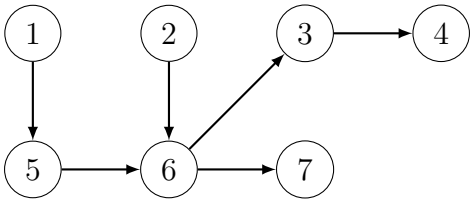


20.4 路径覆盖

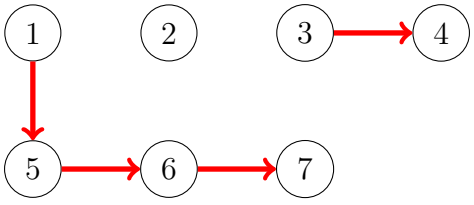
路径覆盖是图中的一组路径，使图中每个结点都至少属于一条路径。对于有向无环图，求最小路径覆盖可以归约为在另一个图中求最大流。

点不相交路径覆盖

在**点不相交路径覆盖**中，每个结点恰好属于一条路径。例如，考虑下图：



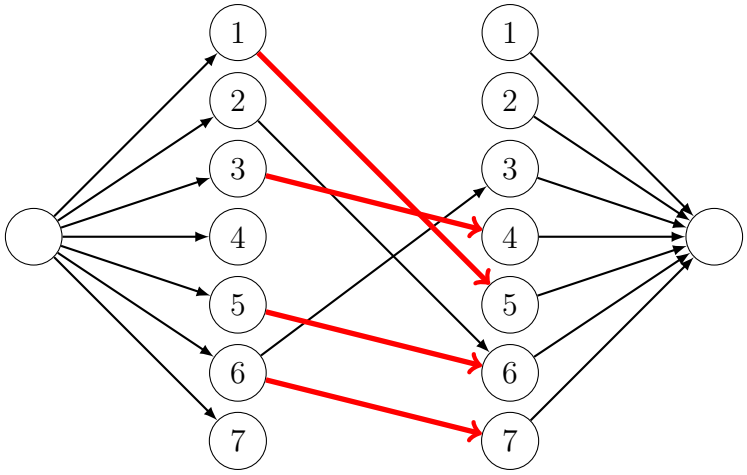
该图的最小点不相交路径覆盖由三条路径组成。例如，可以选择下图所示的路径：



注意，其中一条路径只包含结点 2，因此路径可以不包含任何边。

可以通过构造一个匹配图来求最小点不相交路径覆盖。在匹配图中，原图每个结点都由左、右两个结点表示；若原图中存在一条对应的边，就从左侧结点向右侧结点连边。此外，匹配图还包含源点和汇点，并从源点向所有左侧结点连边、从所有右侧结点向汇点连边。

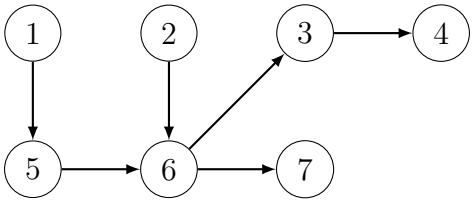
所得图中的最大匹配对应原图中的最小点不相交路径覆盖。例如，上图对应的匹配图含有一个大小为 4 的最大匹配：



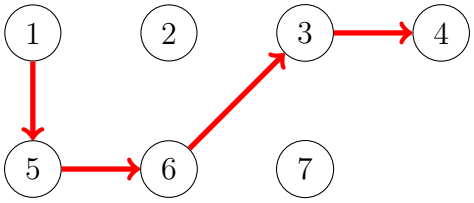
匹配图的最大匹配中，每条边都对应原图最小点不相交路径覆盖中的一条边。因此，最小点不相交路径覆盖的大小为 $n-c$，其中 n 是原图的结点数，c 是最大匹配大小。

一般路径覆盖

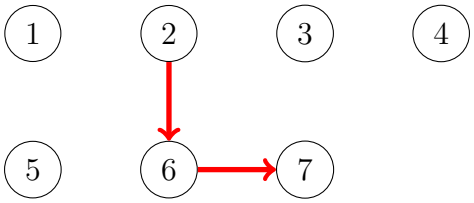
一般路径覆盖允许一个结点属于多条路径。由于结点可以在不同路径中重复使用，最小一般路径覆盖可能小于最小点不相交路径覆盖。再次考虑下图：



该图的最小一般路径覆盖由两条路径组成。例如，第一条路径可以是：

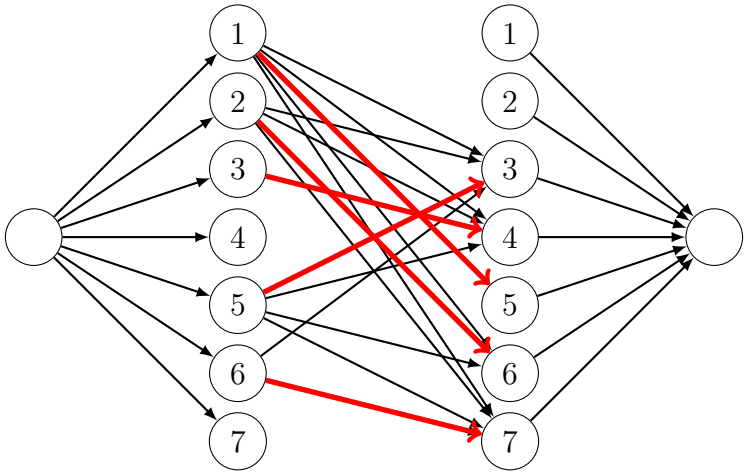


第二条路径可以是：



求最小一般路径覆盖的方法与求最小点不相交路径覆盖几乎相同。只需向匹配图添加一些新边：只要原图中存在一条从 a 到 b 的路径（可能经过多条边），就在匹配图中加入边 $a \to b$。

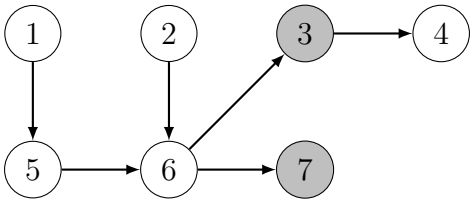
上图对应的匹配图如下：



Dilworth 定理

反链是图中的一个结点集合，其中任意一个结点都无法沿图中的边到达另一个结点。**Dilworth 定理**指出，在有向无环图中，最小一般路径覆盖的大小等于最大反链的大小。

例如，在下图中，结点 3 和结点 7 构成一条反链：



这是一条最大反链，因为无法构造包含三个结点的反链。前面已经看到，该图的最小一般路径覆盖由两条路径组成。

第三部分

高级主题

第二十一章　数论

数论是研究整数的数学分支。数论是一门引人入胜的学科，因为许多与整数有关的问题乍看十分简单，解决起来却非常困难。

例如，考虑方程：

$$x^3 + y^3 + z^3 = 33$$

很容易找到满足该方程的三个实数 x、y、z。例如，可以取

$$\begin{aligned} x &= 3, \\ y &= \sqrt[3]{3}, \\ z &= \sqrt[3]{3}. \end{aligned}$$

然而，在本书原稿写作时，是否存在满足该方程的三个整数 x、y、z 仍是数论中的一个公开问题 [6]。

本章重点介绍数论中的基本概念与算法。除非另有说明，本章中的所有数都假定为整数。

21.1　素数与因数

如果 a 整除 b，就称 a 是 b 的**因数**或**约数**。若 a 是 b 的因数，记作 $a \mid b$；否则记作 $a \nmid b$。例如，24 的因数为 1、2、3、4、6、8、12、24。

若 $n > 1$ 的正因数只有 1 和 n，则称 n 为**素数**。例如，7、19、41 是素数，而 35 不是素数，因为 $5 \cdot 7 = 35$。每个 $n > 1$ 都有唯一的**质因数分解**

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k},$$

其中 $p_1, p_2, \ldots, p_k$ 是互不相同的素数，$\alpha_1, \alpha_2, \ldots, \alpha_k$ 是正整数。例如，84 的质因数分解为

$$84 = 2^2 \cdot 3^1 \cdot 7^1.$$

数 n 的**因数个数**为

$$\tau(n) = \prod_{i=1}^{k} (\alpha_i + 1),$$

因为对于每个素数 p_i，它在一个因数中出现的次数有 $\alpha_i + 1$ 种选择。例如，84 的因数个数为 $\tau(84) = 3 \cdot 2 \cdot 2 = 12$。这些因数是 1、2、3、4、6、7、12、14、21、28、42、84。

数 n 的**因数和**为

$$\sigma(n) = \prod_{i=1}^{k}(1 + p_i + \ldots + p_i^{\alpha_i}) = \prod_{i=1}^{k}\frac{p_i^{a_i+1} - 1}{p_i - 1},$$

后一个等式来自等比数列求和公式。例如，84 的因数和为

$$\sigma(84) = \frac{2^3 - 1}{2 - 1} \cdot \frac{3^2 - 1}{3 - 1} \cdot \frac{7^2 - 1}{7 - 1} = 7 \cdot 4 \cdot 8 = 224.$$

数 n 的**因数乘积**为

$$\mu(n) = n^{\tau(n)/2},$$

因为可以把所有因数配成 $\tau(n)/2$ 对，每对的乘积均为 n。例如，84 的因数可配成 $1 \cdot 84$、$2 \cdot 42$、$3 \cdot 28$ 等，因此因数乘积为 $\mu(84) = 84^6 = 351298031616$。

若 $n = \sigma(n) - n$，即 n 等于它在 1 到 $n - 1$ 之间所有因数之和，则称 n 为**完全数**。例如，28 是完全数，因为 $28 = 1 + 2 + 4 + 7 + 14$。

素数的数量

很容易证明素数有无穷多个。如果素数只有有限多个，就可以构造包含全部素数的集合 $P = \{p_1, p_2, \ldots, p_n\}$，例如 $p_1 = 2$、$p_2 = 3$、$p_3 = 5$，依此类推。然而，利用 P 可以构造一个新的素数

$$p_1 p_2 \cdots p_n + 1$$

它大于 P 中的所有元素。这产生了矛盾，因此素数必有无穷多个。

素数密度

素数密度描述素数在整数中出现的频繁程度。令 $\pi(n)$ 表示 1 到 n 之间的素数个数。例如，$\pi(10) = 4$，因为 1 到 10 之间有 2、3、5、7 共 4 个素数。

可以证明

$$\pi(n) \approx \frac{n}{\ln n},$$

这表明素数相当常见。例如，1 到 10^6 之间的素数个数为 $\pi(10^6) = 78498$，而 $10^6 / \ln 10^6 \approx 72382$。

猜想

关于素数有许多猜想。人们普遍认为这些猜想成立，但至今尚无人能够证明。著名的例子包括：

- **哥德巴赫猜想**：每个偶数 $n > 2$ 都能表示为 $n = a + b$，其中 a 和 b 均为素数。
- **孪生素数猜想**：形如 $\{p, p+2\}$ 且 p 与 $p+2$ 均为素数的数对有无穷多组。
- **Legendre 猜想**：对于任意正整数 n，n^2 与 $(n+1)^2$ 之间总存在一个素数。

基础算法

如果 n 不是素数，就可以写成 $a \cdot b$，其中 $a \leq \sqrt{n}$ 或 $b \leq \sqrt{n}$，因此它在 2 到 $\lfloor \sqrt{n} \rfloor$ 之间一定有一个因数。利用这一观察，可以在 $O(\sqrt{n})$ 时间内判断一个数是否为素数，也可以求出它的质因数分解。

下面的函数 prime 判断给定的 n 是否为素数。函数尝试用 2 到 $\lfloor \sqrt{n} \rfloor$ 之间的所有数去除 n；如果没有一个能够整除 n，则 n 为素数。

```
bool prime(int n) {
    if (n < 2) return false;
    for (int x = 2; x*x <= n; x++) {
        if (n%x == 0) return false;
    }
    return true;
}
```

下面的函数 factors 构造一个保存 n 的质因数分解的向量。函数不断用 n 的质因数除它，并把这些质因数加入向量。当剩余的 n 在 2 到 $\lfloor \sqrt{n} \rfloor$ 之间不再有因数时，过程结束。若此时 $n > 1$，它就是最后一个素因数。

```
vector<int> factors(int n) {
    vector<int> f;
    for (int x = 2; x*x <= n; x++) {
        while (n%x == 0) {
            f.push_back(x);
            n /= x;
```

```
        }
    }
    if (n > 1) f.push_back(n);
    return f;
}
```

注意，每个素因数在向量中出现的次数等于它在原数的质因数分解中的指数。例如，$24 = 2^3 \cdot 3$，因此函数返回 $[2, 2, 2, 3]$。

Eratosthenes 筛法

Eratosthenes 筛法是一种预处理算法。它构造一个数组，使我们能够高效判断 $2 \ldots n$ 之间的给定整数是否为素数；如果不是，还能找到它的一个素因数。

算法构造数组 sieve，使用其中位置 $2, 3, \ldots, n$。sieve$[k] = 0$ 表示 k 是素数；sieve$[k] \neq 0$ 表示 k 不是素数，并且 sieve$[k]$ 是它的一个素因数。

算法依次遍历 $2 \ldots n$。每当发现一个新素数 x，就把 x 的所有倍数 $(2x, 3x, 4x, \ldots)$ 标记为合数，因为它们都能被 x 整除。

例如，当 $n = 20$ 时，数组如下：

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	0	2	0	3	0	2	3	5	0	3	0	7	5	2	0	3	0	5

以下代码实现 Eratosthenes 筛法，并假定 sieve 的所有元素初始均为零。

```
for (int x = 2; x <= n; x++) {
    if (sieve[x]) continue;
    for (int u = 2*x; u <= n; u += x) {
        sieve[u] = x;
    }
}
```

对于每个 x，算法的内层循环执行 n/x 次。因此，算法运行时间的一个上界是调和和

$$\sum_{x=2}^{n} n/x = n/2 + n/3 + n/4 + \cdots + n/n = O(n \log n).$$

实际上，算法更为高效，因为只有当 x 为素数时才会执行内层循环。可以证明，算法的运行时间仅为 $O(n \log \log n)$，非常接近 $O(n)$。

Euclid 算法

数 a 与 b 的**最大公约数** $\gcd(a,b)$，是同时整除 a 和 b 的最大整数；a 与 b 的**最小公倍数** $\operatorname{lcm}(a,b)$，是同时被 a 和 b 整除的最小正整数。例如，$\gcd(24,36)=12$，$\operatorname{lcm}(24,36)=72$。

最大公约数与最小公倍数满足：

$$\operatorname{lcm}(a,b)=\frac{ab}{\gcd(a,b)}$$

Euclid 算法[1]可以高效求出两个数的最大公约数。算法基于公式：

$$\gcd(a,b)=\begin{cases}a & b=0\\ \gcd(b,a \bmod b) & b\neq 0\end{cases}$$

例如，

$$\gcd(24,36)=\gcd(36,24)=\gcd(24,12)=\gcd(12,0)=12.$$

算法可以实现如下：

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a%b);
}
```

可以证明，Euclid 算法的时间复杂度为 $O(\log n)$，其中 $n=\min(a,b)$。当 a、b 是相邻 Fibonacci 数时达到最坏情况。例如，

$$\gcd(13,8)=\gcd(8,5)=\gcd(5,3)=\gcd(3,2)=\gcd(2,1)=\gcd(1,0)=1.$$

Euler 函数

若 $\gcd(a,b)=1$，则称 a 与 b **互质**。**Euler 函数** $\varphi(n)$ 表示 1 到 n 之间与 n 互质的整数个数。例如，$\varphi(12)=4$，因为 1、5、7、11 与 12 互质。

可以根据 n 的质因数分解用以下公式计算 $\varphi(n)$：

$$\varphi(n)=\prod_{i=1}^{k}p_i^{\alpha_i-1}(p_i-1).$$

例如，$\varphi(12)=2^1\cdot(2-1)\cdot3^0\cdot(3-1)=4$。注意，如果 n 是素数，则 $\varphi(n)=n-1$。

[1]Euclid 是约生活于公元前 300 年的希腊数学家。这可能是历史上最早的已知算法。

21.2 模运算

在**模运算**中，数的范围被限制为 $0, 1, 2, \ldots, m-1$，其中 m 是常数。每个数 x 用 $x \bmod m$ 表示，即 x 除以 m 的余数。例如，当 $m = 17$ 时，75 表示为 $75 \bmod 17 = 7$。

通常可以先取模再进行计算。特别地，下列公式成立：

$$\begin{aligned}
(x + y) \bmod m &= (x \bmod m + y \bmod m) \bmod m \\
(x - y) \bmod m &= (x \bmod m - y \bmod m) \bmod m \\
(x \cdot y) \bmod m &= (x \bmod m \cdot y \bmod m) \bmod m \\
x^n \bmod m &= (x \bmod m)^n \bmod m
\end{aligned}$$

快速幂取模

经常需要高效计算 $x^n \bmod m$。利用下面的递归，可以在 $O(\log n)$ 时间内完成：

$$x^n = \begin{cases} 1 & n = 0 \\ x^{n/2} \cdot x^{n/2} & n \text{ is even} \\ x^{n-1} \cdot x & n \text{ is odd} \end{cases}$$

关键在于，当 n 为偶数时，$x^{n/2}$ 只计算一次。因为每当 n 为偶数时都会减半，这就保证了算法的时间复杂度为 $O(\log n)$。

下面的函数计算 $x^n \bmod m$：

```
int modpow(int x, int n, int m) {
    if (n == 0) return 1%m;
    long long u = modpow(x,n/2,m);
    u = (u*u)%m;
    if (n%2 == 1) u = (u*x)%m;
    return u;
}
```

Fermat 定理与 Euler 定理

Fermat 小定理指出，当 m 为素数且 x 与 m 互质时，

$$x^{m-1} \bmod m = 1$$

由此还可得到

$$x^k \bmod m = x^{k \bmod (m-1)} \bmod m.$$

更一般地，**Euler 定理**指出，当 x 与 m 互质时，

$$x^{\varphi(m)} \bmod m = 1$$

Fermat 小定理是 Euler 定理的推论，因为当 m 为素数时，$\varphi(m) = m - 1$。

模逆元

模 m 意义下 x 的逆元是满足下式的数 x^{-1}：

$$xx^{-1} \bmod m = 1.$$

例如，当 $x = 6$、$m = 17$ 时，$x^{-1} = 3$，因为 $6 \cdot 3 \bmod 17 = 1$。

利用模逆元，可以在模 m 意义下进行除法，因为除以 x 等价于乘以 x^{-1}。例如，要计算 $36/6 \bmod 17$，可以计算 $2 \cdot 3 \bmod 17$，因为 $36 \bmod 17 = 2$ 且 $6^{-1} \bmod 17 = 3$。

不过，模逆元并不总是存在。例如，当 $x = 2$、$m = 4$ 时，方程

$$xx^{-1} \bmod m = 1$$

无解，因为 2 的倍数都是偶数，模 4 的余数不可能为 1。事实上，$x^{-1} \bmod m$ 存在当且仅当 x 与 m 互质。

如果模逆元存在，可以用公式

$$x^{-1} = x^{\varphi(m)-1}.$$

当 m 为素数时，公式变为

$$x^{-1} = x^{m-2}.$$

例如，

$$6^{-1} \bmod 17 = 6^{17-2} \bmod 17 = 3.$$

利用快速幂取模算法，可以通过该公式高效计算模逆元。这个公式可由 Euler 定理推出。首先，模逆元应满足：

$$xx^{-1} \bmod m = 1.$$

另一方面，根据 Euler 定理，

$$x^{\varphi(m)} \bmod m = xx^{\varphi(m)-1} \bmod m = 1,$$

因此 x^{-1} 与 $x^{\varphi(m)-1}$ 在模 m 意义下相等。

计算机算术

在程序中，无符号整数按模 2^k 表示，其中 k 是数据类型的位数。常见的结果是：当数值过大时会发生回绕。

例如，在 C++ 中，unsigned int 类型按模 2^{32} 表示。下面的代码声明一个值为 123456789 的 unsigned int 变量，再将其与自身相乘，结果为 $123456789^2 \bmod 2^{32} = 2537071545$。

```
unsigned int x = 123456789;
cout << x*x << "\n"; // 2537071545
```

21.3 求解方程

不定方程

不定方程是形如

$$ax + by = c,$$

的方程，其中 a、b、c 为常数，需要求出 x、y，且方程中的所有数都必须是整数。例如，方程 $5x + 2y = 11$ 的一组解是 $x = 3$、$y = -2$。

利用 Euclid 算法可以高效求解不定方程。可以扩展 Euclid 算法，使其求出满足下式的 x、y：

$$ax + by = \gcd(a, b)$$

当且仅当 c 能被 $\gcd(a, b)$ 整除时，不定方程有整数解。

例如，求满足下式的 x、y：

$$39x + 15y = 12$$

因为 $\gcd(39, 15) = 3$ 且 $3 \mid 12$，所以方程有解。Euclid 算法计算 39 与 15 的最大公约数时，产生如下调用序列：

$$\gcd(39, 15) = \gcd(15, 9) = \gcd(9, 6) = \gcd(6, 3) = \gcd(3, 0) = 3$$

它对应以下等式：

$$\begin{aligned} 39 - 2 \cdot 15 &= 9 \\ 15 - 1 \cdot 9 &= 6 \\ 9 - 1 \cdot 6 &= 3 \end{aligned}$$

利用这些等式可以推出

$$39 \cdot 2 + 15 \cdot (-5) = 3$$

将其乘以 4，得到

$$39 \cdot 8 + 15 \cdot (-20) = 12,$$

所以方程的一组解为 $x = 8$、$y = -20$。

不定方程的解并不唯一；已知一组解后，可以构造无穷多组解。如果 (x, y) 是一组解，那么所有形如

$$(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)})$$

的数对也都是解，其中 k 为任意整数。

中国剩余定理

中国剩余定理用于求解形如

$$\begin{aligned} x &= a_1 \bmod m_1 \\ x &= a_2 \bmod m_2 \\ &\cdots \\ x &= a_n \bmod m_n \end{aligned}$$

的方程组，其中 $m_1, m_2, \ldots, m_n$ 两两互质。

令 x_m^{-1} 表示 x 模 m 的逆元，并令

$$X_k = \frac{m_1 m_2 \cdots m_n}{m_k}.$$

使用这一记号，方程组的一组解为

$$x = a_1 X_1 X_{1m_1}^{-1} + a_2 X_2 X_{2m_2}^{-1} + \cdots + a_n X_n X_{nm_n}^{-1}.$$

在该解中，对于每个 $k = 1, 2, \ldots, n$，都有

$$a_k X_k X_{km_k}^{-1} \bmod m_k = a_k,$$

因为

$$X_k X_{km_k}^{-1} \bmod m_k = 1.$$

而和式中的其他各项都能被 m_k 整除，不影响余数，因此 $x \bmod m_k = a_k$。

例如，方程组

$$\begin{aligned} x &= 3 \bmod 5 \\ x &= 4 \bmod 7 \\ x &= 2 \bmod 3 \end{aligned}$$

的一组解为

$$3 \cdot 21 \cdot 1 + 4 \cdot 15 \cdot 1 + 2 \cdot 35 \cdot 2 = 263.$$

找到一组解 x 后，可以构造无穷多组其他解，因为所有形如

$$x + m_1 m_2 \cdots m_n$$

的数都是解。

21.4 其他结论

Lagrange 四平方定理

Lagrange 四平方定理指出，每个正整数都能表示为四个平方数之和，即 $a^2 + b^2 + c^2 + d^2$。例如，123 可以表示为 $8^2 + 5^2 + 5^2 + 3^2$。

Zeckendorf 定理

Zeckendorf 定理指出，每个正整数都有唯一一种表示，可以写成若干个互不相同且任意两个都不相邻的 Fibonacci 数之和。例如，74 可以表示为 $55 + 13 + 5 + 1$。

勾股数

勾股数是满足勾股定理 $a^2 + b^2 = c^2$ 的三元组 (a, b, c)，即存在一条三边长分别为 a、b、c 的直角三角形。例如，$(3, 4, 5)$ 是一组勾股数。

若 (a, b, c) 是一组勾股数，则当 $k > 1$ 时，所有 (ka, kb, kc) 也都是勾股数。若 a、b、c 的最大公约数为 1，则称这组勾股数为*本原勾股数*。所有勾股数都可由本原勾股数乘以某个倍数 k 得到。

Euclid 公式可以生成所有本原勾股数。每组本原勾股数都形如

$$(n^2 - m^2, 2nm, n^2 + m^2),$$

其中 $0 < m < n$，n 与 m 互质，并且二者至少有一个是偶数。例如，当 $m = 1$、$n = 2$ 时，该公式产生最小的勾股数

$$(2^2 - 1^2, 2 \cdot 2 \cdot 1, 2^2 + 1^2) = (3, 4, 5).$$

Wilson 定理

Wilson 定理指出，n 为素数当且仅当

$$(n-1)! \bmod n = n-1.$$

例如，11 是素数，因为

$$10! \bmod 11 = 10,$$

而 12 不是素数，因为

$$11! \bmod 12 = 0 \neq 11.$$

因此，Wilson 定理可用于判断一个数是否为素数。不过在实践中，该定理无法用于很大的 n，因为当 n 很大时，计算 $(n-1)!$ 十分困难。

第二十二章　组合数学

组合数学研究对对象组合进行计数的方法。通常，目标是在不逐一生成每种组合的情况下，高效计算组合数量。

例如，考虑计算把整数 n 表示为若干正整数之和的方法数。数字 4 共有 8 种表示：

- $1+1+1+1$
- $1+1+2$
- $1+2+1$
- $2+1+1$
- $2+2$
- $3+1$
- $1+3$
- 4

组合计数问题往往可以用递归函数解决。在本题中，定义 $f(n)$ 表示 n 的表示方法数。根据上例，$f(4)=8$。函数值可按如下递推式计算：

$$f(n)=\begin{cases}1 & n=0\\ f(0)+f(1)+\cdots+f(n-1) & n>0\end{cases}$$

边界条件为 $f(0)=1$，因为空和表示数字 0。当 $n>0$ 时，枚举和式的第一个数。如果第一个数为 k，则余下部分有 $f(n-k)$ 种表示。因此，需要对所有满足 $k<n$ 的 $f(n-k)$ 求和。

该函数最初的若干值为：

$$\begin{aligned}f(0) &= 1\\ f(1) &= 1\\ f(2) &= 2\\ f(3) &= 4\\ f(4) &= 8\end{aligned}$$

有时，递推公式可以改写成闭式公式。本题中，

$$f(n)=2^{n-1},$$

这是因为和式中有 $n-1$ 个可能放置加号的位置，可以任意选择其中一个子集。

22.1 二项式系数

二项式系数 $\binom{n}{k}$ 表示从含有 n 个元素的集合中选出 k 个元素组成子集的方法数。例如，$\binom{5}{3}=10$，因为集合 $\{1,2,3,4,5\}$ 有 10 个包含 3 个元素的子集：

$$\{1,2,3\},\{1,2,4\},\{1,2,5\},\{1,3,4\},\{1,3,5\},\{1,4,5\},\{2,3,4\},\{2,3,5\},\{2,4,5\},\{3,4,5\}$$

公式一

二项式系数可以按如下递推式计算：

$$\binom{n}{k}=\binom{n-1}{k-1}+\binom{n-1}{k}$$

其思想是固定集合中的一个元素 x。如果子集包含 x，还需从其余 $n-1$ 个元素中选择 $k-1$ 个；如果子集不包含 x，则需从其余 $n-1$ 个元素中选择 k 个。

递推的边界条件为

$$\binom{n}{0}=\binom{n}{n}=1,$$

因为构造空集以及包含全部元素的子集都恰好只有一种方法。

公式二

二项式系数还可以按下式计算：

$$\binom{n}{k}=\frac{n!}{k!(n-k)!}.$$

n 个元素共有 $n!$ 种排列。枚举所有排列，并把每个排列的前 k 个元素放入子集。由于子集内部和子集外部元素的顺序都不重要，结果需要除以 $k!$ 和 $(n-k)!$。

性质

二项式系数满足

$$\binom{n}{k}=\binom{n}{n-k},$$

因为实际上是把含有 n 个元素的集合分为两个子集：第一个包含 k 个元素，第二个包含 $n-k$ 个元素。

二项式系数之和为

$$\binom{n}{0}+\binom{n}{1}+\binom{n}{2}+\ldots+\binom{n}{n}=2^n.$$

把二项式 $(a+b)$ 展开到 n 次幂，就能看出“二项式系数”这一名称的由来：

$$(a+b)^n=\binom{n}{0}a^nb^0+\binom{n}{1}a^{n-1}b^1+\ldots+\binom{n}{n-1}a^1b^{n-1}+\binom{n}{n}a^0b^n.$$

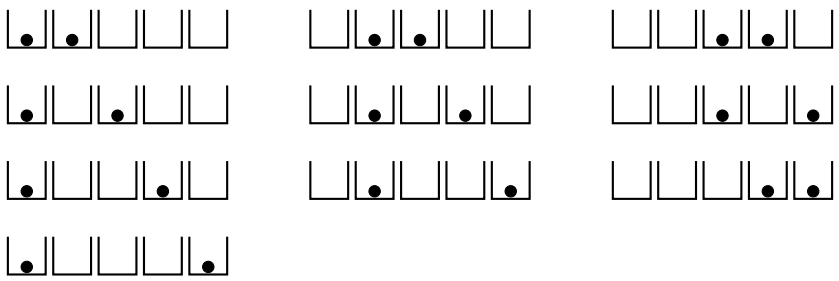
二项式系数也出现在 **Pascal 三角**中，其中每个数等于它上方两个数之和：

$$\begin{array}{ccccccccccc}
 & & & & & 1 & & & & & \\
 & & & & 1 & & 1 & & & & \\
 & & & 1 & & 2 & & 1 & & & \\
 & & 1 & & 3 & & 3 & & 1 & & \\
 & 1 & & 4 & & 6 & & 4 & & 1 & \\
\cdots & & \cdots & & \cdots & & \cdots & & \cdots & & \cdots
\end{array}$$

盒子与球

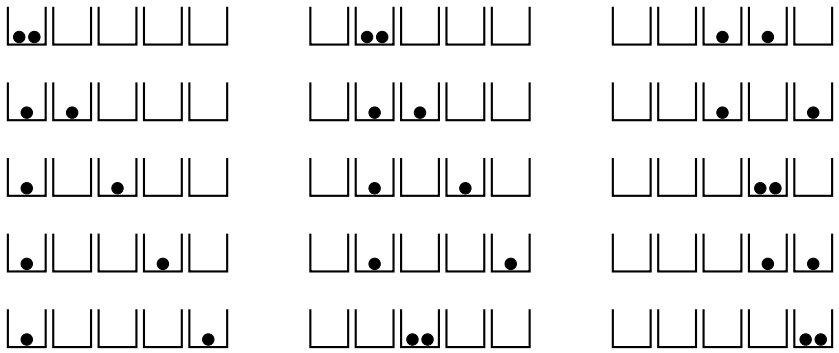
“盒子与球”是一种实用模型，用于计算把 k 个球放入 n 个盒子的方法数。考虑以下三种情形：

情形一：每个盒子至多放一个球。例如，当 $n=5$、$k=2$ 时，共有 10 种方案：



在这种情形下，答案就是二项式系数 $\binom{n}{k}$。

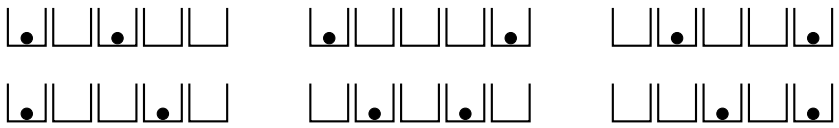
情形二：一个盒子可以放多个球。例如，当 $n=5$、$k=2$ 时，共有 15 种方案：



把球放入盒子的过程可以表示为由符号“o”和“→”组成的字符串。初始时位于最左侧盒子；“o”表示在当前盒子中放一个球，“→”表示向右移动到下一个盒子。

采用这种表示，每种方案对应一个含 k 个“o”和 $n-1$ 个“→”的字符串。例如，上图右上角的方案对应字符串“→ → o → o →”。因此，方案数为 $\binom{k+n-1}{k}$。

情形三：每个盒子至多放一个球，并且任意两个相邻盒子不能同时有球。例如，当 $n=5$、$k=2$ 时，共有 6 种方案：



在这种情形下，可以先放置 k 个球，并在每两个相邻的有球盒子之间预留一个空盒子。余下任务是为剩余空盒子选择位置。这样的空盒子有 $n-2k+1$ 个，可放置的位置有 $k+1$ 个。因此，利用情形二的公式，方案数为 $\binom{n-k+1}{n-2k+1}$。

多项式系数

多项式系数

$$\binom{n}{k_1, k_2, \ldots, k_m} = \frac{n!}{k_1! k_2! \cdots k_m!},$$

表示把 n 个元素划分成大小分别为 $k_1, k_2, \ldots, k_m$ 的子集的方法数，其中 $k_1 + k_2 + \cdots + k_m = n$。多项式系数可以看作二项式系数的推广；当 $m=2$ 时，上式就是二项式系数公式。

22.2 Catalan 数

Catalan 数 C_n 等于由 n 个左括号和 n 个右括号组成的合法括号序列数量。

例如，$C_3 = 5$，因为用三个左括号和三个右括号可以构造以下括号序列：

- ()()()
- (())()
- ()(())
- ((()))
- (()())

括号序列

什么是合法括号序列？以下规则精确定义了所有合法括号序列：

- 空括号序列合法。
- 若序列 A 合法，则序列 (A) 也合法。
- 若序列 A、B 合法，则序列 AB 也合法。

合法括号序列还有另一种刻画：它的任意前缀中，左括号数量都不少于右括号数量；并且完整序列中左右括号数量相等。

公式一

Catalan 数可以用公式

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-i-1}.$$

计算。求和枚举把序列分成两个合法部分的方法，其中第一部分非空且尽可能短。对于每个 i，第一部分包含 $i+1$ 对括号，序列数量是以下两项的乘积：

- C_i：使用第一部分除最外层括号之外的括号构造序列的方法数；
- C_{n-i-1}：使用第二部分的括号构造序列的方法数。

边界条件为 $C_0 = 1$，因为用零对括号可以构造一个空括号序列。

公式二

Catalan 数也可以用二项式系数计算：

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

该公式可解释如下。

由 n 个左括号和 n 个右括号构成的括号序列（不要求合法）共有 $\binom{2n}{n}$ 个。下面计算其中不合法序列的数量。

如果括号序列不合法，它必有一个前缀，其中右括号数量超过左括号数量。把这样一个最短前缀中的每个括号反转。例如，序列 ())()(含有前缀 ())，反转该前缀后得到)((()(。

所得序列包含 $n+1$ 个左括号和 $n-1$ 个右括号。这类序列有 $\binom{2n}{n+1}$ 个，恰好等于不合法括号序列的数量。因此，合法括号序列数为

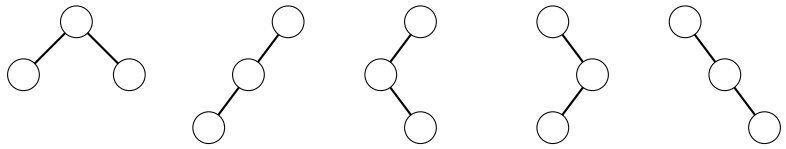
$$\binom{2n}{n} - \binom{2n}{n+1} = \binom{2n}{n} - \frac{n}{n+1}\binom{2n}{n} = \frac{1}{n+1}\binom{2n}{n}.$$

树的计数

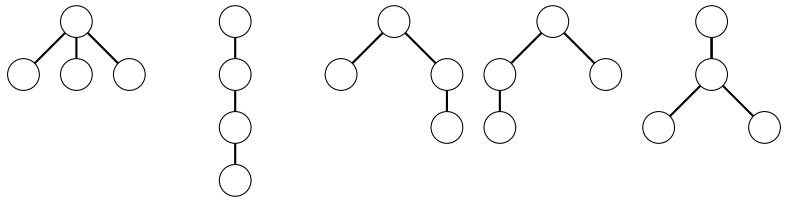
Catalan 数还与树有关：

- 含 n 个结点的二叉树有 C_n 棵；
- 含 n 个结点的有根树有 C_{n-1} 棵。

例如，由 $C_3 = 5$ 可知，对应的二叉树为



而对应的有根树为

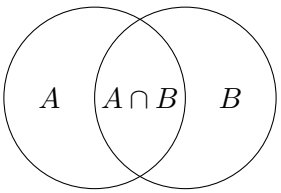


22.3 容斥原理

容斥原理是一种计数技术，可以在已知各交集大小时计算集合并集的大小，也可以反过来计算。一个简单例子是公式

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

其中 A、B 为集合，$|X|$ 表示 X 的大小。该公式可用下图说明：

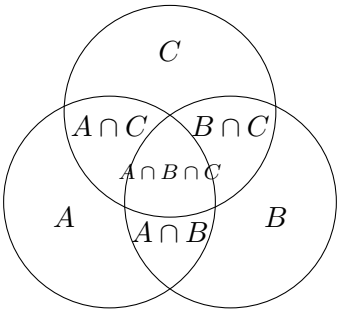


目标是计算并集 $A \cup B$ 的大小，它对应至少属于一个圆的区域面积。由图可见，可以先把 A 与 B 的面积相加，再减去 $A \cap B$ 的面积，从而得到 $A \cup B$ 的面积。

同样的思想也适用于更多集合。当有三个集合时，容斥公式为

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

对应图示为



一般情况下，可以枚举由 $X_1, X_2, \ldots, X_n$ 中若干集合组成的所有交集，计算并集 $X_1 \cup X_2 \cup \cdots \cup X_n$ 的大小。若交集包含奇数个集合，就把它的大小加到答案中；否则从答案中减去。

反过来，也有根据并集大小计算交集大小的类似公式。例如，

$$|A \cap B| = |A| + |B| - |A \cup B|$$

and

$$|A \cap B \cap C| = |A| + |B| + |C| - |A \cup B| - |A \cup C| - |B \cup C| + |A \cup B \cup C|.$$

错排

例如，计算元素 $\{1, 2, \ldots, n\}$ 的**错排**数量，即没有任何元素留在原位置的排列数。当 $n = 3$ 时，共有两个错排：$(2, 3, 1)$ 和 $(3, 1, 2)$。

一种解法是使用容斥原理。令 X_k 为第 k 个位置恰好放置元素 k 的排列集合。当 $n = 3$ 时，各集合如下：

$$\begin{aligned} X_1 &= \{(1,2,3),(1,3,2)\} \\ X_2 &= \{(1,2,3),(3,2,1)\} \\ X_3 &= \{(1,2,3),(2,1,3)\} \end{aligned}$$

利用这些集合，错排数等于

$$n! - |X_1 \cup X_2 \cup \cdots \cup X_n|,$$

因此只需计算并集大小。利用容斥原理，可将其归约为高效计算各交集大小。当 $n = 3$ 时，$|X_1 \cup X_2 \cup X_3|$ 为

$$\begin{aligned} & |X_1| + |X_2| + |X_3| - |X_1 \cap X_2| - |X_1 \cap X_3| - |X_2 \cap X_3| + |X_1 \cap X_2 \cap X_3| \\ = \;& 2 + 2 + 2 - 1 - 1 - 1 + 1 \\ = \;& 4, \end{aligned}$$

所以错排数为 $3! - 4 = 2$。

这个问题也可以不用容斥原理求解。令 $f(n)$ 表示 $\{1,2,\ldots,n\}$ 的错排数，可以使用递推式：

$$f(n)=\begin{cases}0 & n=1\\ 1 & n=2\\ (n-1)(f(n-2)+f(n-1)) & n>2\end{cases}$$

推导该公式时，考虑错排中元素 1 被如何替换。用于替换元素 1 的元素 x 有 $n-1$ 种选择；对每种选择，又有两种情况：

情况一：再用元素 1 替换元素 x。此后只需构造其余 $n-2$ 个元素的错排。

情况二：用 1 以外的某个元素替换元素 x。此时需要构造 $n-1$ 个元素的错排，因为不能再用元素 1 替换元素 x，而所有其他元素仍必须改变位置。

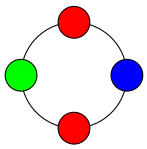
22.4 Burnside 引理

Burnside 引理可用于计算组合数，并让每组相互对称的组合只计入一个代表。Burnside 引理指出，组合数量为

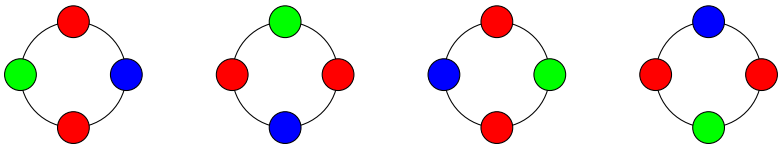
$$\sum_{k=1}^{n}\frac{c(k)}{n},$$

其中共有 n 种改变组合位置的方式，应用第 k 种方式后保持不变的组合有 $c(k)$ 个。

例如，计算由 n 颗珠子组成的项链数量，每颗珠子有 m 种可选颜色。如果两条项链旋转后相同，就认为它们对称。例如，项链



有以下与之对称的项链：



一条项链的位置有 n 种改变方式，因为可以顺时针旋转 $0,1,\ldots,n-1$ 步。旋转 0 步时，所有 m^n 条项链都保持不变；旋转 1 步时，只有全部珠子颜色相同的 m 条项链保持不变。

更一般地，旋转 k 步时，共有

$$m^{\gcd(k,n)}$$

条项链保持不变，其中 $\gcd(k,n)$ 是 k 与 n 的最大公约数。原因是大小为 $\gcd(k,n)$ 的珠子块会相互替换。因此，根据 Burnside 引理，项链数量为

$$\sum_{i=0}^{n-1} \frac{m^{\gcd(i,n)}}{n}.$$

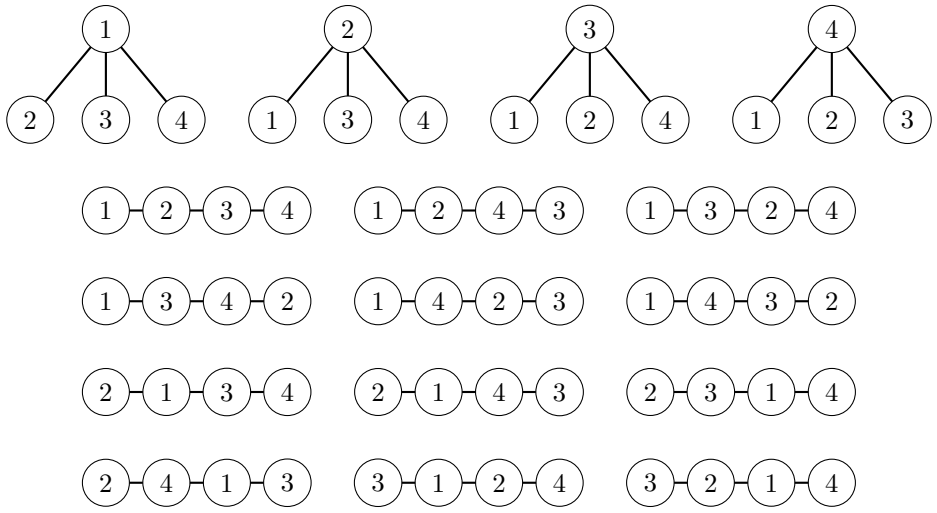
例如，用 3 种颜色构成长度为 4 的项链共有

$$\frac{3^4+3+3^2+3}{4} = 24.$$

22.5 Cayley 公式

Cayley 公式指出，含有 n 个结点的有标号树共有 n^{n-2} 棵。结点编号为 $1,2,\ldots,n$；只要两棵树的结构或标号不同，就视为不同的树。

例如，当 $n=4$ 时，有标号树共有 $4^{4-2}=16$ 棵：

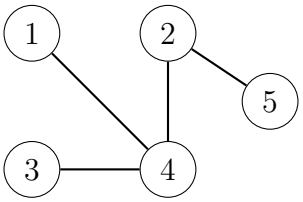


下面说明如何用 Prüfer 编码推导 Cayley 公式。

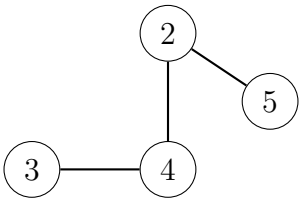
Prüfer 编码

Prüfer 编码是由 $n-2$ 个数构成、用于描述一棵有标号树的序列。构造编码时，要从树中依次删除 $n-2$ 个叶结点。每一步删除标号最小的叶结点，并把它唯一邻居的标号加入编码。

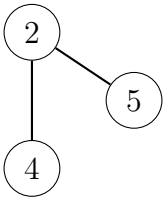
例如，计算下图的 Prüfer 编码：



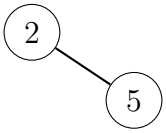
首先删除结点 1，并把结点 4 加入编码：



然后删除结点 3，并把结点 4 加入编码：



最后删除结点 4，并把结点 2 加入编码：



因此，该图的 Prüfer 编码为 $[4, 4, 2]$。

任意树都能构造 Prüfer 编码；更重要的是，也能从 Prüfer 编码恢复原树。因此，含 n 个结点的有标号树数量等于长度为 $n-2$、每项取值为 $1 \ldots n$ 的 Prüfer 编码数量，即 n^{n-2}。

第二十三章　矩阵

矩阵是一种数学概念，对应程序中的二维数组。例如，

$$A = \begin{bmatrix} 6 & 13 & 7 & 4 \\ 7 & 0 & 8 & 2 \\ 9 & 5 & 4 & 18 \end{bmatrix}$$

是一个 3×4 矩阵，即它有 3 行 4 列。记号 $[i, j]$ 表示矩阵中第 i 行、第 j 列的元素。例如，在上述矩阵中，$A[2, 3] = 8$，$A[3, 1] = 9$。

矩阵的一种特殊情形是**向量**，即大小为 $n \times 1$ 的一维矩阵。例如，

$$V = \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix}$$

是一个包含三个元素的向量。

矩阵 A 的**转置** A^T 通过交换 A 的行与列得到，即 $A^T[i, j] = A[j, i]$：

$$A^T = \begin{bmatrix} 6 & 7 & 9 \\ 13 & 0 & 5 \\ 7 & 8 & 4 \\ 4 & 2 & 18 \end{bmatrix}$$

如果矩阵的行数和列数相同，就称它为**方阵**。例如，下面的矩阵是方阵：

$$S = \begin{bmatrix} 3 & 12 & 4 \\ 5 & 9 & 15 \\ 0 & 2 & 4 \end{bmatrix}$$

23.1　运算

当矩阵 A、B 大小相同时，可以定义它们的和 $A + B$。结果矩阵中的每个元素等于 A、B 对应位置元素之和。

例如，

$$\begin{bmatrix} 6 & 1 & 4 \\ 3 & 9 & 2 \end{bmatrix} + \begin{bmatrix} 4 & 9 & 3 \\ 8 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 6+4 & 1+9 & 4+3 \\ 3+8 & 9+1 & 2+3 \end{bmatrix} = \begin{bmatrix} 10 & 10 & 7 \\ 11 & 10 & 5 \end{bmatrix}.$$

矩阵 A 乘以数值 x，表示把 A 的每个元素都乘以 x。例如，

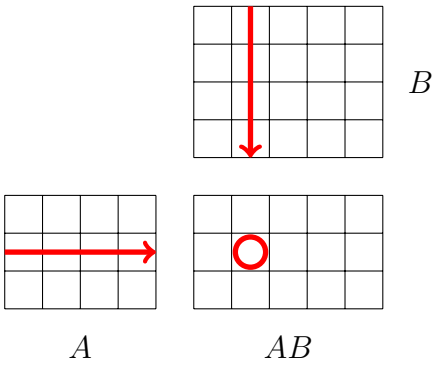
$$2 \cdot \begin{bmatrix} 6 & 1 & 4 \\ 3 & 9 & 2 \end{bmatrix} = \begin{bmatrix} 2 \cdot 6 & 2 \cdot 1 & 2 \cdot 4 \\ 2 \cdot 3 & 2 \cdot 9 & 2 \cdot 2 \end{bmatrix} = \begin{bmatrix} 12 & 2 & 8 \\ 6 & 18 & 4 \end{bmatrix}.$$

矩阵乘法

当 A 的大小为 $a \times n$、B 的大小为 $n \times b$ 时，即 A 的列数等于 B 的行数时，可以定义矩阵乘积 AB。结果是一个 $a \times b$ 矩阵，其元素按公式

$$AB[i,j] = \sum_{k=1}^{n} A[i,k] \cdot B[k,j].$$

计算。也就是说，AB 的每个元素都是 A 的一行与 B 的一列对应元素乘积之和，如下图所示：



例如，

$$\begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} \cdot \begin{bmatrix} 1 & 6 \\ 2 & 9 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 4 \cdot 2 & 1 \cdot 6 + 4 \cdot 9 \\ 3 \cdot 1 + 9 \cdot 2 & 3 \cdot 6 + 9 \cdot 9 \\ 8 \cdot 1 + 6 \cdot 2 & 8 \cdot 6 + 6 \cdot 9 \end{bmatrix} = \begin{bmatrix} 9 & 42 \\ 21 & 99 \\ 20 & 102 \end{bmatrix}.$$

矩阵乘法满足结合律，即 $A(BC) = (AB)C$；但通常不满足交换律，即一般有 $AB \neq BA$。

单位矩阵是主对角线元素全为 1、其他元素全为 0 的方阵。例如，下面是 3×3 单位矩阵：

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

矩阵乘以单位矩阵后保持不变。例如，

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} \quad \text{且} \quad \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix}.$$

使用直接算法，可以在 $O(n^3)$ 时间内计算两个 $n \times n$ 矩阵的乘积。也有更高效的矩阵乘法算法[1]，但它们主要具有理论意义，在算法竞赛中通常没有必要使用。

矩阵幂

当 A 为方阵时，可以定义它的幂 A^k。该定义基于矩阵乘法：

$$A^k = \underbrace{A \cdot A \cdot A \cdots A}_{k \text{ 次}}$$

例如，

$$\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^3 = \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} = \begin{bmatrix} 48 & 165 \\ 33 & 114 \end{bmatrix}.$$

此外，A^0 是单位矩阵。例如，

$$\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

使用第 21.2 节的快速幂算法，可以在 $O(n^3 \log k)$ 时间内高效计算 A^k。例如，

$$\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^8 = \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^4 \cdot \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^4.$$

行列式

当 A 为方阵时，可以定义它的**行列式** $\det(A)$。如果 A 的大小为 1×1，则 $\det(A) = A[1,1]$。更大矩阵的行列式按公式递归计算：

$$\det(A) = \sum_{j=1}^{n} A[1,j]C[1,j],$$

[1]第一个此类算法是 1969 年发表的 Strassen 算法 [63]，时间复杂度为 $O(n^{2.80735})$；本书原稿写作时最优的算法 [27] 为 $O(n^{2.37286})$。

其中 $C[i,j]$ 是 A 在位置 $[i,j]$ 的**代数余子式**，按公式

$$C[i,j] = (-1)^{i+j} \det(M[i,j]),$$

计算；$M[i,j]$ 是从 A 中删除第 i 行和第 j 列所得矩阵。由于代数余子式中含因子 $(-1)^{i+j}$，各项符号正负交替。例如，

$$\det(\begin{bmatrix} 3 & 4 \\ 1 & 6 \end{bmatrix}) = 3 \cdot 6 - 4 \cdot 1 = 14$$

and

$$\det(\begin{bmatrix} 2 & 4 & 3 \\ 5 & 1 & 6 \\ 7 & 2 & 4 \end{bmatrix}) = 2 \cdot \det(\begin{bmatrix} 1 & 6 \\ 2 & 4 \end{bmatrix}) - 4 \cdot \det(\begin{bmatrix} 5 & 6 \\ 7 & 4 \end{bmatrix}) + 3 \cdot \det(\begin{bmatrix} 5 & 1 \\ 7 & 2 \end{bmatrix}) = 81.$$

A 的行列式可以判断是否存在满足 $A \cdot A^{-1} = I$ 的**逆矩阵** A^{-1}，其中 I 为单位矩阵。A^{-1} 存在当且仅当 $\det(A) \neq 0$，并可用公式

$$A^{-1}[i,j] = \frac{C[j,i]}{det(A)}.$$

计算。例如，

$$\underbrace{\begin{bmatrix} 2 & 4 & 3 \\ 5 & 1 & 6 \\ 7 & 2 & 4 \end{bmatrix}}_{A} \cdot \underbrace{\frac{1}{81} \begin{bmatrix} -8 & -10 & 21 \\ 22 & -13 & 3 \\ 3 & 24 & -18 \end{bmatrix}}_{A^{-1}} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{I}.$$

23.2 线性递推

线性递推函数 $f(n)$ 的初始值为 $f(0), f(1), \ldots, f(k-1)$，更大的值按公式

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \ldots + c_k f(n-k),$$

递推计算，其中 $c_1, c_2, \ldots, c_k$ 为常系数。

使用动态规划依次计算 $f(0), f(1), \ldots, f(n)$，可以在 $O(kn)$ 时间内求出任意 $f(n)$。不过，当 k 较小时，利用矩阵运算可在 $O(k^3 \log n)$ 时间内更高效地计算 $f(n)$。

Fibonacci 数

线性递推的一个简单例子是定义 Fibonacci 数的函数：

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

此时 $k = 2$，且 $c_1 = c_2 = 1$。

为了高效计算 Fibonacci 数，把递推公式表示为满足下式的 2×2 方阵 X：

$$X \cdot \begin{bmatrix} f(i) \\ f(i+1) \end{bmatrix} = \begin{bmatrix} f(i+1) \\ f(i+2) \end{bmatrix}$$

也就是说，把 $f(i)$ 和 $f(i+1)$ 作为 X 的“输入”，X 据此计算 $f(i+1)$ 和 $f(i+2)$。满足条件的矩阵为

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

例如，

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} f(5) \\ f(6) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 8 \end{bmatrix} = \begin{bmatrix} 8 \\ 13 \end{bmatrix} = \begin{bmatrix} f(6) \\ f(7) \end{bmatrix}.$$

因此，可以用公式

$$\begin{bmatrix} f(n) \\ f(n+1) \end{bmatrix} = X^n \cdot \begin{bmatrix} f(0) \\ f(1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

计算 $f(n)$。X^n 可以在 $O(\log n)$ 时间内求出，因此 $f(n)$ 也能在 $O(\log n)$ 时间内计算。

一般情形

现在考虑 $f(n)$ 为任意线性递推的一般情形。目标仍是构造满足下式的矩阵 X：

$$X \cdot \begin{bmatrix} f(i) \\ f(i+1) \\ \vdots \\ f(i+k-1) \end{bmatrix} = \begin{bmatrix} f(i+1) \\ f(i+2) \\ \vdots \\ f(i+k) \end{bmatrix}.$$

这样的矩阵为

$$X = \begin{bmatrix} 0 & 1 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 1 \\ c_k & c_{k-1} & c_{k-2} & c_{k-3} & \cdots & c_1 \end{bmatrix}.$$

前 $k-1$ 行中，每行除一个元素为 1 外，其余元素均为 0。这些行依次把 $f(i)$ 替换为 $f(i+1)$、把 $f(i+1)$ 替换为 $f(i+2)$，依此类推。最后一行包含递推式的系数，用于计算新值 $f(i+k)$。

现在，可以用公式

$$\begin{bmatrix} f(n) \\ f(n+1) \\ \vdots \\ f(n+k-1) \end{bmatrix} = X^n \cdot \begin{bmatrix} f(0) \\ f(1) \\ \vdots \\ f(k-1) \end{bmatrix}.$$

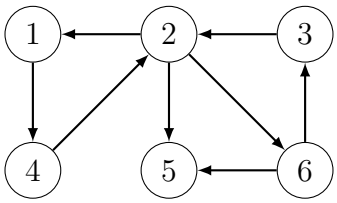
在 $O(k^3 \log n)$ 时间内计算 $f(n)$。

23.3 图与矩阵

路径计数

图的邻接矩阵的幂有一个有趣性质。当 V 是无权图的邻接矩阵时，矩阵 V^n 的元素给出了图中各结点对之间恰好包含 n 条边的路径数量。

例如，对于图



其邻接矩阵为

$$V = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

例如，矩阵

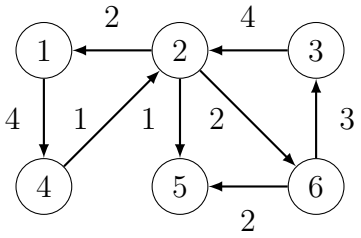
$$V^4 = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 \\ 2 & 0 & 0 & 0 & 2 & 2 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

包含各结点对之间恰好有 4 条边的路径数量。例如，$V^4[2,5] = 2$，因为从结点 2 到结点 5 恰好有两条包含 4 条边的路径：$2 \to 1 \to 4 \to 2 \to 5$ 以及 $2 \to 6 \to 3 \to 2 \to 5$.

最短路

在带权图中利用类似思想，可以计算每对结点之间恰好包含 n 条边的最短路径长度。为此，需要重新定义矩阵乘法，使其不再统计路径数量，而是最小化路径长度。

例如，考虑下图：



构造一个邻接矩阵，其中 ∞ 表示不存在边，其他数值表示边权。矩阵为

$$V = \begin{bmatrix} \infty & \infty & \infty & 4 & \infty & \infty \\ 2 & \infty & \infty & \infty & 1 & 2 \\ \infty & 4 & \infty & \infty & \infty & \infty \\ \infty & 1 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 3 & \infty & 2 & \infty \end{bmatrix}.$$

不再使用公式

$$AB[i,j] = \sum_{k=1}^{n} A[i,k] \cdot B[k,j]$$

而使用公式

$$AB[i,j] = \min_{k=1}^{n} A[i,k] + B[k,j]$$

进行矩阵乘法。也就是说，以取最小值代替求和，以元素相加代替相乘。经过这一修改，矩阵幂就对应图中的最短路。

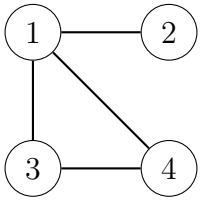
例如，由于

$$V^4 = \begin{bmatrix} \infty & \infty & 10 & 11 & 9 & \infty \\ 9 & \infty & \infty & \infty & 8 & 9 \\ \infty & 11 & \infty & \infty & \infty & \infty \\ \infty & 8 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 12 & 13 & 11 & \infty \end{bmatrix},$$

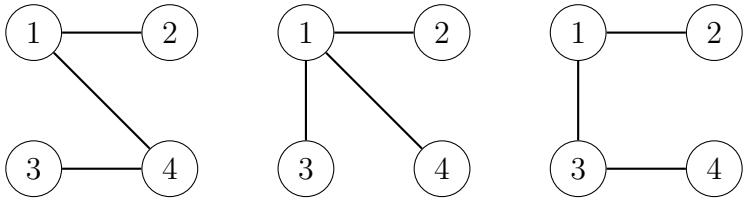
可以得出：从结点 2 到结点 5、恰好包含 4 条边的路径最短长度为 8。这样的一条路径是 $2 \to 1 \to 4 \to 2 \to 5$。

Kirchhoff 定理

Kirchhoff 定理给出了一种方法，可以通过一个特殊矩阵的行列式计算图的生成树数量。例如，图



有三棵生成树：



为了计算生成树数量，构造 **Laplacian 矩阵** L：$L[i,i]$ 为结点 i 的度；若结点 i、j 之间有边，则 $L[i,j] = -1$，否则 $L[i,j] = 0$。上图的 Laplacian 矩阵为：

$$L = \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 1 & 0 & 0 \\ -1 & 0 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}$$

可以证明，从 L 中删除任意一行和任意一列所得矩阵的行列式，就等于生成树数量。例如，删除第一行和第一列后得到

$$\det\left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}\right) = 3.$$

无论从 L 中删除哪一行、哪一列，所得行列式的值始终相同。

注意，第 22.5 节的 Cayley 公式是 Kirchhoff 定理的一个特例，因为对于含 n 个结点的完全图，

$$\det\left(\begin{bmatrix} n-1 & -1 & \cdots & -1 \\ -1 & n-1 & \cdots & -1 \\ \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & \cdots & n-1 \end{bmatrix}\right) = n^{n-2}.$$

第二十四章　概率

概率是 0 到 1 之间的实数，用来表示一个事件发生的可能性。必然事件的概率为 1，不可能事件的概率为 0。事件的概率记作 $P(\cdots)$，括号中的省略号描述该事件。

例如，掷一枚骰子的结果是 1 到 6 之间的整数，每种结果的概率均为 1/6。可以得到以下概率：

- P(“结果为 4”) $= 1/6$
- P(“结果不是 6”) $= 5/6$
- P(“结果为偶数”) $= 1/2$

24.1　概率计算

计算事件概率时，可以使用组合数学，也可以模拟生成事件的过程。例如，计算从洗好的牌中抽出三张点数相同的牌的概率（如 ♠8、♣8、♢8）。

方法一

可以使用公式

$$\frac{\text{目标结果数}}{\text{全部结果数}}.$$

本题的目标结果是三张牌点数相同。这样的结果共有 $13\binom{4}{3}$ 种，因为点数有 13 种选择，从 4 种花色中选择 3 种有 $\binom{4}{3}$ 种方法。

全部结果共有 $\binom{52}{3}$ 种，因为要从 52 张牌中选择 3 张。因此，事件概率为

$$\frac{13\binom{4}{3}}{\binom{52}{3}} = \frac{1}{425}.$$

方法二

另一种计算方法是模拟生成事件的过程。本例抽取三张牌，因此过程分为三步，并要求每一步都成功。

抽取第一张牌必然成功，因为没有任何限制。第二步成功的概率为 3/51，因为剩余 51 张牌中有 3 张与第一张点数相同。类似地，第三步成功的概率为 2/50。

整个过程成功的概率为

$$1 \cdot \frac{3}{51} \cdot \frac{2}{50} = \frac{1}{425}.$$

24.2 事件

概率论中的事件可以表示为集合

$$A \subset X,$$

其中 X 包含所有可能结果，A 是其中一部分结果。例如，掷骰子的结果集合为

$$X = \{1, 2, 3, 4, 5, 6\}.$$

事件“结果为偶数”对应集合

$$A = \{2, 4, 6\}.$$

为每个结果 x 赋予概率 $p(x)$，则事件 A 的概率 $P(A)$ 可按公式

$$P(A) = \sum_{x \in A} p(x).$$

求得，即对事件中各结果的概率求和。例如，掷骰子时，每个结果 x 都有 $p(x) = 1/6$，所以事件“结果为偶数”的概率为

$$p(2) + p(4) + p(6) = 1/2.$$

结果集合 X 中各结果的总概率必须为 1，即 $P(X) = 1$。

由于概率论中的事件是集合，可以使用标准集合运算处理它们：

- **补集** $\bar{A}$ 表示“A 不发生”。例如，掷骰子时，$A = \{2, 4, 6\}$ 的补集为 $\bar{A} = \{1, 3, 5\}$。
- **并集** $A \cup B$ 表示“A 或 B 发生”。例如，$A = \{2, 5\}$ 与 $B = \{4, 5, 6\}$ 的并集为 $A \cup B = \{2, 4, 5, 6\}$。
- **交集** $A \cap B$ 表示“A 和 B 同时发生”。例如，$A = \{2, 5\}$ 与 $B = \{4, 5, 6\}$ 的交集为 $A \cap B = \{5\}$。

补集

补集 $\bar{A}$ 的概率按公式

$$P(\bar{A}) = 1 - P(A).$$

计算。有时可以通过求相反事件，利用补集轻松解决问题。例如，掷骰子十次至少出现一次 6 的概率为

$$1 - (5/6)^{10}.$$

其中 $5/6$ 是单次结果不为 6 的概率，$(5/6)^{10}$ 是十次结果均不为 6 的概率；它的补集就是答案。

并集

并集 $A \cup B$ 的概率按公式

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

计算。例如，掷骰子时，事件

$$A = \text{“结果为偶数”}$$

与

$$B = \text{“结果小于 } 4\text{”}$$

的并集为

$$A \cup B = \text{“结果为偶数或小于 } 4\text{”}\ ,$$

其概率为

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) = 1/2 + 1/2 - 1/6 = 5/6.$$

如果事件 A、B **互斥**，即 $A \cap B$ 为空，则

$$P(A \cup B) = P(A) + P(B).$$

条件概率

条件概率

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

表示在 B 发生的条件下 A 的概率。因此，计算 A 的概率时，只考虑同时属于 B 的结果。

对于前面的集合，

$$P(A|B) = 1/3,$$

因为 B 的结果为 $\{1,2,3\}$，其中一个是偶数。这就是已知结果位于 $1\ldots3$ 时，结果为偶数的概率。

交集

利用条件概率，交集 $A \cap B$ 的概率可按公式

$$P(A \cap B) = P(A)P(B|A).$$

计算。如果

$$P(A|B) = P(A) \quad \text{且} \quad P(B|A) = P(B),$$

则称事件 A、B **相互独立**，即 B 是否发生不改变 A 的概率，反之亦然。此时交集的概率为

$$P(A \cap B) = P(A)P(B).$$

例如，从一副牌中抽牌时，事件

$$A = \text{“花色为梅花”}$$

与

$$B = \text{“点数为 4”}$$

相互独立。因此，事件

$$A \cap B = \text{“抽到梅花 4”}$$

的概率为

$$P(A \cap B) = P(A)P(B) = 1/4 \cdot 1/13 = 1/52.$$

24.3 随机变量

随机变量是由随机过程产生的数值。例如，掷两枚骰子时，可以定义随机变量

$$X = \text{“两次结果之和”}.$$

如果结果为 $[4,6]$（即先掷出 4，再掷出 6），则 X 的值为 10。

用 $P(X = x)$ 表示随机变量 X 取值为 x 的概率。例如，掷两枚骰子时，$P(X = 10) = 3/36$，因为全部结果有 36 种，而得到和 10 的方式有三种：$[4,6]$、$[5,5]$、$[6,4]$。

期望值

期望值 $E[X]$ 表示随机变量 X 的平均值，可计算为

$$\sum_x P(X=x)x,$$

其中 x 遍历 X 的所有可能取值。

例如，掷一枚骰子的期望结果为

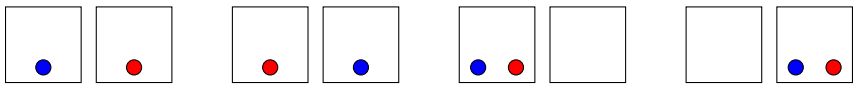
$$1/6 \cdot 1 + 1/6 \cdot 2 + 1/6 \cdot 3 + 1/6 \cdot 4 + 1/6 \cdot 5 + 1/6 \cdot 6 = 7/2.$$

期望值的一个重要性质是**线性性**：$E[X_1 + X_2 + \cdots + X_n]$ 始终等于 $E[X_1] + E[X_2] + \cdots + E[X_n]$。即使这些随机变量相互依赖，公式仍然成立。

例如，掷两枚骰子时，点数和的期望为

$$E[X_1 + X_2] = E[X_1] + E[X_2] = 7/2 + 7/2 = 7.$$

考虑把 n 个球随机放入 n 个盒子，并计算空盒子数量的期望。每个球等概率地放入任意盒子。例如，当 $n=2$ 时，所有可能情况如下：



此时空盒子数的期望为

$$\frac{0+0+1+1}{4} = \frac{1}{2}.$$

一般情况下，某个盒子为空的概率为

$$\left(\frac{n-1}{n}\right)^n,$$

因为没有任何球被放入其中。因此，利用期望的线性性，空盒子数的期望为

$$n \cdot \left(\frac{n-1}{n}\right)^n.$$

概率分布

随机变量 X 的**概率分布**给出 X 每个可能取值的概率，由各个 $P(X=x)$ 组成。例如，掷两枚骰子时，点数和的分布为：

x	2	3	4	5	6	7	8	9	10	11	12
$P(X=x)$	1/36	2/36	3/36	4/36	5/36	6/36	5/36	4/36	3/36	2/36	1/36

在**均匀分布**中，随机变量 X 有 n 个可能取值 $a, a+1, \ldots, b$，每个值的概率均为 $1/n$。例如，掷骰子时，$a=1$、$b=6$，每个 x 都有 $P(X=x)=1/6$。

均匀分布中 X 的期望为

$$E[X]=\frac{a+b}{2}.$$

在**二项分布**中，进行 n 次试验，每次成功的概率为 p。随机变量 X 表示成功次数，取值为 x 的概率为

$$P(X=x)=p^x(1-p)^{n-x}\binom{n}{x},$$

其中 p^x、$(1-p)^{n-x}$ 分别对应成功和失败的试验，$\binom{n}{x}$ 表示选择这些成功试验位置的方法数。

例如，掷骰子十次恰好出现三次 6 的概率为 $(1/6)^3(5/6)^7\binom{10}{3}$。

二项分布中 X 的期望为

$$E[X]=pn.$$

在**几何分布**中，每次试验成功的概率为 p，并持续试验直到第一次成功。随机变量 X 表示所需试验次数，取值为 x 的概率为

$$P(X=x)=(1-p)^{x-1}p,$$

其中 $(1-p)^{x-1}$ 对应前 $x-1$ 次失败，p 对应第 x 次首次成功。

例如，持续掷骰子直到出现 6，恰好掷 4 次的概率为 $(5/6)^3 1/6$。

几何分布中 X 的期望为

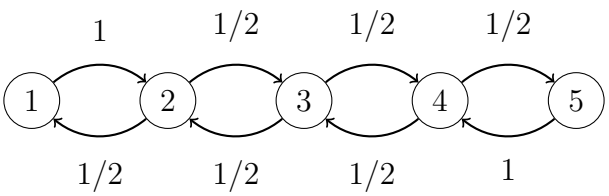
$$E[X]=\frac{1}{p}.$$

24.4 Markov 链

Markov 链是由状态及状态间转移组成的随机过程。对于每个状态，都已知转移到其他状态的概率。Markov 链可以表示为一张图，其中结点是状态，边是转移。

例如，考虑一栋有 n 层的楼，初始位于 1 楼。每一步随机向上或向下走一层，但在 1 楼时必定向上，在 n 楼时必定向下。问经过 k 步后位于 m 楼的概率。

本题中，每一层楼对应 Markov 链的一个状态。当 $n=5$ 时，图如下：



Markov 链的概率分布是向量 $[p_1, p_2, \ldots, p_n]$，其中 p_k 表示当前状态为 k 的概率。始终有 $p_1 + p_2 + \cdots + p_n = 1$。

在上述情形中，初始分布为 $[1, 0, 0, 0, 0]$，因为总是从 1 楼开始。下一步的分布为 $[0, 1, 0, 0, 0]$，因为从 1 楼只能走到 2 楼。再下一步可以向上或向下走一层，所以分布为 $[1/2, 0, 1/2, 0, 0]$，依此类推。

模拟 Markov 链游走的一种高效方法是动态规划：维护概率分布，每一步枚举所有可能转移。利用该方法，可以在 $O(n^2 m)$ 时间内模拟 m 步游走。

Markov 链的转移也可以表示为更新概率分布的矩阵。在上述情形中，矩阵为

$$\begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 1 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix}.$$

用该矩阵乘以概率分布，就得到移动一步后的新分布。例如，可以按下式从分布 $[1, 0, 0, 0, 0]$ 转移到 $[0, 1, 0, 0, 0]$：

$$\begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 1 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

利用矩阵快速幂，可以在 $O(n^3 \log m)$ 时间内计算 m 步后的分布。

24.5 随机化算法

即使问题本身与概率无关，有时也可以利用随机性求解。**随机化算法**就是基于随机性的算法。

Monte Carlo 算法是一类有时可能给出错误答案的随机化算法。要让这种算法实用，出错概率必须足够小。

Las Vegas 算法是一类始终给出正确答案、但运行时间随机变化的随机化算法。目标是让算法以高概率保持高效。

下面介绍三个可以利用随机性解决的示例问题。

顺序统计量

数组的第 k 个**顺序统计量**，是将数组按升序排序后位于第 k 个位置的元素。先排序可以在 $O(n \log n)$ 时间内求出任意顺序统计量，但仅为寻找一个元素，是否真的需要对整个数组排序?

可以使用随机化算法在不排序整个数组的情况下求顺序统计量。该算法称为**快速选择**[1]，它属于 Las Vegas 算法：通常运行时间为 $O(n)$，最坏情况下为 $O(n^2)$。

算法随机选择数组元素 x，把小于 x 的元素移到数组左部，其余元素移到右部。对于 n 个元素，这一步需要 $O(n)$ 时间。假设左部含 a 个元素，右部含 b 个元素。若 $a = k$，则 x 就是第 k 个顺序统计量；若 $a > k$，就在左部递归寻找第 k 个顺序统计量；若 $a < k$，就在右部递归寻找第 $r = k - a$ 个顺序统计量。如此继续，直到找到目标元素。

当每次随机选择 x 时，数组规模在每一步大约减半，因此寻找第 k 个顺序统计量的时间复杂度约为

$$n + n/2 + n/4 + n/8 + \cdots < 2n = O(n).$$

算法最坏情况下仍需 $O(n^2)$ 时间，因为可能每次选到的 x 都是数组中最小或最大的元素之一，从而需要 $O(n)$ 步。不过，这种情况的概率极小，实践中几乎不会发生。

验证矩阵乘法

下一个问题是：当 A、B、C 均为 $n \times n$ 矩阵时，验证 $AB = C$ 是否成立。当然，可以重新计算乘积 AB（基础算法需要 $O(n^3)$ 时间），但验证答案理应有机会比从头计算更容易。

可以使用时间复杂度仅为 $O(n^2)$ 的 Monte Carlo 算法解决该问题[2]。思想很简单：随机选择一个含 n 个元素的向量 X，计算 ABX 和 CX。若 $ABX = CX$，就报告 $AB = C$；否则报告 $AB \neq C$。

算法的时间复杂度为 $O(n^2)$，因为 ABX 和 CX 都能在 $O(n^2)$ 时间内计算。把 ABX 写成 $A(BX)$，只需进行两次 $n \times n$ 矩阵与 $n \times 1$ 矩阵的乘法，即可高效求出它。

---

[1] 1961 年，C. A. R. Hoare 发表了两种平均效率很高的算法：用于数组排序的**快速排序** [36]，以及用于寻找顺序统计量的**快速选择** [37]。

[2] R. M. Freivalds 于 1977 年发表了该算法 [26]，它有时称为**Freivalds 算法**。

该算法的缺点是：报告 $AB = C$ 时，有很小概率出错。例如，

$$\begin{bmatrix} 6 & 8 \\ 1 & 3 \end{bmatrix} \neq \begin{bmatrix} 8 & 7 \\ 3 & 2 \end{bmatrix},$$

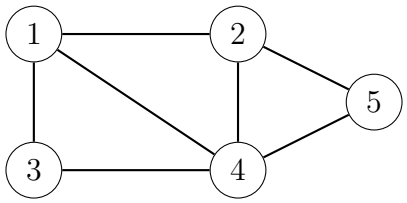
but

$$\begin{bmatrix} 6 & 8 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 8 & 7 \\ 3 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

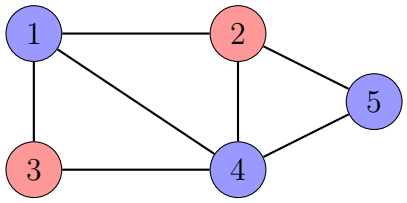
不过在实践中，算法出错的概率很小；在报告 $AB = C$ 之前使用多个随机向量 X 重复验证，可以进一步降低出错概率。

图着色

给定含 n 个结点和 m 条边的图，任务是用两种颜色给结点着色，使至少 $m/2$ 条边的两个端点颜色不同。例如，对于图



一种合法着色如下：



上图有 7 条边，其中 5 条边的两个端点颜色不同，因此该着色合法。

可以使用 Las Vegas 算法解决该问题：不断随机生成着色，直到找到合法着色。在随机着色中，每个结点独立选择颜色，两种颜色的概率均为 1/2。

在随机着色中，一条边的两个端点颜色不同的概率为 1/2。因此，端点颜色不同的边数期望为 $m/2$。由于随机着色在期望意义下就是合法的，实践中很快就能找到一个合法着色。

第二十五章　博弈论

本章重点讨论不含随机因素的双人博弈。目标是判断是否存在一种策略，使我们无论对手如何行动都能获胜，并在存在时找出该策略。

这类博弈存在一套通用理论，可以用 **Nim 理论**进行分析。首先研究玩家从若干堆中取走木棍的简单博弈，再把其中的策略推广到其他博弈。

25.1　博弈状态

考虑这样一个博弈：初始有一堆 n 根木棍，玩家 A、B 轮流行动，由 A 先手。每次必须从堆中取走 1、2 或 3 根木棍，取走最后一根木棍的玩家获胜。

例如，当 $n = 10$ 时，博弈可能按如下过程进行：

- 玩家 A 取走 2 根（剩余 8 根）。
- 玩家 B 取走 3 根（剩余 5 根）。
- 玩家 A 取走 1 根（剩余 4 根）。
- 玩家 B 取走 2 根（剩余 2 根）。
- 玩家 A 取走 2 根并获胜。

该博弈包含状态 $0, 1, 2, \ldots, n$，状态编号表示剩余木棍数量。

必胜态与必败态

必胜态是指当前玩家采用最优策略即可获胜的状态；**必败态**是指对手采用最优策略时当前玩家必定失败的状态。博弈中的每个状态都可以分类为必胜态或必败态。

在上述博弈中，状态 0 显然是必败态，因为当前玩家无法行动。状态 1、2、3 是必胜态，因为可以分别取走 1、2、3 根并立即获胜。状态 4 则是必败态，因为任何行动都会把局面转移到对手的必胜态。

更一般地说，如果存在一步行动能从当前状态转移到必败态，那么当前状态就是必胜态；否则，当前状态就是必败态。利用这一结论，我们可以从无路可走的必败态开始，对博弈的所有状态进行分类。

上述博弈的状态 $0 \ldots 15$ 可分类如下（W 表示必胜态，L 表示必败态）：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
L	W	W	W	L	W	W	W	L	W	W	W	L	W	W	W

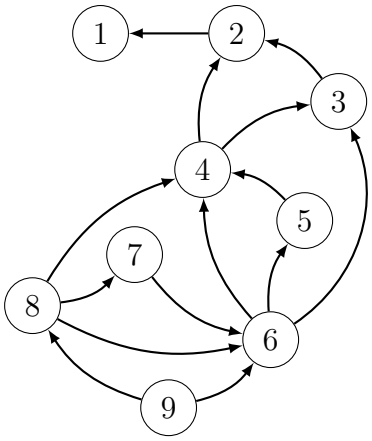
这个博弈很容易分析：当 k 能被 4 整除时，状态 k 是必败态；否则是必胜态。最优策略是每次行动后都让堆中剩余木棍数能被 4 整除。最终木棍会全部取完，而对手失败。

当然，该策略要求轮到我们行动时，木棍数不能被 4 整除。若能被 4 整除，我们便无计可施；只要对手采用最优策略，他就会获胜。

状态图

现在考虑另一种木棍博弈。在状态 k，允许取走任意数量 x 的木棍，其中 $x < k$ 且 x 整除 k。例如，在状态 8 可以取走 1、2 或 4 根，而在状态 7 只能取走 1 根。

下图把该博弈的状态 $1 \ldots 9$ 表示为一张**状态图**，图中的节点表示状态，边表示状态之间的行动：



该博弈的最终状态总是状态 1；由于此时没有合法行动，它是必败态。状态 $1 \ldots 9$ 的分类如下：

1	2	3	4	5	6	7	8	9
L	W	L	W	L	W	L	W	L

令人意外的是，在这个博弈中，所有偶数状态都是必胜态，所有奇数状态都是必败态。

25.2 Nim 博弈

Nim 博弈是一种简单却在博弈论中十分重要的博弈，因为许多其他博弈都可以采用相同的策略。我们先研究 Nim 博弈，再把它的策略推广到其他博弈。

该博弈中有 n 个堆，每堆包含若干根木棍。两名玩家轮流行动；每回合选择一个仍有木棍的堆，并从中取走任意数量的木棍。取走最后一根木棍的玩家获胜。

其状态形如 $[x_1, x_2, \ldots, x_n]$，其中 x_k 表示第 k 堆中的木棍数。例如，$[10, 12, 5]$ 表示有三堆木棍，数量分别为 10、12 和 5。状态 $[0, 0, \ldots, 0]$ 是必败态，因为此时无法再取走木棍；它也总是终止状态。

分析

事实证明，只需计算 **Nim 和** $s = x_1 \oplus x_2 \oplus \cdots \oplus x_n$，就能轻易判定任意 Nim 状态，其中 $\oplus$ 表示异或运算[1]。Nim 和为 0 的状态是必败态，其余状态都是必胜态。例如，$[10, 12, 5]$ 的 Nim 和为 $10 \oplus 12 \oplus 5 = 3$，所以它是必胜态。

这种异或和与该博弈究竟有何联系？考察状态改变时异或和如何变化即可说明这一点。

必败态：终止状态 $[0, 0, \ldots, 0]$ 是必败态，其 Nim 和如预期为 0。在其他必败态中，任何行动都会到达必胜态，因为单个 x_k 改变时 Nim 和也会改变，行动后的 Nim 和因而不再为 0。

必胜态：如果存在一个堆 k 满足 $x_k \oplus s < x_k$，就可以转移到必败态。此时从第 k 堆取走一些木棍，使其剩下 $x_k \oplus s$ 根，即可到达必败态。这样的堆一定存在：在 s 的最高位 1 所在位置，x_k 也有一个二进制位 1。

以状态 $[10, 12, 5]$ 为例。它的 Nim 和为 3，因此是必胜态，所以必然存在一步行动能够到达必败态。下面找出这样的一步。

该状态的 Nim 和如下：

10	1010
12	1100
5	0101
3	0011

在此例中，Nim 和最高位 1 所在的位置上，只有包含 10 根木棍的堆具有二进制位 1：

[1] Nim 博弈的最优策略由 C. L. Bouton 于 1901 年发表 [10]。

10	10$\underline{1}$0
12	1100
5	0101
3	00$\underline{1}$1

该堆的新大小必须为 $10 \oplus 3 = 9$，所以只需取走一根木棍。此后状态变为 $[9, 12, 5]$，它是必败态：

9	1001
12	1100
5	0101
0	0000

反常玩法

在**反常玩法**中，目标恰好相反：取走最后一根木棍的玩家失败。事实证明，反常 Nim 博弈的最优玩法与标准 Nim 博弈几乎相同。

基本思路是先按标准玩法进行反常博弈，只在临近结束时改变策略。当下一步行动后每个堆至多只剩一根木棍时，就切换到新策略。

在标准玩法中，应使行动后恰有一根木棍的堆数为偶数；而在反常玩法中，应使这样的堆数为奇数。

该策略之所以成立，是因为博弈中总会出现需要切换策略的状态，而且它是必胜态：此时恰好只有一个堆含有多于一根木棍，因此 Nim 和不为 0。

25.3 Sprague–Grundy 定理

Sprague–Grundy 定理[2]把 Nim 博弈的策略推广到满足以下条件的所有博弈：

- 两名玩家轮流行动。
- 博弈由若干状态组成，某一状态下可采取的行动与轮到哪名玩家无关。
- 当某名玩家无法行动时，博弈结束。
- 博弈必定会在有限步内结束。
- 玩家完全了解各状态与合法行动，博弈中不存在随机因素。

核心思想是为每个博弈状态计算一个 Grundy 数，它对应于 Nim 博弈中某一堆的木棍数。知道所有状态的 Grundy 数后，就能像 Nim 博弈一样处理该博弈。

[2]该定理由 R. Sprague [61] 和 P. M. Grundy [31] 分别独立发现。

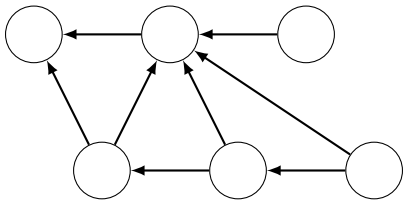
Grundy 数

一个博弈状态的 **Grundy 数**为

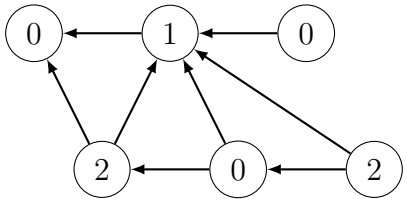
$$\mathrm{mex}(\{g_1, g_2, \ldots, g_n\}),$$

其中 $g_1, g_2, \ldots, g_n$ 是所有可达状态的 Grundy 数，而 mex 函数返回不在该集合中的最小非负整数。例如，$\mathrm{mex}(\{0, 1, 3\}) = 2$。若某状态没有合法行动，其 Grundy 数为 0，因为 $\mathrm{mex}(\emptyset) = 0$。

例如，在下面的状态图中



各状态的 Grundy 数如下：

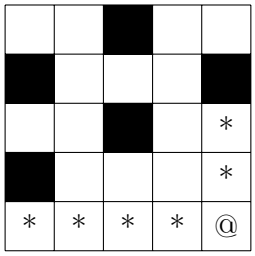


必败态的 Grundy 数为 0，必胜态的 Grundy 数为正数。

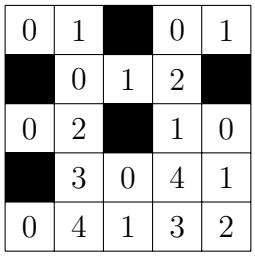
一个状态的 Grundy 数对应于 Nim 博弈中一堆木棍的数量。若 Grundy 数为 0，则只能转移到 Grundy 数为正数的状态；若 Grundy 数为 $x > 0$，则可以转移到 Grundy 数分别为 $0, 1, \ldots, x - 1$ 的状态。

例如，考虑一个玩家在迷宫中移动棋子的博弈。迷宫中的每个方格要么是可通行区域，要么是墙。每回合，玩家必须把棋子向左或向上移动若干格。完成最后一次移动的玩家获胜。

下图给出一种可能的初始状态，其中 @ 表示棋子，* 表示它可以移动到的方格。



该博弈的状态是迷宫中所有可通行的方格。在上述迷宫中，各状态的 Grundy 数如下：



因此，迷宫博弈中的每个状态都对应于 Nim 博弈中的一堆木棍。例如，右下角方格的 Grundy 数为 2，所以它是必胜态。向左移动四格或向上移动两格，都可以到达必败态并赢得博弈。

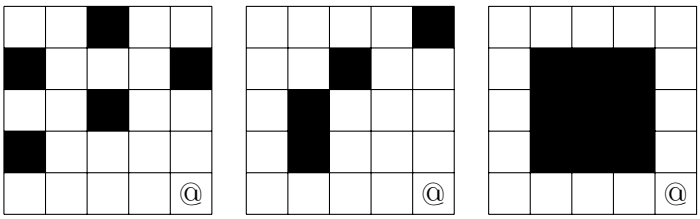
注意，与原始 Nim 博弈不同，这里可能转移到 Grundy 数比当前状态更大的状态。不过，对手总能选择一步行动抵消这种转移，因此无法从必败态中逃脱。

子博弈

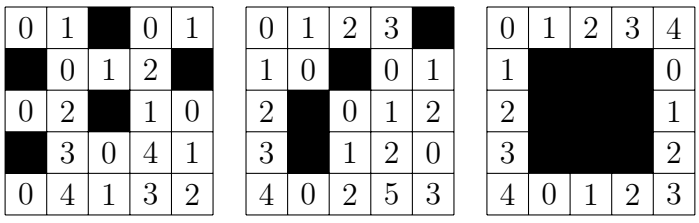
下面假设整个博弈由若干子博弈组成。每回合，玩家先选择一个子博弈，再在其中采取一步行动。当所有子博弈都无法行动时，整个博弈结束。

此时，整个博弈的 Grundy 数等于各子博弈 Grundy 数的 Nim 和。只需计算所有子博弈的 Grundy 数及其 Nim 和，就可以像处理 Nim 博弈一样进行游戏。

例如，考虑一个由三个迷宫组成的博弈。每回合，玩家选择一个迷宫，并在其中移动棋子。假设初始状态如下：



三个迷宫各状态的 Grundy 数如下：



在初始状态中，三个 Grundy 数的 Nim 和为 $2 \oplus 3 \oplus 3 = 2$，因此先手可以获胜。一种最优行动是在第一个迷宫中向上移动两格，使 Nim 和变为 $0 \oplus 3 \oplus 3 = 0$。

Grundy 博弈

有时，博弈中的一步行动会把原博弈分成彼此独立的若干子博弈。此时，原博弈的 Grundy 数为

$$\mathrm{mex}(\{g_1, g_2, \ldots, g_n\}),$$

其中 n 是合法行动的数量，并且

$$g_k = a_{k,1} \oplus a_{k,2} \oplus \ldots \oplus a_{k,m},$$

这里第 k 种行动会产生若干子博弈，其 Grundy 数为 $a_{k,1}, a_{k,2}, \ldots, a_{k,m}$。

这类博弈的一个例子是 **Grundy 博弈**。初始时只有一个包含 n 根木棍的堆。每回合，玩家选择一个堆，把它分成两个非空且大小不同的堆。完成最后一次行动的玩家获胜。

设 $f(n)$ 表示包含 n 根木棍的一个堆所对应的 Grundy 数。枚举把该堆分成两个堆的所有方法，即可计算这个 Grundy 数。例如，当 $n = 8$ 时，可能的划分为 $1 + 7$、$2 + 6$ 和 $3 + 5$，因此

$$f(8) = \mathrm{mex}(\{f(1) \oplus f(7), f(2) \oplus f(6), f(3) \oplus f(5)\}).$$

在该博弈中，$f(n)$ 的值依赖于 $f(1), \ldots, f(n-1)$。边界情况为 $f(1) = f(2) = 0$，因为包含 1 或 2 根木棍的堆无法按要求划分。最初若干个 Grundy 数为：

$$\begin{aligned} f(1) &= 0 \\ f(2) &= 0 \\ f(3) &= 1 \\ f(4) &= 0 \\ f(5) &= 2 \\ f(6) &= 1 \\ f(7) &= 0 \\ f(8) &= 2 \end{aligned}$$

当 $n = 8$ 时 Grundy 数为 2，所以该状态可以获胜。制胜行动是把木棍分成 $1+7$ 两堆，因为 $f(1) \oplus f(7) = 0$。

第二十六章　字符串算法

本章讨论高效的字符串处理算法。许多字符串问题很容易在 $O(n^2)$ 时间内解决，而真正的挑战是找到时间复杂度为 $O(n)$ 或 $O(n \log n)$ 的算法。

例如，字符串处理中的一个基本问题是**模式匹配**：给定一个长度为 n 的字符串和一个长度为 m 的模式串，找出模式串在该字符串中的所有出现位置。例如，模式串 ABC 在字符串 ABABCBABC 中出现两次。

模式匹配很容易用暴力算法在 $O(nm)$ 时间内解决：逐一检查模式串可能出现的所有位置。不过，本章将介绍只需 $O(n+m)$ 时间的更高效算法。

26.1　字符串术语

本章假设字符串采用从 0 开始的下标。因此，长度为 n 的字符串 s 由字符 $\mathrm{s}[0], \mathrm{s}[1], \ldots, \mathrm{s}[n-1]$ 组成。字符串中可能出现的字符集合称为**字符集**。例如，字符集 $\{\mathrm{A}, \mathrm{B}, \ldots, \mathrm{Z}\}$ 由英语大写字母组成。

子串是字符串中一段连续的字符序列。记号 $\mathrm{s}[a \ldots b]$ 表示 s 中从位置 a 开始、到位置 b 结束的子串。长度为 n 的字符串共有 $n(n+1)/2$ 个子串。例如，ABCD 的子串为 A, B, C, D, AB, BC, CD, ABC, BCD and ABCD.

子序列是从字符串中按原有顺序取出的一列字符，这些字符不必连续。长度为 n 的字符串共有 $2^n - 1$ 个非空子序列。例如，ABCD 的子序列为 A, B, C, D, AB, AC, AD, BC, BD, CD, ABC, ABD, ACD, BCD and ABCD.

前缀是从字符串开头开始的子串，**后缀**是到字符串末尾结束的子串。例如，ABCD 的前缀为 A, AB, ABC and ABCD, 而后缀为 D, CD, BCD and ABCD.

把字符串开头的字符依次移到末尾（或反过来），即可生成字符串的**循环移位**。例如，ABCD 的所有循环移位为 ABCD, BCDA, CDAB and DABC.

字符串的**周期**是一个前缀，重复这个前缀即可构造出原字符串；最后一次重复可以不完整，只包含该周期的一个前缀。例如，ABCABCA 的最短周期是 ABC。

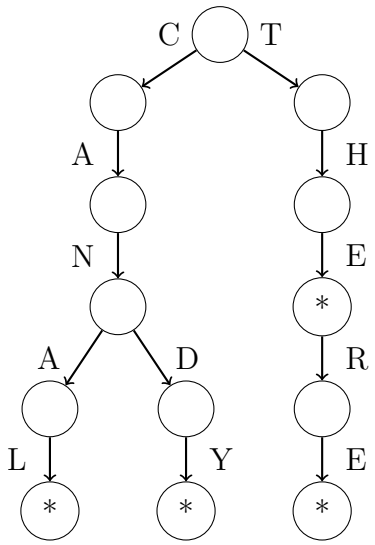
若一个字符串同时是某字符串的前缀和后缀，就称它为该字符串的 **border（边界）**。例如，ABACABA 的 border 为 A、ABA 和 ABACABA。

字符串采用**字典序**（与字母表顺序对应）进行比较。若 $x \neq y$ 且 x 是 y 的前缀，或者存在位置 k，使得对所有 $i < k$ 都有 $x[i] = y[i]$ 且 $x[k] < y[k]$，则 $x < y$。

26.2 Trie 结构

Trie（字典树）是一棵维护字符串集合的有根树。集合中的每个字符串都存储为一条从根节点开始的字符链。若两个字符串拥有公共前缀，它们在树中也会共享一段链。

例如，考虑下面的 Trie：



该 Trie 对应集合 $\{\text{CANAL}, \text{CANDY}, \text{THE}, \text{THERE}\}$。节点中的字符 * 表示集合中有一个字符串在该节点结束。这个标记是必需的，因为一个字符串可能是另一个字符串的前缀。例如，在上面的 Trie 中，THE 是 THERE 的前缀。

要判断 Trie 中是否包含一个长度为 n 的字符串，只需沿着从根节点开始的链行进，时间复杂度为 $O(n)$。向 Trie 中加入一个长度为 n 的字符串同样只需 $O(n)$ 时间：先沿已有链行进，再按需添加新节点。

利用 Trie，可以找出给定字符串中属于该集合的最长前缀。此外，若在每个节点中存储附加信息，还能计算集合中以给定字符串为前缀的字符串数量。

Trie 可以存储在数组中：

```
int trie[N][A];
```

其中 N 是节点数上限（即集合中所有字符串的总长度上限），A 是字符集大小。Trie 的节点依次编号为 $0, 1, 2, \ldots$，根节点编号为 0；trie$[s][c]$ 表示从节点 s 沿字符 c 转移后到达的下一个节点。

26.3 字符串哈希

字符串哈希是一种高效判断两个字符串是否相等的技术[1]。其思想是比较字符串的哈希值，而不是逐个比较字符。

计算哈希值

字符串的**哈希值**是根据其中的字符计算得到的一个数。如果两个字符串相同，它们的哈希值也必然相同，因此可以借助哈希值比较字符串。

实现字符串哈希的常见方法是**多项式哈希**。长度为 n 的字符串 s 的哈希值为

$$(\mathrm{s}[0]A^{n-1} + \mathrm{s}[1]A^{n-2} + \cdots + \mathrm{s}[n-1]A^0) \bmod B,$$

其中 $s[0], s[1], \ldots, s[n-1]$ 解释为 s 中各字符的编码，A 和 B 是预先选定的常数。

例如，ALLEY 中各字符的编码为：

A	L	L	E	Y
65	76	76	69	89

因此，当 $A = 3$、$B = 97$ 时，ALLEY 的哈希值为

$$(65 \cdot 3^4 + 76 \cdot 3^3 + 76 \cdot 3^2 + 69 \cdot 3^1 + 89 \cdot 3^0) \bmod 97 = 52.$$

预处理

采用多项式哈希后，只需先进行 $O(n)$ 的预处理，就能在 $O(1)$ 时间内计算字符串 s 的任意子串哈希值。具体做法是构造数组 h，使 $\mathrm{h}[k]$ 存储前缀 $\mathrm{s}[0 \ldots k]$ 的哈希值。数组元素可按如下递推式计算：

$$\begin{aligned} \mathrm{h}[0] &= \mathrm{s}[0] \\ \mathrm{h}[k] &= (\mathrm{h}[k-1]A + \mathrm{s}[k]) \bmod B \end{aligned}$$

此外，构造数组 p，其中 $\mathrm{p}[k] = A^k \bmod B$：

$$\begin{aligned} \mathrm{p}[0] &= 1 \\ \mathrm{p}[k] &= (\mathrm{p}[k-1]A) \bmod B. \end{aligned}$$

[1]该技术因 Karp–Rabin 模式匹配算法 [42] 而得到推广。

构造这两个数组需要 $O(n)$ 时间。随后，任意子串 $\mathrm{s}[a \ldots b]$ 的哈希值都可在 $O(1)$ 时间内用下式计算：

$$(\mathrm{h}[b] - \mathrm{h}[a-1]\mathrm{p}[b-a+1]) \bmod B$$

这里假设 $a > 0$。若 $a = 0$，哈希值就是 $\mathrm{h}[b]$。

使用哈希值

借助哈希值，可以高效比较字符串。其思想是不再逐个比较字符，而是比较两个字符串的哈希值。若哈希值相等，字符串很可能相等；若哈希值不同，字符串则一定不同。

哈希常常能让暴力算法变得高效。以模式匹配为例：给定字符串 s 和模式串 p，找出 p 在 s 中出现的位置。暴力算法枚举 p 可能出现的所有位置，再逐字符比较，时间复杂度为 $O(n^2)$。

由于该算法实际比较的是字符串的子串，可以用哈希提升效率。采用哈希后，每次只比较子串的哈希值，因而只需 $O(1)$ 时间。最终算法的时间复杂度为 $O(n)$，这也是该问题可能达到的最优时间复杂度。

将哈希与二分查找结合，还可以在对数时间内确定两个字符串的字典序。先通过二分查找计算两字符串公共前缀的长度，再比较公共前缀之后的第一个字符，即可确定顺序。

冲突与参数

比较哈希值时，一个明显风险是发生**哈希冲突**：两个内容不同的字符串却具有相同哈希值。此时，依赖哈希值的算法会误以为字符串相等，从而可能给出错误结果。

由于不同字符串的数量大于不同哈希值的数量，冲突始终可能发生。不过，只要谨慎选择常数 A 和 B，冲突概率就会很小。常见做法是在 10^9 附近随机选取常数，例如：

$$\begin{aligned} A &= 911382323 \\ B &= 972663749 \end{aligned}$$

采用这样的常数计算哈希值时，可以使用 long long 类型，因为乘积 AB 和 BB 都能容纳在 long long 中。但大约 10^9 种不同哈希值真的足够吗？

考虑使用哈希的三种情形：

情形 1：比较字符串 x 与 y。假设所有哈希值等概率出现，发生冲突的概率为 $1/B$。

情形 *2*：把字符串 x 分别与字符串 $y_1, y_2, \ldots, y_n$ 比较。至少发生一次冲突的概率为

$$1 - (1 - \frac{1}{B})^n.$$

情形 *3*：比较字符串 $x_1, x_2, \ldots, x_n$ 的所有两两组合。至少发生一次冲突的概率为

$$1 - \frac{B \cdot (B-1) \cdot (B-2) \cdots (B-n+1)}{B^n}.$$

下表给出 $n = 10^6$ 且 B 取不同值时的冲突概率：

常数 B	情形 1	情形 2	情形 3
10^3	0.001000	1.000000	1.000000
10^6	0.000001	0.632121	1.000000
10^9	0.000000	0.001000	1.000000
10^{12}	0.000000	0.000000	0.393469
10^{15}	0.000000	0.000000	0.000500
10^{18}	0.000000	0.000000	0.000001

表中可以看出，在情形 1 中，当 $B \approx 10^9$ 时冲突概率可以忽略不计。情形 2 中可能发生冲突，但概率仍然很小。然而情形 3 完全不同：当 $B \approx 10^9$ 时，几乎必然发生冲突。

情形 3 中的现象称为**生日悖论**：一个房间里有 n 个人，即使 n 不大，某两个人生日相同的概率也会很高。对应到哈希中，当所有哈希值两两比较时，存在两个相同哈希值的概率很大。

使用不同参数计算多组哈希值，可以降低冲突概率。所有哈希值同时发生冲突的可能性很低。例如，两组参数 $B \approx 10^9$ 的哈希值，大致相当于一组参数 $B \approx 10^{18}$ 的哈希值，冲突概率因而非常小。

有些人使用常数 $B = 2^{32}$ 或 $B = 2^{64}$，这样做很方便，因为 32 位和 64 位整数运算会自然地分别对 2^{32} 和 2^{64} 取模。然而，这不是一个好选择：当常数形如 2^x 时，可以构造出必然产生冲突的输入 [51]。

26.4 Z 算法

长度为 n 的字符串 s 的 **Z 数组** z，对每个 $k = 0, 1, \ldots, n-1$，记录从位置 k 开始且同时是 s 前缀的最长子串长度。因此，$z[k] = p$ 表示 $s[0 \ldots p-1]$ 与 $s[k \ldots k+p-1]$ 相等。利用 Z 数组可以高效解决许多字符串处理问题。

例如，ACBACDACBACBACDA 的 Z 数组如下：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
–	0	0	2	0	0	5	0	0	7	0	0	2	0	0	1

例如，此处 z[6] = 5，因为长度为 5 的子串 ACBAC 是 s 的前缀，而长度为 6 的子串 ACBACB 不是其前缀。

算法说明

下面介绍 **Z 算法**[2]，它能在 $O(n)$ 时间内高效构造 Z 数组。算法从左到右计算 Z 数组，既利用数组中已经存储的信息，也在必要时逐字符比较子串。

为了高效计算 Z 数组，算法维护区间 $[x, y]$，使得 $s[x \dots y]$ 是 s 的前缀，并且 y 尽可能大。由于已知 $s[0 \dots y-x]$ 与 $s[x \dots y]$ 相等，计算位置 $x+1, x+2, \dots, y$ 的 Z 值时便可以利用这一信息。

在每个位置 k，先检查 $z[k-x]$。若 $k+z[k-x] < y$，就能确定 $z[k] = z[k-x]$。若 $k + z[k-x] \geq y$，则已知 $s[0 \dots y-k]$ 与 $s[k \dots y]$ 相等；为了确定 $z[k]$，还需继续逐字符比较子串。算法仍然只需 $O(n)$ 时间，因为比较从位置 $y-k+1$ 和 $y+1$ 开始。

例如，下面构造这个 Z 数组：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
–	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

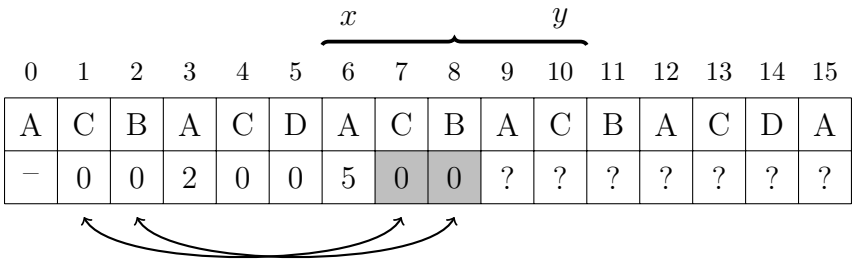
计算出 z[6] = 5 后，当前的 $[x, y]$ 区间为 $[6, 10]$：



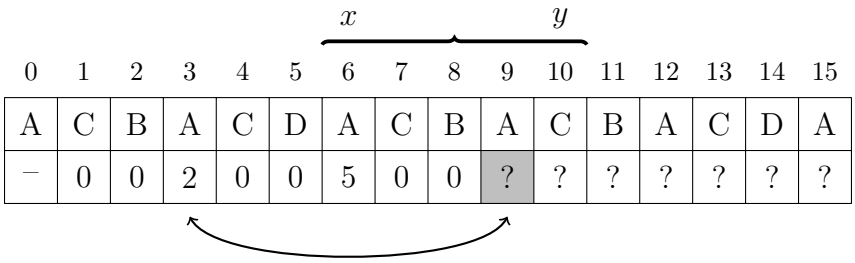
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
–	0	0	2	0	0	5	?	?	?	?	?	?	?	?	?

现在已知 $s[0 \dots 4]$ 与 $s[6 \dots 10]$ 相等，因此可以高效计算后续 Z 数组元素。首先，由于 z[1] = z[2] = 0，立即可知 z[7] = z[8] = 0：

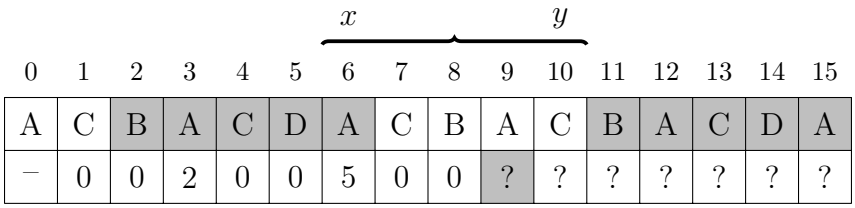
[2]文献 [32] 把 Z 算法描述为已知最简单的线性时间模式匹配方法，并将最初的思想归于文献 [50]。



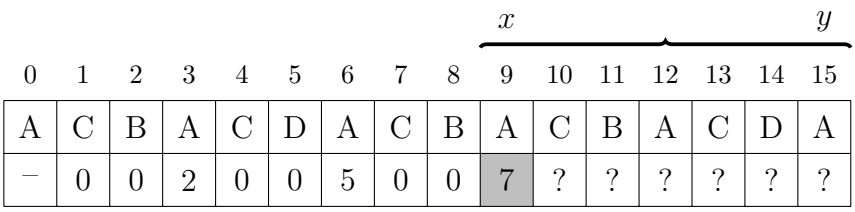
接着，由于 z[3] = 2，可知 z[9] ≥ 2：



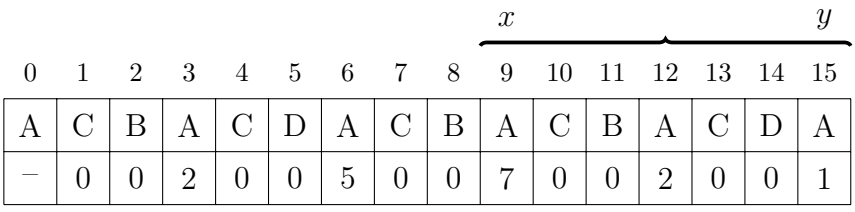
但是，我们没有位置 10 之后的字符串信息，因此需要逐字符比较子串：



结果得到 z[9] = 7，于是新的 $[x, y]$ 区间为 $[9, 15]$：



此后，只需利用 Z 数组中已有的信息，就能确定其余所有元素：



使用 Z 数组

选择字符串哈希还是 Z 算法，往往取决于个人偏好。与哈希不同，Z 算法总能给出确定结果，不存在冲突风险；另一方面，Z 算法更难实现，而且有些问题只能用哈希解决。

再次考虑模式匹配问题：找出模式串 p 在字符串 s 中的所有出现位置。前面已经用字符串哈希高效解决了该问题，Z 算法则提供了另一种方法。

字符串处理中常见的一种思路是把多个字符串用特殊字符分隔并拼接起来。对本问题，可以构造字符串 $p\#s$，其中 p 与 s 之间用不出现在二者中的特殊字符 # 分隔。$p\#s$ 的 Z 数组能够指出 p 在 s 中出现的位置，因为这些位置上的 Z 值等于 p 的长度。

例如，若 s =HATTIVATTI、p =ATT，则 Z 数组如下：

0	1	2	3	4	5	6	7	8	9	10	11	12	13
A	T	T	#	H	A	T	T	I	V	A	T	T	I
–	0	0	0	0	3	0	0	0	0	3	0	0	0

位置 5 和 10 的值均为 3，这表示模式串 ATT 出现在 HATTIVATTI 的相应位置。

所得算法的时间复杂度为线性，因为只需构造 Z 数组并遍历其中的元素。

实现

下面是 Z 算法的简短实现，返回一个表示 Z 数组的向量。

```
vector<int> z(string s) {
    int n = s.size();
    vector<int> z(n);
    int x = 0, y = 0;
    for (int i = 1; i < n; i++) {
        z[i] = max(0,min(z[i-x],y-i+1));
        while (i+z[i] < n && s[z[i]] == s[i+z[i]]) {
            x = i; y = i+z[i]; z[i]++;
        }
    }
    return z;
}
```

第二十七章　平方根算法

平方根算法是指时间复杂度中含有平方根的算法。平方根可以看作“穷人的对数”：$O(\sqrt{n})$ 优于 $O(n)$，但劣于 $O(\log n)$。无论如何，许多平方根算法在实践中速度很快，具有实用价值。

例如，考虑建立一种支持两种数组操作的数据结构：修改指定位置的元素，以及计算给定区间的元素和。此前我们用树状数组和线段树解决过该问题，两种操作的时间复杂度均为 $O(\log n)$。这里采用另一种平方根结构，使单点修改只需 $O(1)$ 时间，区间求和只需 $O(\sqrt{n})$ 时间。

其思想是把数组划分成大小为 $\sqrt{n}$ 的若干块，每个块存储块内元素之和。例如，一个包含 16 个元素的数组可按如下方式划分为每块 4 个元素：

21				17				20				13			
5	8	6	3	2	7	2	6	7	1	7	5	6	2	3	2

在这种结构中，修改数组元素很容易：每次修改后只需更新一个块的元素和，时间复杂度为 $O(1)$。例如，下图展示了某个元素的值及其所在块的元素和如何变化：

21				15				20				13			
5	8	6	3	2	5	2	6	7	1	7	5	6	2	3	2

计算一个区间的元素和时，把区间分为三部分，使答案由两端的单个元素以及中间若干整块的元素和组成：

21				15				20				13			
5	8	6	3	2	5	2	6	7	1	7	5	6	2	3	2

由于单个元素的数量为 $O(\sqrt{n})$，整块的数量也为 $O(\sqrt{n})$，所以求和查询只需 $O(\sqrt{n})$ 时间。把块大小设为 $\sqrt{n}$，是为了在两个量之间取得平衡：数组被分成 $\sqrt{n}$ 个块，每块包含 $\sqrt{n}$ 个元素。

实践中不必严格取 $\sqrt{n}$，也可以使用 k 和 n/k 作为参数，其中 k 与 $\sqrt{n}$ 不同。最优参数取决于具体问题和输入。例如，若算法经常遍历块、却很少检查块内的单个元素，则可以把数组分成 $k < \sqrt{n}$ 个块，每块包含 $n/k > \sqrt{n}$ 个元素。

27.1 组合算法

本节讨论两种把两个算法组合起来的平方根算法。在两种情形中，单独使用任意一种算法都能以 $O(n^2)$ 时间解决问题；而将二者结合后，运行时间只有 $O(n\sqrt{n})$。

分类处理

假设给定一个包含 n 个单元格的二维网格，每个单元格中都有一个字母。任务是找出字母相同且距离最小的两个单元格，其中 (x_1, y_1) 与 (x_2, y_2) 之间的距离为 $|x_1 - x_2| + |y_1 - y_2|$。例如，考虑下面的网格：

A	F	B	A
C	E	G	E
B	D	A	F
A	C	B	D

此例中，两个字母 E 之间的最小距离为 2。

可以分别考虑每个字母来解决问题。这样，问题变成：对一个固定字母 c，计算两个含有该字母的单元格之间的最小距离。考虑以下两种算法：

算法 1：遍历所有包含字母 c 的单元格对，计算它们之间的最小距离。若字母 c 出现于 k 个单元格中，时间复杂度为 $O(k^2)$。

算法 2：从所有包含字母 c 的单元格同时开始广度优先搜索，在 $O(n)$ 时间内求出两个此类单元格之间的最小距离。

一种解法是选择其中一种算法，并用于所有字母。若使用算法 1，运行时间为 $O(n^2)$，因为所有单元格可能都含有同一个字母，此时 $k = n$。若使用算法 2，运行时间同样为 $O(n^2)$，因为所有单元格可能含有不同字母，此时需要进行 n 次搜索。

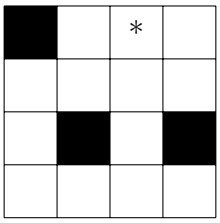
不过，可以组合这两种算法，根据每个字母在网格中的出现次数选用不同算法。假设字母 c 出现 k 次：若 $k \leq \sqrt{n}$，使用算法 1；若 $k > \sqrt{n}$，使用算法 2。这样做后，总运行时间只有 $O(n\sqrt{n})$。

首先，假设对字母 c 使用算法 1。由于 c 在网格中至多出现 $\sqrt{n}$ 次，每个含有 c 的单元格只需与其他单元格比较 $O(\sqrt{n})$ 次，因此处理所有这类单元格共需 $O(n\sqrt{n})$ 时间。再假设对字母 c 使用算法 2。满足条件的字母至多有 $\sqrt{n}$ 个，所以处理这些字母同样只需 $O(n\sqrt{n})$ 时间。

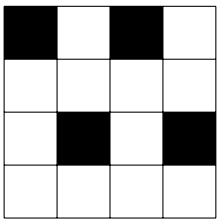
分批处理

下一个问题同样涉及一个包含 n 个单元格的二维网格。初始时，除一个单元格外其余全部为白色。进行 $n-1$ 次操作，每次先计算某个给定白色单元格到黑色单元格的最短距离，再把该白色单元格染成黑色。

例如，考虑下面这次操作：



首先，计算标有 * 的白色单元格到黑色单元格的最短距离。向左移动两步即可到达黑色单元格，所以最短距离为 2。随后把该白色单元格染成黑色：



考虑以下两种算法：

算法 1：使用广度优先搜索，计算每个白色单元格到最近黑色单元格的距离。这需要 $O(n)$ 时间；搜索完成后，可以在 $O(1)$ 时间内查询任意白色单元格到黑色单元格的最短距离。

算法 2：维护一个已经染黑的单元格列表。每次操作时遍历该列表，再把新的单元格加入列表。若列表长度为 k，一次操作需要 $O(k)$ 时间。

把所有操作划分为 $O(\sqrt{n})$ 个批次，每个批次包含 $O(\sqrt{n})$ 次操作，从而组合上述算法。每个批次开始时执行算法 1；随后用算法 2 处理该批次中的操作。批次之间清空算法 2 的列表。每次操作中，到黑色单元格的最短距离取算法 1 与算法 2 所得距离中的较小值。

所得算法的时间复杂度为 $O(n\sqrt{n})$。一方面，算法 1 执行 $O(\sqrt{n})$ 次，每次搜索需要 $O(n)$ 时间；另一方面，在一个批次内使用算法 2 时，列表只含 $O(\sqrt{n})$ 个单元格（批次之间会清空列表），所以每次操作需要 $O(\sqrt{n})$ 时间。

27.2 整数拆分

一些平方根算法基于如下观察：若把正整数 n 表示为若干正整数之和，其中*互不相同*的数最多只有 $O(\sqrt{n})$ 个。这是因为，要让和式包含尽可能多的不同数字，就应选择较小的数。若选择 $1, 2, \ldots, k$，所得总和为

$$\frac{k(k+1)}{2}.$$

因此，不同数字的最大数量为 $k = O(\sqrt{n})$。下面讨论两个可利用这一观察高效解决的问题。

背包

假设给定一列总和为 n 的整数重量，任务是找出由这些重量的任意子集能够组成的所有和。例如，重量为 $\{1, 3, 3\}$ 时，可能的和如下：

- 0（空集）
- 1
- 3
- $1 + 3 = 4$
- $3 + 3 = 6$
- $1 + 3 + 3 = 7$

使用标准背包方法（见第 7.4 节），可以按如下方式解决：定义函数 $\text{possible}(x, k)$，若用前 k 个重量能够组成和 x，其值为 1，否则为 0。由于重量总和为 n，重量个数至多为 n，利用动态规划可在 $O(n^2)$ 时间内计算函数的所有值。

不过，利用不同重量至多只有 $O(\sqrt{n})$ 种这一事实，可以提高算法效率。把相同的重量分组，每组在 $O(n)$ 时间内处理，从而得到时间复杂度为 $O(n\sqrt{n})$ 的算法。

具体做法是使用一个数组，记录由目前已经处理的各组重量能够组成的和。数组含有 n 个元素：若能组成和 k，第 k 个元素为 1，否则为 0。处理一组重量时，从左到右扫描数组，记录利用该组与之前各组能够新组成的和。

字符串构造

给定长度为 n 的字符串 s，以及总长度为 m 的字符串集合 D，考虑计算用 D 中字符串拼接成 s 的方案数。例如，若 $\text{s} = \text{ABAB}$ 且 $D = \{\text{A}, \text{B}, \text{AB}\}$，共有 4 种方案：

- A + B + A + B
- AB + A + B
- A + B + AB
- AB + AB

可以使用动态规划解决该问题。令 count(k) 表示用 D 中字符串构造前缀 s$[0 \ldots k]$ 的方案数，则 count($n-1$) 就是答案；配合 Trie 结构，可以在 $O(n^2)$ 时间内解决问题。

进一步利用字符串哈希，以及 D 中不同字符串长度至多有 $O(\sqrt{m})$ 种这一事实，可以得到更高效的解法。首先构造集合 H，存入 D 中所有字符串的哈希值。随后计算 count(k) 时，遍历所有满足 "D 中存在长度为 p 的字符串" 的 p，计算 s$[k-p+1 \ldots k]$ 的哈希值，并检查它是否属于 H。由于不同字符串长度至多有 $O(\sqrt{m})$ 种，所得算法的运行时间为 $O(n\sqrt{m})$。

27.3 莫队算法

莫队算法[1]适用于许多需要在*静态*数组上处理区间查询的问题，即查询之间数组元素不发生变化。每次查询给出区间 $[a, b]$，要求根据位置 a 到 b 之间的数组元素计算某个值。由于数组是静态的，查询可以按任意顺序处理；莫队算法采用一种特殊顺序，以保证处理效率。

莫队算法维护数组的一个*活动*区间，并始终掌握当前活动区间对应查询的答案。算法逐个处理查询，通过插入和删除元素来移动活动区间的两个端点。若数组包含 n 个元素、共有 n 次查询，且每次插入或删除元素需要 $O(f(n))$ 时间，则算法的时间复杂度为 $O(n\sqrt{n}f(n))$。

莫队算法的关键在于查询的处理顺序：把数组划分为大小为 $k = O(\sqrt{n})$ 的块；若满足下列任一条件，查询 $[a_1, b_1]$ 就在查询 $[a_2, b_2]$ 之前处理：

- $\lfloor a_1/k \rfloor < \lfloor a_2/k \rfloor$ or
- $\lfloor a_1/k \rfloor = \lfloor a_2/k \rfloor$ and $b_1 < b_2$.

因此，左端点位于同一块内的所有查询会连续处理，并按右端点排序。采用这种顺序，算法只会执行 $O(n\sqrt{n})$ 次操作：左端点共移动 $O(n)$ 轮，每轮移动 $O(\sqrt{n})$ 步；右端点共移动 $O(\sqrt{n})$ 轮，每轮移动 $O(n)$ 步。因此，整个算法中两个端点合计移动 $O(n\sqrt{n})$ 步。

[1]据文献 [12] 所述，该算法以中国竞赛程序员莫涛命名，但这一技术更早已见于文献 [44]。

示例

例如，给定一组查询，每个查询对应数组中的一个区间，要求计算每个区间内不同元素的数量。

莫队算法对查询的排序方式始终相同，但如何维护查询答案则取决于具体问题。在本问题中，可以维护数组 count，其中 count[x] 表示元素 x 在活动区间内出现的次数。

从一个查询转移到另一个查询时，活动区间会随之变化。例如，若当前区间为



而下一个区间为



则需要三步：左端点向右移动一步，右端点向右移动两步。

每一步之后都要更新数组 count。加入元素 x 后，把 count[x] 加 1；若更新后 count[x] = 1，查询答案也加 1。类似地，删除元素 x 后，把 count[x] 减 1；若更新后 count[x] = 0，查询答案也减 1。

在本问题中，每一步操作只需 $O(1)$ 时间，所以算法的总时间复杂度为 $O(n\sqrt{n})$。

第二十八章　线段树进阶

线段树是一种用途广泛的数据结构，可以解决大量算法问题。不过，仍有许多与线段树相关的主题尚未涉及。现在来讨论线段树的一些高级变体。

此前，我们通过在树中自底向上行进来实现线段树操作。例如，第 9.3 节中的区间求和实现如下：

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

然而，在更高级的线段树中，往往需要改用自顶向下的方式实现操作。采用这种方式，函数如下：

```
int sum(int a, int b, int k, int x, int y) {
    if (b < x || a > y) return 0;
    if (a <= x && y <= b) return tree[k];
    int d = (x+y)/2;
    return sum(a,b,2*k,x,d) + sum(a,b,2*k+1,d+1,y);
}
```

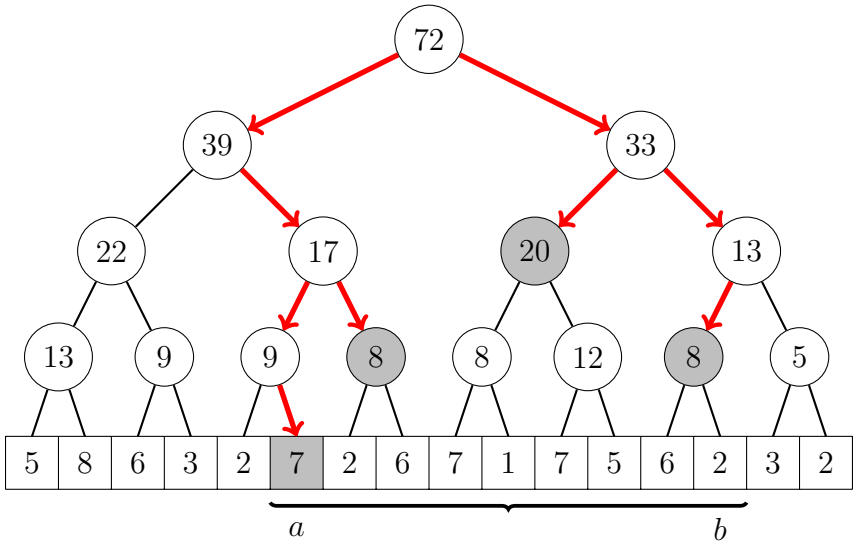
现在可以按如下方式计算任意 $\mathrm{sum}_q(a,b)$（数组区间 $[a,b]$ 的元素和）：

```
int s = sum(a, b, 1, 0, n-1);
```

参数 k 表示当前在 tree 中的位置。初始时从树根开始，因此 $k=1$。区间

$[x,y]$ 与 k 对应，初始为 $[0,n-1]$。计算元素和时，若 $[x,y]$ 位于 $[a,b]$ 之外，其贡献为 0；若 $[x,y]$ 完全包含在 $[a,b]$ 中，则可直接从 tree 取得元素和。若 $[x,y]$ 仅有一部分位于 $[a,b]$ 中，就递归搜索 $[x,y]$ 的左右两半：左半为 $[x,d]$，右半为 $[d+1,y]$，其中 $d=\lfloor\frac{x+y}{2}\rfloor$。

下图展示了计算 $\text{sum}_q(a,b)$ 时的搜索过程。灰色节点表示递归停止、可直接从 tree 中取得元素和的节点。



该实现中的操作同样只需 $O(\log n)$ 时间，因为访问的节点总数为 $O(\log n)$。

28.1 懒惰传播

利用**懒惰传播**，可以构造一棵区间修改与区间查询都只需 $O(\log n)$ 时间的线段树。其思想是自顶向下执行修改和查询，并延迟修改，使其只在必要时才向下传播。

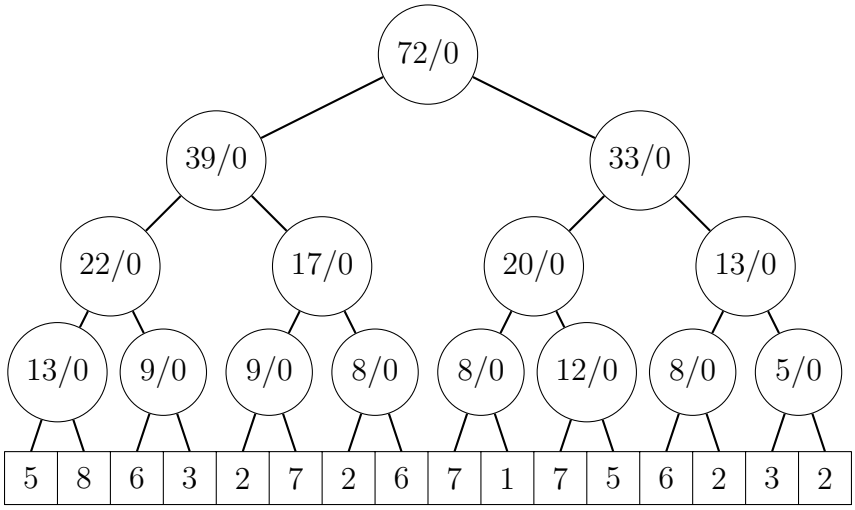
懒标记线段树的节点包含两类信息。与普通线段树一样，每个节点存储对应子数组的元素和或其他值；此外，节点还可能保存尚未传播给子节点的懒修改信息。

区间修改有两类：把区间中的每个数组元素增加某个值，或者把它们赋为某个值。两种操作都可以用类似思想实现，甚至可以构造同时支持二者的线段树。

懒标记线段树

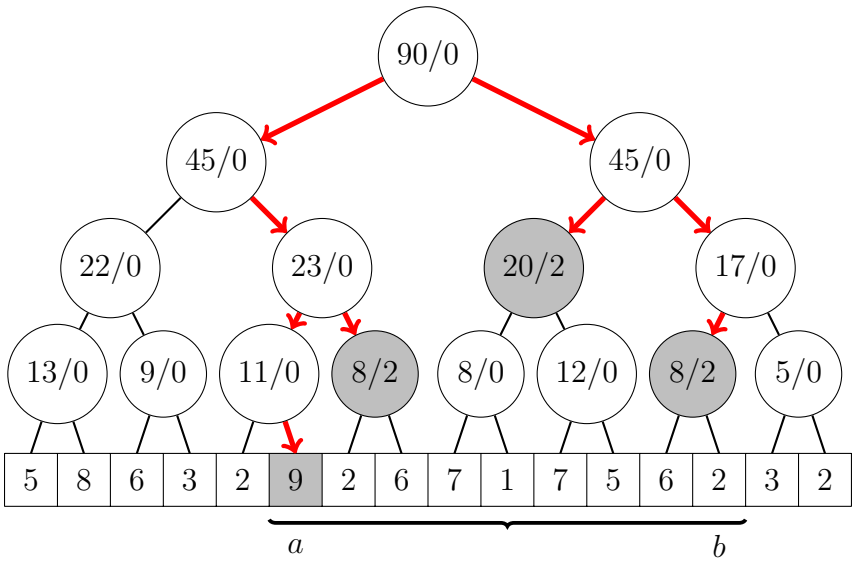
考虑构造一棵支持两种操作的线段树：把 $[a,b]$ 中每个值增加一个常数，以及计算 $[a,b]$ 中的元素和。

构造的树中每个节点保存两个值 s/z：s 表示区间元素和，z 表示懒修改值，即区间内所有值都应增加 z。在下图中，所有节点的 $z=0$，所以当前没有待传播的懒修改。



把 $[a,b]$ 中的元素增加 u 时，从根向叶行进并按如下方式修改节点：若节点区间 $[x,y]$ 完全包含在 $[a,b]$ 中，就把该节点的 z 增加 u 并停止；若 $[x,y]$ 只有一部分属于 $[a,b]$，就把该节点的 s 增加 hu，其中 h 是 $[a,b]$ 与 $[x,y]$ 交集的大小，然后继续递归向下。

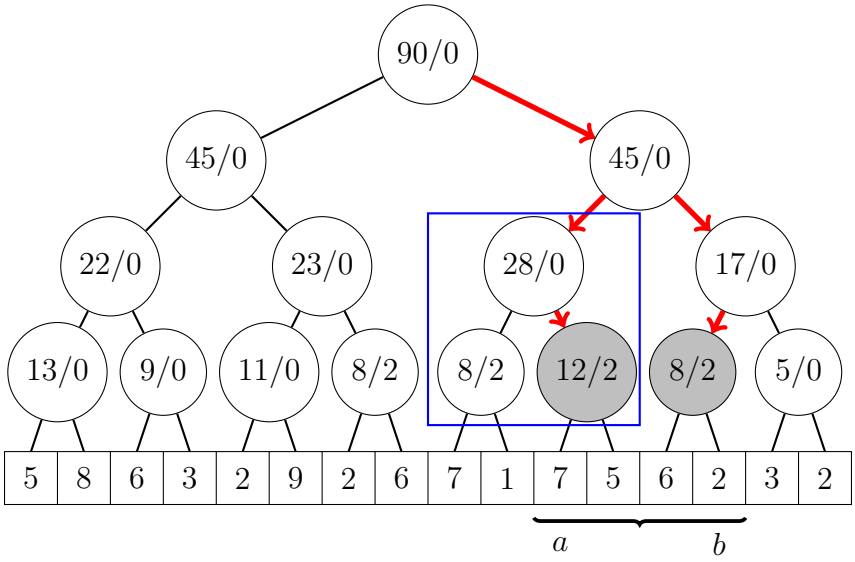
例如，下图展示了把 $[a,b]$ 中的元素增加 2 后的线段树：



计算区间 $[a,b]$ 的元素和时，同样自顶向下遍历线段树。若某节点的区间 $[x,y]$ 完全属于 $[a,b]$，就把该节点的 s 加入答案；否则继续向下递归搜索。

无论修改还是查询，在处理一个节点前，总要先把它的懒修改传播给子节点。核心思想是只在必要时才向下传播修改，从而保证操作始终高效。

下图展示了计算 $\mathrm{sum}_a(a,b)$ 时线段树的变化。矩形框标出了因懒修改向下传播而发生数值变化的节点。



注意，有时需要合并懒修改：当一个已有懒修改的节点又被赋予新的懒修改时，就会出现这种情况。对求和而言，合并很简单，因为修改 z_1 与 z_2 的组合等价于一次修改 z_1+z_2。

多项式修改

懒修改可以推广为使用如下形式的多项式更新区间：

$$p(u)=t_k u^k+t_{k-1}u^{k-1}+\cdots+t_0.$$

此时，区间 $[a,b]$ 中位置 i 的增量为 $p(i-a)$。例如，把多项式 $p(u)=u+1$ 加到 $[a,b]$，表示位置 a 的值增加 1，位置 $a+1$ 的值增加 2，依此类推。

为了支持多项式修改，每个节点保存 $k+2$ 个值，其中 k 是多项式次数。s 表示区间元素和，$z_0,z_1,\ldots,z_k$ 是表示懒修改的多项式系数。

此时，区间 $[x,y]$ 的元素和为

$$s+\sum_{u=0}^{y-x} z_k u^k+z_{k-1}u^{k-1}+\cdots+z_0.$$

利用求和公式可以高效计算这个和。例如，z_0 项对应 $(y-x+1)z_0$，z_1u 项对应

$$z_1(0+1+\cdots+y-x)=z_1\frac{(y-x)(y-x+1)}{2}.$$

在线段树中传播修改时，$p(u)$ 的下标会变化，因为在每个区间 $[x, y]$ 中，取值都按 $u = 0, 1, \ldots, y - x$ 计算。不过这并非问题，因为 $p'(u) = p(u+h)$ 与 $p(u)$ 次数相同。例如，若 $p(u) = t_2u^2 + t_1u - t_0$，则

$$p'(u) = t_2(u+h)^2 + t_1(u+h) - t_0 = t_2u^2 + (2ht_2 + t_1)u + t_2h^2 + t_1h - t_0.$$

28.2 动态线段树

普通线段树是静态的，即每个节点在数组中位置固定，整棵树需要固定数量的内存。**动态线段树**只为算法实际访问的节点分配内存，因而可以节省大量空间。

动态树的节点可以用结构体表示：

```
struct node {
    int value;
    int x, y;
    node *left, *right;
    node(int v, int x, int y) : value(v), x(x), y(y) {}
};
```

其中 value 是节点值，[x,y] 是对应区间，left 和 right 分别指向左右子树。

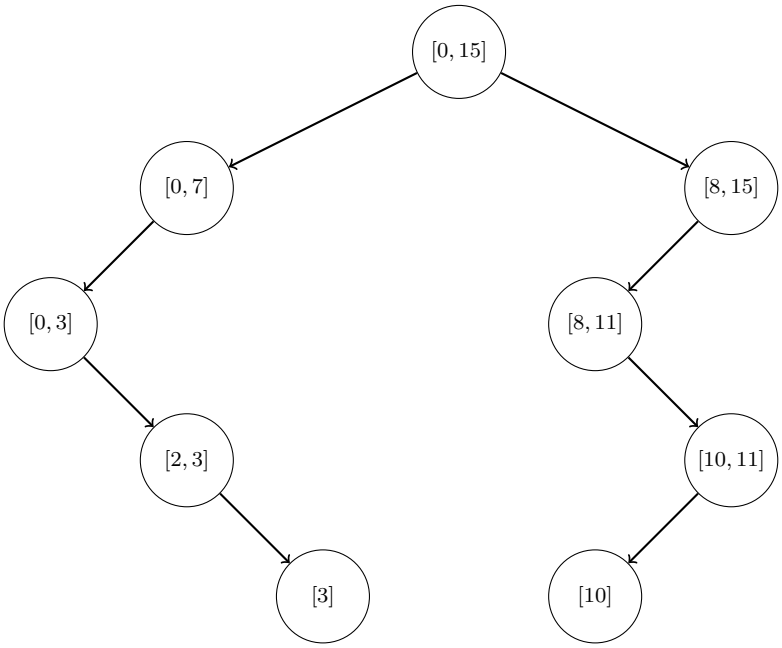
随后可按如下方式创建节点：

```
// create new node
node *x = new node(0, 0, 15);
// change value
x->value = 5;
```

稀疏线段树

当底层数组是稀疏的，即允许的下标范围 $[0, n-1]$ 很大但绝大多数数组元素为零时，动态线段树十分有用。普通线段树使用 $O(n)$ 空间，而动态线段树只使用 $O(k \log n)$ 空间，其中 k 是执行的操作次数。

稀疏线段树初始时只有一个值为零的节点 $[0, n-1]$，表示所有数组元素均为零。发生修改后，再动态向树中加入新节点。例如，若 $n = 16$，且位置 3 和 10 的元素已被修改，则树包含如下节点：



从根到任意叶节点的路径都包含 $O(\log n)$ 个节点，所以每次操作至多向树中加入 $O(\log n)$ 个新节点。因此，执行 k 次操作后，树中至多有 $O(k \log n)$ 个节点。

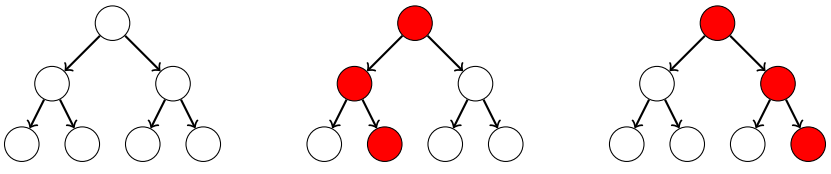
注意，如果算法开始时就知道所有会被修改的元素，则不必使用动态线段树，因为可以配合下标压缩（第 9.4 节）使用普通线段树。但若下标在算法运行过程中才生成，就无法这样做。

可持久化线段树

采用动态实现，还可以构造保存线段树*修改历史*的**可持久化线段树**。这种实现能够高效访问算法执行过程中出现过的所有版本。

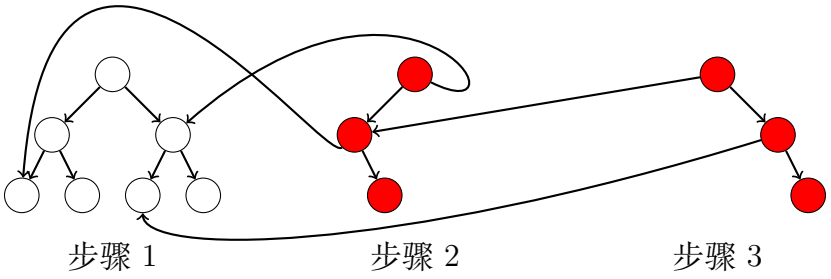
保存修改历史后，由于每个版本的完整结构都得以保留，可以像普通线段树一样在任意旧版本上执行查询。也可以基于旧版本创建新树，再彼此独立地修改。

考虑下面的修改序列，其中红色节点发生变化，其他节点保持不变：



步骤 1　　步骤 2　　步骤 3

每次修改后，树中的大多数节点都保持不变。因此，一种高效保存修改历史的方法，是把历史中的每棵树表示为新节点与旧版本子树的组合。在此例中，修改历史可按如下方式存储：

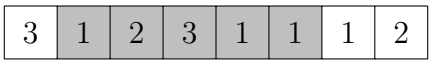


从相应根节点出发沿指针行进，即可重建任意旧版本的树结构。由于每次操作只向树中加入 $O(\log n)$ 个新节点，完整保存线段树的修改历史是可行的。

28.3 节点中的数据结构

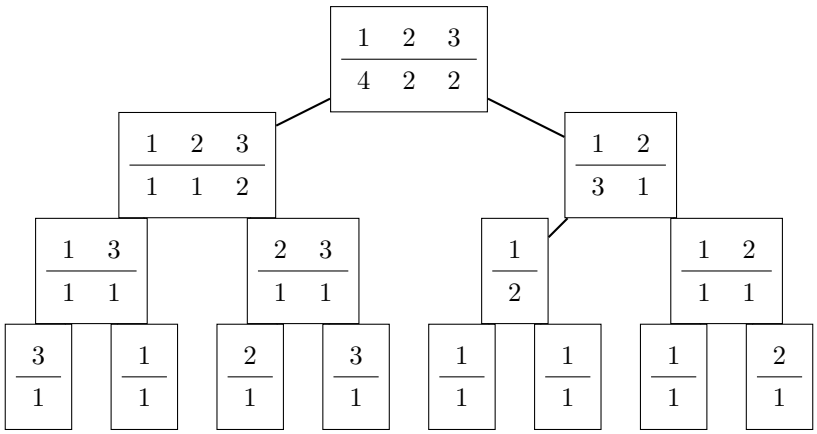
线段树节点不仅可以保存单个值，也可以保存维护对应区间信息的*数据结构*。在这样的树中，操作需要 $O(f(n)\log n)$ 时间，其中 $f(n)$ 是一次操作中处理单个节点所需的时间。

例如，考虑一棵支持“元素 x 在区间 $[a,b]$ 中出现多少次?”这类查询的线段树。下图所示区间中，元素 1 出现了三次：



为支持这类查询，在线段树的每个节点中放置一个数据结构，用于查询任意元素 x 在对应区间内的出现次数。利用这棵树，把属于查询区间的各节点结果合并即可得到答案。

例如，与上述数组对应的线段树如下：



可以让树中每个节点都包含一个 map 结构。此时，处理每个节点需要 $O(\log n)$ 时间，所以一次查询的总时间复杂度为 $O(\log^2 n)$。由于线段树有 $O(\log n)$ 层，每层包含 $O(n)$ 个元素，整棵树使用 $O(n\log n)$ 空间。

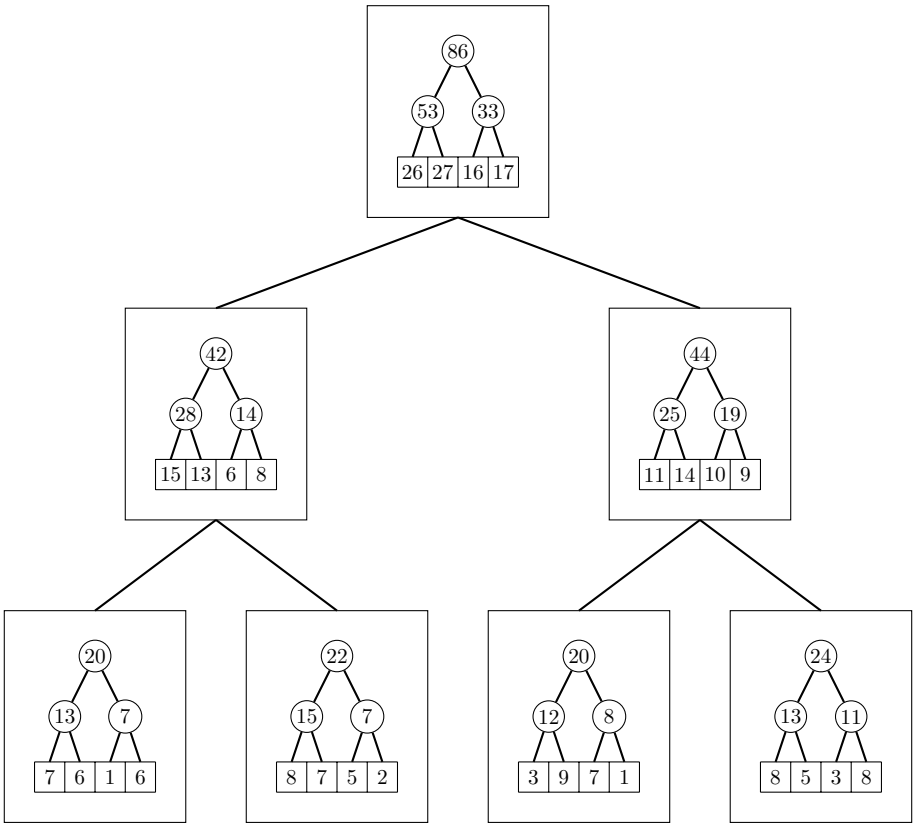
28.4 二维线段树

二维线段树支持对二维数组中矩形子数组的查询。它可以实现为嵌套线段树：外层大树对应数组的各行，每个节点中都包含一棵对应各列的小树。

例如，对如下数组

7	6	1	6
8	7	5	2
3	9	7	1
8	5	3	8

可以通过下面的线段树计算任意子数组的元素和：

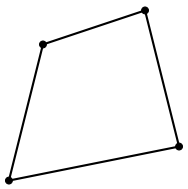


二维线段树的操作需要 $O(\log^2 n)$ 时间，因为大树和每棵小树都包含 $O(\log n)$ 层。整棵树需要 $O(n^2)$ 空间，因为每棵小树包含 $O(n)$ 个值。

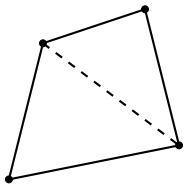
第二十九章　计算几何

在几何问题中，困难往往在于找到一种既便于实现、特殊情况又尽可能少的解题方法。

例如，给定一个四边形（具有四个顶点的多边形）的各个顶点，要求计算其面积。一种可能的输入如下：



一种方法是连接两个相对顶点，用直线把四边形分成两个三角形：

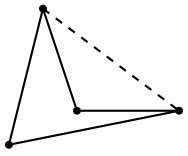


随后只需把两个三角形的面积相加。三角形面积可用**海伦公式**计算：

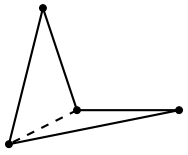
$$\sqrt{s(s-a)(s-b)(s-c)},$$

其中 a、b、c 是三角形三边长度，$s=(a+b+c)/2$。

这种解法可行，但存在一个陷阱：应如何把四边形分成三角形？事实证明，有时不能任意选取一对相对顶点。例如，在下图中，分割线位于四边形外部：



不过，换一条分割线就可以：



人很容易判断哪条分割线正确，但计算机处理起来并不容易。

不过，可以改用一种更便于编程的方法。存在如下通用公式：

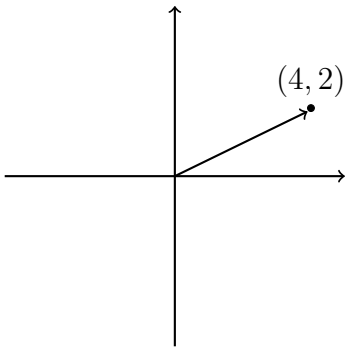
$$x_1y_2 - x_2y_1 + x_2y_3 - x_3y_2 + x_3y_4 - x_4y_3 + x_4y_1 - x_1y_4,$$

它可以计算顶点为 (x_1, y_1), (x_2, y_2), (x_3, y_3) and (x_4, y_4) 的四边形面积。该公式易于实现，没有特殊情况，而且还能推广到所有多边形。

29.1 复数

复数形如 $x + yi$，其中 $i = \sqrt{-1}$ 是**虚数单位**。复数的几何意义是表示二维点 (x, y)，或从原点指向 (x, y) 的向量。

例如，$4 + 2i$ 对应下面的点和向量：



C++ 的复数类 complex 在解决几何问题时十分有用。它可以用复数表示点和向量，并提供多种适用于几何计算的工具。

在下面的代码中，C 是坐标类型，P 是点或向量的类型。代码还定义了宏 X 和 Y，用于访问 x、y 坐标。

```
typedef long long C;
typedef complex<C> P;
#define X real()
#define Y imag()
```

例如，下面的代码定义点 $p = (4, 2)$，并输出其 x、y 坐标：

```
P p = {4,2};
cout << p.X << " " << p.Y << "\n"; // 4 2
```

下面的代码定义向量 $v = (3, 1)$ 和 $u = (2, 2)$，再计算它们的和 $s = v + u$。

```
P v = {3,1};
P u = {2,2};
P s = v+u;
cout << s.X << " " << s.Y << "\n"; // 5 3
```

实践中，坐标通常适合用 long long（整数）或 long double（实数）表示。应尽可能使用整数，因为整数计算是精确的。若必须使用实数，比较数值时要考虑精度误差。判断实数 a 与 b 是否相等的一种安全方法是检查 $|a-b|<\epsilon$，其中 ϵ 是很小的数（例如 $\epsilon=10^{-9}$）。

函数

以下示例中的坐标类型为 long double。

函数 abs(v) 用公式 $\sqrt{x^2+y^2}$ 计算向量 $v=(x,y)$ 的长度 $|v|$。它也可用于计算点 (x_1,y_1) 与 (x_2,y_2) 之间的距离，因为该距离等于向量 (x_2-x_1,y_2-y_1) 的长度。

下面的代码计算点 $(4,2)$ 与 $(3,-1)$ 之间的距离：

```
P a = {4,2};
P b = {3,-1};
cout << abs(b-a) << "\n"; // 3.16228
```

函数 arg(v) 计算向量 $v=(x,y)$ 相对于 x 轴的夹角。返回值以弧度表示，r 弧度等于 $180r/\pi$ 度。指向右方的向量角度为 0；顺时针旋转时角度减小，逆时针旋转时角度增大。

函数 polar(s,a) 构造长度为 s、方向角为 a 的向量。向量乘以一个长度为 1、方向角为 a 的向量，即可将其旋转 a 弧度。

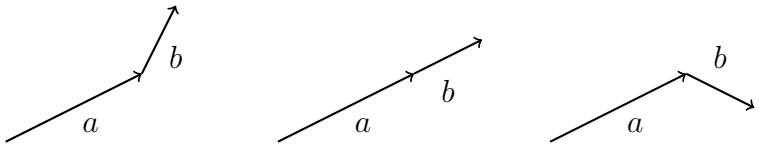
下面的代码先计算向量 $(4,2)$ 的方向角，再将它逆时针旋转 $1/2$ 弧度，最后重新计算方向角：

```
P v = {4,2};
cout << arg(v) << "\n"; // 0.463648
v *= polar(1.0,0.5);
cout << arg(v) << "\n"; // 0.963648
```

29.2 点与直线

向量 $a=(x_1,y_1)$ 与 $b=(x_2,y_2)$ 的**叉积** $a\times b$ 为 $x_1y_2-x_2y_1$。把 b 接在 a 之后时，叉积可以判断 b 是向左转（正值）、不转向（零）还是向右转（负值）。

下图展示了这三种情况：



$a\times b=6$　　$a\times b=0$　　$a\times b=-8$

例如，在第一种情况中 $a=(4,2)$、$b=(1,2)$。下面用 complex 类计算叉积：

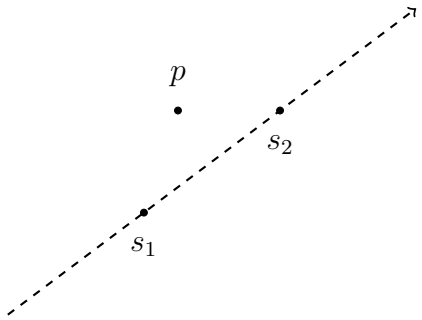
```
P a = {4,2};
P b = {1,2};
C p = (conj(a)*b).Y; // 6
```

上述代码之所以成立，是因为函数 conj 会把向量的 y 坐标取反；将向量 $(x_1,-y_1)$ 与 (x_2,y_2) 相乘后，结果的 y 坐标正是 $x_1y_2-x_2y_1$。

点的位置

叉积可用于判断一个点位于直线的左侧还是右侧。假设直线经过 s_1、s_2，观察方向从 s_1 指向 s_2，待判断的点为 p。

例如，下图中的 p 位于直线左侧：

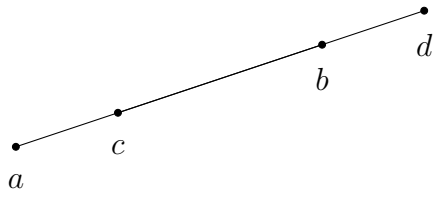


叉积 $(p-s_1)\times(p-s_2)$ 可以判断点 p 的位置。若叉积为正，p 位于左侧；若为负，p 位于右侧；若为零，则 s_1、s_2、p 三点共线。

线段相交

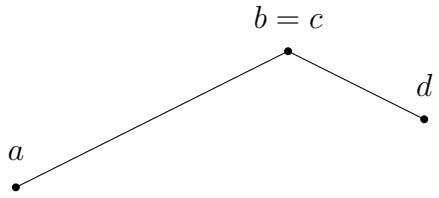
下面考虑如何判断两条线段 ab 与 cd 是否相交。可能情况如下。

情况 *1*：两条线段位于同一直线上，并且彼此重叠。此时有无穷多个交点。例如，在下图中，c 与 b 之间的所有点都是交点：



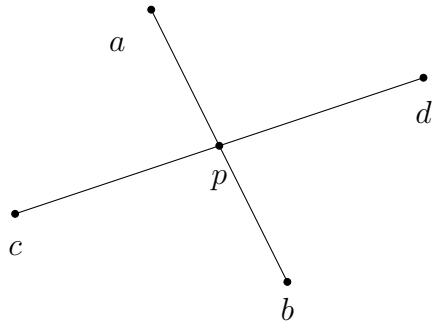
这种情况下，可以用叉积检查所有点是否共线，再对各点排序并判断两条线段是否重叠。

情况 *2*：两条线段有一个公共端点，且它是唯一交点。例如，下图中的交点为 $b = c$：



这种情况很容易检查，因为交点只有四种可能：$a = c$、$a = d$、$b = c$ 或 $b = d$。

情况 *3*：恰有一个交点，且它不是任何线段的端点。下图中，点 p 是交点：



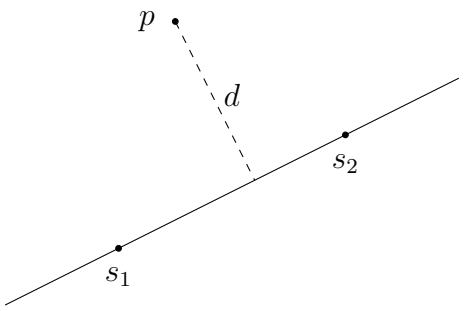
这种情况下，两条线段相交当且仅当 c、d 分居直线 ab 两侧，同时 a、b 分居直线 cd 两侧。可以用叉积检查这些条件。

点到直线的距离

叉积的另一项用途是用下式计算三角形面积：

$$\frac{|(a-c)\times(b-c)|}{2},$$

其中 a、b、c 是三角形顶点。由此可以推导点到直线最短距离的公式。例如，下图中的 d 是点 p 到由 s_1、s_2 定义的直线的最短距离：

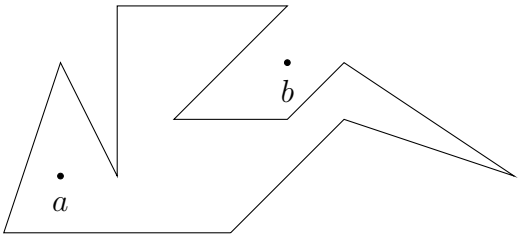


顶点为 s_1、s_2、p 的三角形面积可以用两种方式计算，既是 $\frac{1}{2}|s_2 - s_1|d$，也是 $\frac{1}{2}((s_1 - p) \times (s_2 - p))$。因此最短距离为

$$d = \frac{(s_1 - p) \times (s_2 - p)}{|s_2 - s_1|}.$$

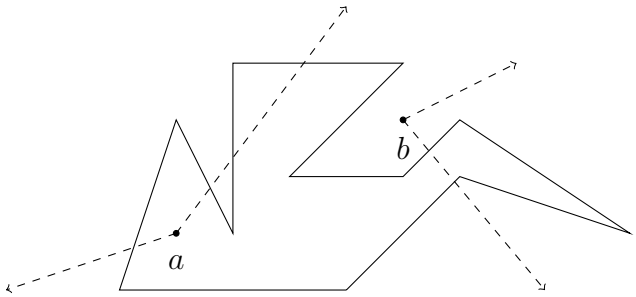
点在多边形内

下面考虑如何判断一个点位于多边形内部还是外部。例如，下图中点 a 在多边形内部，点 b 在外部。



一种方便的解法是从该点向任意方向发出一条射线，并计算它与多边形边界相交的次数。若次数为奇数，点在多边形内部；若为偶数，点在多边形外部。

例如，可以发出如下射线：



从 a 发出的两条射线分别与多边形边界相交 1 次和 3 次，所以 a 在多边形内部。相应地，从 b 发出的两条射线分别相交 0 次和 2 次，所以 b 在多边形外部。

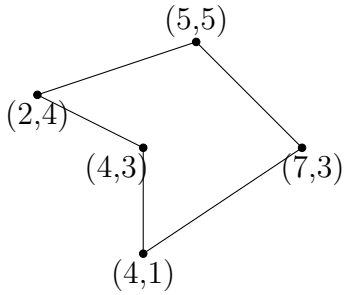
29.3 多边形面积

计算多边形面积的通用公式有时称为**鞋带公式**:

$$\frac{1}{2}|\sum_{i=1}^{n-1}(p_i \times p_{i+1})| = \frac{1}{2}|\sum_{i=1}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)|,$$

其中顶点按多边形边界上的相邻顺序排列为 $p_1 = (x_1, y_1)$、$p_2 = (x_2, y_2)$、...、$p_n = (x_n, y_n)$，并且首尾顶点相同，即 $p_1 = p_n$。

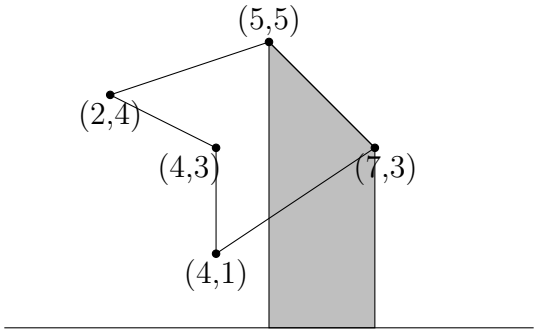
例如，下列多边形的面积



为

$$\frac{|(2 \cdot 5 - 5 \cdot 4) + (5 \cdot 3 - 7 \cdot 5) + (7 \cdot 1 - 4 \cdot 3) + (4 \cdot 3 - 4 \cdot 1) + (4 \cdot 4 - 2 \cdot 3)|}{2} = 17/2.$$

该公式的思想是遍历若干梯形，其中一条边是多边形的一条边，另一条边位于水平直线 $y = 0$ 上。例如:



这种梯形的面积为

$$(x_{i+1} - x_i)\frac{y_i + y_{i+1}}{2},$$

其中多边形的两个顶点为 p_i 与 p_{i+1}。若 $x_{i+1} > x_i$，面积取正；若 $x_{i+1} < x_i$，面积取负。

多边形面积是所有此类梯形面积之和，由此得到公式

$$|\sum_{i=1}^{n-1}(x_{i+1} - x_i)\frac{y_i + y_{i+1}}{2}| = \frac{1}{2}|\sum_{i=1}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)|.$$

注意，需要对总和取绝对值，因为沿多边形边界按顺时针或逆时针方向遍历时，总和可能分别为正或为负。

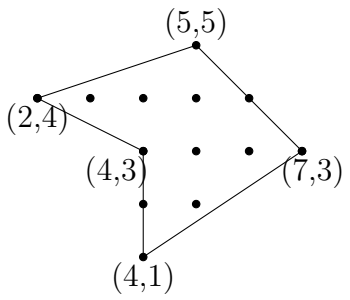
Pick 定理

若多边形所有顶点的坐标均为整数，**Pick 定理**提供了另一种计算面积的方法。根据 Pick 定理，多边形面积为

$$a + b/2 - 1,$$

其中 a 是多边形内部的整点数，b 是边界上的整点数。

例如，下列多边形的面积



为 $6 + 7/2 - 1 = 17/2$。

29.4 距离函数

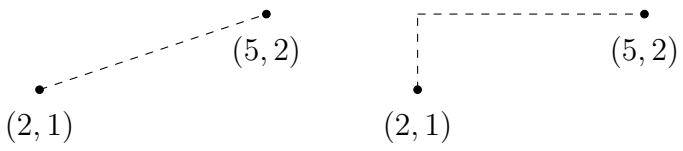
距离函数定义两点之间的距离。常用的距离函数是**欧几里得距离**，点 (x_1, y_1) 与 (x_2, y_2) 之间的距离为

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

另一种距离函数是**曼哈顿距离**，点 (x_1, y_1) 与 (x_2, y_2) 之间的距离为

$$|x_1 - x_2| + |y_1 - y_2|.$$

例如，考虑下图：



欧几里得距离　　曼哈顿距离

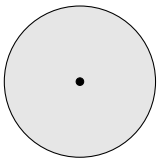
两点间的欧几里得距离为

$$\sqrt{(5-2)^2 + (2-1)^2} = \sqrt{10}$$

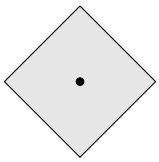
曼哈顿距离为

$$|5-2|+|2-1|=4.$$

下图分别展示在欧几里得距离与曼哈顿距离下，到中心点距离不超过 1 的区域：



欧几里得距离

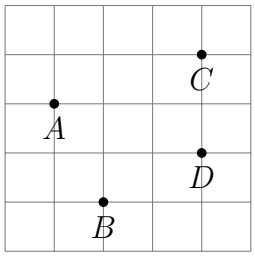


曼哈顿距离

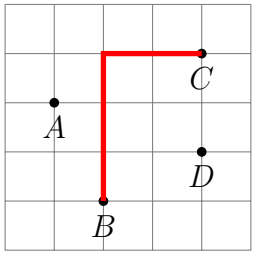
旋转坐标系

有些问题用曼哈顿距离代替欧几里得距离后更容易解决。例如，给定二维平面上的 n 个点，要求计算任意两点之间的最大曼哈顿距离。

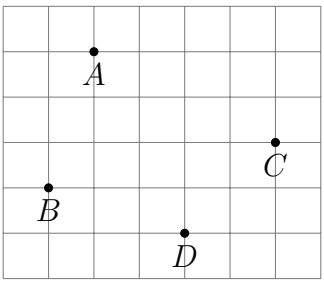
考虑下面这组点：



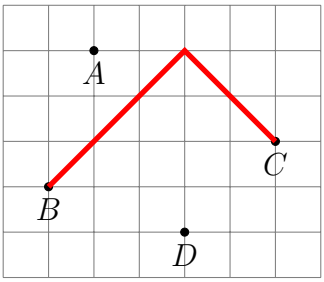
点 B 与 C 之间的曼哈顿距离最大，为 5：



处理曼哈顿距离时，一种实用技巧是把整个坐标系旋转 45 度，使点 (x,y) 变为 $(x+y,y-x)$。例如，旋转上述各点后得到：



最大距离如下所示：



考虑两点 $p_1 = (x_1, y_1)$ 和 $p_2 = (x_2, y_2)$，旋转后的坐标分别为 $p'_1 = (x'_1, y'_1)$ 和 $p'_2 = (x'_2, y'_2)$。此时，p_1 与 p_2 之间的曼哈顿距离有两种等价表示：

$$|x_1 - x_2| + |y_1 - y_2| = \max(|x'_1 - x'_2|, |y'_1 - y'_2|)$$

例如，若 $p_1 = (1, 0)$、$p_2 = (3, 3)$，旋转后坐标为 $p'_1 = (1, -1)$、$p'_2 = (6, 0)$，曼哈顿距离为

$$|1 - 3| + |0 - 3| = \max(|1 - 6|, |-1 - 0|) = 5.$$

旋转坐标后可以分别考虑 x、y 坐标，从而简化曼哈顿距离的处理。要最大化两点之间的曼哈顿距离，应找到旋转后坐标使下式最大的两个点：

$$\max(|x'_1 - x'_2|, |y'_1 - y'_2|).$$

这很容易做到，因为旋转后坐标的水平差或垂直差中必有一个达到最大值。

第三十章　扫描线算法

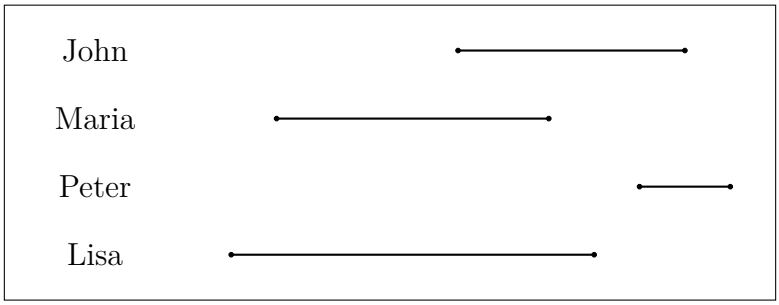
许多几何问题可以用**扫描线**算法解决。这类算法把问题实例表示为平面上若干点所对应的事件，再按事件的 x 坐标或 y 坐标递增处理。

例如，某公司有 n 名员工，已知每名员工某天到达和离开办公室的时间，要求计算同一时刻在办公室内的最多人数。

可以把每名员工建模为两个事件，分别对应到达和离开时间。对事件排序后依次处理，并维护办公室中的人数。例如，下表

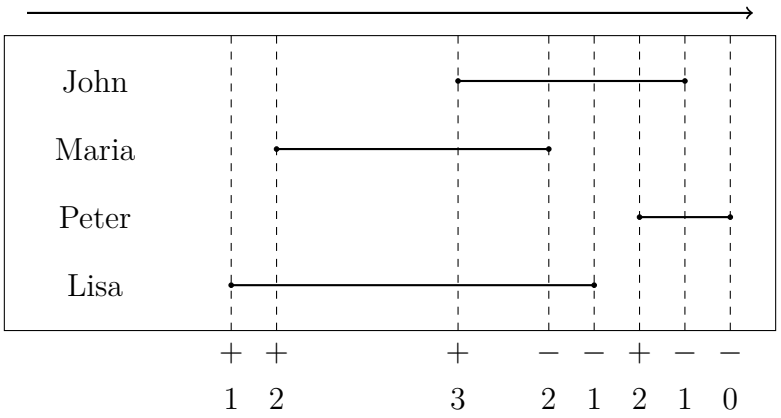
员工	到达时间	离开时间
John	10	15
Maria	6	12
Peter	14	16
Lisa	5	13

对应如下事件：



从左到右处理事件并维护一个计数器。每当有人到达，计数器加一；有人离开，计数器减一。算法执行过程中计数器的最大值就是答案。

在此例中，事件处理过程如下：

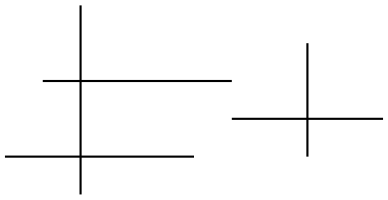


符号 + 和 − 分别表示计数器增加或减少，下方显示计数器的值。从 John 到达至 Maria 离开之间，计数器达到最大值 3。

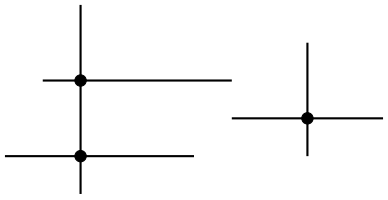
算法运行时间为 $O(n \log n)$，其中事件排序需要 $O(n \log n)$ 时间，其余部分需要 $O(n)$ 时间。

30.1 交点

给定 n 条水平或竖直线段，考虑计算交点总数的问题。例如，线段如下时



共有三个交点：

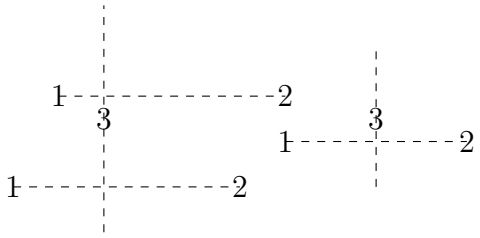


枚举所有线段对并检查是否相交，即可在 $O(n^2)$ 时间内解决问题。不过，结合扫描线算法和区间查询数据结构，可以把复杂度改进到 $O(n \log n)$。

其思想是从左到右处理线段端点，并关注三类事件：

(1) 水平线段开始
(2) 水平线段结束
(3) 竖直线段

该示例对应的事件如下：



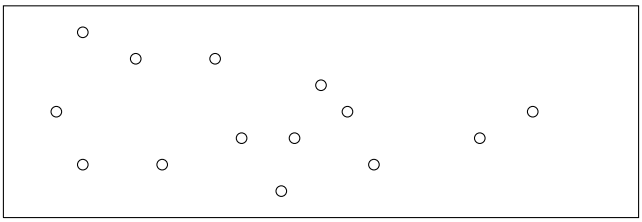
从左到右处理事件，并用一种数据结构维护当前活动水平线段的 y 坐标集合。遇到事件 1 时，把线段的 y 坐标加入集合；遇到事件 2 时，将其从集合中删除。

交点在事件 3 处统计。若存在从 y_1 到 y_2 的竖直线段，就统计 y 坐标介于 y_1 与 y_2 之间的活动水平线段数量，并把该数量加入交点总数。

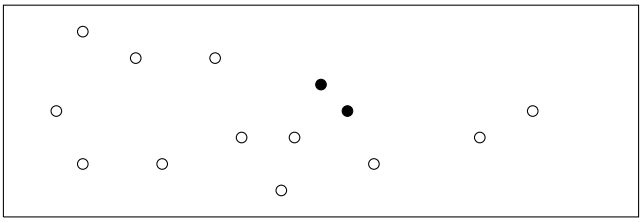
可以使用树状数组或线段树存储水平线段的 y 坐标，必要时配合下标压缩。使用这些结构后，每个事件的处理时间为 $O(\log n)$，算法总运行时间为 $O(n \log n)$。

30.2 最近点对问题

给定 n 个点，下面的问题是找出欧几里得距离最小的两个点。例如，若点集为



则应找出下图中的两个点：

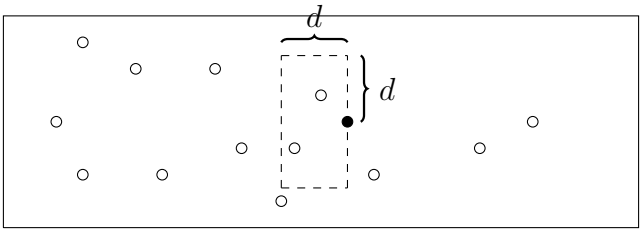


这是另一个可以用扫描线算法在 $O(n \log n)$ 时间内解决的问题[1]。从左到右遍历各点，并维护 d，表示目前见过的点对之间的最小距离。对每个点，找出其左侧距离最近的点；若距离小于 d，就得到新的最小距离并更新 d。

[1]此外，还有一种 $O(n \log n)$ 的分治算法 [56]：把点分成两组，并递归解决两组中的问题。

若当前点为 (x, y)，且其左侧存在距离小于 d 的点，则该点的 x 坐标必在 $[x-d, x]$ 内，y 坐标必在 $[y-d, y+d]$ 内。因此只需考虑落在这些范围中的点，算法由此变得高效。

例如，下图虚线区域包含所有可能与活动点距离小于 d 的点：



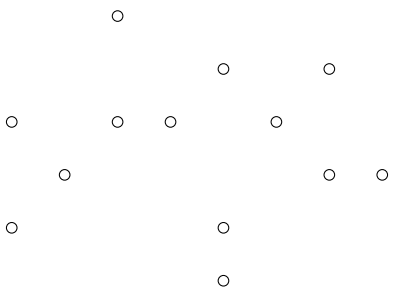
算法的效率建立在该区域始终只包含 $O(1)$ 个点这一事实之上。维护 x 坐标位于 $[x-d, x]$ 内、并按 y 坐标递增排列的点集，即可在 $O(\log n)$ 时间内遍历这些候选点。

算法时间复杂度为 $O(n \log n)$，因为需要遍历 n 个点，并为每个点在 $O(\log n)$ 时间内找出其左侧最近点。

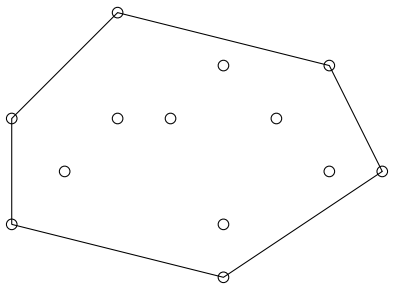
30.3 凸包问题

凸包是包含给定点集中所有点的最小凸多边形。凸性是指，多边形任意两个顶点之间的线段都完全位于多边形内部。

例如，对下列点集



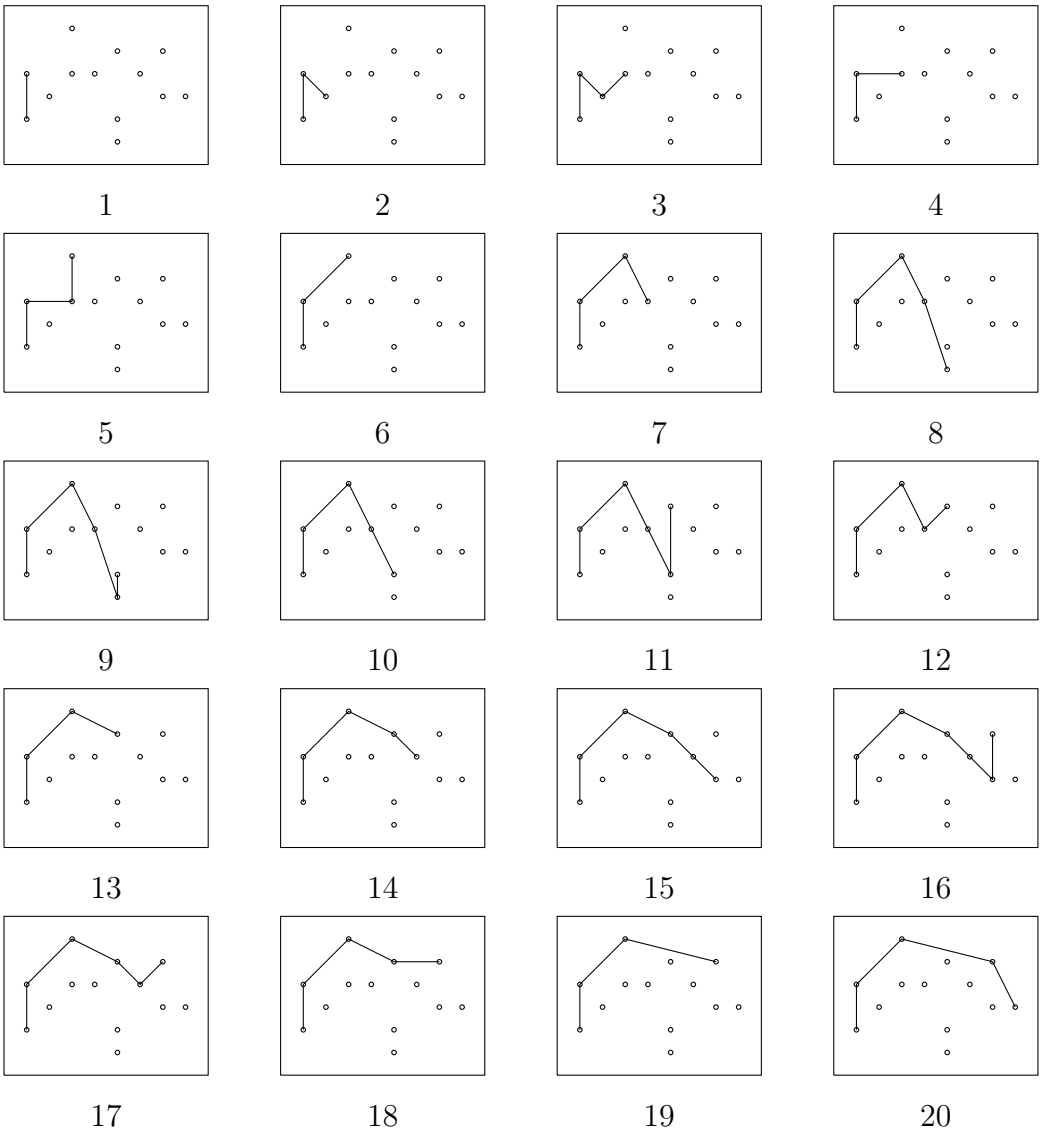
其凸包如下：



Andrew 算法 [3] 提供了一种在 $O(n \log n)$ 时间内构造点集凸包的简便方法。算法先找出最左点与最右点，再分两部分构造凸包：先构造上凸壳，再构造下凸壳。两部分过程相似，因此只需关注上凸壳的构造。

首先以 x 坐标为第一关键字、y 坐标为第二关键字对点排序。随后依次遍历各点并加入凸包。每次加入点后，都要确保凸包最后一条线段不会向左转；只要出现左转，就反复删除凸包中的倒数第二个点。

下列各图展示了 Andrew 算法的执行过程：



参考文献

[1] A. V. Aho, J. E. Hopcroft and J. Ullman. *Data Structures and Algorithms*, Addison-Wesley, 1983.

[2] R. K. Ahuja and J. B. Orlin. Distance directed augmenting path algorithms for maximum flow and parametric maximum flow problems. *Naval Research Logistics*, 38(3):413–430, 1991.

[3] A. M. Andrew. Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5):216–219, 1979.

[4] B. Aspvall, M. F. Plass and R. E. Tarjan. A linear-time algorithm for testing the truth of certain quantified boolean formulas. *Information Processing Letters*, 8(3):121–123, 1979.

[5] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

[6] M. Beck, E. Pine, W. Tarrat and K. Y. Jensen. New integer representations as the sum of three cubes. *Mathematics of Computation*, 76(259):1683–1690, 2007.

[7] M. A. Bender and M. Farach-Colton. The LCA problem revisited. In *Latin American Symposium on Theoretical Informatics*, 88–94, 2000.

[8] J. Bentley. *Programming Pearls*. Addison-Wesley, 1999 (2nd edition).

[9] J. Bentley and D. Wood. An optimal worst case algorithm for reporting intersections of rectangles. *IEEE Transactions on Computers*, C-29(7):571–577, 1980.

[10] C. L. Bouton. Nim, a game with a complete mathematical theory. *Annals of Mathematics*, 3(1/4):35–39, 1901.

[11] Croatian Open Competition in Informatics, http://hsin.hr/coci/

[12] Codeforces: On "Mo's algorithm", http://codeforces.com/blog/entry/20032

[13] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, MIT Press, 2009 (3rd edition).

[14] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

[15] K. Diks et al. *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions*, University of Warsaw, 2012.

[16] M. Dima and R. Ceterchi. Efficient range minimum queries using binary indexed trees. *Olympiad in Informatics*, 9(1):39–44, 2015.

[17] J. Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17(3):449–467, 1965.

[18] J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.

[19] S. Even, A. Itai and A. Shamir. On the complexity of time table and multicommodity flow problems. *16th Annual Symposium on Foundations of Computer Science*, 184–193, 1975.

[20] D. Fanding. A faster algorithm for shortest-path – SPFA. *Journal of Southwest Jiaotong University*, 2, 1994.

[21] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.

[22] J. Fischer and V. Heun. Theoretical and practical improvements on the RMQ-problem, with applications to LCA and LCE. In *Annual Symposium on Combinatorial Pattern Matching*, 36–48, 2006.

[23] R. W. Floyd Algorithm 97: shortest path. *Communications of the ACM*, 5(6):345, 1962.

[24] L. R. Ford. Network flow theory. RAND Corporation, Santa Monica, California, 1956.

[25] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8(3):399–404, 1956.

[26] R. Freivalds. Probabilistic machines can use less running time. In *IFIP congress*, 839–842, 1977.

[27] F. Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, 296–303, 2014.

[28] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman and Company, 1979.

[29] Google Code Jam Statistics (2017), https://www.go-hero.net/jam/17

[30] A. Grønlund and S. Pettie. Threesomes, degenerates, and love triangles. In *Proceedings of the 55th Annual Symposium on Foundations of Computer Science*, 621–630, 2014.

[31] P. M. Grundy. Mathematics and games. *Eureka*, 2(5):6–8, 1939.

[32] D. Gusfield. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.

[33] S. Halim and F. Halim. *Competitive Programming 3: The New Lower Bound of Programming Contests*, 2013.

[34] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.

[35] C. Hierholzer and C. Wiener. Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. *Mathematische Annalen*, 6(1), 30–32, 1873.

[36] C. A. R. Hoare. Algorithm 64: Quicksort. *Communications of the ACM*, 4(7):321, 1961.

[37] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.

[38] J. E. Hopcroft and J. D. Ullman. A linear list merging algorithm. Technical report, Cornell University, 1971.

[39] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM*, 21(2):277–292, 1974.

[40] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.

[41] The International Olympiad in Informatics Syllabus, https://people.ksp.sk/~misof/ioi-syllabus/

[42] R. M. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.

[43] P. W. Kasteleyn. The statistics of dimers on a lattice: I. The number of dimer arrangements on a quadratic lattice. *Physica*, 27(12):1209–1225, 1961.

[44] C. Kent, G. M. Landau and M. Ziv-Ukelson. On the complexity of sparse exon assembly. *Journal of Computational Biology*, 13(5):1013–1027, 2006.

[45] J. Kleinberg and É. Tardos. *Algorithm Design*, Pearson, 2005.

[46] D. E. Knuth. *The Art of Computer Programming. Volume 2: Seminumerical Algorithms*, Addison–Wesley, 1998 (3rd edition).

[47] D. E. Knuth. *The Art of Computer Programming. Volume 3: Sorting and Searching*, Addison–Wesley, 1998 (2nd edition).

[48] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.

[49] V. I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics doklady*, 10(8):707–710, 1966.

[50] M. G. Main and R. J. Lorentz. An $O(n \log n)$ algorithm for finding all repetitions in a string. *Journal of Algorithms*, 5(3):422–432, 1984.

[51] J. Pachocki and J. Radoszewski. Where to use and how not to use polynomial string hashing. *Olympiads in Informatics*, 7(1):90–100, 2013.

[52] I. Parberry. An efficient algorithm for the Knight's tour problem. *Discrete Applied Mathematics*, 73(3):251–260, 1997.

[53] D. Pearson. A polynomial-time algorithm for the change-making problem. *Operations Research Letters*, 33(3):231–234, 2005.

[54] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.

[55] 27-Queens Puzzle: Massively Parallel Enumeration and Solution Counting. https://github.com/preusser/q27

[56] M. I. Shamos and D. Hoey. Closest-point problems. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, 151–162, 1975.

[57] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.

[58] S. S. Skiena. *The Algorithm Design Manual*, Springer, 2008 (2nd edition).

[59] S. S. Skiena and M. A. Revilla. *Programming Challenges: The Programming Contest Training Manual*, Springer, 2003.

[60] SZKOpuł, https://szkopul.edu.pl/

[61] R. Sprague. Über mathematische Kampfspiele. *Tohoku Mathematical Journal*, 41:438–444, 1935.

[62] P. Stańczyk. *Algorytmika praktyczna w konkursach Informatycznych*, MSc thesis, University of Warsaw, 2006.

[63] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.

[64] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[65] R. E. Tarjan. Applications of path compression on balanced trees. *Journal of the ACM*, 26(4):690–715, 1979.

[66] R. E. Tarjan and U. Vishkin. Finding biconnected componemts and computing tree functions in logarithmic parallel time. In *Proceedings of the 25th Annual Symposium on Foundations of Computer Science*, 12–20, 1984.

[67] H. N. V. Temperley and M. E. Fisher. Dimer problem in statistical mechanics – an exact result. *Philosophical Magazine*, 6(68):1061–1063, 1961.

[68] USA Computing Olympiad, http://www.usaco.org/

[69] H. C. von Warnsdorf. *Des Rösselsprunges einfachste und allgemeinste Lösung*. Schmalkalden, 1823.

[70] S. Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.